

# **Real VLA**

**로봇공학자의 관점에서 본 Vision-Language-Action**

Giseop Kim

Author: Giseop Kim

Contact: [gsk@dgist.ac.kr](mailto:gsk@dgist.ac.kr)

Build date: 2026-05-17

Format: 170mm × 240mm technical manuscript

# 서문

이 책은 VLA를 처음 배우는 일반 AI 독자를 위한 입문서가 아니다. 이 책은 로봇공학, 제어, 로봇러닝, 컴퓨터비전 배경을 가진 대학원생과 연구자가 Vision-Language-Action system을 engineering 관점에서 해석하기 위한 field manual이다.

이 책의 중심 질문은 “어떤 모델이 가장 큰가”가 아니라 “언어 조건부 perception과 learned action이 실제 로봇의 state estimation, controller, safety monitor, data engine, evaluation protocol과 어떤 책임 경계로 연결되는가”이다. 따라서 이 책은 model leaderboard를 만들지 않는다. 대신 VLA 논문과 시스템을 읽을 때 반드시 물어야 할 interface, evidence, failure mode를 정리한다.

## 이 책의 독자

이 책은 다음 독자를 기준으로 한다.

- 로봇공학, 제어, motion planning, robot learning 중 적어도 하나의 배경을 가진 대학원생
- VLM/LLM 기반 embodied AI 논문을 실제 robot deployment 관점에서 읽고 싶은 연구자
- VLA model을 사용하거나 fine-tuning할 때 action representation, controller interface, evaluation report를 설계해야 하는 엔지니어

이 책은 classical robotics textbook, reinforcement learning textbook, transformer textbook을 대체하지 않는다. 필요한 개념은 다시 정의하지만, 목적은 개념 자체의 교과서적 완결성이 아니라 Real VLA system claim을 검토하는 기준을 만드는 것이다.

## 이 책에서 다루지 않는 것

- VLM/LLM 일반론 전체를 설명하지 않는다.
- 강화학습과 최적제어의 표준 교과서 내용을 처음부터 증명하지 않는다.
- 특정 VLA 모델의 leaderboard 순위를 고정하지 않는다.
- 실제 배치 evidence 없이 architecture novelty 만으로 capability를 판정하지 않는다.

## 어떻게 읽을 것인가

Part I은 Real VLA가 올라갈 robot substrate를 고정한다. Part II는 imitation, RL, visuomotor policy, VLM grounding을 VLA-specific failure 관점에서 읽는다. Part III는 planner와 robotics transformer에서 action representation, data engine, adaptation으로 이어지는 현대 VLA의 중심부이다. Part IV는 evaluation, hierarchy, real-time, safety를 통해 deployment evidence를 다룬다. Part V는 humanoid, whole-book synthesis, future direction으로 범위를 넓힌다.

각 장을 읽을 때는 다음 네 질문을 유지한다.

1. 이 장에서 바뀌는 interface는 무엇인가?
2. 이 장의 주장은 어떤 변수, constraint, data record, metric으로 검증되는가?
3. running example인 “머그컵을 drying rack에 올리고 유리컵은 건드리지 말라”에서 어떤 failure를 설명하는가?
4. 논문을 읽을 때 어떤 evidence가 없으면 claim을 약하게 봐야 하는가?

## Notation and Glossary

## 주요 notation

| 기호             | 이 책에서의 의미                                                                                       |
|----------------|-------------------------------------------------------------------------------------------------|
| $\mathbf{o}_t$ | time $t$ 의 observation. RGB, depth, tactile, proprioception, language context를 포함할 수 있다.        |
| $\mathbf{s}_t$ | 실제 physical state. object pose, robot configuration, contact state, hidden disturbance 등을 포함한다. |
| $\mathbf{b}_t$ | state uncertainty를 포함한 belief 또는 internal estimate.                                             |
| $\ell$         | language instruction 또는 task specification.                                                     |
| $\mathbf{a}_t$ | VLA policy가 내는 policy-level action. token, vector, waypoint, chunk, distribution일 수 있다.         |
| $\mathbf{u}_t$ | controller, driver, actuator가 실행하는 control command. velocity, torque, impedance target 등이다.     |
| $\mathbf{q}_t$ | robot joint configuration 또는 generalized coordinate.                                            |
| $C_t$          | constraint set. collision, contact, joint limit, safety envelope를 포함한다.                         |
| $h_t$          | history 또는 memory state. observation/action/outcome의 temporal context이다.                        |

## Notation discipline

이 책에서 반복되는 기호는 전역 의미를 우선한다. 특히  $\mathbf{a}_t$ 는 VLA policy의 policy-level action,  $\mathbf{u}_t$ 는 controller command,  $\mathbf{q}_t$ 는 robot configuration으로 고정한다. symbolic planner의 discrete action은 필요하면  $\alpha_k$ , skill은  $\sigma_i$ , runtime record는  $\rho_t$ , metadata는  $m_i$ , contact mode는  $\mu_t$ 로 쓴다. 같은 문자 하나로 state, skill, query, reward, record를 동시에 나타내지 않는 것이 원칙이다.

일부 수식은 theorem이 아니라 diagnostic notation이다. 예를 들어 claim tuple, evidence vector, executability gap은 system을 읽기 위한 구조화된 표기이지, 모든 장에서 동일하게 측정되는 scalar metric이 아니다. 그런 경우 본문에서 “diagnostic notation” 또는 “설계 원칙을 나타내는 notation”이라고 명시한다.

# 주요 용어

---

| Term                  | 이 책에서의 의미                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------|
| VLM                   | image/text semantic model. 장면을 설명하거나 언어 질의를 처리하지만, 그 자체가 robot execution을 보장하지는 않는다.           |
| VLA policy            | observation, language, state estimate로부터 policy-level action을 내는 learned policy.               |
| Real VLA system       | VLA policy, controller, monitor, safety, data, evaluation까지 포함한 책임 구조.                         |
| Action representation | $\mathbf{a}_t$ 의 형식. token, continuous vector, waypoint, chunk, diffusion/flow sample 등이 포함된다. |
| Control boundary      | $\mathbf{a}_t$ 가 $\mathbf{u}_t$ 로 바뀌는 경계. 이 경계가 불명확하면 논문 claim이 실제 robot behavior로 해석되지 않는다.   |
| Data engine           | data collection, curation, labeling, failure mining, evaluation replay가 반복되는 system loop.      |
| Safety shield         | action 또는 controller reference를 safe set으로 projection, reject, revalidate하는 runtime layer.     |

---

# Contents

---

|                                              |           |
|----------------------------------------------|-----------|
| 서문 . . . . .                                 | iii       |
| 이 책의 독자 . . . . .                            | iii       |
| 이 책에서 다루지 않는 것 . . . . .                     | iv        |
| 어떻게 읽을 것인가 . . . . .                         | iv        |
| <b>Notation and Glossary . . . . .</b>       | <b>iv</b> |
| 주요 notation . . . . .                        | v         |
| Notation discipline . . . . .                | v         |
| 주요 용어 . . . . .                              | vi        |
| <b>0 이 책의 기획 의도 . . . . .</b>                | <b>1</b>  |
| 0.1 왜 이 책이 필요한가 . . . . .                    | 1         |
| 0.2 VLA를 만든 계보 . . . . .                     | 2         |
| 0.3 이 책이 아닌 것 . . . . .                      | 4         |
| 0.4 대상 독자와 전제 지식 . . . . .                   | 5         |
| 0.5 책 전체의 판정 기준 . . . . .                    | 6         |
| 0.6 Evidence tier와 citation policy . . . . . | 7         |
| 0.7 Real VLA를 시스템으로 보는 이유 . . . . .          | 8         |
| 0.8 장 구성의 의도 . . . . .                       | 9         |
| 0.8.1 각 챕터의 존재 이유 . . . . .                  | 9         |
| 0.9 공학적 엄밀성의 기준 . . . . .                    | 13        |
| 0.10 이 책을 읽는 방법 . . . . .                    | 14        |
| 0.11 요약 . . . . .                            | 14        |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| 0.12 Further reading and exercises . . . . .                | 15        |
| <b>I Real VLA Stack</b>                                     | <b>16</b> |
| <b>1 Real VLA란 무엇인가 . . . . .</b>                           | <b>17</b> |
| 1.1 VLA는 더 큰 VLM이 아니다 . . . . .                             | 17        |
| 1.1.1 정책 action과 제어 입력 . . . . .                            | 18        |
| 1.2 언어 목표에서 모터 명령까지 . . . . .                               | 19        |
| 1.2.1 Task semantics . . . . .                              | 19        |
| 1.2.2 Embodied reasoning . . . . .                          | 19        |
| 1.2.3 Policy-level action . . . . .                         | 20        |
| 1.2.4 Control command . . . . .                             | 20        |
| 1.3 다섯 개의 책임 layer . . . . .                                | 21        |
| 1.3.1 Embodied reasoning layer . . . . .                    | 21        |
| 1.3.2 VLA policy layer . . . . .                            | 22        |
| 1.3.3 Motion/control layer . . . . .                        | 22        |
| 1.3.4 Safety/assurance layer . . . . .                      | 22        |
| 1.3.5 Data/evaluation layer . . . . .                       | 22        |
| 1.4 인터페이스 계약: observation, state, action, control . . . . . | 23        |
| 1.5 역사적 수렴: 제어, 계획, 로봇러닝, VLM/LLM . . . . .                 | 24        |
| 1.6 현대 VLA system pattern . . . . .                         | 24        |
| 1.7 Data, evaluation, safety는 appendices가 아니다 . . . . .     | 25        |
| 1.7.1 Robot data의 구조 . . . . .                              | 25        |
| 1.7.2 Benchmark success와 robot capability . . . . .         | 26        |
| 1.7.3 Safety as stack property . . . . .                    | 27        |
| 1.8 VLA 논문을 읽는 여섯 가지 질문 . . . . .                           | 27        |
| 1.9 이 장이 고정하는 것 . . . . .                                   | 28        |
| 1.10 Further reading and exercises . . . . .                | 30        |
| <b>2 State, Dynamics, and Estimation . . . . .</b>          | <b>31</b> |
| 2.1 로봇이 본다는 것은 상태를 안다는 뜻이 아니다 . . . . .                     | 31        |
| 2.2 State-space formulation . . . . .                       | 32        |



|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| 2.3      | Partial observability . . . . .                      | 33        |
| 2.4      | VLA policy가 belief를 사용하는 방식 . . . . .                | 34        |
| 2.5      | Dynamics and controller-relevant state . . . . .     | 34        |
| 2.6      | Latency and time alignment . . . . .                 | 35        |
| 2.7      | Calibration and coordinate frames . . . . .          | 36        |
| 2.8      | State uncertainty and safety . . . . .               | 37        |
| 2.9      | Estimation failure modes . . . . .                   | 38        |
| 2.10     | Further reading and exercises . . . . .              | 38        |
| 2.11     | Robotician's Takeaway . . . . .                      | 39        |
| <b>3</b> | <b>Control Interfaces . . . . .</b>                  | <b>40</b> |
| 3.1      | Control interface가 필요한 이유 . . . . .                  | 40        |
| 3.2      | Joint-space interfaces . . . . .                     | 41        |
| 3.2.1    | Joint position target . . . . .                      | 41        |
| 3.2.2    | Joint velocity target . . . . .                      | 41        |
| 3.2.3    | Torque command . . . . .                             | 42        |
| 3.3      | Cartesian interfaces . . . . .                       | 42        |
| 3.3.1    | End-effector target pose . . . . .                   | 42        |
| 3.3.2    | Delta pose . . . . .                                 | 43        |
| 3.4      | Waypoint and trajectory interfaces . . . . .         | 44        |
| 3.5      | Impedance and operational-space interfaces . . . . . | 44        |
| 3.5.1    | Impedance target . . . . .                           | 44        |
| 3.5.2    | Operational-space control . . . . .                  | 45        |
| 3.6      | MPC reference interface . . . . .                    | 45        |
| 3.7      | Safety projection . . . . .                          | 45        |
| 3.8      | Control interface별 failure mode . . . . .            | 46        |
| 3.9      | Evaluation log에 남겨야 할 control metadata . . . . .     | 47        |
| 3.10     | Further reading and exercises . . . . .              | 47        |
| 3.11     | Robotician's Takeaway . . . . .                      | 48        |

|           |                                               |           |
|-----------|-----------------------------------------------|-----------|
| <b>4</b>  | <b>Contact, Force, and Manipulation</b>       | <b>49</b> |
| 4.1       | Manipulation은 접촉 문제다                          | 49        |
| 4.2       | Contact state and mode                        | 50        |
| 4.3       | Contact force and friction cone               | 51        |
| 4.4       | Hybrid position/force control                 | 52        |
| 4.5       | Impedance and admittance                      | 52        |
| 4.6       | Grasp stability and slip                      | 53        |
| 4.7       | Tactile and force sensing                     | 54        |
| 4.8       | Examples: insertion, pushing, and opening     | 55        |
| 4.9       | Safety filter for contact                     | 55        |
| 4.10      | Evaluation metadata                           | 56        |
| 4.11      | Further reading and exercises                 | 57        |
| 4.12      | 요약                                            | 57        |
| <b>5</b>  | <b>Planning, Motion Planning, and TAMP</b>    | <b>59</b> |
| 5.1       | Planning은 action sequence의 문제다                | 59        |
| 5.2       | Preconditions and effects                     | 60        |
| 5.3       | Search and heuristic                          | 61        |
| 5.4       | Motion planning in configuration space        | 62        |
| 5.5       | Sampling-based planning                       | 62        |
| 5.6       | Trajectory optimization                       | 63        |
| 5.7       | Task and motion planning                      | 63        |
| 5.8       | VLA as planner, scorer, or executor           | 64        |
| 5.9       | Closed-loop replanning                        | 65        |
| 5.10      | Evaluation metadata                           | 65        |
| 5.11      | Further reading and exercises                 | 66        |
| 5.12      | 요약                                            | 66        |
| <b>II</b> | <b>Learning Foundations</b>                   | <b>67</b> |
| <b>6</b>  | <b>Imitation Learning and Covariate Shift</b> | <b>68</b> |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 6.1      | Demonstration data . . . . .                                   | 68        |
| 6.2      | Behavior cloning . . . . .                                     | 69        |
| 6.3      | Open-loop accuracy is not closed-loop success . . . . .        | 70        |
| 6.4      | Covariate shift . . . . .                                      | 71        |
| 6.5      | Recovery as a learned behavior . . . . .                       | 71        |
| 6.6      | DAgger and dataset aggregation . . . . .                       | 72        |
| 6.7      | Intervention and correction labels . . . . .                   | 72        |
| 6.8      | Success-only data의 위험 . . . . .                                | 73        |
| 6.9      | Action representation and imitation difficulty . . . . .       | 74        |
| 6.10     | Data engine view . . . . .                                     | 74        |
| 6.11     | Further reading and exercises . . . . .                        | 75        |
| 6.12     | 요약 . . . . .                                                   | 75        |
| <b>7</b> | <b>RL, Optimal Control, and Control as Inference . . . . .</b> | <b>77</b> |
| 7.1      | Why imitation is not enough . . . . .                          | 77        |
| 7.2      | Language-conditioned MDP . . . . .                             | 78        |
| 7.3      | Return, value, and Q function . . . . .                        | 79        |
| 7.4      | Reward is not just success . . . . .                           | 79        |
| 7.5      | Finite-horizon optimal control . . . . .                       | 80        |
| 7.6      | Constrained RL and safety . . . . .                            | 80        |
| 7.7      | Maximum entropy RL . . . . .                                   | 81        |
| 7.8      | Control as inference . . . . .                                 | 81        |
| 7.9      | Offline RL and robot data . . . . .                            | 82        |
| 7.10     | Evaluation metadata . . . . .                                  | 82        |
| 7.11     | Further reading and exercises . . . . .                        | 83        |
| 7.12     | 요약 . . . . .                                                   | 83        |
| <b>8</b> | <b>Visuomotor Policies . . . . .</b>                           | <b>85</b> |
| 8.1      | From vision to action . . . . .                                | 85        |
| 8.2      | Visual observation is not state . . . . .                      | 86        |
| 8.3      | Closed-loop visual feedback . . . . .                          | 87        |

|            |                                                               |            |
|------------|---------------------------------------------------------------|------------|
| 8.4        | Temporal memory . . . . .                                     | 87         |
| 8.5        | Attention and transformer policies . . . . .                  | 88         |
| 8.6        | Real-robot grasping . . . . .                                 | 88         |
| 8.7        | Pushing, reaching, insertion . . . . .                        | 89         |
| 8.8        | Action chunking and bimanual visuomotor policy . . . . .      | 90         |
| 8.9        | From visuomotor policy to VLA . . . . .                       | 90         |
| 8.10       | Evaluation metadata . . . . .                                 | 90         |
| 8.11       | Further reading and exercises . . . . .                       | 92         |
| 8.12       | 요약 . . . . .                                                  | 92         |
| <b>9</b>   | <b>Transformers, VLMs, and Grounding . . . . .</b>            | <b>93</b>  |
| 9.1        | Why VLM matters for VLA . . . . .                             | 93         |
| 9.2        | Tokenization . . . . .                                        | 94         |
| 9.3        | Self-attention . . . . .                                      | 95         |
| 9.4        | Contrastive image-text alignment . . . . .                    | 95         |
| 9.5        | Multimodal fusion . . . . .                                   | 96         |
| 9.6        | Four kinds of grounding . . . . .                             | 96         |
| 9.7        | From VLM prior to VLA policy . . . . .                        | 97         |
| 9.8        | Grounding gap . . . . .                                       | 98         |
| 9.9        | Evaluation metadata . . . . .                                 | 98         |
| 9.10       | Further reading and exercises . . . . .                       | 99         |
| 9.11       | 요약 . . . . .                                                  | 99         |
| <b>III</b> | <b>From Planner to VLA . . . . .</b>                          | <b>101</b> |
| <b>10</b>  | <b>LLM/VLM as Robot Planner . . . . .</b>                     | <b>102</b> |
| 10.1       | Planner로서의 LLM/VLM . . . . .                                  | 102        |
| 10.2       | Skill library와 interface . . . . .                            | 103        |
| 10.3       | SayCan: language likelihood와 affordance . . . . .             | 104        |
| 10.4       | Code as Policies: plan text에서 executable program으로 . . . . .  | 105        |
| 10.5       | Inner Monologue: feedback을 포함한 closed-loop planning . . . . . | 106        |

|           |                                            |            |
|-----------|--------------------------------------------|------------|
| 10.6      | VoxPoser: value map과 model-based planning  | 107        |
| 10.7      | Executability gap                          | 109        |
| 10.8      | Planner+skills에서 VLA로                      | 110        |
| 10.9      | 평가 관점                                      | 111        |
| 10.10     | Further reading and exercises              | 111        |
| 10.11     | 요약                                         | 111        |
| <b>11</b> | <b>From Planner + Skills to VLA</b>        | <b>113</b> |
| 11.1      | Planner + skills 구조의 기본형                   | 113        |
| 11.2      | Skill boundary의 병목                         | 114        |
| 11.3      | VLA가 흡수하는 responsibility                   | 115        |
| 11.4      | Interface spectrum                         | 116        |
| 11.5      | Latent skill과 hierarchical policy          | 116        |
| 11.6      | Residual VLA                               | 117        |
| 11.7      | Closed-loop feedback의 위치 변화                | 118        |
| 11.8      | Middle ground로서의 hierarchical VLA          | 118        |
| 11.9      | VLA로 넘어가는 조건                               | 119        |
| 11.10     | Further reading and exercises              | 119        |
| 11.11     | 요약                                         | 120        |
| <b>12</b> | <b>Robotics Transformer Era</b>            | <b>121</b> |
| 12.1      | 전환점: action as sequence prediction         | 121        |
| 12.2      | Language-conditioned imitation learning    | 122        |
| 12.3      | Gato와 generalist agent 관점                  | 123        |
| 12.4      | RT-1: real robot data와 action token        | 123        |
| 12.5      | PaLM-E: embodied multimodal language model | 124        |
| 12.6      | RT-2: action을 token처럼 다루기                  | 124        |
| 12.7      | RoboCat과 multi-embodiment adaptation       | 125        |
| 12.8      | Robotics Transformer 계열 비교                 | 125        |
| 12.9      | 왜 이것이 Real VLA의 시작인가                       | 126        |
| 12.10     | 2024-2025 흐름으로 이어지는 질문                     | 127        |

|           |                                                                |            |
|-----------|----------------------------------------------------------------|------------|
| 12.11     | Further reading and exercises . . . . .                        | 128        |
| 12.12     | 요약 . . . . .                                                   | 128        |
| <b>13</b> | <b>Action Representation Is the Control Boundary . . . . .</b> | <b>129</b> |
| 13.1      | Why action representation is the control boundary . . . . .    | 129        |
| 13.2      | Action representation spectrum . . . . .                       | 130        |
| 13.3      | Canonical VLA case comparison . . . . .                        | 130        |
| 13.4      | Discrete action tokens . . . . .                               | 133        |
| 13.5      | Continuous regression and normalization . . . . .              | 134        |
| 13.6      | Waypoint, delta pose, and trajectory target . . . . .          | 135        |
| 13.7      | Action chunking as temporal interface . . . . .                | 135        |
| 13.8      | Diffusion policies and multimodal actions . . . . .            | 136        |
| 13.9      | Flow matching and action expert . . . . .                      | 137        |
| 13.10     | Action tokenizers and compression . . . . .                    | 137        |
| 13.11     | Choosing an interface . . . . .                                | 138        |
| 13.12     | Failure modes . . . . .                                        | 138        |
| 13.13     | Checklist . . . . .                                            | 138        |
| 13.14     | Running example: mug, rack, and glass . . . . .                | 140        |
| 13.15     | Further reading and exercises . . . . .                        | 140        |
| 13.16     | 요약 . . . . .                                                   | 141        |
| <b>14</b> | <b>Data Engines . . . . .</b>                                  | <b>142</b> |
| 14.1      | Robot data is not web data . . . . .                           | 142        |
| 14.2      | Data engine loop . . . . .                                     | 143        |
| 14.3      | Data record schema . . . . .                                   | 144        |
| 14.4      | Robot dataset card . . . . .                                   | 144        |
| 14.5      | Demonstration collection . . . . .                             | 146        |
| 14.6      | Action normalization and embodiment metadata . . . . .         | 147        |
| 14.7      | Failure, correction, and recovery data . . . . .               | 148        |
| 14.8      | Simulation, synthetic data, and human video . . . . .          | 148        |
| 14.9      | Curation and relabeling . . . . .                              | 149        |

|           |                                                      |            |
|-----------|------------------------------------------------------|------------|
| 14.10     | Data quality metrics . . . . .                       | 149        |
| 14.11     | Data engine and safety . . . . .                     | 149        |
| 14.12     | Further reading and exercises . . . . .              | 150        |
| 14.13     | 요약 . . . . .                                         | 150        |
| <b>15</b> | <b>Adaptation and Fine-Tuning . . . . .</b>          | <b>151</b> |
| 15.1      | Why adaptation is unavoidable . . . . .              | 151        |
| 15.2      | Adaptation targets . . . . .                         | 152        |
| 15.3      | Action head adaptation . . . . .                     | 152        |
| 15.4      | Embodiment adapter . . . . .                         | 153        |
| 15.5      | LoRA, adapter tuning, and full fine-tuning . . . . . | 154        |
| 15.6      | Fine-tuning objective . . . . .                      | 154        |
| 15.7      | Data mixture for adaptation . . . . .                | 155        |
| 15.8      | Action normalization and calibration . . . . .       | 155        |
| 15.9      | Validation protocol . . . . .                        | 155        |
| 15.10     | Case-study roles . . . . .                           | 156        |
| 15.11     | Deployment checklist . . . . .                       | 156        |
| 15.12     | Further reading and exercises . . . . .              | 157        |
| 15.13     | 요약 . . . . .                                         | 158        |
| <b>IV</b> | <b>Deployment Evidence . . . . .</b>                 | <b>159</b> |
| <b>16</b> | <b>Evaluation and Benchmarking . . . . .</b>         | <b>160</b> |
| 16.1      | Why success rate is not enough . . . . .             | 160        |
| 16.2      | Benchmark families . . . . .                         | 161        |
| 16.3      | Evaluation protocol schema . . . . .                 | 162        |
| 16.4      | Action-dependent metrics . . . . .                   | 162        |
| 16.5      | Robustness and distribution shift . . . . .          | 162        |
| 16.6      | Failure taxonomy . . . . .                           | 163        |
| 16.7      | Statistical reporting . . . . .                      | 164        |
| 16.8      | Evaluation harnesses and reproducibility . . . . .   | 164        |

|           |                                                          |            |
|-----------|----------------------------------------------------------|------------|
| 16.9      | Benchmark artifacts . . . . .                            | 164        |
| 16.10     | Real robot reporting standard . . . . .                  | 165        |
| 16.11     | Further reading and exercises . . . . .                  | 166        |
| 16.12     | 요약 . . . . .                                             | 167        |
| <b>17</b> | <b>Hierarchical VLA and Embodied Reasoning . . . . .</b> | <b>168</b> |
| 17.1      | Why hierarchy returns . . . . .                          | 168        |
| 17.2      | Embodied reasoning is not text reasoning . . . . .       | 169        |
| 17.3      | Options view of hierarchical VLA . . . . .               | 170        |
| 17.4      | Interface contract between reasoning and VLA . . . . .   | 171        |
| 17.5      | Progress detection and success estimation . . . . .      | 172        |
| 17.6      | VLA plus TAMP . . . . .                                  | 173        |
| 17.7      | Tool and function calling . . . . .                      | 173        |
| 17.8      | World models and memory . . . . .                        | 174        |
| 17.9      | Monolithic versus hierarchical design . . . . .          | 174        |
| 17.10     | Safety gates . . . . .                                   | 175        |
| 17.11     | Logging and evaluation . . . . .                         | 176        |
| 17.12     | Design checklist . . . . .                               | 177        |
| 17.13     | Further reading and exercises . . . . .                  | 177        |
| 17.14     | 요약 . . . . .                                             | 178        |
| <b>18</b> | <b>Real-Time and Hardware-Aware VLA . . . . .</b>        | <b>179</b> |
| 18.1      | Latency is part of the dynamics . . . . .                | 179        |
| 18.2      | Latency budget . . . . .                                 | 180        |
| 18.3      | Time-scale hierarchy . . . . .                           | 181        |
| 18.4      | Action chunking . . . . .                                | 182        |
| 18.5      | Asynchronous execution . . . . .                         | 183        |
| 18.6      | Hardware constraints . . . . .                           | 183        |
| 18.7      | On-device versus cloud inference . . . . .               | 183        |
| 18.8      | Compression and its robot-specific risks . . . . .       | 185        |
| 18.9      | Watchdog and fallback controller . . . . .               | 186        |



|           |                                                            |            |
|-----------|------------------------------------------------------------|------------|
| 18.10     | Timing report standard . . . . .                           | 186        |
| 18.11     | Design checklist . . . . .                                 | 187        |
| 18.12     | Further reading and exercises . . . . .                    | 188        |
| 18.13     | 요약 . . . . .                                               | 188        |
| <b>19</b> | <b>Safety and Assurance . . . . .</b>                      | <b>190</b> |
| 19.1      | Safety is a stack property . . . . .                       | 190        |
| 19.2      | Unsafe success and safe failure . . . . .                  | 191        |
| 19.3      | Risk model . . . . .                                       | 192        |
| 19.4      | Semantic safety . . . . .                                  | 192        |
| 19.5      | Geometric and dynamic safety . . . . .                     | 193        |
| 19.6      | Safety shield . . . . .                                    | 193        |
| 19.7      | Runtime monitor . . . . .                                  | 194        |
| 19.8      | Human supervision . . . . .                                | 195        |
| 19.9      | Data safety . . . . .                                      | 195        |
| 19.10     | Red-teaming and safety benchmarks . . . . .                | 195        |
| 19.11     | Assurance case . . . . .                                   | 197        |
| 19.12     | Incident report standard . . . . .                         | 197        |
| 19.13     | Further reading and exercises . . . . .                    | 198        |
| 19.14     | 요약 . . . . .                                               | 198        |
| <b>V</b>  | <b>Beyond Manipulation . . . . .</b>                       | <b>200</b> |
| <b>20</b> | <b>Humanoids and Whole-Body VLA . . . . .</b>              | <b>201</b> |
| 20.1      | Humanoid VLA is not high-dimensional arm control . . . . . | 201        |
| 20.2      | Whole-body state . . . . .                                 | 202        |
| 20.3      | Balance and support constraints . . . . .                  | 203        |
| 20.4      | Interface to whole-body controller . . . . .               | 203        |
| 20.5      | Locomotion-manipulation coupling . . . . .                 | 204        |
| 20.6      | Bimanual and dexterous control . . . . .                   | 205        |
| 20.7      | Dual-system architecture . . . . .                         | 205        |

---

|           |                                                    |            |
|-----------|----------------------------------------------------|------------|
| 20.8      | Humanoid data engine . . . . .                     | 208        |
| 20.9      | Safety envelope . . . . .                          | 209        |
| 20.10     | Evaluation . . . . .                               | 209        |
| 20.11     | Design checklist . . . . .                         | 210        |
| 20.12     | Further reading and exercises . . . . .            | 210        |
| 20.13     | 요약 . . . . .                                       | 210        |
| <b>21</b> | <b>What We Have Learned . . . . .</b>              | <b>212</b> |
| 21.1      | The thesis revisited . . . . .                     | 212        |
| 21.2      | Part I: robot substrate . . . . .                  | 213        |
| 21.3      | Part II: learning foundations . . . . .            | 214        |
| 21.4      | Part III: from planner to VLA . . . . .            | 215        |
| 21.5      | Part IV: deployment evidence . . . . .             | 216        |
| 21.6      | Part V: beyond manipulation . . . . .              | 217        |
| 21.7      | Notation recap . . . . .                           | 217        |
| 21.8      | The system loop . . . . .                          | 217        |
| 21.9      | Running example revisited . . . . .                | 218        |
| 21.10     | Twelve questions for reading VLA papers . . . . .  | 219        |
| 21.11     | Ten propositions . . . . .                         | 220        |
| 21.12     | Further reading and exercises . . . . .            | 221        |
| 21.13     | What remains . . . . .                             | 222        |
| <b>22</b> | <b>Future Directions . . . . .</b>                 | <b>223</b> |
| 22.1      | Beyond bigger VLMs . . . . .                       | 223        |
| 22.2      | World models . . . . .                             | 224        |
| 22.3      | Information gathering . . . . .                    | 225        |
| 22.4      | Self-improving data loop . . . . .                 | 225        |
| 22.5      | Safe online adaptation . . . . .                   | 225        |
| 22.6      | Multi-embodiment transfer . . . . .                | 226        |
| 22.7      | Safety assurance as a research primitive . . . . . | 227        |
| 22.8      | Hardware-aware scaling . . . . .                   | 228        |

|       |                                                |            |
|-------|------------------------------------------------|------------|
| 22.9  | Evaluation becomes infrastructure . . . . .    | 228        |
| 22.10 | Research agenda . . . . .                      | 229        |
| 22.11 | What should roboticists learn? . . . . .       | 229        |
| 22.12 | Further reading and exercises . . . . .        | 230        |
| 22.13 | Closing synthesis . . . . .                    | 230        |
|       | <b>Instructor and Editorial Note . . . . .</b> | <b>232</b> |
|       | <b>참고 문헌 . . . . .</b>                         | <b>234</b> |

## 제0장

# 이 책의 기획 의도

## 학습 목표

이 장은 기술 개념을 본격적으로 설명하기 전에, 이 책이 어떤 문제의식으로 쓰였는지 고정한다. 이 장을 읽고 나면 독자는 다음을 이해해야 한다.

1. 이 책이 VLA를 단일 모델이 아니라 실제 로봇 시스템의 책임 구조로 다루는 이유를 설명할 수 있다.
2. Vision-language model, robot learning policy, controller, planner, data engine, safety layer를 분리해서 읽어야 하는 이유를 이해한다.
3. 이 책에서 말하는 “real”의 의미가 benchmark 성능이 아니라 물리 실행, 실패 기록, 안전 보장, 재현 가능한 평가와 연결됨을 이해한다.
4. 뒤 장들을 model catalog가 아니라 engineering contract의 연속으로 읽을 수 있다.

## 0.1 왜 이 책이 필요한가

Vision-Language-Action(VLA)은 로봇 연구에서 빠르게 중심 주제가 되고 있다. 큰 vision-language model은 장면을 읽고, 언어 지시를 해석하고, 행동을 예측하는 방향으로 확장되고 있다. 그러나 로봇공학자의 관점에서 중요한 질문은 조금 다르다. “어떤 모델이 가장 큰가?” 또는 “어떤 benchmark에서 점수가 높은가?”가 아니라 다음 질문이 먼저 와야 한다.

언어와 시각으로부터 나온 action이 실제 로봇의 물리 실행, 제어 안정성, 안전 조건, 실패 복구, 데이터 기록, 평가 주장으로 어떻게 연결되는가?

이 책은 이 질문을 다루기 위해 쓰였다. VLA를 단순히 VLM 위에 action head를 붙인 모델로 설명하면 핵심 문제가 빠진다. 로봇은 문장을 출력하는 시스템이 아니라, 상태를 추정하고, 접촉을 만들고, actuator limit 안에서 움직이고, 실패하면 멈추거나 복구해야 하는 물리 시스템이다. 따라서 Real VLA는 모델 구조만으로 완성되지 않는다. Real VLA는 다음 mapping 전체를 책임지는 시스템이다.

$$(\mathbf{o}_{1:t}, \ell, \mathbf{b}_t, \mathcal{C}_t) \longrightarrow (\mathbf{a}_t, \mathbf{u}_t, r_t, e_t), \quad (0.1)$$

여기서  $\mathbf{o}_{1:t}$ 는 observation history,  $\ell$ 는 language instruction 또는 task specification,  $\mathbf{b}_t$ 는 state belief,  $\mathcal{C}_t$ 는 physical/safety constraint이다. 출력의  $\mathbf{a}_t$ 는 policy-level action,  $\mathbf{u}_t$ 는 controller command,  $r_t$ 는 runtime record,  $e_t$ 는 evaluation evidence이다. 이 책의 핵심은  $\mathbf{a}_t$  하나가 아니라 이 전체 tuple을 보는 데 있다.

### 이 책의 중심 명제

Real VLA는 로봇공학을 대체하는 foundation model이 아니다. Real VLA는 perception, language, planning, control, data, evaluation, safety를 하나의 embodied system 안에서 다시 연결하는 interface이다.

## 0.2 VLA를 만든 계보

Real VLA는 갑자기 등장한 단일 모델 계열이 아니다. 현재의 VLA는 classical control, state estimation, symbolic planning, motion planning, robot learning, VLM/LLM, robot data scaling, action representation 연구가 겹치면서 만들어진 문제 설정이다. 표 1은 이 책이 어떤 역사적 흐름을 하나의 engineering contract로 다시 읽는지 요약한다.

표 1: VLA를 만든 계보: control에서 embodied foundation model까지

| 시기          | 핵심 질문                                  | 대표 흐름                                                                                                     | 이 책에서의 역할                                                                           |
|-------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1950s–1980s | 로봇을 안정적으로 움직이는가?                       | dynamic programming, Kalman filtering, LQR/MPC, impedance control, operational-space control [1, 7, 6, 8] | VLA action이 결국 물리 제어계로 들어간다는 사실을 고정한다.                                              |
| 1960s–2000s | 복잡한 목표를 어떻게 계획하는가?                     | A*, STRIPS, PDDL, GraphPlan, heuristic planning [11]                                                      | 언어 목표를 executable sub-goal과 precondition/effect로 읽는 기준을 제공한다.                       |
| 1990s–2010s | symbolic goal을 연속공간 motion으로 바꾸는가?     | PRM, RRT/RRT*, trajectory optimization, TAMP [12, 13, 14]                                                 | semantic proposal이 geometry, grasp, collision, trajectory feasibility를 통과해야 함을 보인다. |
| 2010s       | 시각에서 바로 action을 배울 수 있는가?              | DAGger, guided policy search, visual foresight, diffusion policy [15, 19, 20, 40]                         | VLA 이전의 image-to-action 학습과 covariate shift 문제를 연결한다.                               |
| 2017–2021   | 언어와 시각 표현을 크게 학습할 수 있는가?               | Transformer, ViT, CLIP [23, 24, 25]                                                                       | VLA backbone의 representation substrate를 만든다.                                        |
| 2022        | LLM/VLM을 planner처럼 쓸 수 있는가?            | SayCan, Code as Policies, Inner Monologue, VoxPoser [28, 29, 30, 31]                                      | language reasoning과 robot affordance, skill library, controller 호출의 경계를 드러낸다.       |
| 2022–2023   | robot action도 trans-former가 예측할 수 있는가? | BC-Z, Gato, RT-1, PaLM-E, RT-2, RoboCat [33, 32, 34, 35, 36, 37]                                          | VLA라는 문제 설정이 action token, robot data, embodied multimodal model로 구체화된다.            |

다음 쪽에 계속

표 1: VLA를 만든 계보 계속

| 시기        | 핵심 질문                         | 대표 흐름                                                                                      | 이 책에서의 역할                                                                               |
|-----------|-------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 2023–2024 | robot data도 scaling 되는가?      | Open X-Embodiment, BridgeData V2, DROID, Octo, OpenVLA [38, 49, 50, 41, 42]                | model scale보다 embodiment, action schema, controller metadata, failure label이 중요해짐을 보인다. |
| 2024–2025 | token action만으로 충분한가?         | ACT/ALOHA, RDT-1B, $\pi_0$ , OpenVLA-OFT, FAST [39, 46, 43, 45, 44]                        | action representation을 output format이 아니라 control boundary로 읽게 만든다.                     |
| 2025–2026 | 실제 배치 가능한 embodied system 인가? | $\pi_{0.5}$ , GR00T N1, VLA Foundry, safety/evaluation/data-engine 흐름 [48, 47, 56, 51, 55] | real-time, safety, evaluation, humanoid deployment를 VLA claim의 일부로 끌어들인다.               |

이 명제를 받아들이면 책의 구조가 자연스럽게 결정된다. 먼저 state, dynamics, estimation, control, manipulation, planning을 다시 확인해야 한다. 그 다음 imitation learning, reinforcement learning, visuomotor policy, VLM, LLM planner, VLA policy를 연결해야 한다. 마지막으로 action representation, data engine, fine-tuning, evaluation, hierarchy, real-time deployment, safety, humanoid embodiment, whole-book synthesis, future direction을 다루어야 한다. 이 순서는 역사적 유형의 순서가 아니라 책임이 달히는 순서이다.

### 0.3 이 책이 아닌 것

이 책은 VLA 논문 목록을 나열하는 survey가 아니다. 특정 모델의 leaderboard 순위를 외우기 위한 책도 아니다. 또한 classical robotics를 낡은 배경지식으로 밀어내고 foundation model이 모든 것을 해결한다는 이야기를 하려는 책도 아니다.

### 모델 이름은 시스템 책임을 대신하지 못한다

어떤 VLA 모델이 강력한 backbone을 사용하더라도, action representation, controller interface, timing budget, data mixture, safety gate, evaluation protocol이 명시되지 않으면 실제 로봇 시스템의 capability claim은 불완전하다.

이 책은 다음 세 가지 설명 방식을 피한다.

1. **VLM 확장론**: VLA를 VLM에 action token만 추가한 구조로 설명하는 방식.
2. **Benchmark 환원론**: benchmark success rate를 곧 robot capability로 읽는 방식.
3. **End-to-end 만능론**: controller, planner, estimator, safety monitor를 모두 암묵적 network 내부로 넣으면 문제가 사라진다고 보는 방식.

대신 이 책은 각 장에서 다음 질문을 반복한다.

이 방법은 어떤 입력을 받아 어떤 출력을 만들며, 그 출력은 다음 layer에서 어떤 가정 하에 실행 가능한가?

이 질문은 단순하지만 강력하다. VLA 연구가 실제 robot deployment로 가까워질수록 모든 주장은 이 질문을 통과해야 한다.

## 0.4 대상 독자와 전제 지식

이 책은 로봇공학, 제어, planning, robot learning, embodied AI를 연구하거나 수업에서 다루는 독자를 대상으로 한다. 이상적인 독자는 다음 배경 중 일부를 이미 알고 있다.

- rigid-body kinematics와 configuration space의 기본 개념
- feedback control, trajectory tracking, impedance 또는 operational-space control의 기본 개념



- supervised learning, imitation learning, reinforcement learning의 기본 notation
- transformer, VLM, LLM의 기본 구조
- robot dataset, benchmark, real-robot experiment의 기본 형식

그러나 이 책은 각 분야를 처음부터 교과서적으로 다시 쓰지 않는다. 목적은 control textbook, planning textbook, deep learning textbook을 대체하는 것이 아니다. 목적은 이 분야들이 Real VLA system 안에서 어떤 책임을 갖는지 재배치하는 것이다.

따라서 독자는 다음 관점으로 읽어야 한다. 어떤 장이 classical topic을 다루더라도, 그것은 과거 기술을 복습하기 위해서가 아니라 VLA가 물리 시스템에 들어갈 때 남는 constraint를 보기 위해서이다. 어떤 장이 foundation model을 다루더라도, 그것은 모델 규모를 찬양하기 위해서가 아니라 action, planning, data, safety interface를 해석하기 위해서이다.

## 0.5 책 전체의 판정 기준

이 책에서 하나의 VLA system claim은 다음 조건을 만족할수록 강하다고 본다.

$$\mathcal{K} = \{\mathcal{M}, \mathcal{A}, \mathcal{P}, \mathcal{U}, \mathcal{D}, \mathcal{E}, \mathcal{S}\}, \quad (0.2)$$

여기서  $\mathcal{M}$ 은 model,  $\mathcal{A}$ 는 action representation,  $\mathcal{P}$ 는 planner 또는 policy interface,  $\mathcal{U}$ 는 low-level control interface,  $\mathcal{D}$ 는 data engine,  $\mathcal{E}$ 는 evaluation protocol,  $\mathcal{S}$ 는 safety/assurance layer이다. 이 중 하나라도 비어 있으면 claim은 제한된다.

이 표는 뒤 장 전체의 읽기 규칙이다. 예를 들어 Chapter 13에서 action representation을 다룰 때는  $\mathcal{A}$ 와  $\mathcal{U}$ 의 경계를 본다. Chapter 14와 16에서는  $\mathcal{D}$ 와  $\mathcal{E}$ 를 본다. Chapter 19에서는  $\mathcal{S}$ 를 독립된 afterthought가 아니라 system claim의 일부로 다룬다.

표 2: Real VLA claim을 읽는 기준

| 항목         | 질문                                                  | 약한 claim의 전형적 형태                      |
|------------|-----------------------------------------------------|---------------------------------------|
| Model      | 어떤 observation과 language context를 처리하는가?            | backbone 이름만 제시함                      |
| Action     | action이 token, pose, delta, chunk, skill 중 무엇인가?    | action semantics가 불명확함                |
| Control    | action이 어떤 controller command로 변환되는가?               | controller가 black box로 남음             |
| Planning   | long-horizon constraint와 recovery를 어떻게 다루는가?        | plausible plan을 executable plan처럼 제시함 |
| Data       | 어떤 embodiment와 failure가 데이터에 들어갔는가?                 | success trajectory만 강조함               |
| Evaluation | 어떤 protocol에서 어떤 confidence로 검증했는가?                 | benchmark score만 제시함                  |
| Safety     | unsafe success, near miss, intervention을 어떻게 기록하는가? | 성공률과 안전성을 분리하지 않음                     |

## 0.6 Evidence tier와 citation policy

VLA 분야는 빠르게 움직인다. 그래서 이 책은 모든 reference를 같은 무게로 읽지 않는다. Kalman filtering, motion planning, operational-space control, impedance control처럼 확립된 이론은 textbook 또는 고전 논문을 기준으로 둔다 [1, 6, 7, 11]. 반면 OpenVLA,  $\pi_0$ , FAST, OpenVLA-OFT, GR00T N1 같은 최신 VLA 사례는 대개 arXiv, project page, released code, company technical report의 형태로 먼저 등장한다 [42, 43, 44, 45, 47]. 따라서 최신 사례는 “증명된 표준”이 아니라 “현재 공개된 evidence로 읽는 case”로 다룬다.

### Evidence tier

- **Tier A:** textbook, established theory, widely reproduced classical result.
- **Tier B:** peer-reviewed paper 또는 널리 재현된 benchmark paper.
- **Tier C:** arXiv preprint, project page, released code가 있는 최신 연구.

- **Tier D:** company technical report, official blog, demo 중심 발표.

Tier C/D 사례는 engineering insight를 얻기 위해 분석하되, real-robot deployment evidence가 부족하면 capability claim을 강하게 읽지 않는다.

이 정책은 최신성을 낮추기 위한 장치다. 오히려 최신 VLA 흐름을 더 정확히 읽기 위한 장치다. 예를 들어 action representation 개선 논문은 Chapter 13의 control boundary 관점에서 매우 중요하지만, 그 claim이 real robot task, simulation task, throughput benchmark 중 어디에서 닫혔는지는 별도로 기록해야 한다.

## 0.7 Real VLA를 시스템으로 보는 이유

로봇 시스템은 여러 rate로 동작한다. VLM inference는 수 Hz 이하일 수 있고, policy head는 수 Hz에서 수십 Hz로 동작할 수 있으며, low-level controller는 수백 Hz에서 kHz로 동작한다. Safety monitor와 state estimator는 또 다른 rate를 가진다. 이 차이를 무시하면 VLA를 실제 하드웨어에 올릴 때 문제가 생긴다.

이를 간단히 쓰면 다음과 같다.

$$\mathbf{b}_t = \text{Estimate}(\mathbf{b}_{t-1}, \mathbf{o}_t, \mathbf{u}_{t-1}), \quad (0.3)$$

$$z_t = \text{Reason}(\ell, \mathbf{b}_t, m_t), \quad (0.4)$$

$$\mathbf{a}_t = \pi_\theta(\mathbf{o}_t, z_t, \mathbf{q}_t), \quad (0.5)$$

$$\mathbf{u}_t = \kappa(\mathbf{a}_t, \hat{\mathbf{s}}_t, \mathcal{C}_t), \quad (0.6)$$

$$\rho_t = \text{Monitor}(\ell, \mathbf{a}_t, \mathbf{u}_t, \mathbf{o}_{t+1}). \quad (0.7)$$

$z_t$ 는 embodied reasoning 결과,  $m_t$ 는 memory 또는 retrieved context,  $\rho_t$ 는 run-time monitor output이다. Real VLA에서 중요한 것은 이 식의 각 항이 실제 system에서 누가 계산하고, 어느 주기로 갱신하며, 실패 시 어떤 fallback을 갖는지이다.

이 관점은 end-to-end learning과 모순되지 않는다. learned policy가 더 많은 책임을 흡수할수록 interface는 단순해질 수 있다. 그러나 책임이 사라지는 것은

아니다. 학습된 network 내부로 들어간 책임은 evaluation, logging, stress test를 통해 다시 외부에서 검증되어야 한다.

## 0.8 장 구성의 의도

이 책은 크게 다섯 부분으로 읽을 수 있다.

1. **Robotics foundation:** state, dynamics, estimation, control, contact, planning을 Real VLA의 실행 조건으로 재정리한다.
2. **Learning foundation:** imitation learning, reinforcement learning, visuomotor policy가 VLA policy의 직접 조상임을 설명한다.
3. **Language and foundation model bridge:** transformers, VLMs, LLM/VLM planner, planner+skills 구조가 VLA로 어떻게 이어지는지 다룬다.
4. **VLA system core:** action representation, data engine, adaptation, evaluation protocol을 통해 model claim을 system claim으로 바꾼다.
5. **Deployment and assurance:** hierarchy, real-time constraint, safety, humanoid embodiment, whole-book synthesis, future direction을 다룬다.

이 구분은 독립된 분야 소개가 아니다. 각 부분은 다음 부분의 전제 조건이다. Control을 모르면 action representation을 해석하기 어렵고, planning을 모르면 LLM/VLM planner의 한계를 읽기 어렵다. Imitation learning과 covariate shift를 모르면 VLA data engine의 의미가 약해진다. Evaluation protocol을 모르면 benchmark success와 robot capability를 분리할 수 없다.

### 0.8.1 각 챕터의 존재 이유

각 장은 독립적인 주제 소개가 아니라 하나의 Real VLA claim을 닫기 위한 역할을 갖는다. 표 3은 각 장이 왜 필요한지, 그리고 뒤 장을 읽을 때 어떤 해석 기준을 제공하는지 정리한다.

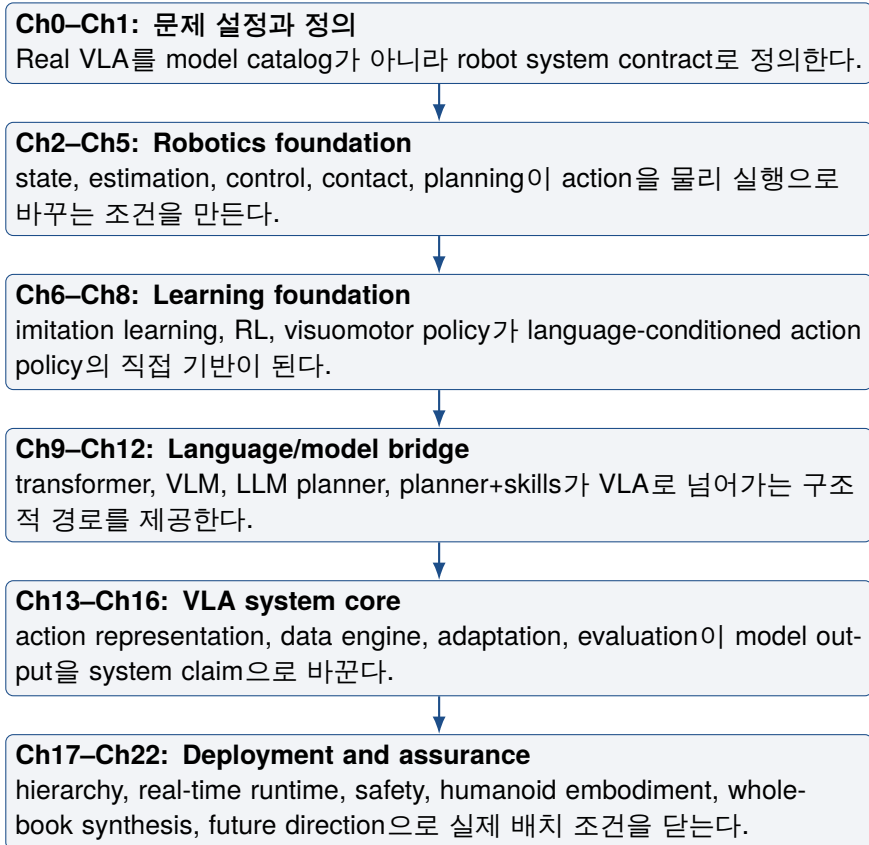


그림 1: 전체 장 구성의 의존 관계. 위쪽 장은 아래쪽 장의 배경지식이 아니라 interface와 claim을 해석하기 위한 전제 조건이다.

표 3: 각 챕터의 존재 이유와 후속 장의 해석 기준

| 장 | 제목             | 존재 이유                                                 | 후속 장의 해석 기준                       |
|---|----------------|-------------------------------------------------------|-----------------------------------|
| 0 | 이 책의 기획 의도     | 책 전체를 model survey가 아니라 engineering contract로 읽게 만든다. | Real VLA claim을 평가하는 기준           |
| 1 | Real VLA란 무엇인가 | VLA를 VLM 확장이 아니라 robot system 문제로 재정의한다.              | responsibility layer와 기본 notation |

다음 페이지에 계속

표 3 계속

| 장  | 제목                                            | 존재 이유                                                              | 후속 장의 해석 기준                               |
|----|-----------------------------------------------|--------------------------------------------------------------------|-------------------------------------------|
| 2  | State, Dynamics, and Estimation               | policy 입력이 단순 image가 아니라 추정된 state와 belief 위에 있음을 보인다.             | $s, b$ , observation/state 구분             |
| 3  | Control Interfaces                            | policy action과 controller command의 차이를 고정한다.                       | $a_t \rightarrow u_t$ interface           |
| 4  | Contact, Force, and Manipulation              | manipulation이 geometry만이 아니라 contact와 force constraint 문제임을 보인다.   | contact-aware action 해석                   |
| 5  | Planning, Motion Planning, and TAMP           | language goal이 executable plan이 되려면 feasibility check가 필요함을 보인다.   | subgoal, constraint, recovery planning    |
| 6  | Imitation Learning and Covariate Shift        | VLA policy 학습의 기본 문제가 distribution shift임을 설명한다.                   | demonstration quality와 rollout failure 관점 |
| 7  | RL, Optimal Control, and Control as Inference | reward, objective, uncertainty, feedback을 action policy 관점으로 연결한다. | policy optimization과 safety trade-off     |
| 8  | Visuomotor Policies                           | language가 불기 전 image-to-action policy의 구조와 한계를 정리한다.               | observation-to-action backbone 관점         |
| 9  | Transformers, VLMs, and Grounding             | VLM이 VLA에 주는 semantic prior와 grounding gap을 분리한다.                  | language/vision representation 관점         |
| 10 | LLM/VLM as Robot Planner                      | 큰 모델을 planner로 쓸 때 plausible plan과 executable plan의 차이를 보인다.       | planner proposal과 verifier 구조             |
| 11 | From Planner + Skills to VLA                  | skill library 기반 구조가 learned VLA policy로 넘어가는 경계를 설명한다.            | skill, action, policy의 spectrum           |

다음 페이지에 계속

표 3 계속

| 장  | 제목                                      | 존재 이유                                                                  | 후속 장의 해석 기준                                  |
|----|-----------------------------------------|------------------------------------------------------------------------|----------------------------------------------|
| 12 | Robotics Transformer Era                | RT 계열과 multi-embodiment learning 이 action token 관점을 만든 흐름을 정리한다.       | sequence prediction 으로서의 action              |
| 13 | Action Representation                   | VLA 출력이 무엇인지 정의하는 핵심 API 장이다.                                          | token, pose, delta, chunk, trajectory target |
| 14 | Data Engines                            | VLA 성능이 data collection, correction, failure mining loop에 의존함을 보인다.    | dataset schema와 data quality 기준              |
| 15 | Adaptation and Fine-Tuning              | pretrained VLA가 특정 robot/task로 옮겨갈 때 필요한 adaptation 문제를 정리한다.          | fine-tuning과 deployment claim의 경계            |
| 16 | Evaluation and Benchmarking             | benchmark success와 robot capability를 분리해서 주장하는 법을 정한다.                 | evaluation protocol과 confidence              |
| 17 | Hierarchical VLA and Embodied Reasoning | 단일 policy가 아니라 reasoning layer와 action layer가 나뉘는 구조를 다룬다.             | hierarchy, monitor, re-plan trigger          |
| 18 | Real-Time and Hardware-Aware VLA        | VLA가 실제 hardware timing, memory, compute budget을 만족해야 함을 보인다.          | latency budget과 runtime architecture         |
| 19 | Safety and Assurance                    | success와 safety를 분리하고, unsafe success를 시스템 claim에서 다룬다.                | safety gate, incident log, assurance case    |
| 20 | Humanoids and Whole-Body VLA            | whole-body embodiment에서는 balance, contact, locomotion이 action 의미를 바꾼다. | whole-body action/evaluation 조건              |

다음 페이지에 계속

표 3 계속

| 장  | 제목                   | 존재 이유                                                                      | 후속 장의 해석 기준                 |
|----|----------------------|----------------------------------------------------------------------------|-----------------------------|
| 21 | What We Have Learned | 책 전체에서 배운 state, action, control, data, evaluation, safety의 연결 구조를 다시 묶는다. | Real VLA system contract 요약 |
| 22 | Future Directions    | 앞 장의 responsibility layer를 기반으로 연구 방향과 남은 병목을 정리한다.                        | 향후 연구 problem statement     |

이 표에서 중요한 점은 장 번호가 단순 난이도 순서가 아니라는 것이다. Chapter 13은 중간에 있지만 사실상 책 전체의 action API를 정의한다. Chapter 14와 16은 뒤쪽에 있지만 부록이 아니라 claim을 검증하는 핵심 장이다. Chapter 19 역시 deployment 이후에 붙는 윤리 장이 아니라, Chapter 1에서 정의한 Real VLA system의 필수 layer이다.

## 0.9 공학적 엄밀성의 기준

이 책은 비유 중심의 설명을 피한다. 필요한 경우 직관적 설명을 사용하지만, 핵심 주장은 가능한 한 변수, mapping, interface, metric, failure mode로 표현한다. 특히 다음 네 가지를 엄격히 구분한다.

1. **Task:** 언어 또는 symbolic form으로 표현된 목표.
2. **Policy action:** learned policy가 내는 action object.
3. **Controller command:** robot driver 또는 servo loop가 받는 물리 명령.
4. **Evidence:** claim을 뒷받침하는 log, metric, protocol, failure record.

이 구분을 notation으로 쓰면 하나의 deployment episode는 다음 기록으로 남아야 한다.

$$\tau = \{(\mathbf{o}_t, \mathbf{b}_t, z_t, \mathbf{a}_t, \mathbf{u}_t, y_t, \rho_t)\}_{t=1}^T, \quad (0.8)$$



여기서  $y_t$ 는 outcome signal 또는 event label이다. 단순한 success/failure label 만으로는 충분하지 않다. 성공했더라도 near miss가 있었는지, 사람 intervention이 있었는지, controller saturation이 있었는지, planner timeout이 있었는지 기록해야 한다. 이 기록이 없으면 다음 data collection, fine-tuning, safety review, benchmark comparison이 모두 약해진다.

## 0.10 이 책을 읽는 방법

첫 번째로, 각 장을 독립된 주제 소개로 읽지 말고 하나의 system stack을 두껍게 만드는 과정으로 읽어야 한다. Chapter 2의 state estimation은 Chapter 8의 visuomotor policy 입력을 정의하고, Chapter 13의 action representation은 Chapter 18의 real-time deployment와 연결된다.

두 번째로, 논문을 읽을 때도 같은 기준을 적용해야 한다. 어떤 논문이 VLA라고 주장하면 다음 질문을 해보아야 한다.

1. action은 정확히 무엇인가?
2. action은 어떤 controller 또는 simulator interface로 들어가는가?
3. data는 어떤 embodiment, sensor, reset, intervention policy에서 왔는가?
4. evaluation은 model ability와 system engineering을 분리해 보여주는가?
5. failure mode와 unsafe success가 기록되는가?

세 번째로, classical robotics와 foundation model을 경쟁 관계로 읽지 않아야 한다. 실제 시스템에서는 둘 중 하나가 사라지는 것이 아니라, 책임의 위치가 바뀐다. Learned model이 planner 역할을 일부 흡수할 수 있고, controller가 action error를 흡수할 수 있으며, safety monitor가 policy output을 수정할 수 있다. 중요한 것은 어떤 책임이 어디에 있고, 그 책임이 어떻게 검증되는가이다.

## 0.11 요약

이 책의 목적은 Real VLA를 model architecture가 아니라 robot system architecture로 읽게 만드는 것이다. VLA의 핵심 질문은 “어떤 backbone을 썼는가”에서 끝

나지 않는다. 중요한 질문은 language, vision, policy action, controller command, safety decision, data record, evaluation claim이 어떻게 닫힌 loop를 이루는가이다.

따라서 이 책은 control, planning, robot learning, embodied foundation model, data engine, evaluation, safety를 한 줄로 연결한다. 이 연결이 없으면 VLA는 demo 또는 benchmark model에 머문다. 이 연결이 생길 때 VLA는 실제 로봇공학의 대상이 된다.

## 0.12 Further reading and exercises

**Further reading** Chapter 1의 Real VLA definition, Chapter 13의 action contract, Chapter 16의 evaluation vector를 함께 읽어라. 세 장이 이 책의 기본 판정 기준이다.

**Review question** VLA claim을  $\mathcal{K} = \{M, A, P, U, D, E, S\}$ 로 읽으면, model-only claim에서 가장 먼저 빠지는 항목은 무엇인가?

**Design exercise** 최근 VLA 논문 하나를 골라 model, action, controller, data, evaluation, safety 항목을 한 줄씩 채워라. 비어 있는 항목이 그 논문의 약한 claim이다.

다음 장에서는 이 관점을 바탕으로 Real VLA가 무엇인지 정의한다. Chapter 1은 VLA를 VLM보다 큰 robot system problem으로 재정의하고, language goal에서 motor command까지 이어지는 책임 layer를 구체화한다.

제 I 편

# **Real VLA Stack**

## 제1장

# Real VLA란 무엇인가

## 학습 목표

이 장의 목표는 VLA를 “큰 VLM에 action head를 붙인 모델”로 이해하는 관점을 정리하고, 실제 로봇 시스템에서 필요한 인터페이스를 명확히 분리하는 것이다. 이 장을 마치면 독자는 다음 질문에 답할 수 있어야 한다.

1. VLM, VLA policy, Real VLA system의 차이를 설명할 수 있다.
2. 정책 수준 action  $a_t$ 와 제어 입력  $u_t$ 를 구분할 수 있다.
3. 언어 명령이 실제 로봇 명령으로 바뀌는 경로를 layer 단위로 분해할 수 있다.
4. VLA 논문을 backbone, action representation, data, control interface, evaluation, safety 기준으로 읽을 수 있다.

## 1.1 VLA는 더 큰 VLM이 아니다

Vision-language model(VLM)은 이미지나 비디오와 텍스트 사이의 의미적 대응을 학습한다. 예를 들어 VLM은 장면 속 물체를 설명하거나, 이미지에 대한 질문에 답하거나, 자연어 지시를 문장 수준에서 해석할 수 있다. 그러나 로봇에서 필요한 것은 텍스트 답변이 아니라 물리적 실행이다. 로봇은 관측한 장면과 주어진 언어 지시를 동역학, 접촉, 지연, 센서 오차, actuator limit, 안전 제약을 가진 물리 행동으로 바꾸어야 한다.

따라서 VLA를 단순히 “VLM 뒤에 action head를 붙인 모델”이라고 정의하면 중요한 부분이 사라진다. Action head가 어떤 형식의 행동을 내는지, 그 행동이

어떤 controller로 들어가는지, 안전 layer가 무엇을 검사하는지, 실패했을 때 누가 멈추고 누가 복구하는지, 실험 결과가 어떤 evidence로 기록되는지가 모두 Real VLA의 일부이다.

### 핵심 정의: Real VLA

이 책에서 Real VLA는 특정 모델 이름이 아니다. Real VLA는 language-conditioned perception과 learned action을 실제 로봇의 물리 실행, monitoring, evaluation, safety assurance로 연결하는 책임 구조이다.

이 관점은 고전 로봇공학을 대체하지 않는다. 오히려 고전 로봇공학의 개념을 더 선명하게 만든다. 안정성, 제약 만족, 접촉 제어, 모션 feasibility, state estimation, failure recovery는 VLM의 파라미터 수가 커져도 사라지지 않는다. VLA가 실제 로봇에 가까워질수록 이 문제들은 더 중요해진다.

#### 1.1.1 정책 action과 제어 입력

이 장에서 가장 먼저 고정할 notation은 정책 수준 action과 제어 입력의 차이이다. VLA policy가 출력하는 action을  $\mathbf{a}_t$  라 쓰고, 실제 servo loop 또는 robot driver가 받는 제어 입력을  $\mathbf{u}_t$  라 쓰자. 일반적으로

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{o}_t, \mathbf{q}_t, \ell, \mathbf{b}_t), \quad (1.1)$$

이다. 여기서  $\mathbf{o}_t$ 는 sensor observation,  $\mathbf{q}_t$ 는 robot configuration,  $\ell$ 는 언어 지시 또는 subgoal,  $\mathbf{b}_t$ 는 state belief 또는 memory를 뜻한다. 그러나  $\mathbf{a}_t$ 는 actuator command가 아니다. 로봇에 실제로 들어가는 명령은 보통 다음 변환을 거친다.

$$\mathbf{u}_t = \kappa(\mathbf{a}_t, \hat{\mathbf{s}}_t, \mathcal{C}_t), \quad (1.2)$$

여기서  $\hat{\mathbf{s}}_t$ 는 추정 상태이고,  $\mathcal{C}_t$ 는 kinematic constraint, collision constraint, force limit, workspace bound, safety condition 등을 포함한다.

**주의: VLA는 controller replacement가 아니다**

VLA policy는 grasp, waypoint, delta pose, action token, action chunk를 제안할 수 있다. 하지만 그 자체가 collision-free trajectory, stable contact, actuator-safe torque, distribution shift recovery를 보장하지 않는다. Real VLA engineering의 큰 부분은  $a_t \rightarrow u_t$  경계에 있다.

## 1.2 언어 목표에서 모터 명령까지

다음 명령을 보자.

머그컵을 drying rack 위에 올려라. 단, 옆의 유리컵은 건드리지 마라.

이 문장은 간단해 보이지만 실제 로봇 시스템에서는 여러 종류의 정보로 분해되어야 한다.

### 1.2.1 Task semantics

첫 번째 변환은 language에서 task semantics로 가는 변환이다. “머그컵”은 조작 대상이고, “drying rack”은 목표 위치 또는 목표 relation이며, “유리컵을 건드리지 말라”는 hard constraint이다. “올려라”라는 동사는 grasp, lift, transport, place, release, verify로 이어지는 작업 구조를 암시한다. 즉 자연어 지시는 action 자체가 아니라 action을 조직하는 task-level specification이다.

### 1.2.2 Embodied reasoning

두 번째 변환은 task semantics에서 embodied reasoning으로 가는 변환이다. 시스템은 머그컵과 유리컵을 찾고, pose 또는 affordance를 추정하고, drying rack의 가능한 placement region을 계산하며, 유리컵 주변 forbidden region을 설정해야 한다. 이 단계는 LLM, VLM, embodied reasoning model, scene graph, memory, planner, tool call을 사용할 수 있다. 그러나 출력은 여전히 motor command가 아니다.

### 1.2.3 Policy-level action

세 번째 변환은 embodied reasoning 결과를 policy-level action으로 바꾸는 것이다. VLA policy는 observation, language 또는 subgoal, proprioception, short memory를 받아 다음 action object를 출력한다. 이 action object는 end-effector delta, waypoint, discrete action token, short action chunk, continuous action distribution 등일 수 있다. 이 출력이  $a_t$ 이다.

### 1.2.4 Control command

네 번째 변환은  $a_t$ 를 controller target 또는 low-level command로 바꾸는 것이다. 예를 들어 VLA가 end-effector delta pose를 냈다면 motion/control layer는 현재 robot state, coordinate frame, kinematic limit, obstacle geometry, controller rate를 고려하여 이를 해석해야 한다. Operational-space controller, impedance controller, MPC, trajectory optimizer, whole-body controller 등이 여기서  $u_t$ 를 만든다.

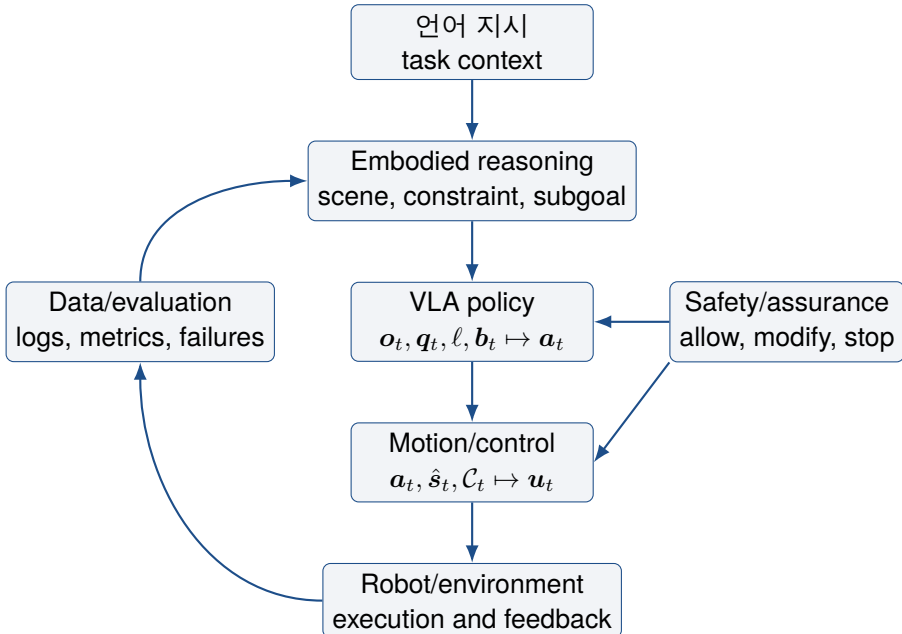


그림 2: Real VLA system stack. 이 그림은 특정 구현 pipeline이 아니라 책임 구조를 나타낸다.

### 1.3 다섯 개의 책임 layer

Real VLA를 시스템으로 읽기 위해 이 책은 다섯 개의 layer를 사용한다. 중요한 것은 layer가 반드시 별도 module이어야 한다는 뜻이 아니라, 책임이 사라지지 않아야 한다는 뜻이다.

표 4: Real VLA stack의 입력, 출력, 책임

| Layer              | 핵심 질문                           | 대표 입력                                          | 대표 출력                                        |
|--------------------|---------------------------------|------------------------------------------------|----------------------------------------------|
| Embodied reasoning | 무엇을 해야 하는가?                     | instruction, scene, memory, task context       | subgoal, plan, constraint, tool/ policy call |
| VLA policy         | 지금 어떤 action을 제안할 것인가?          | observation, language/ subgoal, proprioception | policy-level action $a_t$                    |
| Motion/control     | action을 어떻게 물리 motion으로 바꿀 것인가? | $a_t$ , robot state, constraints               | controller command $u_t$                     |
| Safety/assurance   | 허용, 수정, 정지, 복구 중 무엇을 할 것인가?     | instruction, plan, action, runtime signals     | allow, modify, stop, recover, handoff        |
| Data/evaluation    | 어떤 evidence가 생성되었는가?            | logs, actions, states, failures, interventions | dataset record, metric, regression test      |

#### 1.3.1 Embodied reasoning layer

Embodied reasoning layer는 intent를 구조화한다. 이 layer는 scene graph, memory, planner, VLM/LLM, skill library를 사용할 수 있다. 하지만 좋은 plan은 physical execution의 필요조건일 수는 있어도 충분조건은 아니다. “유리컵을 피해서 머그컵을 옮겨라”라는 subgoal이 있다고 해서 자동으로 collision-free trajectory와 stable grasp가 생기지는 않는다.



### 1.3.2 VLA policy layer

VLA policy layer는 이 책에서 “VLA”라는 단어가 가장 좁은 의미로 쓰이는 위치이다. 이 layer는 observation, language 또는 subgoal, robot state를 받아 policy-level action object를 낸다. 이 action object는 action type, coordinate frame, rate, horizon, scaling, downstream controller를 요구하는 interface object이다.

### 1.3.3 Motion/control layer

Motion/control layer는 단순 executor가 아니다. 같은 “오른쪽으로 이동”이라는 action도 base frame 기준인지 end-effector frame 기준인지, normalized vector인지 metric pose인지, velocity command인지 target pose인지에 따라 전혀 다르게 실행된다. 따라서 controller는 learned policy의 하위 부품이 아니라 physical feasibility를 해석하는 별도 책임 layer이다.

### 1.3.4 Safety/assurance layer

Safety는 마지막 wrapper가 아니다. 위험한 instruction을 거절하는 semantic safety, forbidden region을 검사하는 geometric safety, 속도와 힘 제한을 다루는 dynamic safety, 실행 중 이상을 감지하는 runtime assurance, 사람의 intervention을 가능하게 하는 handoff mechanism이 모두 필요하다. Runtime safety decision을

$$r_t = \sigma(\ell, \hat{s}_t, \mathbf{a}_t, \mathbf{u}_t, h_t) \in \{\text{allow, modify, stop, ask}\} \quad (1.3)$$

로 쓸 수 있다. 여기서  $h_t$ 는 최근 sensor history와 system log이다.

### 1.3.5 Data/evaluation layer

Data/evaluation layer는 deployment 이후 문서화 절차가 아니다. VLA의 성능 주장은 데이터와 평가 protocol 위에 선다. 어떤 robot, 어떤 camera, 어떤 gripper, 어떤 action representation, 어떤 controller, 어떤 randomization, 어떤 reset/intervention policy를 썼는지에 따라 같은 success rate의 의미가 달라진다.

## 1.4 인터페이스 계약: observation, state, action, control

Real VLA stack은 interface가 명확할 때만 분석 가능하다. 첫 번째 구분은 observation과 state이다.  $\mathbf{o}_t$ 는 camera image, depth, tactile signal, proprioception 등 system이 관측한 값이다.  $\mathbf{s}_t$ 는 실제 환경과 로봇의 물리 상태이다. 실제 상태는 대개 직접 관측되지 않는다. 따라서 system은 state estimate  $\hat{\mathbf{s}}_t$  또는 belief  $\mathbf{b}_t$  위에서 작동한다.

$$\mathbf{b}_t = \eta(\mathbf{b}_{t-1}, \mathbf{o}_t, \mathbf{u}_{t-1}), \quad (1.4)$$

와 같이 belief update를 쓸 수 있다. 여기서  $\eta$ 는 estimator, filter, learned state representation, memory update를 포괄한다.

두 번째 구분은 action representation이다. VLA action은 다음처럼 다양한 형태를 가질 수 있다.

표 5: 대표 action representation과 system-level implication

| Action type           | 장점                                   | 주의할 점                                         |
|-----------------------|--------------------------------------|-----------------------------------------------|
| Action token          | VLM/LLM sequence modeling과 잘 맞음      | token의 물리 의미와 controller mapping이 불명확할 수 있음   |
| Continuous delta      | controller와 직접 연결하기 쉬움               | scaling, normalization, coordinate frame에 민감함 |
| Waypoint              | planning/control integration이 쉬움     | downstream planner와 collision checking 부담이 커짐 |
| Action chunk          | temporal consistency를 줄 수 있음         | interruption과 recovery granularity가 거칠어질 수 있음 |
| Diffusion/flow output | multimodal action distribution 표현 가능 | sampling latency, safety projection이 필요함      |

Action representation은 formatting 문제가 아니다. 무엇을  $\mathbf{a}_t$ 로 정의하는지에 따라 policy가 학습하는 대상, controller가 해석해야 할 대상, safety layer가 검사할 수 있는 대상, evaluator가 기록해야 할 대상이 모두 달라진다.

## 1.5 역사적 수렴: 제어, 계획, 로봇러닝, VLM/LLM

Real VLA는 갑자기 등장한 분야가 아니다. 네 흐름이 수렴한 결과로 보는 것이 정확하다.

1. **Control and estimation:** state, feedback, stability, contact, latency, constraints.
2. **Planning and TAMP:** task structure, feasibility, long-horizon composition, recovery.
3. **Robot learning:** visuomotor policy, imitation, RL, dataset coverage, distribution shift.
4. **VLM/LLM/foundation models:** semantic grounding, language-conditioned reasoning, tool use.

Control과 estimation은 VLA가 물리계에 들어갈 때 필요한 언어이다. VLA가 end-effector delta pose를 내든, waypoint를 내든, action chunk를 내든, 로봇은 여전히 kinematics, dynamics, contact, actuator limit 안에서 움직인다.

Planning과 TAMP도 현재형 도구이다. 자연어 instruction은 종종 long-horizon structure를 갖는다. LLM 또는 VLM이 planning function 일부를 흡수할 수는 있지만, planning problem 자체가 사라지는 것은 아니다. 문제는 PDDL을 반드시 쓰느냐가 아니라 task structure, constraint, feasibility, recovery가 system 어딘가에 존재하느냐이다.

Robot learning은 VLA의 직접 조상이다. VLA 이전에도 behavior cloning, DAgger, guided policy search, visual RL, large-scale grasping은 image-to-action mapping을 학습하려고 했다. VLA가 새롭게 가져온 것은 여기에 language grounding과 foundation-model representation을 결합했다는 점이다.

## 1.6 현대 VLA system pattern

현대 VLA system은 다음 pattern으로 읽으면 좋다. 이것은 완성된 taxonomy가 아니라 논문을 읽기 위한 engineering map이다.

표 6: 현대 VLA system pattern

| Pattern                    | 핵심 기여                                                    | 과대해석 위험                                     |
|----------------------------|----------------------------------------------------------|---------------------------------------------|
| VLA policy                 | vision/language/proprioception에서 action을 직접 예측           | policy를 full robot system으로 오해함             |
| Action-distribution model  | token, chunk, diffusion, flow 등 action 생성 구조를 설계         | controller와 safety assumption이 숨겨짐          |
| ER + VLA orchestration     | reasoning model이 task decomposition 또는 VLA/skill call 수행 | reasoning competence를 motor competence로 오해함 |
| Humanoid/ whole-body stack | loco-manipulation, balance, whole-body control까지 확장      | demo를 broad deployment evidence로 오해함        |
| Open-source training stack | fine-tuning, action tokenizer, benchmark harness 제공      | local deployment가 해결되었다고 오해함                |

이 pattern들은 모두 같은 결론으로 이어진다. VLA 연구의 핵심은 backbone 크기만이 아니다. 어떤 action representation을 쓰는지, 어떤 controller에 연결되는지, 어떤 data record가 남는지, 어떤 evaluation protocol이 claim을 지지하는지가 함께 중요하다.

## 1.7 Data, evaluation, safety는 appendices가 아니다

Real VLA claim은 architecture만으로 성립하지 않는다. Policy가 image와 language를 받아 action을 낼 수 있어도, 실제 capability는 data regime, evaluation protocol, safety layer에 의존한다.

### 1.7.1 Robot data의 구조

Robot trajectory는 단순한 observation sequence가 아니다. 다음 record를 포함해야 해석 가능하다.

$$\begin{aligned}
 \mathcal{D}_i &= (\ell_i, O_i, Q_i, A_i, U_i, m_i, y_i, e_i), \\
 O_i &= \{\mathbf{o}_t\}_{t=1}^{T_i}, & Q_i &= \{\mathbf{q}_t\}_{t=1}^{T_i}, \\
 A_i &= \{\mathbf{a}_t\}_{t=1}^{T_i}, & U_i &= \{\mathbf{u}_t\}_{t=1}^{T_i}.
 \end{aligned} \tag{1.5}$$

여기서  $m_i$  는 robot, controller, action type, rate 같은 metadata 이고,  $y_i$  는 success, failure, intervention label 이며,  $e_i$  는 safety event, reset policy, normalization 기록이다. 이 정보가 없으면 같은 demonstration도 다르게 해석된다. 5 Hz Cartesian delta action을 compliant controller가 tracking 한 trajectory와 50 Hz joint target을 position controller가 tracking 한 trajectory는 같은 training signal 이 아니다.

### 1.7.2 Benchmark success와 robot capability

Benchmark는 필요하다. 공통 task, 비교 가능한 score, regression test를 제공하기 때문이다. 그러나 benchmark success가 곧 robot capability는 아니다. Benchmark는 일부 변수를 고정하여 특정 차원을 측정한다. Deployment는 여러 차원을 동시에 노출한다.

머그컵 예제에서 Real VLA evaluation은 단순히 “머그컵이 rack 위에 올라갔는가”만 묻지 않는다. 최소한 다음 항목을 봐야 한다.

- mug가 목표 region에 놓였는가?
- glass와 contact가 없었는가?
- contact force와 velocity limit이 유지되었는가?
- human intervention이 있었는가?
- slip이나 실패를 감지하고 recover했는가?
- layout, lighting, distractor 변화에서 재현되는가?
- $\mathbf{a}_t$  와  $\mathbf{u}_t$  의 interface assumption이 기록되었는가?

Evaluation vector를 다음처럼 정의할 수 있다.

$$\mathbf{m} = [p_{\text{succ}}, p_{\text{safe}}, \tau_{\text{latency}}, n_{\text{intervention}}, r_{\text{recovery}}, \Delta_{\text{shift}}] . \quad (1.6)$$

여기서  $p_{\text{succ}}$  만 보고 capability를 주장하면 stack의 중요한 failure mode가 가려진다.

**Benchmark success는 robot capability가 아니다**

Benchmark result는 system이 특정 protocol을 만족했다는 뜻이다. Robot capability는 더 넓은 claim이다. Protocol, randomization, failure taxonomy, intervention policy, safety event, controller assumption이 보이지 않으면 number의 의미도 제한된다.

**1.7.3 Safety as stack property**

Safety는 polite language model이나 collision checker 하나로 환원되지 않는다. Real VLA safety는 semantic, geometric, dynamic, runtime, data, human-supervision 책임을 포함한다.

표 7: Safety responsibility by layer

| Layer     | Safety question           | Example failure                |
|-----------|---------------------------|--------------------------------|
| Semantic  | instruction이 허용 가능한가?     | constraint를 preference로 해석함    |
| Geometric | 경로와 목표가 충돌 없이 가능한가?       | glass 근처 forbidden region을 무시함 |
| Dynamic   | 속도, 힘, torque가 제한 안에 있는가? | place 동작 중 overshoot 발생        |
| Runtime   | 실패를 충분히 빨리 감지하는가?         | contact 후에야 stop signal 발생     |
| Data      | unsafe data가 학습/평가에 섞였는가? | hidden reset이 success로 기록됨     |
| Human     | 사람이 중단하거나 인계받을 수 있는가?     | intervention path가 없음          |

**1.8 VLA 논문을 읽는 여섯 가지 질문**

이 책의 나머지 장에서는 다음 reading protocol을 반복적으로 사용한다. 어떤 VLA 논문이 robot capability를 주장할 때, 먼저 다음 여섯 질문을 던져야 한다.

1. **Backbone**은 무엇인가? VLM, LLM, diffusion model, transformer policy, flow model, hierarchical planner, hybrid architecture 중 무엇인가?
2. **Action representation**은 무엇인가? token, Cartesian delta, pose, waypoint, action chunk, joint target, torque command, latent action 중 무엇을 출력하는가?
3. **Data**는 무엇으로 구성되는가? real robot trajectory, simulation, teleoperation, scripted demonstration, human video, synthetic data, web pretraining 중 무엇인가?
4. **Control interface**는 무엇인가? policy output을 어떤 controller가 받는가? control frequency와 tracking assumption은 명시되었는가?
5. **Evaluation protocol**은 무엇인가? trial count, randomization, distribution shift, failure taxonomy, intervention policy가 보이는가?
6. **Safety/deployment evidence**는 무엇인가? instruction filtering, geometric check, runtime monitor, stop/recovery, human override, audit log가 있는가?

이 질문들은 단순한 비기술적 요약표가 아니다. 어떤 interface가 바뀌었고 어떤 evidence가 추가되었는지 분리하기 위한 engineering rubric이다. 어떤 논문의 실제 기여가 더 큰 backbone이 아니라 action tokenizer일 수 있고, 다른 논문의 핵심은 data mixture 또는 evaluation protocol일 수 있다.

## 1.9 이 장이 고정하는 것

이 장은 다음 vocabulary를 고정한다.

- Real VLA system: language-conditioned perception을 monitored robot behavior로 바꾸는 전체 책임 구조.
- Embodied reasoning layer: task context, constraint, memory, subgoal을 구조화하는 layer.
- VLA policy layer: observation, language/subgoal, robot state를 policy-level action  $a_t$ 로 바꾸는 layer.

- Action/control boundary:  $a_t$ 와  $u_t$ 의 구분.
- Safety/assurance layer: allow, modify, stop, recover, ask-human 결정을 내리는 layer.
- Data/evaluation layer: evidence와 metric, failure taxonomy를 정의하는 layer.

후속 장은 이 vocabulary를 바탕으로 전개된다. Chapter 13은 action representation과 control interface를 다룬다. Chapter 14는 robot data engine을 다룬다. Chapter 16은 evaluation protocol을 다룬다. Chapter 19는 safety, assurance, deployment를 다룬다.

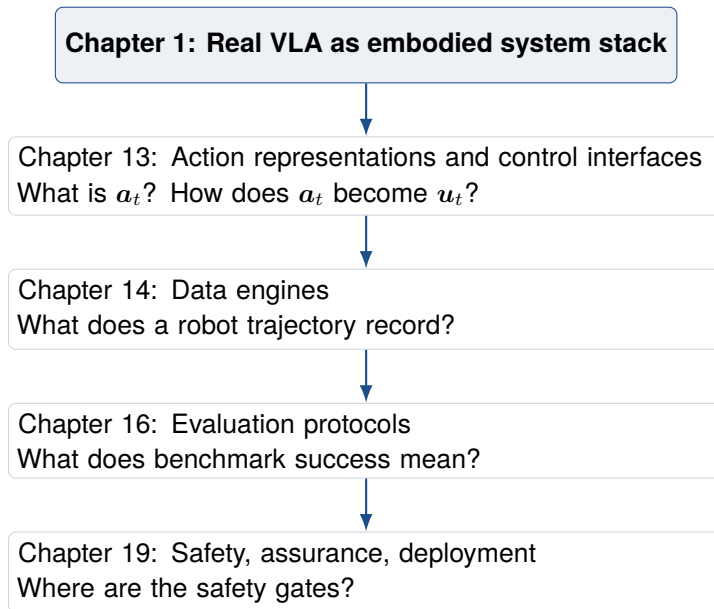


그림 3: Chapter 1이 후속 장에 넘기는 dependency map.

## 요약

Real VLA는 하나의 model family가 아니라 책임 구조이다. 핵심은 VLA policy가 내는  $a_t$ 를 실제 robot command  $u_t$ 로 바꾸는 interface, 그 과정에서 필요한 state estimation과 control, safety gate, data/evaluation protocol을 함께 보는 것이다.



이 장의 목적은 최신 system 이름을 나열하는 것이 아니라 후속 장에서 사용할 engineering vocabulary를 고정하는 것이다.

## 1.10 Further reading and exercises

**Further reading** OpenVLA,  $\pi_0$ , FAST, OpenVLA-OFT를 Chapter 1의 responsibility layer 관점에서 다시 읽어라. 모델 이름보다 action/control/data/evidence boundary를 먼저 확인한다.

**Review question**  $a_t$ 와  $u_t$ 를 구분하지 않는 VLA 논문에서는 어떤 failure attribution 문제가 생기는가?

**Design exercise** mug/rack/glass task에 대해 perception, policy action, controller command, safety monitor, evaluation evidence를 각각 한 문장으로 정의하라.

## 제2장

# State, Dynamics, and Estimation

## 학습 목표

Chapter 1은 Real VLA를 다섯 개의 책임 layer로 정의했다. Chapter 2는 그 중 observation과 state의 차이를 기술적으로 고정한다. 이 장의 목표는 다음과 같다.

1. observation  $\mathbf{o}_t$ 와 physical state  $\mathbf{s}_t$ 의 차이를 설명한다.
2. VLA policy, estimator, controller가 서로 다른 시간축과 정보 집합을 사용한다는 점을 명확히 한다.
3. partial observability, latency, calibration error가 왜 VLA failure mode가 되는지 정식화한다.
4. Chapter 3의 control interface와 Chapter 13의 action representation 논의를 위한 state notation을 준비한다.

## 2.1 로봇이 본다는 것은 상태를 안다는 뜻이 아니다

VLA 논문에서 흔히 “image와 instruction을 입력으로 action을 출력한다”고 말한다. 이 표현은 모델 구조를 간단히 설명할 때는 유용하지만, 실제 로봇 시스템을 설명하기에는 부족하다. 카메라 image는 observation이다. 로봇이 제어해야 하는 것은 physical state이다. 두 개는 같지 않다.

예를 들어 머그컵이 보인다고 해서 로봇이 머그컵의 6D pose, 표면 마찰, 내부 액체 유무, gripper와의 접촉 상태, 유리컵과의 최소 거리, 카메라 timestamp

와 controller timestamp의 오차를 모두 알고 있다는 뜻은 아니다. VLM이 장면을 잘 설명해도 그 설명은 상태 추정을 대체하지 않는다.

### Observation과 state

Observation  $\mathbf{o}_t$ 는 센서가 시점  $t$ 에서 제공한 측정값이다. State  $\mathbf{s}_t$ 는 로봇과 환경의 물리 상태이다. 대부분의 실제 로봇 문제에서  $\mathbf{s}_t$ 는 직접 관측되지 않으며, 시스템은 estimate  $\hat{\mathbf{s}}_t$  또는 belief  $\mathbf{b}_t$ 를 사용한다.

Real VLA에서 이 구분이 중요한 이유는 단순하다. VLA policy는 주로 observation-conditioned policy로 학습되지만, controller와 safety layer는 state-dependent constraint를 요구한다. 즉 policy가 보는 것과 controller가 필요로 하는 것이 다르다.

## 2.2 State-space formulation

가장 기본적인 로봇 시스템 모델은 다음 state-space form으로 쓸 수 있다 [1, 2].

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{u}_t, \mathbf{w}_t), \quad (2.1)$$

$$\mathbf{o}_t = h(\mathbf{s}_t, \mathbf{v}_t). \quad (2.2)$$

여기서  $f$ 는 dynamics,  $h$ 는 observation model,  $\mathbf{w}_t$ 는 process noise,  $\mathbf{v}_t$ 는 measurement noise이다. VLA policy가 직접 사용하는 입력은 보통  $\mathbf{o}_t$  또는 observation history  $\mathbf{o}_{\leq t}$ 이다. 그러나 실제 dynamics는  $\mathbf{s}_t$ 와 control command  $\mathbf{u}_t$ 에 의해 전개된다.

Manipulator의 경우 state는 다음처럼 구성될 수 있다.

$$\mathbf{s}_t = \left[ \mathbf{q}_t^\top, \dot{\mathbf{q}}_t^\top, \mathbf{x}_{ee,t}^\top, \mathbf{x}_{obj,t}^\top, \mathbf{c}_t^\top \right]^\top, \quad (2.3)$$

여기서  $\mathbf{q}_t$ 는 joint configuration,  $\dot{\mathbf{q}}_t$ 는 joint velocity,  $\mathbf{x}_{ee,t}$ 는 end-effector pose,  $\mathbf{x}_{obj,t}$ 는 object pose,  $\mathbf{c}_t$ 는 contact-related variable이다. 모든 항이 항상 직접 관측되는 것은 아니다.

VLA policy는 이 전체 state 대신 다음과 같은 입력을 받을 수 있다.

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{I}_t, \mathbf{p}_t, \ell, m_t), \quad (2.4)$$

여기서  $\mathbf{I}_t$ 는 image observation,  $\mathbf{p}_t$ 는 proprioception,  $\ell$ 는 language instruction,  $m_t$ 는 memory이다. 이 식은 [equation \(2.1\)](#)와 [equation \(2.2\)](#)을 제거하지 않는다. 단지 policy가 사용하는 정보 집합을 나타낼 뿐이다.

## 2.3 Partial observability

로봇 조작은 대부분 partially observable problem이다. 관측되지 않는 변수는 많다.

- 물체의 정확한 6D pose와 covariance
- 접촉 여부와 접촉 force
- gripper slip
- 가려진 obstacle
- 표면 마찰과 object compliance
- actuator delay와 backlash
- 사람의 의도 또는 workspace 침입 가능성

따라서 Real VLA 시스템은 observation을 곧바로 state로 취급하면 안 된다. 더 일반적으로는 belief state를 사용한다.

$$\mathbf{b}_t = p(\mathbf{s}_t \mid \mathbf{o}_{1:t}, \mathbf{u}_{1:t-1}, \ell). \quad (2.5)$$

Belief update는 다음과 같은 recursive form으로 쓸 수 있다.

$$\bar{\mathbf{b}}_t(\mathbf{s}_t) = \int p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{u}_{t-1}) \mathbf{b}_{t-1}(\mathbf{s}_{t-1}) d\mathbf{s}_{t-1}, \quad (2.6)$$

$$\mathbf{b}_t(\mathbf{s}_t) = \eta p(\mathbf{o}_t \mid \mathbf{s}_t) \bar{\mathbf{b}}_t(\mathbf{s}_t). \quad (2.7)$$

여기서 [equation \(2.6\)](#)는 prediction step이고, [equation \(2.7\)](#)는 correction step이다. EKF, UKF, particle filter, visual-inertial odometry, learned state estimator는 이 구조의 다른 구현으로 볼 수 있다.

## 2.4 VLA policy가 belief를 사용하는 방식

VLA policy가 명시적으로  $\mathbf{b}_t$ 를 입력으로 받지 않더라도, 시스템 수준에서는 belief가 존재한다. 예를 들어 다음 세 가지 구현이 가능하다.

1. **Implicit belief:** policy가 image history와 proprioception history를 직접 받아 내부 hidden state로 불확실성을 흡수한다.
2. **Explicit estimator:** 별도 state estimator가  $\hat{\mathbf{s}}_t$  또는 covariance  $\Sigma_t$ 를 계산하고, policy 또는 controller가 이를 사용한다.
3. **Hybrid interface:** VLA policy는 image와 language를 사용하고, controller/safety layer는 estimator가 만든 geometric state와 uncertainty를 사용한다.

Real VLA 책에서 중요한 것은 어떤 방식이 항상 옳다는 결론이 아니다. 중요한 것은 interface를 명시하는 것이다. Policy가 uncertainty를 직접 다루는가? Controller가 별도 state estimate를 받는가? Safety layer가 covariance를 볼 수 있는가? Evaluation log에 estimator failure가 기록되는가?

## 2.5 Dynamics and controller-relevant state

Controller가 필요로 하는 state는 policy가 보는 observation보다 더 엄격하다. Manipulator dynamics는 일반적으로 다음과 같이 쓸 수 있다.

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \boldsymbol{\tau}_{contact} = \boldsymbol{\tau}. \quad (2.8)$$

여기서  $\mathbf{M}$ 은 inertia matrix,  $\mathbf{C}$ 는 Coriolis/centrifugal term,  $\mathbf{g}$ 는 gravity,  $\boldsymbol{\tau}_{contact}$ 는 contact force가 joint torque로 반영된 항,  $\boldsymbol{\tau}$ 는 actuator torque이다.

표 8: State information 이 stack 에서 사용되는 위치

| Layer              | 필요한 state 정보                                             | state error 가 만드는 failure                       |
|--------------------|----------------------------------------------------------|-------------------------------------------------|
| Embodied reasoning | object identity, relation, approximate pose              | 잘못된 object reference 또는 infeasible subgoal      |
| VLA policy         | image, proprioception, short history, estimated target   | action drift, wrong grasp, layout shift failure |
| Motion/control     | joint state, end-effector pose, velocity, constraint     | tracking error, collision, unstable motion      |
| Safety/assurance   | forbidden region, distance, force, velocity, uncertainty | late stop, unsafe recovery, false accept        |
| Data/evaluation    | logged state, timestamp, intervention, failure label     | benchmark artifact, hidden estimator failure    |

VLA policy 가 waypoint 나 delta pose 를 출력하더라도, 실제 robot 은 [equation \(2.8\)](#)의 제약 아래 움직인다. 따라서 controller는 적어도 다음 정보를 필요로 한다.

- 현재 joint configuration  $\mathbf{q}_t$  와 velocity  $\dot{\mathbf{q}}_t$
- end-effector pose 와 Jacobian
- obstacle 또는 workspace constraint
- velocity, acceleration, torque limit
- contact state 또는 force estimate

이 정보가 잘못되면 VLA policy 가 semantic 하게 맞는 action 을 내도 물리 실행은 실패할 수 있다.

## 2.6 Latency and time alignment

VLA 시스템에서 time index 는 자주 무시되지만, 실제 deployment 에서는 핵심 변수이다. Camera frame, proprioception, VLA inference, safety monitor, controller loop 는 서로 다른 rate 로 동작한다.

표 9: Real VLA stack의 대표 time scale

| Component               | 대표 rate 또는 지연                 | 영향                                      |
|-------------------------|-------------------------------|-----------------------------------------|
| Camera / vision encoder | 10–60 Hz, frame latency 존재    | object pose와 실제 pose 사이 time skew 발생    |
| VLA inference           | model 크기에 따라 수 Hz 이하일 수 있음    | high-rate correction 이 어려움              |
| Controller loop         | 100 Hz–1 kHz                  | fast stabilization과 contact response 담당 |
| Safety monitor          | event에 따라 high-rate 필요        | 늦으면 stop이 의미 없어짐                        |
| Data logger             | multi-rate synchronization 필요 | failure analysis 와 reproducibility에 영향  |

측정 시각과 실행 시각이 다르면 policy는 오래된 observation에 대해 action을 낼 수 있다. 이를 단순하게 쓰면

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{o}_{t-\delta_o}, \mathbf{q}_{t-\delta_q}, \ell), \quad (2.9)$$

이다. 여기서  $\delta_o$ 와  $\delta_q$ 는 observation과 proprioception의 effective delay이다. Controller가 실제로 받는 명령은 다시 inference delay와 communication delay를 거친다.

$$\mathbf{u}_t = \kappa(\mathbf{a}_{t-\delta_{\pi}}, \hat{\mathbf{s}}_{t-\delta_s}, \mathcal{C}_{t-\delta_c}). \quad (2.10)$$

이 식은 추상적이지만 중요한 engineering message를 준다. VLA가 “현재 장면”을 보고 있다고 말하려면 timestamp alignment를 증명해야 한다. 특히 contact-rich manipulation과 human-proximate robot에서는 delay가 safety failure로 직결 이어질 수 있다.

## 2.7 Calibration and coordinate frames

State estimation은 센서 값을 추정 state로 바꾸는 문제일 뿐 아니라 coordinate frame을 맞추는 문제이기도 하다. VLA action이 end-effector delta인지, camera frame waypoint인지, world frame pose인지에 따라 controller interpretation이 달라진다.

대표적으로 camera frame의 point  $x^C$ 를 robot base frame으로 바꾸려면

$$x^B = T_{BC}x^C \quad (2.11)$$

와 같은 extrinsic transform이 필요하다. 여기서  $T_{BC} \in SE(3)$ 는 camera-to-base transform이다. Calibration error가 있으면 policy가 맞는 object를 선택해도 controller는 잘못된 위치로 움직인다.

Frame mismatch는 VLA action representation과 직접 연결된다.

- Action token은 frame 정보를 token vocabulary에 암묵적으로 숨길 수 있다.
- Continuous delta는 scale과 frame convention을 명시해야 한다.
- Waypoint는 world frame, base frame, camera frame 중 어느 frame인지 명확해야 한다.
- Whole-body action은 base motion과 arm motion의 frame coupling을 다룬다.

따라서 Chapter 13의 action representation 비교는 Chapter 2의 state/frame notation 위에 세워진다.

## 2.8 State uncertainty and safety

Safety layer는 단순히 estimated state만 보면 부족하다. Uncertainty도 봐야 한다. 물체 위치 추정치가 같아도 covariance가 큰 경우와 작은 경우는 다른 safety decision을 요구한다.

장애물과 end-effector 사이 signed distance를  $d(\hat{s}_t)$ 라 쓰자. Uncertainty를 반영한 conservative safety margin은 다음처럼 둘 수 있다.

$$d(\hat{s}_t) - \lambda \sqrt{J_d \Sigma_t J_d^T} \geq d_{\min}, \quad (2.12)$$

여기서  $\Sigma_t$ 는 state covariance,  $J_d$ 는 distance function의 local Jacobian,  $\lambda$ 는 confidence margin,  $d_{\min}$ 은 최소 안전 거리이다.

이 식은 safety가 단순 rule이 아니라 state estimation과 연결된다는 점을 보여준다. VLA policy가 semantic하게 “glass를 피하라”를 이해해도, glass pose uncertainty가 크면 안전한 motion을 계산할 수 없다.



## 2.9 Estimation failure modes

Real VLA에서 estimation failure는 model failure처럼 보일 수 있다. 그러나 원인을 분리해야 한다.

표 10: Estimation-related failure mode

| Failure mode            | 증상                           | 확인할 log                                              |
|-------------------------|------------------------------|------------------------------------------------------|
| Object pose error       | grasp가 물체 옆을 지나감             | pose estimate, image timestamp, calibration          |
| State latency           | moving object를 늦게 따라감        | sensor/control timestamp, inference latency          |
| Frame mismatch          | 일정 방향으로 systematic offset 발생 | camera-base extrinsic, action frame convention       |
| Contact unobservability | grasp 후 slip을 감지하지 못함        | tactile/force sensor, gripper current, contact label |
| Hidden intervention     | success처럼 보이거나 사람이 중간에 수정    | intervention log, reset policy, video audit          |

이 표는 Chapter 16의 evaluation protocol과 연결된다. Benchmark가 final success만 기록하면 estimation failure가 policy success 또는 policy failure로 잘못 분류될 수 있다.

### Case study: VLA failure처럼 보이는 calibration failure

카메라 extrinsic이 몇 cm 어긋나면 VLA는 target object를 맞게 찾고도 gripper가 항상 물체 옆을 지나갈 수 있다. 이 실패는 language grounding 실패도, action head 실패도 아니다. Real VLA report는 camera-base calibration, timestamp, frame convention, pose covariance를 함께 기록해야 한다.

## 2.10 Further reading and exercises

**Further reading** Kalman filtering과 Probabilistic Robotics의 belief-state 관점을 읽고, VLA observation pipeline이 어떤 state estimate를 암묵적

으로 요구하는지 확인하라 [1, 2].

**Review question** image-conditioned policy가 있다고 해서 state estimation 문제가 사라지지 않는 이유는 무엇인가?

**Design exercise** mug/rack/glass task에서  $\mathbf{o}_t$ ,  $\hat{\mathbf{s}}_t$ ,  $\mathbf{b}_t$ , calibration metadata, safety constraint를 분리한 state record를 설계하라.

## 2.11 Robotician's Takeaway

### Chapter 2 Takeaway

VLA가 observation을 입력으로 받는다는 사실은 state estimation 문제가 사라졌다는 뜻이 아니다. Real VLA system은  $\mathbf{o}_t$ ,  $\hat{\mathbf{s}}_t$ ,  $\mathbf{b}_t$ ,  $\mathbf{a}_t$ ,  $\mathbf{u}_t$ 가 어떤 시간축과 frame에서 연결되는지 명시해야 한다. 이 interface가 불명확하면 policy, controller, safety, evaluation failure를 구분할 수 없다.

다음 장은 이 상태 추정과 시간 정렬 위에서 VLA output이 어떤 controller interface를 통과해야 하는지 다룬다.

## 제3장

## Control Interfaces

## 학습 목표

이 장은 VLA policy가 출력한  $\mathbf{a}_t$ 가 어떤 제어 interface를 거쳐 실제 robot command  $\mathbf{u}_t$ 가 되는지 다룬다. Chapter 2가 state와 observation의 차이를 고정했다면, Chapter 3은 action proposal과 controller command의 차이를 고정한다.

1. VLA action representation과 controller interface를 분리한다.
2. joint, Cartesian, waypoint, trajectory, impedance, MPC reference의 차이를 설명한다.
3.  $\mathbf{a}_t \rightarrow \mathbf{u}_t$  mapping이 safety와 evaluation에 미치는 영향을 정식화한다.
4. Chapter 13의 action representation 논의를 위한 control-side 기준을 제공한다.

## 3.1 Control interface가 필요한 이유

VLA policy가 “action을 출력한다”는 말은 충분하지 않다. Action이 무엇인지, 어떤 rate로 나오며, 어떤 coordinate frame을 쓰고, 어떤 controller가 그 action을 해석하는지 명시해야 한다. 실제 로봇은 neural network output을 그대로 실행하지 않는다. 대부분의 경우 VLA output은 controller reference, waypoint, target pose, short trajectory, 또는 high-level skill call이다 [6, 7, 8].

Chapter 1의 notation을 다시 쓰면, VLA policy는

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{o}_t, \mathbf{q}_t, \ell, \mathbf{b}_t) \quad (3.1)$$

를 출력한다. 하지만 robot driver는 보통  $\mathbf{a}_t$ 가 아니라

$$\mathbf{u}_t = \kappa(\mathbf{a}_t, \hat{\mathbf{s}}_t, \mathcal{C}_t) \quad (3.2)$$

를 받는다. 여기서  $\kappa$ 는 단순한 software wrapper가 아니다. 그것은 action representation을 물리적 command로 해석하는 interface contract이다.

#### Control interface

Control interface는 policy-level action  $\mathbf{a}_t$ 가 robot hardware에 전달되기 전에 통과하는 해석 경계이다. 이 경계는 coordinate frame, unit, rate, limit, tracking law, safety filter, failure semantics를 포함한다.

## 3.2 Joint-space interfaces

가장 직접적인 interface는 joint-space command이다. Robot configuration을  $\mathbf{q} \in \mathbb{R}^n$ 이라 할 때, VLA 또는 downstream module은 target joint position, joint velocity, 또는 torque를 낼 수 있다.

### 3.2.1 Joint position target

Joint position interface는 다음과 같이 쓸 수 있다.

$$\mathbf{a}_t = \mathbf{q}_{t+H}^{ref}, \quad \mathbf{u}_t = K_p(\mathbf{q}_{t+H}^{ref} - \mathbf{q}_t) - K_d\dot{\mathbf{q}}_t. \quad (3.3)$$

이 interface는 단순하고 많은 robot API와 잘 맞는다. 그러나 VLA가 joint target을 직접 내면 morphology dependency가 매우 커진다. 같은 task라도 robot arm의 joint dimension, joint limit, kinematic chain이 다르면 action space가 달라진다. Multi-embodiment VLA에서 joint-space output이 어려운 이유가 여기에 있다.

### 3.2.2 Joint velocity target

Velocity command는 다음처럼 표현된다.

$$\mathbf{a}_t = \dot{\mathbf{q}}_t^{ref}, \quad \mathbf{u}_t = \dot{\mathbf{q}}_t^{ref}. \quad (3.4)$$

단순해 보이지만, velocity command는 integration drift와 limit handling이 중요하다. Velocity를 오래 적분하면 joint limit, self-collision, workspace boundary를 넘을 수 있다. 따라서 velocity interface는 safety projection과 함께 쓰여야 한다.

### 3.2.3 Torque command

Torque-level interface는 가장 낮은 수준의 command이다.

$$\mathbf{a}_t = \boldsymbol{\tau}_t^{ref}, \quad \mathbf{u}_t = \boldsymbol{\tau}_t^{ref}. \quad (3.5)$$

Torque command는 표현력이 크지만 risk도 크다. 고주파 안정성, latency, actuator saturation, contact instability를 직접 다루어야 한다. VLA policy가 torque를 직접 출력하려면 control frequency, safety certification, dynamics generalization 문제가 매우 어려워진다.

#### Torque output의 위험

Torque는 robot hardware에 가장 가까운 action이다. 학습 policy가 torque를 직접 내면 controller가 흡수하던 안정성 책임이 policy로 이동한다. 이 경우 inference delay, out-of-distribution state, contact uncertainty가 곧바로 safety issue가 된다.

## 3.3 Cartesian interfaces

대부분의 manipulation task는 joint보다 task-space에서 표현하기 쉽다. End-effector pose를  $\mathbf{x}_{ee} \in SE(3)$  또는 local coordinate vector로 쓰면, VLA output은 target pose 또는 delta pose가 될 수 있다.

### 3.3.1 End-effector target pose

End-effector target pose interface는 다음처럼 쓸 수 있다.

$$\mathbf{a}_t = \mathbf{x}_{ee,t+H}^{ref}. \quad (3.6)$$

Controller는 inverse kinematics 또는 operational-space control을 통해 이를 joint command로 바꾼다.

$$\dot{\mathbf{q}}_t^{ref} = J(\mathbf{q}_t)^\dagger K_x(\mathbf{x}_{ee,t+H}^{ref} - \mathbf{x}_{ee,t}). \quad (3.7)$$

여기서  $J(\mathbf{q}_t)$ 는 Jacobian,  $J^\dagger$ 는 pseudo-inverse이다.

이 interface는 robot morphology를 일부 숨길 수 있다. Policy는 end-effector motion을 내고, robot-specific kinematics는 controller가 처리한다. 그러나 singularity, joint limit, collision, orientation ambiguity는 여전히 controller가 해결해야 한다.

### 3.3.2 Delta pose

Delta pose interface는 VLA에서 자주 사용된다.

$$\mathbf{a}_t = \Delta \mathbf{x}_{ee,t}, \quad \mathbf{x}_{ee,t+1}^{ref} = \mathbf{x}_{ee,t} \oplus \Delta \mathbf{x}_{ee,t}. \quad (3.8)$$

여기서  $\oplus$ 는 pose composition을 의미한다. Delta pose는 local correction에 유리하지만, scale과 coordinate frame에 민감하다. Camera frame 기준 delta 인지, base frame 기준 delta 인지, end-effector frame 기준 delta 인지 명시하지 않으면 같은 vector가 완전히 다른 motion이 된다.

표 11: Cartesian interface에서 반드시 명시해야 할 항목

| 항목                         | 이유                                                                              |
|----------------------------|---------------------------------------------------------------------------------|
| Frame                      | base, camera, end-effector frame 중 무엇인지에 따라 motion 방향이 달라짐                      |
| Unit and scale             | normalized action 인지 metric meter/radian 인지 구분 필요                               |
| Rate                       | delta가 control step당 변화인지 VLA inference step당 변화인지 명시 필요                        |
| Orientation representation | Euler, quaternion, axis-angle, 6D representation의 singularity와 normalization 문제 |
| Limit handling             | workspace, velocity, acceleration, joint limit projection 필요                    |

### 3.4 Waypoint and trajectory interfaces

Waypoint interface는 policy가 단일 command 대신 중간 목표를 제안하는 구조이다.

$$\mathbf{a}_t = \mathbf{w}_{t:t+K} = \{\mathbf{x}_1^{ref}, \dots, \mathbf{x}_K^{ref}\}. \quad (3.9)$$

Motion planner 또는 trajectory optimizer는 waypoint를 연결해 시간 매개화된 trajectory를 만든다.

$$\xi^* = \arg \min_{\xi} J(\xi; \mathbf{w}_{t:t+K}) \quad \text{s.t.} \quad \xi(\tau) \in \mathcal{X}_{free}, \dot{\xi}(\tau) \in \mathcal{V}, \xi(0) = \mathbf{s}_t. \quad (3.10)$$

Waypoint interface의 장점은 policy와 motion planning의 역할을 분리한다는 점이다. Policy는 semantic target 또는 local waypoint를 내고, planner는 collision-free path와 timing을 만든다. 단점은 planner failure가 policy failure처럼 보일 수 있다는 점이다. Evaluation log는 waypoint feasibility, planner failure, controller tracking failure를 분리해야 한다.

### 3.5 Impedance and operational-space interfaces

Contact-rich manipulation에서는 position tracking만으로 부족하다. 로봇이 물체를 밀거나, 잡거나, 삽입하거나, 사람 가까이에서 움직일 때는 compliance가 필요하다.

#### 3.5.1 Impedance target

Impedance control은 end-effector가 가상의 spring-damper처럼 동작하도록 한다.

$$\mathbf{F}_{cmd} = K_x(\mathbf{x}^{ref} - \mathbf{x}) + D_x(\dot{\mathbf{x}}^{ref} - \dot{\mathbf{x}}) + \mathbf{F}^{ff}. \quad (3.11)$$

이때 VLA policy는 position target뿐 아니라 stiffness, damping, force reference를 제안할 수도 있다.

$$\mathbf{a}_t = (\mathbf{x}_t^{ref}, K_{x,t}, D_{x,t}, \mathbf{F}_t^{ff}). \quad (3.12)$$

이 interface는 manipulation에 강력하지만 safety risk도 크다. Stiffness를 잘못

선택하면 contact force가 커지고, damping이 부족하면 oscillation이 생긴다. 따라서 impedance parameter를 VLA가 직접 내는 경우에는 safety bound가 필요하다.

### 3.5.2 Operational-space control

Operational-space control은 task-space force를 joint torque로 변환한다.

$$\boldsymbol{\tau} = \mathbf{J}(\mathbf{q})^\top \mathbf{F}_{cmd} + \mathbf{N}(\mathbf{q})^\top \boldsymbol{\tau}_{null} + \mathbf{g}(\mathbf{q}). \quad (3.13)$$

여기서  $\mathbf{N}$ 은 null-space projection이다. 이 구조는 VLA policy가 task-space target을 내고, controller가 robot-specific dynamics를 처리하는 좋은 예이다.

## 3.6 MPC reference interface

Model predictive control(MPC)은 VLA output을 reference 또는 constraint로 받아 finite horizon optimization을 수행할 수 있다.

$$\begin{aligned} \mathbf{u}_{t:t+H-1}^* = \arg \min_{\mathbf{u}_{t:t+H-1}} & \sum_{k=0}^H \ell_x(\mathbf{s}_{t+k}, \mathbf{a}_t) + \sum_{k=0}^{H-1} \ell_u(\mathbf{u}_{t+k}) \\ \text{s.t. } & \mathbf{s}_{t+k+1} = f(\mathbf{s}_{t+k}, \mathbf{u}_{t+k}), \\ & \mathbf{s}_{t+k} \in \mathcal{X}_{safe}, \quad \mathbf{u}_{t+k} \in \mathcal{U}. \end{aligned} \quad (3.14)$$

여기서  $\mathbf{a}_t$ 는 target pose, cost parameter, forbidden region, subgoal, 또는 terminal set으로 들어갈 수 있다. MPC interface의 장점은 constraint와 prediction을 명시적으로 다룰 수 있다는 점이다. 단점은 model mismatch, optimization time, solver failure를 관리해야 한다는 점이다.

## 3.7 Safety projection

VLA output을 그대로 controller에 넣지 않고 safety set으로 project할 수 있다. 가장 단순한 형태는 다음과 같다.

$$\tilde{\mathbf{a}}_t = \arg \min_{\mathbf{a} \in \mathcal{A}_{safe}(\hat{\mathbf{s}}_t)} \|\mathbf{a} - \mathbf{a}_t\|^2. \quad (3.15)$$



이후 controller는  $\tilde{\mathbf{a}}_t$ 를 사용한다.

$$\mathbf{u}_t = \kappa(\tilde{\mathbf{a}}_t, \hat{\mathbf{s}}_t, \mathcal{C}_t). \quad (3.16)$$

이 방식은 action space가 metric structure를 가질 때 유용하다. Continuous delta나 waypoint는 projection이 비교적 자연스럽다. 반면 action token은 물리적 distance metric이 명확하지 않기 때문에 projection이 어렵다. 이것이 action representation과 safety가 연결되는 지점이다.

### 3.8 Control interface별 failure mode

표 12: Control interface별 대표 failure mode

| Interface         | 강점                               | 대표 failure mode                                         |
|-------------------|----------------------------------|---------------------------------------------------------|
| Joint position    | robot API와 단순 연결                 | morphology-specific, joint limit violation              |
| Joint velocity    | reactive correction 가능           | integration drift, accumulated unsafe motion            |
| Torque            | full dynamics control 가능         | latency와 OOD state에서 unstable behavior                  |
| End-effector pose | task-space 표현이 직관적               | IK failure, singularity, frame mismatch                 |
| Delta pose        | local closed-loop correction에 적합 | scale/frame ambiguity, drift                            |
| Waypoint          | planner와 결합 가능                   | infeasible waypoint, planner/controller blame ambiguity |
| Impedance target  | contact-rich task에 적합            | stiffness/force unsafe setting                          |
| MPC reference     | constraint와 horizon을 명시 가능       | model mismatch, solver latency                          |

이 표는 Chapter 13의 action taxonomy를 평가할 때 그대로 다시 사용된다. 어떤 action representation도 단독으로 좋거나 나쁘지 않다. 중요한 것은 task, controller, safety monitor, data log가 같은 interface contract를 공유하는가이다.

### 3.9 Evaluation log에 남겨야 할 control metadata

VLA paper가 real robot result를 보고할 때 control interface metadata가 빠지면 결과 해석이 불가능해진다. 최소한 다음 항목은 필요하다.

- action type: token, delta pose, waypoint, velocity, torque 등
- coordinate frame: base, camera, end-effector, world
- VLA inference rate와 controller rate
- downstream controller type
- action scaling과 normalization
- limit handling과 safety projection
- planner 또는 controller failure count
- human intervention과 reset policy

이 metadata가 있어야 benchmark success와 robot capability를 분리할 수 있다. 같은 success rate라도 torque-level policy와 waypoint+MPC system은 전혀 다른 evidence를 제공한다.

#### Case study: 같은 action, 다른 controller claim

“move 5 cm right”라는 action은 Cartesian delta controller에서는 local servo command이고, waypoint planner에서는 collision-checked target이며, impedance controller에서는 contact-compliant reference가 된다. 따라서 action string이 같아도 system claim은 다르다. 논문은 action syntax가 아니라 downstream controller를 밝혀야 한다.

### 3.10 Further reading and exercises

**Further reading** Operational-space control, impedance control, MPC를 Chapter 13의 action representation taxonomy와 함께 읽어라 [6, 7, 8].

**Review question** 왜 VLA output  $\mathbf{a}_t$ 를 robot command  $\mathbf{u}_t$ 와 동일시하면 safety와 evaluation 해석이 약해지는가?

**Design exercise** 하나의 tabletop task에 대해 delta pose, waypoint, impedance target 세 가지 controller interface를 설계하고 각각 필요한 meta-data를 비교하라.

### 3.11 Robotician's Takeaway

#### Chapter 3 Takeaway

VLA action은 output tensor가 아니라 control interface object이다.  $\mathbf{a}_t$ 의 의미는 downstream controller  $\kappa$ , state estimate  $\hat{\mathbf{s}}_t$ , safety constraint  $\mathcal{C}_t$ , command rate가 함께 정의할 때만 결정된다. 따라서 Real VLA 논문을 읽을 때는 model architecture만 보지 말고,  $\mathbf{a}_t$ 가 어떤 controller를 통해  $\mathbf{u}_t$ 가 되는지 확인해야 한다.

다음 장은 이 control interface가 contact, force, manipulation과 만날 때 어떤 문제가 생기는지 다룬다.

## 제4장

# Contact, Force, and Manipulation

## 학습 목표

이 장은 조작 VLA에서 접촉이 왜 별도 장으로 다루어져야 하는지를 설명한다. Chapter 2는 state와 observation의 차이를 다루었고, Chapter 3은 policy-level action과 controller command의 차이를 다루었다. Chapter 4의 초점은 action이 물체와 환경에 닿는 순간 발생하는 물리 제약이다.

1. contact mode, contact force, friction cone을 수식으로 표현한다.
2. 위치 제어만으로는 contact-rich manipulation을 설명할 수 없음을 보인다.
3. hybrid position/force control, impedance, admittance를 VLA interface 관점에서 정리한다.
4. grasp, slip, insertion, pushing, opening에서 필요한 관측과 평가 metadata를 정의한다.

## 4.1 Manipulation은 접촉 문제다

자연어 명령은 조작을 간단한 동사로 표현한다. “컵을 집어라”, “서랍을 열어라”, “peg를 hole에 넣어라”, “천을 집어라”와 같은 명령은 high-level semantics에서는 단순해 보인다. 그러나 로봇이 실행해야 하는 문제는 기하학적 이동만이 아니다. gripper가 물체에 닿는 순간부터 문제는 contact mode, normal force, tangential friction, compliance, slip, deformation, sensing delay의 문제로 바뀐다 [10, 7].

Real VLA stack에서 policy는 다음과 같이 action proposal을 낼 수 있다.

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{o}_t, \mathbf{q}_t, \ell, \mathbf{b}_t). \quad (4.1)$$

하지만 contact-rich task에서 성공 여부는  $\mathbf{a}_t$ 가 목표 pose를 향하는가만으로 결정되지 않는다. 실제 실행은 접촉력과 상대 운동을 포함한다.

$$\mathbf{u}_t = \kappa(\mathbf{a}_t, \hat{\mathbf{s}}_t, \mathcal{C}_t, \hat{\boldsymbol{\lambda}}_t), \quad (4.2)$$

여기서  $\hat{\boldsymbol{\lambda}}_t$ 는 추정된 접촉력 또는 접촉 impulse이고,  $\mathcal{C}_t$ 는 collision, force limit, joint limit, task constraint를 포함하는 constraint set이다.

### Contact-rich manipulation

Contact-rich manipulation은 task outcome이 자유공간의 end-effector trajectory보다 접촉 상태와 접촉력에 더 민감한 조작 문제이다. 대표적인 예는 grasping, insertion, pushing, pulling, wiping, folding, door/drawer opening, tool use이다.

따라서 contact-rich VLA를 평가할 때는 semantic correctness와 geometric tracking을 분리해야 한다. 모델이 올바른 물체를 선택하고 올바른 방향으로 움직였더라도, 접촉력이 과도하거나 slip을 감지하지 못하거나 insertion contact를 잘못 해석하면 task는 실패한다.

## 4.2 Contact state and mode

접촉을 다루기 위해 먼저 상태를 확장한다. 자유공간에서 로봇 상태를  $\mathbf{s}_t$ 로 쓰면, 접촉 작업에서는 contact mode  $m_t$ 와 contact force  $\boldsymbol{\lambda}_t$ 를 함께 고려해야 한다.

$$\mathbf{s}_t^c = (\mathbf{s}_t, m_t, \boldsymbol{\lambda}_t). \quad (4.3)$$

contact mode는 이산 변수로 표현할 수 있다.

$$m_t \in \{\text{no-contact, sticking, sliding, impact, rolling}\}. \quad (4.4)$$

이 표현은 단순한 표기상의 확장이 아니다. 같은 end-effector pose라도 contact mode가 다르면 필요한 제어가 달라진다. 예를 들어 peg insertion에서 end-effector 위치가 hole 근처에 있어도, peg가 rim에 걸려 sliding 중인지, 정렬되어 sticking 중인지, 충돌 후 튕겨 나오는 중인지에 따라 다음 action은 달라져야 한다.

#### Vision-only observation의 한계

카메라 영상은 물체 identity와 approximate pose를 제공할 수 있지만 접촉력, stick-slip transition, hidden contact, micro-slip을 항상 제공하지 않는다. 접촉 작업에서  $\mathbf{o}_t$ 가 image만으로 구성되면 belief  $\mathbf{b}_t$ 는 물리적으로 중요한 변수를 놓칠 수 있다.

Real VLA에서는 contact mode가 명시적으로 추정될 수도 있고, recurrent policy나 history-conditioned policy 내부에 암묵적으로 표현될 수도 있다. 중요한 점은 시스템 설계와 평가에서 이 변수를 숨기지 않는 것이다.

### 4.3 Contact force and friction cone

단일 접촉점에서 접촉력을 normal component와 tangential component로 분해하자. 접촉 normal을  $\mathbf{n}$ , 접촉력을  $\mathbf{f}$ 라 하면,

$$\mathbf{f}_n = \mathbf{n}^\top \mathbf{f}, \quad \mathbf{f}_t = \mathbf{f} - \mathbf{f}_n \mathbf{n}. \quad (4.5)$$

비침투 조건은 보통 normal force가 음수가 되지 않아야 함을 요구한다.

$$\mathbf{f}_n \geq 0. \quad (4.6)$$

Coulomb friction cone은 다음과 같이 쓸 수 있다.

$$\|\mathbf{f}_t\|_2 \leq \mu \mathbf{f}_n, \quad (4.7)$$

여기서  $\mu$ 는 friction coefficient이다. 식 (4.7)이 깨지면 접촉은 sticking을 유지하지 못하고 sliding으로 전이된다.

VLA policy가 “물체를 더 세게 잡아라”라는 식의 semantic instruction을 처리

한다고 해도, 실제 safety bound는 다음과 같은 수치 조건으로 표현되어야 한다.

$$f_n^{min} \leq f_n \leq f_n^{max}, \quad \|\mathbf{f}_t\|_2 \leq \mu f_n. \quad (4.8)$$

너무 작은 normal force는 slip을 만들고, 너무 큰 normal force는 물체 파손 또는 사람 접촉 위험을 만든다.

## 4.4 Hybrid position/force control

접촉 작업에서는 모든 방향을 위치로 제어하는 것이 적절하지 않다. 어떤 방향은 위치를 맞춰야 하고, 어떤 방향은 힘을 맞춰야 한다. Hybrid position/force control은 이 구분을 selection matrix로 표현한다. task-space displacement를  $\mathbf{x}$ , force를  $\mathbf{F}$ 라 하고, 위치 제어 방향 선택 행렬을  $S_p$ , 힘 제어 방향 선택 행렬을  $S_f$ 라 하자.

$$S_p + S_f = I, \quad S_p S_f = 0. \quad (4.9)$$

하나의 단순한 control law는 다음 형태를 갖는다.

$$\mathbf{F}_{cmd} = S_p K_x (\mathbf{x}^{ref} - \mathbf{x}) + S_f (\mathbf{F}^{ref} - \mathbf{F}). \quad (4.10)$$

이 식은 VLA interface 해석에 직접적인 의미를 갖는다. VLA가 end-effector target  $\mathbf{x}^{ref}$ 만 출력하면 시스템은 force direction을 어떻게 다룰지 별도로 정해야 한다. 반대로 VLA가 contact mode 또는 force target까지 출력한다면 action은 다음과 같이 확장된다.

$$\mathbf{a}_t = (\mathbf{x}_t^{ref}, \mathbf{F}_t^{ref}, S_{p,t}, S_{f,t}, m_t^{ref}). \quad (4.11)$$

이 표현은 강력하지만 label 수집과 safety 검증이 어려워진다. 따라서 실제 시스템에서는 VLA가 contact mode를 직접 예측하기보다, primitive 또는 controller preset을 선택하게 하는 구조도 자주 사용된다.

## 4.5 Impedance and admittance

Chapter 3에서는 impedance를 control interface의 하나로 소개했다. 여기서는 contact 관점에서 그 의미를 다시 정리한다. Impedance control은 위치 오차를

바로 큰 힘으로 바꾸지 않고, 원하는 동적 관계를 설정한다.

$$M_d(\ddot{\mathbf{x}} - \ddot{\mathbf{x}}^{ref}) + D_d(\dot{\mathbf{x}} - \dot{\mathbf{x}}^{ref}) + K_d(\mathbf{x} - \mathbf{x}^{ref}) = \mathbf{F}_{ext}. \quad (4.12)$$

여기서  $M_d$ ,  $D_d$ ,  $K_d$ 는 desired inertia, damping, stiffness 이고,  $\mathbf{F}_{ext}$ 는 외력이다. 높은 stiffness는 정확한 위치 추적에는 유리하지만 contact force overshoot를 만들 수 있다. 낮은 stiffness는 안전하고 compliant하지만 task precision이 낮아질 수 있다.

Admittance control은 측정된 힘을 받아 reference motion을 수정한다.

$$M_a\ddot{\mathbf{x}}^{ref} + D_a\dot{\mathbf{x}}^{ref} + K_a(\mathbf{x}^{ref} - \mathbf{x}^{nom}) = \mathbf{F}_{meas}. \quad (4.13)$$

force/torque sensor가 좋은 경우 admittance는 stiff robot에서도 compliant behavior를 만들 수 있다. 그러나 force sensing delay와 noise가 크면 oscillation이 발생할 수 있다.

표 13: 접촉 제어 방식과 VLA 결합 방식

| 방식                    | VLA가 줄 수 있는 입력                                     | 주의할 점                                       |
|-----------------------|----------------------------------------------------|---------------------------------------------|
| Hybrid position/force | contact mode, force direction, target force        | selection matrix 오류는 잘못된 방향의 힘 제어를 만든다      |
| Impedance             | target pose, stiffness, damping, feedforward force | stiffness bound와 contact safety limit이 필요하다 |
| Admittance            | nominal motion, force-response gain                | sensor delay와 noise가 안정성을 좌우한다              |
| Skill preset          | insert, push, wipe, open 같은 primitive 선택           | primitive coverage 밖의 task에서는 일반화가 제한된다     |

## 4.6 Grasp stability and slip

Grasping은 물체를 선택하고 gripper를 닫는 문제가 아니라 물체와 gripper 사이의 wrench balance 문제이다. 접촉점  $i$ 에서 접촉력이  $\mathbf{f}_i$ 이고 접촉 위치가  $\mathbf{r}_i$ 라면



물체에 작용하는 wrench는

$$\mathbf{w} = \sum_i \begin{bmatrix} \mathbf{f}_i \\ \mathbf{r}_i \times \mathbf{f}_i \end{bmatrix}. \quad (4.14)$$

외부 wrench  $\mathbf{w}_{ext}$ 를 상쇄하려면 다음 조건을 만족해야 한다.

$$\mathbf{w} + \mathbf{w}_{ext} = \mathbf{0}, \quad \mathbf{f}_i \in \mathcal{F}_i, \quad (4.15)$$

여기서  $\mathcal{F}_i$ 는 각 접촉점의 friction cone이다.

Slip은 접촉면의 tangential force가 friction limit에 가까워질 때 발생한다. 단 순한 slip margin은 다음과 같이 정의할 수 있다.

$$\rho_i = \mu_i f_{n,i} - \|\mathbf{f}_{t,i}\|_2. \quad (4.16)$$

$\rho_i > 0$ 이면 sticking margin이 있고,  $\rho_i \leq 0$ 이면 slip 위험이 있다. VLA가 action을 생성하더라도 grasp controller는 이 margin을 감시해야 한다.

#### Contact success

조작에서 성공은 object pose가 목표에 도달했는가만으로 정의되면 부족하다. 접촉 성공은 slip margin, force bound, collision absence, object damage absence, recovery behavior까지 포함해야 한다.

## 4.7 Tactile and force sensing

접촉 정보는 vision만으로 안정적으로 복원되기 어렵다. force/torque sensor, tactile array, joint torque estimate, motor current, gripper position error는 모두 contact observation이 될 수 있다. VLA 관점에서는 observation vector를 다음처럼 확장할 수 있다.

$$\mathbf{o}_t = (\mathbf{I}_t, \mathbf{d}_t, \mathbf{q}_t, \dot{\mathbf{q}}_t, \mathbf{f}_t^{ee}, \mathbf{z}_t^{tactile}, \mathbf{g}_t), \quad (4.17)$$

여기서  $\mathbf{I}_t$ 는 image,  $\mathbf{d}_t$ 는 depth,  $\mathbf{f}_t^{ee}$ 는 end-effector force/torque,  $\mathbf{z}_t^{tactile}$ 는 tactile feature,  $\mathbf{g}_t$ 는 gripper state이다.

센서가 추가되면 모델이 자동으로 contact를 이해한다고 보면 안 된다. sampling rate, time alignment, calibration, sensor saturation, tactile-to-action latency가 함께 기록되어야 한다. Chapter 2의 time alignment 문제는 contact sensing에서 더 심각해진다.

## 4.8 Examples: insertion, pushing, and opening

Contact-rich manipulation은 task마다 필요한 물리 변수가 다르다.

표 14: 대표 contact-rich task와 필요한 물리 정보

| Task           | 필요한 정보                                                                   | 대표 failure                                           |
|----------------|--------------------------------------------------------------------------|------------------------------------------------------|
| Peg insertion  | hole pose, chamfer, contact normal, axial force, lateral force           | rim contact, jamming, excessive insertion force      |
| Pushing        | object support friction, contact point, pushing direction, slip state    | object rotation, loss of contact, obstacle collision |
| Drawer opening | handle grasp, pulling direction, joint constraint, friction, force limit | handle slip, wrong pull direction, over-force        |
| Wiping         | surface normal, contact pressure, coverage path, compliance              | too low pressure, too high pressure, missed area     |
| Cloth folding  | deformable state, pinch point, tension, self-occlusion                   | hidden fold error, slip, irreversible wrinkle        |

이 표의 목적은 task taxonomy가 아니라 evaluation taxonomy이다. 같은 VLA architecture라도 contact profile이 다르면 필요한 observation과 controller가 달라진다. 따라서 “manipulation benchmark”라는 하나의 score만으로는 시스템의 실제 능력을 설명하기 어렵다.

## 4.9 Safety filter for contact

접촉 안전은 collision avoidance보다 좁은 문제가 아니다. collision이 없더라도 힘이 과하면 위험하고, 힘이 작아도 slip으로 인해 위험한 상태가 될 수 있다. 하

나의 단순한 safety filter는 다음과 같이 쓸 수 있다.

$$\tilde{\mathbf{a}}_t = \arg \min_{\mathbf{a} \in \mathcal{A}} \|\mathbf{a} - \mathbf{a}_t\|^2 \quad \text{s.t.} \quad f_n(\mathbf{a}, \hat{\mathbf{s}}_t) \leq f_n^{max}, \quad \rho_i(\mathbf{a}, \hat{\mathbf{s}}_t) \geq \rho^{min}. \quad (4.18)$$

이 식은 실제 구현에서는 conservative approximation으로 바뀐다. force prediction이 정확하지 않으면 measured force threshold, controller-side torque limit, stop condition, human supervision이 함께 사용된다.

VLA policy가 contact mode를 잘못 판단했을 때 safety layer는 세 가지 중 하나를 선택해야 한다.

$$d_t \in \{\text{allow, modify, stop/recover}\}. \quad (4.19)$$

contact-rich task에서 recovery는 필수 기능이다. insertion 실패 후 살짝 빼고 다시 정렬하기, grasp slip 후 재과지하기, drawer opening에서 handle을 다시 잡기 같은 행동은 단순 success/failure label보다 중요한 system capability이다.

## 4.10 Evaluation metadata

Contact-rich VLA 실험은 최소한 다음 metadata를 기록해야 한다.

- contact sensor type: force/torque, tactile, joint torque estimate, motor current
- controller type: position, hybrid force/position, impedance, admittance, MPC
- control frequency and VLA inference frequency
- maximum measured force and torque
- force threshold and stop condition
- slip or contact-loss detection rule
- recovery attempts and recovery success rate
- object damage, environment collision, human intervention count

이 metadata가 없으면 benchmark score를 해석하기 어렵다. 어떤 모델은 semantic grounding이 좋아서 성공할 수도 있고, 어떤 모델은 controller가 contact

error를 흡수해서 성공할 수도 있다. Real VLA 연구에서는 이 둘을 분리해 기록해야 한다.

#### Case study: vision-only success와 contact failure

VLA가 peg와 hole을 정확히 찾더라도 insertion 중 접촉력이 증가하고 미세 정렬이 실패하면 task는 멈춘다. 이 경우 image-text grounding은 성공했지만 contact mode와 compliance control이 실패한 것이다. success rate만 보고하면 grounding과 contact execution을 구분할 수 없다.

## 4.11 Further reading and exercises

**Further reading** Mason의 manipulation mechanics와 impedance control을 함께 읽고, language-conditioned manipulation이 왜 contact mechanics를 피할 수 없는지 확인하라 [10, 7].

**Review question** contact-rich task에서 “object를 정확히 보았다”와 “object를 안전하게 조작했다”는 왜 다른 claim인가?

**Design exercise** drawer opening task에 대해 contact sensor, force threshold, slip/contact-loss label, recovery action을 포함한 evaluation log를 설계하라.

## 4.12 요약

Contact는 Real VLA에서 예외 상황이 아니라 manipulation action의 기본 문법이다. VLA가 language와 vision을 통해 task intent를 잘 이해하더라도, 접촉 순간의 force, friction, compliance, slip, hidden state를 다루지 못하면 실제 조작은 실패한다. 따라서 contact-rich VLA는 policy architecture만으로 정의되지 않는다. 그것은 contact-aware observation, controller interface, force safety, recovery, evaluation log가 함께 있는 system이다.

다음 장에서는 이 물리적 실행 문제 위에 planning과 motion planning, 그리고 task and motion planning을 올린다. 접촉을 고려한 action이 가능하려면, 먼저

어떤 subgoal과 constraint를 어떤 순서로 달성해야 하는지 결정해야 하기 때문이다.

## 제5장

# Planning, Motion Planning, and TAMP

## 학습 목표

이 장은 Real VLA에서 planning layer가 왜 사라지지 않는지 설명한다. VLA가 action을 직접 생성하더라도 long-horizon task에서는 goal decomposition, pre-condition, feasibility, recovery, constraint satisfaction이 필요하다. 이 장의 목표는 classical planning을 복고적으로 소개하는 것이 아니라, VLA가 실제 로봇 시스템 안에서 planner와 어떻게 결합되는지 이해하는 것이다.

1. symbolic planning, motion planning, TAMP의 문제 정의를 구분한다.
2. language instruction이 plan, subgoal, constraint로 바뀌는 과정을 정식화한다.
3. continuous feasibility가 왜 LLM/VLA planning의 병목인지 설명한다.
4. VLA planner, affordance model, motion planner, controller가 하나의 loop로 연결되는 구조를 정리한다.

## 5.1 Planning은 action sequence의 문제다

자연어 명령은 종종 하나의 문장으로 주어진다. 그러나 로봇이 실행해야 하는 것은 보통 하나의 action이 아니라 조건을 만족하는 action sequence이다. 예를 들어 “테이블 위 컵을 식기 건조대에 놓아라”라는 명령은 object grounding, grasp

feasibility, collision-free transport, stable placement, release verification을 포함한다. 이 문제를 바로 continuous action vector 하나로 바꾸면 중요한 구조가 사라진다. task planning 문제는 다음 tuple로 표현할 수 있다 [11, 14].

$$\mathcal{P} = (\mathcal{S}, \mathcal{A}, s_0, \mathcal{G}, \gamma). \quad (5.1)$$

여기서  $\mathcal{S}$ 는 symbolic state set,  $\mathcal{A}$ 는 symbolic action set,  $s_0$ 는 initial state,  $\mathcal{G}$ 는 goal condition,  $\gamma$ 는 transition function이다. 하나의 plan은 action sequence이다.

$$\pi^{plan} = (a_1, a_2, \dots, a_K), \quad s_K \in \mathcal{G}. \quad (5.2)$$

이때  $a_k$ 는 VLA policy의 continuous action  $a_t$ 와 구분하기 위해 symbolic action으로 읽어야 한다. 예를 들어 `pick(mug)`, `place(mug, rack)`, `open(drawer)`는 symbolic action이고, end-effector delta pose나 gripper command는 controller가 실행하는 action이다.

### Task planning

Task planning은 목표를 달성하기 위한 discrete action sequence를 찾는 문제이다. 이 계층은 무엇을 어떤 순서로 해야 하는지 다루지만, 각 action이 실제 로봇의 configuration space에서 가능한지는 별도 검사가 필요하다.

## 5.2 Preconditions and effects

고전 symbolic planning의 핵심은 action을 precondition과 effect로 표현하는 것이다. action  $a$ 가 state  $s$ 에서 실행 가능하려면 precondition이 만족되어야 한다.

$$\text{Pre}(a) \subseteq s. \quad (5.3)$$

action을 실행한 뒤의 state는 add effect와 delete effect로 갱신된다.

$$\gamma(s, a) = (s \setminus \text{Del}(a)) \cup \text{Add}(a). \quad (5.4)$$

로봇 조작 예제에서 `pick(mug)`의 precondition은 다음과 같은 symbolic pred-

icate로 구성될 수 있다.

$$\text{Pre}(\text{pick}(\text{mug})) = \{\text{visible}(\text{mug}), \text{reachable}(\text{mug}), \text{graspable}(\text{mug}), \text{gripper-empty}\}. \quad (5.5)$$

문제는 **reachable**이나 **graspable**이 순수 symbolic predicate가 아니라는 점이다. 이것들은 robot geometry, scene geometry, grasp sampler, IK, collision checking, contact condition에 의해 결정된다. 이 지점에서 task planning과 motion planning이 결합된다.

### 5.3 Search and heuristic

Planning은 search problem으로 볼 수 있다. 각 node는 symbolic state이고, edge는 action이다. 비용이 있는 planning에서는 다음 목적을 최소화한다.

$$\pi^{plan^*} = \arg \min_{\pi^{plan}} \sum_{k=1}^K c(s_{k-1}, a_k) \quad \text{s.t.} \quad s_K \in \mathcal{G}. \quad (5.6)$$

A\*와 같은 heuristic search는 다음 priority를 사용한다.

$$F(n) = G(n) + H(n), \quad (5.7)$$

여기서  $G(n)$ 은 현재 node까지의 cost,  $H(n)$ 은 goal까지의 추정 cost이다.

LLM/VLM planner를 이 관점으로 보면, 큰 모델은 heuristic 또는 proposal generator 역할을 할 수 있다. 즉, 어떤 action sequence가 plausible한지 제안할 수 있다. 그러나 plausible plan과 executable plan은 다르다. Real VLA에서는 language model proposal이 feasibility checker, affordance model, motion planner를 통과해야 한다.

#### Plausible plan은 executable plan이 아니다

LLM이 “컵을 집고 선반에 놓아라”라는 plan을 생성할 수 있어도, 컵이 reachable한지, grasp가 가능한지, 선반까지 collision-free path가 있는지, 놓은 뒤 안정적인지 보장하지 않는다. 자연어 계획은 physical feasibility check를 대체하지 못한다.



## 5.4 Motion planning in configuration space

Motion planning은 continuous configuration space에서 충돌 없는 경로를 찾는 문제이다. 로봇 configuration을  $\mathbf{q} \in \mathcal{Q}$ 라 하자. 장애물과 self-collision을 피하는 자유공간은

$$\mathcal{Q}_{free} = \{\mathbf{q} \in \mathcal{Q} \mid \text{Collision}(\mathbf{q}) = 0\}. \quad (5.8)$$

motion planning 문제는 시작 configuration  $\mathbf{q}_{start}$ 와 목표 set  $\mathcal{Q}_{goal}$  사이의 경로를 찾는 것이다.

$$\xi : [0, 1] \rightarrow \mathcal{Q}_{free}, \quad \xi(0) = \mathbf{q}_{start}, \quad \xi(1) \in \mathcal{Q}_{goal}. \quad (5.9)$$

VLA가 end-effector target이나 waypoint를 출력한다면, 그것은 보통  $\mathcal{Q}_{goal}$  또는 intermediate constraint를 정의한다. 하지만 실제 path  $\xi$ 는 collision, joint limit, velocity limit, singularity, workspace bound를 만족해야 한다.

## 5.5 Sampling-based planning

PRM이나 RRT 계열의 sampling-based planner는 continuous space를 직접 격자로 나누지 않고 sampled configuration을 이용해 graph 또는 tree를 만든다. 단순화하면 roadmap planner는 다음 graph를 만든다.

$$\begin{aligned} \mathcal{G}_{roadmap} &= (V, E), & V &\subset \mathcal{Q}_{free}, \\ E &= \{(q_i, q_j) \mid \text{LocalPlan}(q_i, q_j) \subset \mathcal{Q}_{free}\}. \end{aligned} \quad (5.10)$$

RRT 계열은 시작 configuration에서 tree를 확장한다.

$$q_{new} = \text{Steer}(q_{near}, q_{rand}, \eta), \quad q_{new} \in \mathcal{Q}_{free}. \quad (5.11)$$

이 방법들은 high-dimensional configuration space에서 유용하지만, VLA와 결합할 때 두 가지 문제가 생긴다. 첫째, planner가 실패했을 때 그것이 VLA의 잘못된지, goal pose의 잘못된지, scene estimate의 잘못된지 구분해야 한다. 둘째, planner의 computation time이 VLA inference loop와 맞아야 한다. 따라서 planner output은 trajectory뿐 아니라 failure reason을 함께 반환해야 한다.

## 5.6 Trajectory optimization

Trajectory optimization은 경로를 변수로 두고 objective와 constraint를 최적화한다.

$$\begin{aligned} \xi^* = \arg \min_{\xi} \quad & J_{task}(\xi; \mathbf{a}_t) + \lambda_s J_{smooth}(\xi) + \lambda_c J_{collision}(\xi) \\ \text{s.t.} \quad & \xi(\tau) \in \mathcal{Q}_{free}, \\ & \dot{\xi}(\tau) \in \mathcal{V}, \quad \ddot{\xi}(\tau) \in \mathcal{A}. \end{aligned} \quad (5.12)$$

Trajectory optimization은 VLA가 생성한 goal, cost map, affordance map, forbidden region을 objective나 constraint로 넣기 좋다. 예를 들어 “유리컵을 건드리지 말라”는 instruction은 forbidden region  $\mathcal{R}_{forbid}$ 로 바뀔 수 있고, collision 또는 distance constraint로 들어갈 수 있다.

$$d(\xi(\tau), \mathcal{R}_{forbid}) \geq d_{min} \quad \forall \tau. \quad (5.13)$$

## 5.7 Task and motion planning

TAMP는 symbolic action sequence와 continuous parameter를 함께 찾는 문제이다. symbolic action  $a_k$ 는 grasp, placement, path, timing 같은 continuous parameter  $\theta_k$ 를 필요로 한다.

$$(a_{1:K}^*, \theta_{1:K}^*) = \arg \min_{a_{1:K}, \theta_{1:K}} \sum_{k=1}^K C(a_k, \theta_k) \quad \text{s.t.} \quad \Phi(a_{1:K}, \theta_{1:K}; \hat{\mathbf{s}}_0) = 1. \quad (5.14)$$

여기서  $\Phi$ 는 symbolic precondition, geometric feasibility, collision checking, grasp validity, placement stability를 모두 포함하는 feasibility predicate이다.

예를 들어 **pick(mug)**는 단순한 symbolic action처럼 보이지만 실제로는 grasp pose  $g$ , pregrasp pose  $p$ , approach trajectory  $\xi$ , gripper force  $f_g$ 를 필요로 한다.

$$\theta_{\text{pick}} = (g, p, \xi, f_g). \quad (5.15)$$

따라서 TAMP는 “무엇을 할 것인가”와 “그것을 어떻게 물리적으로 실행할 것인가”를 동시에 다룬다.

표 15: Planning 계층별 책임

| 계층                      | 입력/출력                                                                    | 대표 failure                                                           |
|-------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------|
| Task planning           | symbolic state, goal $\rightarrow$ action sequence                       | missing precondition, wrong ordering, no recovery branch             |
| Motion planning         | start/goal configuration, geometry $\rightarrow$ collision-free path     | IK failure, narrow passage, collision, timeout                       |
| Trajectory optimization | cost, constraints $\rightarrow$ smooth feasible trajectory               | local minimum, constraint violation, solver latency                  |
| TAMP                    | symbolic action + continuous parameter search                            | sampled grasp/pose infeasible, combinatorial explosion               |
| VLA/ER planning         | language, image, memory $\rightarrow$ subgoal, affordance, plan proposal | plausible but infeasible plan, hallucinated object, unsafe ambiguity |

## 5.8 VLA as planner, scorer, or executor

VLA와 planning의 결합 방식은 하나가 아니다. Real VLA stack에서 큰 모델은 다음 네 역할 중 하나를 맡을 수 있다.

1. **Planner:** language와 scene을 보고 symbolic 또는 semi-symbolic plan을 제안한다.
2. **Scorer:** 여러 skill 또는 action 후보의 affordance score를 계산한다.
3. **Constraint generator:** forbidden region, target relation, success condition을 만든다.
4. **Executor:** observation과 subgoal을 받아 continuous action을 생성한다.

이 역할들은 한 시스템 안에서 함께 존재할 수 있다.

$$\ell, \mathbf{o}_t \xrightarrow{\text{ER/VLM}} (g_{1:K}, \mathcal{C}_{1:K}) \xrightarrow{\text{TAMP/Motion Planner}} (\xi_{1:K}, r_{1:K}) \xrightarrow{\text{VLA/Controller}} \mathbf{u}_t. \quad (5.16)$$

여기서  $g_{1:K}$ 는 subgoal sequence,  $\mathcal{C}_{1:K}$ 는 constraint sequence,  $\xi_{1:K}$ 는 feasible trajectory,  $r_{1:K}$ 는 controller reference이다.

## 5.9 Closed-loop replanning

실제 로봇 실행은 open-loop plan execution 이 아니다. perception error, object motion, human intervention, contact failure, planner timeout 이 발생하면 replanning 이 필요하다. 하나의 receding planning loop는 다음과 같이 쓸 수 있다.

$$\begin{aligned}
 \mathbf{b}_t &= \text{UpdateBelief}(\mathbf{b}_{t-1}, \mathbf{o}_t), \\
 p_t &= \text{Plan}(\ell, \mathbf{b}_t, \mathcal{C}_t), \\
 \mathbf{a}_t &= \pi_\theta(\mathbf{o}_t, \mathbf{q}_t, p_t), \\
 \mathbf{u}_t &= \kappa(\mathbf{a}_t, \hat{\mathbf{s}}_t, \mathcal{C}_t).
 \end{aligned} \tag{5.17}$$

이 loop에서 VLA는 plan을 매 step 다시 만들 수도 있고, plan은 느리게 갱신하고 VLA policy만 빠르게 실행할 수도 있다. 핵심은 plan, action, controller, safety가 서로 다른 rate로 동작할 수 있다는 점이다.

## 5.10 Evaluation metadata

Planning이 포함된 VLA 실험은 단순 success rate 만으로 평가하면 부족하다. 최소한 다음 항목을 기록해야 한다.

- plan length and number of replans
- symbolic failure reason: missing object, unsatisfied precondition, no valid action
- motion planning failure reason: IK failure, collision, timeout, narrow passage
- trajectory optimization status: converged, infeasible, constraint violation
- VLA role: planner, scorer, constraint generator, executor
- recovery branch and human intervention count
- time budget: ER/VLM inference, planning, motion planning, controller loop

이 기록이 없으면 “VLA가 실패했다”라는 문장은 너무 거칠다. 실패는 semantic parsing, object grounding, symbolic precondition, grasp sampling, IK, collision checking, controller tracking, contact safety 중 어느 곳에서든 발생할 수 있다.

**Case study: SayCan과 VoxPoser를 feasibility 관점에서 읽기**

SayCan은 language likelihood와 affordance score를 결합해 executable skill을 고르려 했고, VoxPoser는 language-conditioned value map을 통해 motion optimization이 사용할 field를 만들었다 [28, 31]. 두 사례 모두 핵심은 planner가 자연어 plan을 쓰는지가 아니라, semantic proposal이 feasibility check와 어떻게 만나는가이다.

## 5.11 Further reading and exercises

**Further reading** Sampling-based motion planning과 TAMP를 LLM/VLM planner 논문과 함께 읽어라 [11, 12, 13, 14].

**Review question** LLM이 plan text를 잘 생성해도 robot execution plan이 되지 않는 이유는 무엇인가?

**Design exercise** mug/rack/glass task를 symbolic precondition, motion feasibility, contact constraint, recovery branch로 나누어 plan schema를 작성하라.

## 5.12 요약

Planning은 Real VLA에서 과거의 모듈이 아니라 현재의 responsibility layer이다. LLM/VLM/VLA는 plan proposal, affordance scoring, constraint generation, action execution을 강하게 만들 수 있다. 그러나 physical feasibility, collision checking, contact constraint, recovery planning은 여전히 시스템 안에 명시적으로 존재해야 한다. 강한 Real VLA는 end-to-end policy와 planner 중 하나를 선택하는 시스템이 아니라, semantic reasoning과 geometric feasibility를 closed-loop로 연결하는 시스템이다.

다음 장에서는 policy를 어떻게 학습하는가로 이동한다. planning이 무엇을 해야 하는지와 어떤 경로가 가능한지를 다룬다면, imitation learning은 실제 trajectory에서 action policy를 어떻게 배우는지 다룬다.

## 제 II 편

# Learning Foundations

## 제6장

# Imitation Learning and Covariate Shift

## 학습 목표

이 장은 VLA 이전의 로봇러닝에서 가장 직접적인 계보인 imitation learning을 다룬다. VLA는 foundation model처럼 보이지만, 로봇 action을 학습한다는 점에서 여전히 demonstration, behavior cloning, rollout distribution, correction data의 문제를 가진다.

1. behavior cloning objective를 정의한다.
2. open-loop supervised learning과 closed-loop robot deployment의 차이를 설명한다.
3. covariate shift와 error compounding을 수식으로 정리한다.
4. DAgger, intervention, recovery data가 VLA fine-tuning에서 왜 중요한지 설명한다.

## 6.1 Demonstration data

로봇 imitation learning의 입력은 demonstration dataset이다. 하나의 trajectory를 다음과 같이 쓰자 [15, 19].

$$\tau_i = \{(o_{i,t}, \mathbf{q}_{i,t}, \ell_i, \mathbf{a}_{i,t}^E)\}_{t=1}^{T_i}. \quad (6.1)$$

여기서  $\mathbf{a}_{i,t}^E$  는 expert action 이다. VLA setting 에서는  $\mathbf{o}_{i,t}$  가 image, depth, proprioception, tactile signal, history 를 포함할 수 있고,  $\ell_i$  는 language instruction 또는 subgoal 이다. 전체 dataset 은

$$\mathcal{D}_E = \{\tau_i\}_{i=1}^N. \quad (6.2)$$

중요한 점은 trajectory 가 단순한 image-action pair 가 아니라는 것이다. trajectory 는 embodiment, controller, action representation, reset procedure, safety rule, annotation scheme 에 묶여 있다. 같은 “pick” demonstration 이라도 action 이 joint position 인지, end-effector delta 인지, waypoint 인지, action chunk 인지 에 따라 학습 문제가 달라진다.

## 6.2 Behavior cloning

Behavior cloning 은 expert action 을 supervised learning target 으로 둔다. 가장 단순한 objective 는 다음과 같다.

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathbf{o}, \mathbf{q}, \ell, \mathbf{a}^E) \sim \mathcal{D}_E} [\mathcal{L}(\pi_{\theta}(\mathbf{o}, \mathbf{q}, \ell), \mathbf{a}^E)]. \quad (6.3)$$

연속 action 에서는  $\ell_2$  loss 나 negative log-likelihood 가 쓰일 수 있다.

$$\mathcal{L}_{reg} = \|\pi_{\theta}(\mathbf{o}, \mathbf{q}, \ell) - \mathbf{a}^E\|_2^2. \quad (6.4)$$

확률적 policy 에서는 expert action 의 likelihood 를 최대화한다.

$$\mathcal{L}_{nll} = -\log p_{\theta}(\mathbf{a}^E \mid \mathbf{o}, \mathbf{q}, \ell). \quad (6.5)$$

VLA 에서는 action distribution 이 단봉일 필요가 없다. 같은 instruction 과 observation 에서도 여러 grasp, 여러 path, 여러 correction 이 가능하다. 따라서 단순 regression loss 는 multi-modal action 을 평균내는 문제를 만들 수 있다. 이 문제는 diffusion policy, flow matching, discretized action token, mixture density 같은 표현으로 이어진다. 자세한 action representation 은 Chapter 13 에서 다룬다.



### Behavior cloning

Behavior cloning은 expert trajectory에서 observation-condition-action mapping을 supervised learning으로 학습하는 imitation learning 방법이다. 장점은 단순성과 데이터 효율이고, 핵심 약점은 closed-loop rollout 중 학습 분포를 벗어날 수 있다는 점이다.

## 6.3 Open-loop accuracy is not closed-loop success

Behavior cloning loss는 expert dataset distribution 위에서 계산된다. 하지만 deployment에서 policy가 방문하는 state distribution은 expert distribution과 다를 수 있다. expert가 만드는 state distribution을  $d_E(s)$ , learned policy가 만드는 distribution을  $d_\theta(s)$ 라 하자. 학습 objective는 대개

$$\mathbb{E}_{s \sim d_E}[\ell(\pi_\theta(s), \pi_E(s))] \quad (6.6)$$

를 줄인다. 그러나 deployment risk는

$$\mathbb{E}_{s \sim d_\theta}[\ell(\pi_\theta(s), \pi_E(s))] \quad (6.7)$$

에 가깝다.

두 distribution이 같으면 supervised learning으로 충분할 수 있다. 문제는 작은 action error가 다음 observation을 바꾸고, 바뀐 observation에서 다시 더 큰 error가 발생한다는 점이다. 이를 covariate shift라고 부른다.

### Open-loop metric의 한계

validation set에서 action prediction error가 낮아도 real robot rollout success가 높다는 보장은 없다. validation set은 보통 expert가 방문한 상태이고, 배치 중 policy가 만드는 off-distribution 상태를 충분히 포함하지 않는다.

## 6.4 Covariate shift

closed-loop rollout을 다음 동역학으로 쓰자.

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t, \mathbf{w}_t), \quad \mathbf{a}_t = \pi_\theta(\mathbf{o}_t, \mathbf{q}_t, \ell). \quad (6.8)$$

expert policy는  $\pi_E$  이고 learned policy는  $\pi_\theta$  이다. 한 step error probability를

$$\epsilon = \Pr_{\mathbf{s} \sim d_E} [\pi_\theta(\mathbf{s}) \neq \pi_E(\mathbf{s})] \quad (6.9)$$

라고 하면, horizon이 길어질수록 누적 실패 확률은 커진다. 매우 단순한 union bound만 써도

$$\Pr[\text{at least one error in } T \text{ steps}] \leq T\epsilon. \quad (6.10)$$

실제 문제는 이보다 더 나쁘다. early error가 policy를 expert dataset 밖의 상태로 보내면 다음 step의 error probability 자체가 증가한다.

VLA에서는 covariate shift가 여러 형태로 나타난다.

- object pose가 demonstration보다 조금 벗어난 상태
- gripper가 half-closed인 상태
- failed grasp 이후 물체가 기울어진 상태
- language instruction은 같지만 scene relation이 바뀐 상태
- contact 후 slip이 발생했지만 image에서는 잘 보이지 않는 상태

## 6.5 Recovery as a learned behavior

많은 dataset은 성공 trajectory 중심으로 구성된다. 하지만 real robot은 실패 직전과 실패 직후의 상태를 자주 만난다. 따라서 policy는 nominal action뿐 아니라 recovery action도 배워야 한다. recovery policy를 명시적으로 쓰면 다음과 같은 gating 구조가 된다.

$$\mathbf{a}_t = \begin{cases} \pi_\theta^{nom}(\mathbf{o}_t, \mathbf{q}_t, \ell), & r_t = 0, \\ \pi_\phi^{rec}(\mathbf{o}_t, \mathbf{q}_t, \ell, e_t), & r_t = 1, \end{cases} \quad (6.11)$$

여기서  $r_t$ 는 recovery mode indicator이고,  $e_t$ 는 failure estimate이다. 하나의 큰 VLA policy가 nominal과 recovery를 모두 학습할 수도 있지만, 데이터에는 recovery context가 명시적으로 들어가지 않는다.

### Recovery data

Recovery data는 실패 후 원래 task를 계속 수행하기 위해 필요한 correction trajectory이다. 예를 들어 regrasp, pull-back-and-retry, re-localize, ask-human, stop-safe 같은 행동이 포함된다.

## 6.6 DAgger and dataset aggregation

DAgger의 핵심은 learned policy가 실제로 방문하는 상태에서 expert label을 다시 모으는 것이다. 반복  $k$ 에서 policy  $\pi_{\theta_k}$ 를 rollout하여 방문 상태  $s \sim d_{\theta_k}$ 를 수집하고, expert action을 query한다.

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \{(\mathbf{o}_t, \mathbf{q}_t, \ell, \pi_E(\mathbf{o}_t, \mathbf{q}_t, \ell))\}_{s_t \sim d_{\theta_k}}. \quad (6.12)$$

그 다음 aggregated dataset 위에서 다시 policy를 학습한다.

$$\theta_{k+1} = \arg \min_{\theta} \mathbb{E}_{(\mathbf{o}, \mathbf{q}, \ell, \mathbf{a}^E) \sim \mathcal{D}_{k+1}} [\mathcal{L}(\pi_{\theta}(\mathbf{o}, \mathbf{q}, \ell), \mathbf{a}^E)]. \quad (6.13)$$

Real VLA에서 DAgger를 그대로 적용하기는 쉽지 않다. expert query가 비싸고, robot rollout이 느리며, unsafe state에서 expert intervention이 필요하다. 그러나 원리는 중요하다. 좋은 VLA dataset은 성공 demonstration만 모으는 것이 아니라 policy가 실제로 실패하는 state를 찾아내고, 그 state에서 어떤 correction이 필요한지 기록해야 한다.

## 6.7 Intervention and correction labels

실제 로봇 데이터 수집에서는 사람이 teleoperation, shared autonomy, emergency stop, verbal correction으로 개입할 수 있다. 이 개입은 단순히 나쁜 rollout을 제거

할 근거가 아니라 학습 signal 이다. trajectory record를 다음처럼 확장할 수 있다.

$$\tau_i = \{(\mathbf{o}_t, \mathbf{q}_t, \ell, \mathbf{a}_t, h_t, c_t, y_t)\}_{t=1}^{T_i}. \quad (6.14)$$

여기서  $h_t$  는 human intervention flag,  $c_t$  는 correction action 또는 instruction,  $y_t$  는 success/failure/progress label 이다.

표 16: VLA imitation dataset에 필요한 label

| Label               | 의미                                           | 쓰임                         |
|---------------------|----------------------------------------------|----------------------------|
| success label       | task 종료 시 성공 여부                              | benchmark와 reward model 학습 |
| progress label      | subgoal 진행 상태                                | long-horizon monitoring    |
| intervention flag   | 사람이 개입한 시점                                   | 위험 상태 탐지와 data filtering   |
| correction action   | expert 가 바꾼 action                           | DAGger-style aggregation   |
| failure mode        | grasp fail, collision, slip, time-out 등      | targeted data collection   |
| controller metadata | action type, controller, rate, safety filter | action representation 해석   |

이 label들이 없으면 VLA fine-tuning은 성공 trajectory의 평균적인 action만 모방하게 된다. 반대로 correction과 failure mode가 있으면 data engine은 어떤 state를 더 수집해야 하는지 알 수 있다.

## 6.8 Success-only data의 위험

성공 trajectory만 모으면 dataset은 깨끗해 보인다. 그러나 성공 data만으로는 다음 질문에 답하기 어렵다.

- policy가 실패 직전에 어떤 warning signal을 보아야 하는가?
- 실패 후 어떤 recovery action이 가능한가?
- 어떤 failure가 perception 문제이고 어떤 failure가 control interface 문제인가?
- user instruction이 ambiguous할 때 stop/ask behavior를 어떻게 배울 것인가?

실패 data는 모델을 나쁘게 만드는 noise가 아니다. 제대로 annotation되면 failure data는 safety, recovery, uncertainty estimation의 핵심 자원이다.

## 6.9 Action representation and imitation difficulty

같은 behavior cloning 이라도 action representation에 따라 난이도가 달라진다.

표 17: Action representation과 imitation learning 난이도

| Action type           | 장점                            | imitation risk                              |
|-----------------------|-------------------------------|---------------------------------------------|
| Joint position        | robot API와 직접 연결 가능           | embodiment-specific, joint limit 근처 error   |
| End-effector delta    | multi-robot transfer에 유리      | frame/scale ambiguity, contact force 미반영    |
| Waypoint              | planner와 결합 가능                | planner failure와 policy failure가 섞임         |
| Action chunk          | temporal consistency 증가       | interruption과 recovery가 어려움                 |
| Discrete token        | language-model pipeline과 잘 맞음 | quantization error, continuous precision 손실 |
| Diffusion/flow action | multi-modal action 표현 가능      | sampling latency와 controller rate 문제        |

따라서 imitation learning chapter는 단순히 loss function의 장이 아니다. 어떤 action interface를 imitation target으로 삼을지 결정하는 장이다.

## 6.10 Data engine view

Real VLA에서 imitation learning은 one-shot training recipe가 아니라 data engine이다. 실행 중 실패를 발견하고, 그 실패를 분류하고, 필요한 correction을 수집하고, 다시 fine-tuning하거나 evaluation set에 넣는 loop가 필요하다.

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup \mathcal{D}_t^{success} \cup \mathcal{D}_t^{failure} \cup \mathcal{D}_t^{correction}. \quad (6.15)$$

이 loop는 Chapter 14에서 더 자세히 다룰 data engine의 기본 형태이다.

**Case study: success-only data가 recovery를 지우는 방식**

성공 trajectory 만 모으면 policy는 실패 직전 state와 recovery action을 보지 못한다. 실제 rollout에서 gripper가 미끄러지거나 object pose가 살짝 어긋나면 policy는 expert distribution 밖의 상태에 놓인다. intervention과 correction label이 없는 dataset은 offline accuracy가 높아도 deployment recovery를 설명하지 못한다.

## 6.11 Further reading and exercises

**Further reading** DAgger와 guided policy search를 읽고, VLA fine-tuning에서 failure/correction data가 왜 demonstration보다 중요한 순간이 있는지 확인하라 [15, 19].

**Review question** behavior cloning이 expert distribution에서는 강하지만 robot rollout에서는 약해지는 이유는 무엇인가?

**Design exercise** 하나의 manipulation task에 대해 success, failure, correction, intervention, recovery record를 포함한 data collection protocol을 설계하라.

## 6.12 요약

Imitation learning은 VLA 이전의 낡은 방법이 아니라 VLA의 핵심 학습 기반이다. Behavior cloning은 강력하지만 expert distribution 위에서만 학습한다. 실제 로봇은 learned policy가 만든 상태분포 위에서 움직이므로 covariate shift와 compounding error가 발생한다. 따라서 Real VLA dataset은 성공 demonstration만이 아니라 failure, intervention, correction, recovery, controller metadata를 포함해야 한다. 이 관점이 없으면 VLA fine-tuning은 높은 offline score와 낮은 real rollout success 사이의 간극을 반복하게 된다.

다음 장에서는 imitation 만으로 충분하지 않을 때 필요한 reward, cost, constraint, exploration의 언어, 즉 reinforcement learning과 optimal control로 넘어간다.

## 제7장

# RL, Optimal Control, and Control as Inference

## 학습 목표

이 장은 demonstration을 넘어 policy를 개선하려면 어떤 수학적 언어가 필요한지 다룬다. Chapter 6이 expert trajectory를 모방하는 문제를 다루었다면, Chapter 7은 reward, cost, constraint, uncertainty, exploration, action distribution을 다룬다. VLA 시대에도 이 개념들은 사라지지 않는다. 단지 language instruction과 visual context가 state와 objective 위에 추가되었을 뿐이다.

1. language-conditioned RL 문제를 MDP/POMDP 관점에서 정식화한다.
2. return, value, Q function, Bellman relation을 VLA policy와 연결한다.
3. finite-horizon optimal control과 MPC가 VLA controller layer에서 갖는 의미를 설명한다.
4. maximum entropy RL과 control as inference가 action-distribution VLA를 이해하는 데 주는 관점을 정리한다.

## 7.1 Why imitation is not enough

Behavior cloning은 expert가 잘하는 구간에서는 강력하다. 그러나 expert data가 충분하지 않거나 task objective가 명시적인 cost/constraint를 포함하면 imitation 만으로는 부족하다 [16, 17, 18]. 예를 들어 “컵을 식기건조대에 놓아라”라는 task



에서 우리는 성공 여부, 충돌 회피, 힘 제한, 시간, smoothness, recovery 성공률을 함께 보고 싶다. 이런 요구는 supervised loss 만으로 표현하기 어렵다.

정책이 실제 환경과 상호작용하면서 성능을 개선하려면 다음 질문이 필요하다.

- 어떤 outcome이 좋은가?
- 어떤 constraint를 절대 위반하면 안 되는가?
- exploration은 얼마나 허용 가능한가?
- offline robot data만으로 policy improvement를 할 수 있는가?
- VLA가 낸 action distribution은 어떤 cost나 reward와 일관되는가?

## 7.2 Language-conditioned MDP

가장 단순하게는 language-conditioned robot control을 MDP로 쓸 수 있다.

$$\mathcal{M}_\ell = (\mathcal{S}, \mathcal{A}, P, r_\ell, \gamma). \quad (7.1)$$

여기서  $\ell$ 는 language instruction 또는 task embedding 이고,  $r_\ell$ 는 그 instruction에 따라 달라지는 reward function이다. policy는

$$\mathbf{a}_t \sim \pi_\theta(\mathbf{a} \mid \mathbf{o}_t, \mathbf{q}_t, \ell, \mathbf{b}_t) \quad (7.2)$$

로 표현할 수 있다.

실제 로봇은 완전한 state  $\mathbf{s}_t$ 를 보지 못하므로 POMDP가 더 정확하다. 이때 policy는 observation history 또는 belief를 사용한다.

$$\mathbf{b}_t = \eta p(\mathbf{o}_t \mid \mathbf{s}_t) \int p(\mathbf{s}_t \mid \mathbf{s}_{t-1}, \mathbf{a}_{t-1}) \mathbf{b}_{t-1} d\mathbf{s}_{t-1}. \quad (7.3)$$

VLA에서 memory, recurrent state, video history, tactile history는 모두  $\mathbf{b}_t$ 를 근사하는 방법으로 볼 수 있다.

### 7.3 Return, value, and Q function

policy  $\pi_\theta$ 의 discounted return은 다음과 같다.

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[ \sum_{t=0}^T \gamma^t r_\ell(\mathbf{s}_t, \mathbf{a}_t) \right]. \quad (7.4)$$

value function은 state 또는 belief에서 기대되는 return이다.

$$V^\pi(\mathbf{s}) = \mathbb{E}_\pi \left[ \sum_{k=0}^{\infty} \gamma^k r(\mathbf{s}_{t+k}, \mathbf{a}_{t+k}) \mid \mathbf{s}_t = \mathbf{s} \right]. \quad (7.5)$$

Q function은 특정 action을 먼저 선택한 뒤의 기대 return이다.

$$Q^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim P} [V^\pi(\mathbf{s}')]. \quad (7.6)$$

Bellman optimality relation은

$$Q^*(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim P} \left[ \max_{\mathbf{a}'} Q^*(\mathbf{s}', \mathbf{a}') \right] \quad (7.7)$$

로 쓸 수 있다.

VLA 시스템에서  $Q$ 나 value model은 action을 직접 내는 policy와 다른 역할을 할 수 있다. 예를 들어 LLM/VLM planner가 여러 skill 후보를 만들고, value/affordance model이 “이 robot이 지금 할 수 있는가”를 평가할 수 있다. 이 구조는 VLA를 단독 policy가 아니라 proposal과 scoring의 결합으로 본다.

### 7.4 Reward is not just success

robot task reward는 단순 success indicator만으로 충분하지 않을 때가 많다.

$$r_t = r_t^{task} - \lambda_c c_t^{collision} - \lambda_f c_t^{force} - \lambda_u \|\mathbf{u}_t\|^2 - \lambda_j c_t^{jerk}. \quad (7.8)$$

여기서 task success는 positive term이고, collision, force, control effort, jerk는 penalty term이다. Language-conditioned reward model은 instruction과 observation을 보고 progress 또는 success를 추정할 수 있다.

$$\hat{r}_t = R_\psi(\mathbf{o}_t, \ell, \mathbf{a}_t, \mathbf{o}_{t+1}). \quad (7.9)$$

하지만 learned reward는 곧바로 안전한 objective가 아니다. reward model이 misspecified되면 policy는 실제 task success가 아니라 reward model의 빈틈을 최소화할 수 있다. Real VLA에서는 reward model, success detector, safety monitor를 분리해서 기록해야 한다.

## 7.5 Finite-horizon optimal control

Optimal control은 dynamics와 cost가 주어졌을 때 control sequence를 계산한다. finite horizon 문제는

$$\begin{aligned} \mathbf{u}_{0:T-1}^* &= \arg \min_{\mathbf{u}_{0:T-1}} \sum_{t=0}^{T-1} \ell(\mathbf{s}_t, \mathbf{u}_t; \ell) + \ell_T(\mathbf{s}_T; \ell) \\ \text{s.t. } \quad &\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{u}_t), \\ &\mathbf{s}_t \in \mathcal{X}, \quad \mathbf{u}_t \in \mathcal{U}. \end{aligned} \quad (7.10)$$

MPC는 이 문제를 매 시점 다시 푸는 receding-horizon control이다. VLA가 내는 target이나 subgoal은 cost의 parameter가 될 수 있다.

$$\ell(\mathbf{s}_t, \mathbf{u}_t; \mathbf{a}_t^{VLA}) = \|\mathbf{x}_{ee,t} - \mathbf{x}^{ref}(\mathbf{a}_t^{VLA})\|_Q^2 + \|\mathbf{u}_t\|_R^2. \quad (7.11)$$

이 관점에서 VLA는 low-level torque를 직접 예측하지 않아도 된다. VLA는 objective, reference, constraint, terminal condition을 제공하고, optimal control layer가 dynamics와 limit을 처리할 수 있다.

## 7.6 Constrained RL and safety

로봇에서는 reward maximization만으로 충분하지 않다. constraint violation은 높은 reward로 보상될 수 없는 경우가 많다. constrained MDP는 다음 문제로 쓸 수 있다.

$$\begin{aligned} \max_{\pi} \quad &\mathbb{E}_{\pi} \left[ \sum_t \gamma^t r_t \right] \\ \text{s.t. } \quad &\mathbb{E}_{\pi} \left[ \sum_t \gamma^t c_t^i \right] \leq d_i, \quad i = 1, \dots, m. \end{aligned} \quad (7.12)$$

여기서  $c_t^i$ 는 collision, force excess, workspace violation, human-proximity risk 같은 cost이다.

Real VLA에서는 이 constraint가 여러 layer에 걸쳐 구현된다. semantic safety filter는 instruction을 검사하고, geometric planner는 forbidden region을 피하며, controller는 torque/force limit을 지키고, runtime monitor는 stop/recover를 결정한다. RL objective는 이 stack을 대체하지 않는다.

## 7.7 Maximum entropy RL

Maximum entropy RL은 return뿐만 아니라 policy entropy를 함께 최적화한다.

$$J_{\text{MaxEnt}}(\pi) = \mathbb{E}_{\pi} \left[ \sum_t r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t)) \right]. \quad (7.13)$$

entropy term은 여러 가능한 action을 유지하게 하며, multimodal behavior를 다루는 데 유용하다.

VLA의 action generation에서도 이 관점은 중요하다. 조작 task에서 하나의 observation에 대해 하나의 정답 action만 있는 경우는 드물다. 여러 grasp, 여러 approach direction, 여러 recovery path가 가능하다. 따라서 action distribution을 유지하는 policy가 평균 action을 내는 regression policy보다 더 적합할 수 있다.

## 7.8 Control as inference

Control as inference는 optimal control을 probabilistic inference 문제로 본다. trajectory  $\tau$ 에 대해 optimality variable  $\mathcal{O}$ 를 도입하고,

$$p(\mathcal{O}_t = 1 | \mathbf{s}_t, \mathbf{a}_t) \propto \exp(r(\mathbf{s}_t, \mathbf{a}_t)) \quad (7.14)$$

라고 두면, 좋은 trajectory posterior는

$$p(\tau | \mathcal{O}_{0:T} = 1) \propto p(\tau) \exp \left( \sum_{t=0}^T r(\mathbf{s}_t, \mathbf{a}_t) \right). \quad (7.15)$$

가 된다.

이 관점은 diffusion policy 나 flow-based action model을 이해하는 데 도움이 된다. policy는 단일 action을 회귀하는 함수가 아니라, task와 observation 조건 아래에서 좋은 trajectory 또는 action chunk의 distribution을 샘플링하는 모델이 될 수 있다.

$$\mathbf{a}_{t:t+H} \sim p_{\theta}(\mathbf{a}_{t:t+H} \mid \mathbf{o}_{\leq t}, \ell). \quad (7.16)$$

## 7.9 Offline RL and robot data

Real robot online RL은 비용과 위험이 크다. 따라서 VLA 시대에는 offline robot dataset에서 policy를 개선하려는 압력이 크다. offline RL objective는 dataset distribution 안에서 policy improvement를 시도한다.

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \mathcal{D}}[Q_{\phi}(\mathbf{s}, \pi_{\theta}(\mathbf{s}))] - \beta D(\pi_{\theta}(\cdot \mid \mathbf{s}), \pi_{\mathcal{D}}(\cdot \mid \mathbf{s})). \quad (7.17)$$

두 번째 항은 policy가 dataset support 밖으로 나가는 것을 막는 regularization으로 볼 수 있다.

VLA fine-tuning에서도 같은 문제가 있다. 거대한 model이 dataset에 없는 action을 그럴듯하게 생성할 수 있지만, 그것이 robot에서 안전하다는 보장은 없다. 따라서 offline VLA improvement에는 conservative policy update, validation rollout, simulator/hardware-in-the-loop evaluation이 필요하다.

## 7.10 Evaluation metadata

RL 또는 optimal-control component가 들어간 VLA 실험은 다음 항목을 기록해야 한다.

- reward source: hand-coded, learned reward, success detector, human preference
- constraint set: collision, force, velocity, workspace, semantic safety
- online interaction budget and safety intervention count
- offline dataset coverage and out-of-distribution action filter
- policy update method: BC, offline RL, on-policy RL, MPC-guided update

- action distribution type: Gaussian, token, mixture, diffusion, flow, chunk
- evaluation split: offline validation, simulator rollout, real robot rollout

### Case study: reward model과 safety monitor를 분리해야 하는 이유

learned reward가 task progress를 잘 예측해도 collision, force, human proximity 같은 hard constraint를 항상 보장하지는 않는다. Real VLA에서는 reward maximization과 safety filtering을 같은 scalar objective로 합치기보다, reward/cost/constraint/monitor를 분리해 기록하는 편이 검증 가능하다.

## 7.11 Further reading and exercises

**Further reading** Sutton and Barto, SAC, CQL을 읽고 reward, entropy, conservative update가 VLA action distribution에 어떤 의미를 갖는지 연결하라 [16, 17, 18].

**Review question** offline VLA improvement에서 dataset support 밖의 action이 위험한 이유는 무엇인가?

**Design exercise** language-conditioned task 하나에 대해 reward, hard constraint, intervention rule, offline validation rule을 분리한 objective schema를 작성하라.

## 7.12 요약

RL과 optimal control은 VLA의 반대편에 있는 낯은 이론이 아니다. 이들은 VLA가 목표, 제약, 불확실성, action distribution을 어떻게 다루어야 하는지 설명하는 수학적 언어다. Behavior cloning은 expert action을 모방하지만, RL과 optimal control은 무엇이 좋은 행동인지, 어떤 constraint를 지켜야 하는지, 어떤 action distribution이 합리적인지 묻는다. Real VLA는 이 관점을 safety와 data engine 안에 넣어야 한다.

다음 장에서는 language가 붙기 전부터 존재했던 image-to-action 계보, 즉 visuomotor policy를 다룬다.

## 제8장

# Visuomotor Policies

## 학습 목표

이 장은 언어가 로봇 policy의 중심에 들어오기 전부터 존재했던 image-to-action 계보를 다룬다. VLA는 language-conditioned policy이지만, 실제 action을 생성하고 실행하는 하위 문제는 여전히 visuomotor control의 문제이다. 카메라가 본 장면, proprioception, gripper state, 과거 관측, task condition을 이용해 다음 action을 내는 구조가 오늘날 VLA의 직접적인 전신이다.

1. visuomotor policy를 observation-to-action mapping으로 정의한다.
2. visual observation이 partial, delayed, viewpoint-dependent state라는 점을 설명한다.
3. temporal memory, recurrent policy, attention, transformer policy가 왜 필요한지 정리한다.
4. real-robot grasping, pushing, reaching, insertion에서 필요한 evaluation meta-data를 정의한다.

## 8.1 From vision to action

Visuomotor policy는 시각 관측과 로봇 상태를 받아 action을 출력하는 policy이다 [19, 20, 21, 22].

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{I}_t, \mathbf{q}_t, \mathbf{g}_t), \quad (8.1)$$



여기서  $I_t$ 는 camera image,  $q_t$ 는 robot configuration 또는 proprioception,  $g_t$ 는 goal representation이다. goal은 목표 이미지, one-hot task label, language embedding, keypoint, affordance map 등 여러 형태가 될 수 있다.

VLA 관점에서 보면 language가 추가되기 전의 policy는 다음과 같은 구조였다.

$$a_t = \pi_\theta(I_t, q_t, z^{task}), \quad (8.2)$$

여기서  $z^{task}$ 는 task id, goal image, desired pose, 또는 learned task embedding이다. VLA는 이 자리에 language instruction  $\ell$  또는 VLM embedding을 넣는다.

### Visuomotor policy

Visuomotor policy는 visual observation과 robot state를 조건으로 하여 robot action을 생성하는 policy이다. VLA는 여기에 language-conditioned task semantics를 추가한 형태로 볼 수 있지만, visual feedback, action timing, controller interface, distribution shift 문제는 그대로 남는다.

## 8.2 Visual observation is not state

카메라 영상은 풍부하지만 완전한 state가 아니다. 물체가 보인다고 해서 정확한 6-DoF pose, mass distribution, friction, contact state, occluded obstacle을 아는 것은 아니다. Chapter 2에서 다룬 observation/state 구분은 visuomotor policy에서 가장 직접적으로 나타난다.

visual encoder를  $\phi_\theta$ 라 하면,

$$h_t = \phi_\theta(I_t), \quad a_t = \pi_\theta(h_t, q_t). \quad (8.3)$$

하지만  $h_t$ 가 충분한 state representation인지는 task와 sensor setup에 따라 다르다. camera calibration, viewpoint, lighting, occlusion, motion blur, rolling shutter, depth noise가 모두 action 품질에 영향을 준다.

### Image-to-action의 숨은 가정

Image-to-action policy는 image가 task-relevant state를 충분히 포함한다는 가정을 둔다. 이 가정이 깨지면 policy는 시각적으로 그럴듯한 action을 내더라도 contact, force, hidden object, timing 문제에서 실패한다.

## 8.3 Closed-loop visual feedback

open-loop trajectory를 한 번 생성해서 실행하는 방식은 작은 perturbation에 약하다. Visuomotor policy의 강점은 매 step 관측을 다시 보고 action을 수정할 수 있다는 점이다.

$$\mathbf{a}_t = \pi_\theta(\mathbf{o}_t, \mathbf{q}_t), \quad \mathbf{s}_{t+1} = f(\mathbf{s}_t, \kappa(\mathbf{a}_t, \hat{\mathbf{s}}_t), \mathbf{w}_t). \quad (8.4)$$

이 closed-loop 구조는 reaching, grasping, pushing 처럼 visual correction이 필요한 task에서 중요하다.

Visual servoing의 고전적 형태는 feature error를 줄이는 control이다. image feature를  $\mathbf{y}$ , target feature를  $\mathbf{y}^{ref}$  라 하면,

$$\dot{\mathbf{q}} = -K J_{img}(\mathbf{q})^\dagger (\mathbf{y} - \mathbf{y}^{ref}), \quad (8.5)$$

여기서  $J_{img}$ 는 image feature velocity와 joint velocity를 연결하는 interaction matrix이다. Learned visuomotor policy는 이 mapping을 명시적으로 쓰지 않을 수 있지만, 실제로는 visual error를 action correction으로 바꾸는 문제를 학습한다.

## 8.4 Temporal memory

단일 image는 속도, 접촉 전후 변화, occluded state를 충분히 알려주지 않는다. 따라서 policy는 history를 사용할 수 있다.

$$\mathbf{a}_t = \pi_\theta(\mathbf{o}_{t-H:t}, \mathbf{q}_{t-H:t}, \mathbf{g}_t). \quad (8.6)$$

recurrent policy는 hidden state를 갱신한다.

$$\mathbf{m}_t = F_\theta(\mathbf{m}_{t-1}, \mathbf{o}_t, \mathbf{q}_t), \quad \mathbf{a}_t = G_\theta(\mathbf{m}_t, \mathbf{g}_t). \quad (8.7)$$

VLA에서도 memory는 optional feature가 아니다. language instruction이 길거나 task가 multi-stage일 때, policy는 현재 frame뿐 아니라 이전 subgoal, grasp 시도, contact event, failure signal을 기억해야 한다.

## 8.5 Attention and transformer policies

CNN 기반 encoder는 image feature를 만들고, MLP 또는 recurrent head가 action을 출력하는 구조가 오래 쓰였다. Transformer policy는 image patch, proprioception token, action history, task token을 하나의 sequence로 처리할 수 있다.

$$\mathbf{Z}_t = [z_{1:P}^{img}, z_t^{prop}, z^{task}, z_{t-H:t}^{hist}], \quad \mathbf{H}_t = \text{Transformer}_\theta(\mathbf{Z}_t). \quad (8.8)$$

action head는 transformer output에서 action distribution을 만든다.

$$p_\theta(\mathbf{a}_t \mid \mathbf{o}_{\leq t}, \ell) = \text{ActionHead}_\theta(\mathbf{H}_t). \quad (8.9)$$

이 구조는 VLA와 자연스럽게 연결된다. language token과 image token을 같은 transformer에 넣을 수 있고, robot action을 token, continuous vector, chunk, diffusion target으로 출력할 수 있다. 그러나 transformer backbone이 들어왔다고 해서 controller rate, latency, contact feedback 문제가 사라지는 것은 아니다.

## 8.6 Real-robot grasping

Grasping은 visuomotor policy의 대표적 testbed이다. image에서 grasp point, approach direction, gripper width, lift action을 예측하는 구조는 다음처럼 쓸 수 있다.

$$\mathbf{a}_t = (\mathbf{p}_t^{grasp}, \mathbf{R}_t^{grasp}, w_t, \alpha_t), \quad (8.10)$$

여기서  $\mathbf{p}^{grasp}$ 는 grasp position,  $\mathbf{R}^{grasp}$ 는 grasp orientation,  $w$ 는 gripper width,  $\alpha$ 는 execute/open/close 같은 discrete mode이다.

grasp policy의 실패는 여러 계층에서 발생한다.

- perception failure: object segmentation 또는 pose 추정 실패
- action failure: grasp pose가 물체 geometry와 맞지 않음

- control failure: approach trajectory 나 gripper timing 문제
- contact failure: slip, insufficient normal force, collision
- recovery failure: 실패 후 재과하지 못함

따라서 grasp success rate 만으로는 policy를 충분히 평가할 수 없다. grasp attempt count, slip detection, lift success, placement success, recovery success를 분리해야 한다.

## 8.7 Pushing, reaching, insertion

Visuomotor policy는 grasping뿐 아니라 pushing, reaching, insertion에서도 사용된다. 각 task는 서로 다른 observation/action coupling을 가진다.

표 18: Visuomotor task 별 policy 요구사항

| Task      | 필요한 관측                                        | 대표 action/failure                         |
|-----------|-----------------------------------------------|-------------------------------------------|
| Reaching  | target pose, end-effector pose, obstacle      | overshoot, depth error, calibration error |
| Grasping  | object geometry, gripper state, contact cue   | bad grasp point, slip, collision          |
| Pushing   | object pose, support friction, contact point  | object rotation, loss of contact          |
| Insertion | hole pose, peg pose, contact force, alignment | jamming, rim contact, excessive force     |
| Wiping    | surface normal, coverage state, pressure      | missed area, too high/low pressure        |

이 표는 VLA에서도 그대로 중요하다. language instruction이 task를 지정할 수는 있지만, 각 task에서 필요한 visual feedback과 controller interface는 다르다.

## 8.8 Action chunking and bimanual visuomotor policy

고주파 control을 매 step 큰 모델로 생성하기 어렵기 때문에, policy가 짧은 horizon의 action chunk를 출력하는 구조가 많이 쓰인다.

$$\mathbf{A}_{t:t+H} = \pi_{\theta}(\mathbf{o}_{\leq t}, \mathbf{q}_t, \ell). \quad (8.11)$$

실행 중에는 chunk의 앞부분만 쓰고 다음 관측에서 다시 chunk를 생성할 수 있다.

$$\mathbf{u}_t = \kappa(\mathbf{A}_{t:t+H}[0], \hat{\mathbf{s}}_t). \quad (8.12)$$

Bimanual manipulation에서는 action dimension이 커진다.

$$\mathbf{a}_t = (\mathbf{a}_t^{left}, \mathbf{a}_t^{right}, \mathbf{a}_t^{sync}). \quad (8.13)$$

여기서  $\mathbf{a}^{sync}$ 는 두 arm 사이의 timing, relative pose, force balance를 나타낼 수 있다. cloth folding, cable manipulation, handover 같은 task에서는 이 synchronization이 핵심이다.

## 8.9 From visuomotor policy to VLA

VLA는 visuomotor policy에 language와 foundation-model representation을 추가한다.

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{I}_t, \mathbf{q}_t, \ell, \mathbf{b}_t). \quad (8.14)$$

하지만 이 변화가 image-to-action 문제를 제거하지는 않는다. Language는 무엇을 해야 하는지 알려주지만, 어떻게 grasp할지, 어떤 frame에서 delta pose를 낼지, contact 후 어떤 correction을 할지, controller가 어떤 rate로 추적할지는 여전히 system design 문제이다.

## 8.10 Evaluation metadata

Visuomotor policy 또는 VLA policy를 평가할 때 다음 metadata가 필요하다.

- camera setup: viewpoint, calibration, frame rate, latency

표 19: Visuomotor policy와 VLA의 차이

| 항목             | Visuomotor policy                                 | VLA policy                                              |
|----------------|---------------------------------------------------|---------------------------------------------------------|
| Task condition | task id, goal image, goal pose                    | language instruction, VLM embedding, subgoal            |
| Observation    | image, depth, proprioception                      | multimodal observation plus language/history            |
| Action         | delta pose, joint command, waypoint, chunk        | token, continuous action, chunk, trajectory, skill call |
| Generalization | visual/task distribution 안의 일 변화                  | open-vocabulary semantics와 embodiment transfer 기대       |
| Remaining risk | distribution shift, calibration, contact, latency | 동일한 physical risk plus semantic ambiguity               |

- proprioception and gripper state availability
- history length and memory architecture
- action type: delta pose, joint target, waypoint, chunk, token
- controller type and control frequency
- task-specific failure mode: grasp slip, insertion jamming, pushing loss of contact
- recovery policy and number of attempts
- train/test split: object, background, lighting, viewpoint, layout shift

#### Case study: ACT/Diffusion Policy에서 VLA action chunk로

ACT와 Diffusion Policy는 language 중심 VLA가 아니더라도 action chunk, multimodal action, temporal consistency가 robot policy에서 왜 중요한지 보여준다 [39, 40]. VLA가 chunk나 diffusion/flow action을 쓰는 이유는 단순히 최신 생성모델을 적용하기 위해서가 아니라, visuomotor policy가 오래 겪어 온 multimodality와 timing 문제를 다루기 위해서다.

## 8.11 Further reading and exercises

- Further reading** End-to-end visuomotor policy, visual foresight, Transporter, CLI-Port, ACT/Diffusion Policy를 VLA의 실행 계보로 읽어라 [19, 20, 21, 22, 39, 40].
- Review question** language가 추가되어도 image-to-action failure mode가 사라지지 않는 이유는 무엇인가?
- Design exercise** 하나의 pick-place task에 대해 camera setup, history length, action chunk horizon, controller rate, recovery label을 포함한 visuomotor evaluation sheet를 작성하라.

## 8.12 요약

Visuomotor policy는 VLA의 이전 단계가 아니라 VLA의 실행 중심부에 남아 있는 문제다. Language와 VLM backbone은 task semantics와 open-vocabulary grounding을 제공하지만, robot action은 여전히 image, proprioception, history, controller interface, contact feedback 위에서 생성된다. 따라서 VLA를 이해하려면 transformer와 language model만 볼 수 없고, image-to-action policy가 real robot에서 어떤 실패를 겪어 왔는지 함께 이해해야 한다.

다음 장에서는 language와 vision representation 자체로 넘어간다. Transformer, VLM, grounding은 VLA가 open-vocabulary task를 이해하게 만든 기반이지만, 그 grounding이 robot action grounding과 어떻게 다른지 구분해야 한다.

## 제9장

# Transformers, VLMs, and Grounding

## 학습 목표

이 장은 Transformer와 Vision-Language Model(VLM)이 VLA의 backbone으로 들어오는 경로를 설명한다. 목적은 Transformer 일반론을 길게 반복하는 것이 아니다. Real VLA 관점에서 필요한 질문은 더 구체적이다. Tokenization은 robot action representation에 어떤 제약을 주는가? VLM의 image-text grounding은 robot manipulation grounding과 어떻게 다른가? Web-scale semantic prior는 어떤 부분을 도와주고, 어떤 부분은 여전히 robot data와 controller가 맡아야 하는가?

1. tokenization과 self-attention을 VLA에 필요한 수준으로 정리한다.
2. ViT와 VLM의 multimodal representation을 설명한다.
3. contrastive image-text alignment와 open-vocabulary recognition의 의미를 정식화한다.
4. semantic grounding, visual grounding, affordance grounding, physical grounding을 구분한다.

## 9.1 Why VLM matters for VLA

VLA가 가능해진 배경은 로봇공학 내부에만 있지 않다. Transformer는 sequence modeling을 확장했고, ViT는 이미지를 patch token sequence로 바꾸었으며, VLM은 image와 text를 같은 representation space에서 정렬했다 [23, 24, 25, 26, 27]. 이



흐름은 로봇 policy가 open-vocabulary instruction과 visual scene을 조건으로 action을 생성할 수 있게 만든 핵심 배경이다.

하지만 VLM은 그 자체로 VLA가 아니다. VLM은 보통 다음 함수를 학습한다.

$$\mathbf{h}^{vlm} = F_{\theta}(\mathbf{I}, \mathbf{x}^{text}), \quad (9.1)$$

여기서  $\mathbf{I}$ 는 image,  $\mathbf{x}^{text}$ 는 text token sequence이다. VLA는 이 representation을 robot action으로 연결해야 한다.

$$\mathbf{a}_t = G_{\phi}(\mathbf{h}_t^{vlm}, \mathbf{q}_t, \mathbf{b}_t). \quad (9.2)$$

식 (9.2)의  $G_{\phi}$ 가 바로 robot-specific policy, action head, planner, controller interface이다. 이 부분이 없으면 VLM은 scene을 설명할 수 있어도 robot을 움직이는 model은 아니다.

## 9.2 Tokenization

Transformer는 입력을 token sequence로 본다. text는 subword token sequence로 표현된다.

$$\mathbf{x}^{text} = (x_1, x_2, \dots, x_N), \quad x_i \in \mathcal{V}. \quad (9.3)$$

image도 patch token으로 바꿀 수 있다. image  $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ 를  $P \times P$  patch로 나누면 patch 수는

$$N_p = \frac{HW}{P^2}. \quad (9.4)$$

각 patch를 embedding으로 projection하여

$$\mathbf{z}_i^{img} = E_{img} \text{vec}(\mathbf{I}_i) + \mathbf{p}_i \quad (9.5)$$

를 만든다. 여기서  $\mathbf{p}_i$ 는 positional embedding이다.

VLA에서 tokenization의 의미는 더 넓다. language token, image patch token, proprioception token, history token, action token이 하나의 sequence에 들어갈 수 있다.

$$\mathbf{Z} = [\mathbf{z}_{1:N}^{text}, \mathbf{z}_{1:P}^{img}, \mathbf{z}^{prop}, \mathbf{z}^{hist}, \mathbf{z}_{1:K}^{act}]. \quad (9.6)$$

이 구조는 flexible 하지만, robot action이 token으로 잘 표현되는지는 별도 문제이다. Discrete token은 VLM/LLM pipeline과 잘 맞지만, continuous contact-rich manipulation에서는 quantization, latency, smoothness 문제가 생길 수 있다.

### 9.3 Self-attention

Self-attention은 token 간 관계를 계산한다. token embedding matrix를  $X \in \mathbb{R}^{N \times d}$ 라 하면,

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \quad (9.7)$$

attention output은

$$\text{Attn}(X) = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} \right) V. \quad (9.8)$$

VLA에서 attention의 장점은 서로 다른 modality 사이의 관계를 모델링할 수 있다는 점이다. 예를 들어 “빨간 컵을 잡아라”라는 text token은 image patch의 red object region과 연결될 수 있고, “잡아라”라는 verb는 gripper action과 연결될 수 있다. 그러나 attention weight가 높다고 해서 물리적으로 grasp 가능한 것은 아니다. Attention은 representation relation이고, feasibility는 robot geometry와 contact constraint의 문제이다.

### 9.4 Contrastive image-text alignment

CLIP-style contrastive learning은 image encoder와 text encoder를 같은 embedding space에 정렬한다. image embedding을  $u_i$ , text embedding을  $v_j$ 라 하면 similarity는

$$s_{ij} = \frac{u_i^\top v_j}{\|u_i\|_2 \|v_j\|_2}. \quad (9.9)$$

batch 안의 paired image-text를 맞추는 contrastive loss는 다음처럼 쓸 수 있다.

$$\mathcal{L}_{i2t} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^B \exp(s_{ij}/\tau)}. \quad (9.10)$$

text-to-image 방향 loss도 대칭적으로 더할 수 있다.

이 objective는 image와 caption 사이의 semantic alignment를 강하게 만든다. 그래서 VLM은 zero-shot object recognition, semantic retrieval, open-vocabulary grounding에서 강하다. 그러나 이 loss는 grasp stability, friction, mass, force limit, controller compatibility를 직접 학습하지 않는다.

## 9.5 Multimodal fusion

VLM은 image token과 text token을 여러 방식으로 결합할 수 있다. 간단한 late fusion은

$$\mathbf{h} = \text{Fuse}(\phi_I(\mathbf{I}), \phi_T(\mathbf{x}^{text})) \quad (9.11)$$

로 쓸 수 있다. Interleaved multimodal model은 image/video/text token을 하나의 sequence로 처리한다.

$$\mathbf{Z} = [\mathbf{z}_{1:n_1}^{text}, \mathbf{z}_{1:p_1}^{img}, \mathbf{z}_{n_1+1:n_2}^{text}, \mathbf{z}_{p_1+1:p_2}^{img}]. \quad (9.12)$$

Robot setting에서는 여기에 proprioception과 action history가 추가되어야 한다.

$$\mathbf{h}_t^{robot} = F_\theta(\mathbf{I}_{t-H:t}, \mathbf{x}^{text}, \mathbf{q}_{t-H:t}, \mathbf{a}_{t-H:t}). \quad (9.13)$$

이 식은 VLA가 단순 VLM fine-tuning이 아니라는 점을 보여준다. Robot policy에는 body state와 action history가 필요하다.

## 9.6 Four kinds of grounding

Grounding이라는 단어는 자주 너무 넓게 쓰인다. Real VLA에서는 최소 네 가지 grounding을 구분해야 한다.

VLM이 주로 강한 영역은 semantic grounding과 visual grounding이다. Robot deployment에서 필요한 것은 affordance grounding과 physical/action grounding이다. 예를 들어 VLM이 “서랍 손잡이”를 찾을 수 있어도, 그 손잡이가 현재 robot reachability 안에 있는지, 어떤 방향으로 당겨야 하는지, 필요한 force가 얼마인지, gripper가 slip 없이 잡을 수 있는지는 별도 문제이다.

표 20: Grounding의 네 가지 의미

| 종류                     | 의미                            | VLA에서 필요한 추가 조건                                         |
|------------------------|-------------------------------|---------------------------------------------------------|
| Semantic grounding     | 단어와 개념을 연결한다                  | instruction이 task와 constraint로 변환되어야 한다                 |
| Visual grounding       | 단어와 image region/object를 연결한다 | object pose, affordance, occlusion을 다뤄야 한다              |
| Affordance grounding   | object가 어떤 action을 허용하는지 추정한다 | robot morphology와 current state에 의존한다                   |
| Physical/<br>grounding | action이 실제 물리 결과로 이어진다        | controller, contact, force, safety, feedback loop가 필요하다 |

## 9.7 From VLM prior to VLA policy

VLM prior는 VLA policy에 여러 방식으로 들어간다.

1. visual encoder로 사용되어 object와 relation feature를 제공한다.
2. language encoder로 사용되어 instruction embedding을 만든다.
3. embodied reasoning layer로 사용되어 subgoal, constraint, plan proposal을 만든다.
4. action policy backbone으로 fine-tuning되어 action token 또는 action chunk를 생성한다.

이 구조를 하나의 equation으로 쓰면 다음과 같다.

$$\mathbf{a}_t = \pi_{\phi}(F_{\theta}^{vlm}(\mathbf{I}_t, \ell), \mathbf{q}_t, \mathbf{b}_t). \quad (9.14)$$

여기서  $\theta$ 는 web-scale 또는 multimodal pretraining에서 온 parameter이고,  $\phi$ 는 robot action head 또는 adapter parameter일 수 있다.

VLA 설계의 핵심 질문은  $\theta$ 와  $\phi$ 의 경계를 어디에 둘 것인가이다. VLM backbone을 고정하고 action head만 학습할 수도 있고, multimodal backbone 전체를 robot data로 fine-tuning할 수도 있다. 전자는 안정적이지만 embodiment-specific

adaptation이 제한될 수 있고, 후자는 강력하지만 데이터와 safety 검증 비용이 커진다.

## 9.8 Grounding gap

VLM이 잘하는 것과 robot이 해야 하는 것 사이에는 gap이 있다.

$$\text{GroundingGap} = \text{Semantics}(\mathbf{I}, \ell) - \text{ExecutableAction}(\mathbf{s}, \mathbf{q}, \mathbf{C}). \quad (9.15)$$

이 식은 엄밀한 metric이 아니라 책임 경계를 표현하는 notation이다. 첫 항은 VLM이 강한 semantic interpretation이고, 두 번째 항은 robot state, constraint, controller를 만족하는 executable action이다.

이 gap은 다음 failure로 나타난다.

- object는 맞게 찾았지만 graspable pose를 잘못 추정한다.
- instruction은 이해했지만 forbidden region을 hard constraint로 처리하지 않는다.
- open-vocabulary object는 인식하지만 robot reachability를 모른다.
- action token은 생성하지만 controller가 고주파로 추적할 수 없다.
- caption-level relation은 맞지만 contact force나 hinge direction을 모른다.

## 9.9 Evaluation metadata

VLM-to-robot grounding을 평가하려면 일반 VLM benchmark와 다른 metadata가 필요하다.

- object grounding accuracy and pose error
- relation grounding: left/right/on/in/behind/inside 같은 spatial relation
- affordance prediction: graspable, pushable, openable, reachable
- executable action success: controller tracking, contact success, recovery

- semantic ambiguity handling: ask, abstain, clarify, safe stop
- distribution shift: object category, viewpoint, lighting, layout, distractor
- physical metadata: robot morphology, camera calibration, controller, force limit

이 metadata가 없으면 VLM의 높은 recognition score를 robot capability로 착각하기 쉽다.

#### Case study: semantic grounding과 action grounding의 차이

VLM이 “red mug”를 정확히 찾고 “put it in the rack”을 이해해도, mug pose covariance, rack slot reachability, gripper approach direction, forbidden glass region을 action constraint로 변환하지 못하면 robot task는 실패한다. 이 장의 핵심은 VLM grounding이 action grounding의 입력일 뿐이라는 점이다.

## 9.10 Further reading and exercises

**Further reading** Transformer, ViT, CLIP, Flamingo, BLIP-2를 VLA backbone의 representation source로 읽되, physical grounding과 구분하라 [23, 24, 25, 26, 27].

**Review question** attention map이나 image-text score가 높아도 robot affordance claim이 되지 않는 이유는 무엇인가?

**Design exercise** 하나의 instruction에 대해 semantic grounding, visual grounding, affordance grounding, physical/action grounding을 분리한 annotation sheet를 설계하라.

## 9.11 요약

Transformer와 VLM은 VLA의 핵심 표현학습 기반이다. Tokenization과 attention은 image, text, proprioception, history, action을 하나의 sequence로 결합할 수 있게 한다. Contrastive VLM은 open-vocabulary semantic grounding을 제공한다. 그러

나 semantic grounding은 physical/action grounding이 아니다. Real VLA는 VLM prior를 robot state, controller, contact, safety, evaluation loop와 연결해야 한다.

다음 장에서는 이 VLM/LLM을 robot planner로 사용하던 흐름을 다룬다. 그 시기는 완전한 VLA는 아니었지만, 오늘날 hierarchical VLA와 embodied reasoning architecture의 직접적인 조상이다.

## 제 III 편

# From Planner to VLA



## 제10장

# LLM/VLM as Robot Planner

### 학습 목표

이 장은 VLA 이전 단계에서 LLM과 VLM이 로봇 시스템 안에서 어떤 역할을 맡았는지 정리한다. 여기서 중요한 점은 LLM/VLM planner가 곧 VLA policy라는 뜻이 아니라는 것이다. 대부분의 시스템에서 LLM은 high-level plan, skill 선택, 코드 생성, scene query, success check, human feedback 처리 같은 역할을 맡고, 실제 운동은 기존 skill, controller, motion planner, perception module이 수행한다. 따라서 이 장의 목적은 language reasoning과 physical execution 사이의 interface를 명확히 정의하는 것이다.

1. plan, skill, controller의 계층 구조를 구분한다.
2. LLM/VLM planner가 왜 executable plan을 보장하지 못하는지 설명한다.
3. SayCan, Code as Policies, Inner Monologue, VoxPoser류 구조를 하나의 interface 관점으로 정식화한다.
4. 이 흐름이 hierarchical VLA와 embodied reasoning으로 이어지는 이유를 정리한다.

### 10.1 Planner로서의 LLM/VLM

로봇에게 자연어 명령  $\ell$ 이 주어졌다고 하자. 예를 들어 “테이블 위의 빨간 컵을 집어서 싱크대 옆에 놓아라”라는 명령은 하나의 motor command가 아니다. 이 명령은 object detection, pose estimation, reachability check, grasp selection, collision avoidance, force control, release condition까지 포함하는 복합 task이다. LLM

planner는 이런 task를 다음과 같은 plan으로 분해하려고 한다.

$$P_t = (g_1, g_2, \dots, g_K), \quad g_k \in \mathcal{G}. \quad (10.1)$$

여기서  $g_k$ 는 subgoal, symbolic action, skill call, 또는 executable code fragment가 될 수 있다. 가장 단순한 형태는 다음 mapping이다.

$$P_t = M_\theta^{plan}(\ell, \mathbf{o}_t, \mathcal{M}, \mathcal{S}), \quad (10.2)$$

여기서  $\mathcal{M}$ 은 scene memory 또는 world model,  $\mathcal{S}$ 는 사용 가능한 skill library이다. 중요한 것은  $P_t$ 가 아직 low-level robot action이 아니라는 점이다. 실제 robot command는 plan을 executor가 해석한 뒤 생성된다.

$$\mathbf{u}_t = E_\psi(P_t, \mathbf{b}_t, \mathbf{q}_t, \mathcal{C}), \quad (10.3)$$

여기서  $\mathcal{C}$ 는 controller, motion planner, safety checker의 집합이다. 이 구조는 Chapter 5의 planning과 Chapter 3의 control interface를 language model과 연결한 형태이다.

이때 LLM/VLM의 장점은 semantic prior이다. LLM은 물체의 일반적 용도, task 순서, language ambiguity 처리, tool selection에서 강하다. 그러나 robot execution은 state-dependent 문제이다. 현재 컵이 집을 수 있는 자세인지, gripper가 접근 가능한지, 중간 경로가 collision-free인지, 접촉 후 힘이 안정적인지는 language prior만으로 결정되지 않는다.

## 10.2 Skill library와 interface

LLM planner가 직접 joint torque를 출력하지 않는다면, 시스템은 skill library를 필요로 한다. skill을 다음 tuple로 정의하자.

$$s_i = (\text{name}_i, \text{pre}_i, \text{post}_i, \pi_i, \Sigma_i), \quad (10.4)$$

여기서  $\text{name}_i$ 는 language로 호출 가능한 이름,  $\text{pre}_i$ 는 precondition,  $\text{post}_i$ 는 기대되는 effect,  $\pi_i$ 는 실제 controller 또는 policy,  $\Sigma_i$ 는 실패 조건과 안전 제약이다. LLM은 주로  $\text{name}_i$ 와 textual description을 보고 skill을 선택하지만, 실행 가능성은  $\text{pre}_i$ 와  $\Sigma_i$ 가 현재 belief  $\mathbf{b}_t$ 에서 만족되는지로 판단해야 한다.

$$\text{Executable}(s_i, \mathbf{b}_t, \mathbf{q}_t) = \mathbb{I}[\text{pre}_i(\mathbf{b}_t) = 1 \wedge \Sigma_i(\mathbf{b}_t, \mathbf{q}_t) = 1]. \quad (10.5)$$

이 식은 단순하지만 핵심적인 분리를 보여준다. LLM이 어떤 skill을 말할 수 있다는 사실과 그 skill이 현재 로봇에게 실행 가능하다는 사실은 다르다.

표 21: LLM/VLM planner 구조에서 module 별 책임

| Module                   | Input                                   | Responsibility                                            |
|--------------------------|-----------------------------------------|-----------------------------------------------------------|
| LLM planner              | instruction, context, skill description | task decomposition, skill ordering, tool routing          |
| VLM/scene module         | image, text query, object list          | object grounding, relation extraction, state description  |
| Affordance/ value module | belief, candidate skill                 | executability, success probability, reachability estimate |
| Skill executor           | selected skill, parameters              | trajectory generation, controller call, termination check |
| Safety monitor           | robot state, constraints                | stop condition, collision/force/limit violation handling  |

이 책임 분리가 없으면 LLM planner는 시스템 설계 관점에서 위험해진다. plan text는 사람이 보기에는 그럴듯할 수 있지만, 로봇에 연결되는 순간 plan은 resource allocation, motion feasibility, contact stability, failure recovery의 문제로 바뀐다.

### 10.3 SayCan: language likelihood와 affordance

SayCan류 구조의 핵심은 “무엇을 해야 하는가”와 “무엇을 할 수 있는가”를 곱해서 skill을 고르는 것이다. LLM은 task context에서 적절한 skill의 language likelihood를 제공한다.

$$p_L(s_i | \ell, c_t) = P_{\theta}^{LLM}(s_i | \ell, c_t), \quad (10.6)$$

여기서  $c_t$  는 task history, scene description, 사용 가능한 skill 목록이다. 별도의 affordance 또는 value function 은 현재 상태에서 해당 skill 이 성공할 가능성을 추정한다.

$$p_A(s_i | \mathbf{b}_t) = P_\phi^{aff}(success | s_i, \mathbf{b}_t). \quad (10.7)$$

최종 선택은 보통 다음 형태로 표현할 수 있다.

$$s_t^* = \arg \max_{s_i \in S} p_L(s_i | \ell, c_t) p_A(s_i | \mathbf{b}_t). \quad (10.8)$$

이 정식화의 장점은 명확하다. LLM이 semantic coherence를 제공하고, robot-specific value가 physical feasibility를 보정한다. 예를 들어 “쓰레기를 버려라”라는 명령에서 LLM은 pick, move, open bin, place 같은 순서를 제안할 수 있다. 그러나 bin이 닫혀 있는지, 로봇 손이 쓰레기를 grasp할 수 있는지, 현재 위치에서 접근 가능한지는 affordance model이 판단해야 한다.

한계도 명확하다. 첫째,  $p_A$ 가 skill 수준의 성공확률이면 skill 내부 trajectory의 세부 실패를 충분히 설명하지 못한다. 둘째,  $S$ 에 없는 행동은 생성할 수 없다. 셋째,  $p_L$ 과  $p_A$ 의 곱은 calibration 문제가 있다. LLM score와 affordance score가 서로 다른 의미의 확률이면 단순 곱이 잘 calibrated decision rule이라고 보장할 수 없다.

## 10.4 Code as Policies: plan text에서 executable program으로

Code as Policies류 접근은 LLM의 출력을 자연어 plan이 아니라 program으로 만든다. 로봇 시스템은 미리 정의된 API를 제공하고, LLM은 그 API를 호출하는 code를 생성한다.

$$c_t = M_\theta^{code}(\ell, \mathbf{o}_t, \mathcal{A}_{api}, d), \quad (10.9)$$

여기서  $\mathcal{A}_{api}$ 는 사용 가능한 robot API 목록,  $d$ 는 example, documentation, prompt context이다. 생성된 code는 다음과 같이 실행된다.

$$\tau_{0:T}, r = \text{Exec}(c_t, \mathbf{b}_t, C), \quad (10.10)$$

여기서  $\tau_{0:T}$ 는 실행 trajectory,  $r$ 은 success/failure report이다.

이 구조의 장점은 compositionality이다. LLM은 loop, conditional, function call, geometric helper를 조합할 수 있다. 또한 code는 natural language plan보다 system boundary가 분명하다. 특정 함수가 어떤 argument를 받고 어떤 return을 내는지 정의할 수 있기 때문이다. 그러나 code generation은 별도의 safety problem을 만든다. 생성된 program이 API contract를 어길 수 있고, unchecked loop, invalid coordinate, unsafe motion request를 만들 수 있다.

따라서 code-based planner는 최소한 다음 검증 단계를 가져야 한다.

$$\text{Accept}(c_t) = \mathbb{I}[\text{TypeCheck}(c_t) \wedge \text{APICheck}(c_t) \wedge \text{SafetyCheck}(c_t, \mathbf{b}_t)]. \quad (10.11)$$

로봇에서는 code가 syntactically valid하다는 것만으로 충분하지 않다. code가 요청하는 motion이 robot limit, obstacle, force constraint를 만족해야 한다.

Engineering note: generated code is an interface, not authority

LLM이 생성한 code는 robot authority가 아니라 proposal이다. 실행 권한은 type checker, API checker, motion feasibility checker, safety monitor를 통과한 뒤에만 부여되어야 한다. 이 분리는 실험실 demo와 실제 로봇 시스템의 가장 큰 차이 중 하나다.

## 10.5 Inner Monologue: feedback을 포함한 closed-loop planning

초기 language planner의 약점은 open-loop plan이었다. 실제 로봇은 plan을 실행하는 동안 perception이 바뀌고, object pose가 틀어지고, grasp가 실패하고, 사람의 추가 지시가 들어온다. Inner Monologue류 구조는 planner가 observation과 feedback을 계속 읽으며 plan을 수정하는 형태를 취한다.

$$P_t = M_{\theta}^{plan}(\ell, h_t, \mathbf{o}_t), \quad (10.12)$$

$$r_t = \text{Execute}(P_t, \mathbf{b}_t), \quad (10.13)$$

$$h_{t+1} = \text{Update}(h_t, \mathbf{o}_{t+1}, r_t, f_t), \quad (10.14)$$

여기서  $h_t$ 는 language-level history,  $r_t$ 는 execution result,  $f_t$ 는 human feedback 또는 success detector output이다.

closed-loop 구조의 핵심은 planner가 실패를 text로 다시 받아들인다는 점이다. 예를 들어 “컵을 집으려 했지만 gripper가 미끄러졌다”는 feedback은 다음 plan에서 grasp pose, approach direction, force parameter를 바꾸는 근거가 된다. 그러나 feedback이 language로 들어온다고 해서 문제가 해결되는 것은 아니다. feedback에는 latency, perception error, ambiguous diagnosis가 포함된다. 실패 원인을 잘못 해석하면 LLM은 같은 실패를 다른 말로 반복할 수 있다.

따라서 closed-loop planner는 failure report를 구조화해야 한다.

$$e_t = (\text{stage}, \text{symptom}, \text{measured\_state}, \text{constraint}, \text{confidence}). \quad (10.15)$$

이 형식이 있으면 LLM은 단순히 “실패했다”가 아니라 “grasp approach stage에서 normal force가 threshold보다 낮았다” 같은 정보를 사용할 수 있다. 이것이 Chapter 19의 safety and assurance와 연결된다.

## 10.6 VoxPoser: value map과 model-based planning

VoxPoser류 구조는 LLM/VLM을 직접 action generator로 쓰지 않고, scene에 대한 value field 또는 constraint field를 만드는 데 사용한다. 예를 들어 language instruction과 visual scene이 주어지면, system은 3D workspace 위에 goal-relevant value map을 생성한다.

$$V_\theta(\mathbf{x}) = F_\theta^{vlm}(\mathbf{x}, \ell, \mathbf{o}_t), \quad \mathbf{x} \in \mathcal{X}. \quad (10.16)$$

그 다음 trajectory optimizer 또는 motion planner가 이 field를 사용해 trajectory를 찾는다.

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} \left[ \sum_{k=0}^T V_\theta(\mathbf{x}_k) - \lambda C(\tau, \mathbf{b}_t, \mathbf{q}_t) \right]. \quad (10.17)$$

여기서  $C$ 는 collision, smoothness, joint limit, contact constraint 등 robot-specific cost이다.

이 접근은 VLM의 semantic grounding과 classical planning의 feasibility checking을 분리한다. VLM은 “어디가 중요한가”를 알려주고, planner는 “어떻게 갈

수 있는가”를 묻는다. Real VLA 관점에서 이 구조는 매우 중요하다. 왜냐하면 VLA의 action head가 모든 것을 직접 학습하지 않아도, language-conditioned spatial field와 model-based optimizer의 결합으로 강한 system을 만들 수 있음을 보여주기 때문이다.

하지만 value map 방식도 한계가 있다. 첫째, value가 높다고 해서 contact-rich manipulation이 안정적이라는 뜻은 아니다. 둘째, 3D reconstruction이나 object segmentation error가 value field에 그대로 들어간다. 셋째, optimizer가 local optimum에 빠지면 language model은 그 실패를 직접 보정하지 못할 수 있다.

### Case study: SayCan과 VoxPoser를 같은 rubric으로 읽기

SayCan은 language likelihood와 affordance score를 결합해 skill을 선택한다 [28]. 따라서 핵심 interface는  $\ell, \mathbf{b}_t \rightarrow s_i$  이고, controller는 skill 내부에 숨는다. VoxPoser는 VLM이 3D value map 또는 constraint field를 만들고, optimizer가 trajectory를 찾는다 [31]. 따라서 핵심 interface는  $\ell, \mathbf{o}_t \rightarrow V(\mathbf{x}) \rightarrow \tau^*$  이다.

| 항목                | SayCan                                     | VoxPoser                                        |
|-------------------|--------------------------------------------|-------------------------------------------------|
| Model claim       | language prior와 affordance의 결합             | language-conditioned spatial field 생성           |
| Action boundary   | discrete skill selection                   | value field에서 trajectory optimization으로 연결      |
| Hidden assumption | skill library가 충분하고 calibrated되어야 함        | 3D scene reconstruction과 optimizer가 충분히 좋아야 함   |
| Real VLA verdict  | planner proposal과 feasibility score의 분리 사례 | semantic grounding과 model-based planning의 분리 사례 |

두 사례는 모두 LLM/VLM이 robot controller를 대체하지 않는다는 점을 보여준다. 차이는 proposal의 형식이다. 하나는 skill index를 제안하고, 다른 하나는 workspace field를 제안한다. 그래서 두 논문의 성능 비교보다 더 중요한 질문은 어떤 proposal이 어떤 verifier와 controller를 요구하는가

이다.

## 10.7 Executability gap

LLM/VLM planner의 중심 문제는 executability gap이다. semantic plan은 task intent를 잘 반영할 수 있지만, robot execution은 constraints를 만족해야 한다. 이를 다음과 같이 표현할 수 있다.

$$\Delta_{exec} = \text{Plausible}(P_t \mid \ell, o_t) - \text{Feasible}(P_t \mid \mathbf{b}_t, \mathbf{q}_t, \mathcal{C}). \quad (10.18)$$

이 식은 엄밀한 metric이 아니라 설계 원칙을 나타내는 notation이다. 첫 항은 language/vision model이 잘하는 영역이고, 두 번째 항은 robot system이 검증해야 하는 영역이다.

표 22: LLM/VLM planner의 failure mode

| Failure mode             | 원인                                     | 필요한 보완                                              |
|--------------------------|----------------------------------------|-----------------------------------------------------|
| Non-executable step      | skill library에 없는 action을 생성           | skill schema, tool constraint, fallback planner     |
| Wrong grounding          | object 또는 relation을 잘못 연결              | VLM verification, 3D perception, uncertainty report |
| Feasibility miss         | reachability/ collision/ contact 조건 무시 | motion planner, affordance model, safety checker    |
| Open-loop drift          | execution 중 scene이 바뀜                  | observation feedback, re-planning, success detector |
| Miscalibrated confidence | LLM이 plausible answer를 과신              | abstention, score calibration, human query          |

이 gap을 줄이는 방법은 LLM을 더 크게 만드는 것만이 아니다. interface를 구조화하고, robot state를 명시하고, executable skill을 제한하고, feedback을 닫고, 실패를 측정 가능한 report로 되돌려야 한다.



## 10.8 Planner+skills에서 VLA로

LLM/VLM planner 시대의 구조는 다음과 같이 요약할 수 있다.

$$\ell, \mathbf{o}_t \xrightarrow{\text{LLM/VLM}} P_t \xrightarrow{\text{skill/controller}} \mathbf{u}_t. \quad (10.19)$$

VLA는 이 중 일부 boundary를 end-to-end representation learning으로 흡수한다.

$$\mathbf{a}_t = \pi_{\theta}(\ell, \mathbf{o}_t, \mathbf{b}_t). \quad (10.20)$$

그러나 이 식이 모든 hierarchy를 없앤다는 뜻은 아니다. 오히려 현대 VLA 시스템은 direct action head, planner, skill library, safety checker, embodied reasoning module을 다시 조합하는 방향으로 발전한다. 따라서 LLM/VLM planner의 교혼은 폐기되지 않는다. 다음 네 가지가 그대로 남는다.

1. Robot action은 language plausibility가 아니라 physical feasibility로 검증되어야 한다.
2. Skill boundary는 data efficiency와 generalization을 동시에 결정한다.
3. Feedback loop 없이는 long-horizon task에서 error accumulation을 피하기 어렵다.
4. Safety와 assurance는 planner output 이후의 후처리가 아니라 planner interface의 일부여야 한다.

## 10.9 평가 관점

LLM/VLM planner를 평가할 때는 단순히 plan text가 그럴듯한지를 보면 안 된다. 최소한 다음 지표들을 분리해야 한다.

$$R_{semantic} = \frac{\#\{\text{task intent preserved}\}}{\#\{\text{episodes}\}}, \quad (10.21)$$

$$R_{exec} = \frac{\#\{\text{plan executed without interface violation}\}}{\#\{\text{episodes}\}}, \quad (10.22)$$

$$R_{success} = \frac{\#\{\text{task success}\}}{\#\{\text{episodes}\}}, \quad (10.23)$$

$$R_{recover} = \frac{\#\{\text{failure recovered}\}}{\#\{\text{recoverable failures}\}}. \quad (10.24)$$

이 네 지표는 서로 다르다.  $R_{semantic}$ 이 높아도  $R_{exec}$ 가 낮으면 plan은 말만 맞고 실행되지 않는다.  $R_{exec}$ 가 높아도  $R_{success}$ 가 낮으면 skill은 호출되지만 task objective를 달성하지 못한다.  $R_{recover}$ 는 closed-loop planner의 핵심 지표다.

## 10.10 Further reading and exercises

**Further reading** SayCan, Code as Policies, Inner Monologue, VoxPoser를 planner-output interface 기준으로 비교하라 [28, 29, 30, 31].

**Review question** LLM/VLM planner가 semantic plan을 잘 만들어도 executability gap이 남는 이유는 무엇인가?

**Design exercise** 하나의 drawer-opening task에 대해 planner proposal, feasibility checker, skill library, feedback channel을 나누어 system diagram을 작성하라.

## 10.11 요약

LLM/VLM as Robot Planner 흐름은 VLA 이전의 과도기적 기술이지만, Real VLA를 이해하는 데 필수적이다. 이 흐름은 language model이 robot system에 들어갈 때 어떤 interface가 필요한지 보여주었다. LLM은 task를 분해하고, VLM은 scene

을 해석하고, affordance model은 실행 가능성을 평가하고, controller는 실제 운동을 만든다. 이 역할 분리가 명확할수록 system은 설명 가능하고 검증 가능하다.

다음 장에서는 이 planner+skills 구조가 왜 충분하지 않았는지, 그리고 어떻게 action을 포함한 VLA policy로 넘어가게 되었는지 다룬다. 핵심 질문은 다음과 같다. skill library의 경계가 너무 강하면 generalization이 막히고, end-to-end policy로 너무 많이 넘기면 safety와 control interface가 약해진다. Real VLA는 이들 중 하나를 단순히 선택하는 문제가 아니라, 어느 boundary를 학습하고 어느 boundary를 시스템적으로 고정할지 결정하는 설계 문제다.

## 제11장

# From Planner + Skills to VLA

## 학습 목표

이 장은 planner와 skill library를 결합한 구조가 왜 VLA로 이어졌는지 설명한다. Chapter 10에서는 LLM/VLM이 task를 분해하고, 기존 skill과 controller를 호출하는 구조를 다루었다. 그 구조는 해석 가능하고 안전장치를 붙이기 쉽지만, skill boundary가 너무 단단하면 새로운 행동을 만들기 어렵다. 반대로 end-to-end VLA policy는 perception, language, action을 하나의 mapping으로 학습할 수 있지만, control boundary와 safety 검증을 약화시킬 수 있다.

1. planner+skills 구조의 engineering benefit과 bottleneck을 구분한다.
2. skill boundary를 learned policy가 흡수할 때 생기는 장점과 위험을 설명한다.
3. skill call, waypoint, action chunk, residual command를 VLA interface spectrum으로 정리한다.
4. hierarchical VLA가 pure planning과 pure end-to-end 사이의 실용적 middle ground임을 이해한다.

## 11.1 Planner + skills 구조의 기본형

Planner+skills 시스템은 다음 세 단계로 쓸 수 있다.

$$\ell, \mathbf{o}_t \xrightarrow{M_\theta^{plan}} P_t \xrightarrow{\text{Select}} s_t \xrightarrow{\pi_{s_t}} \mathbf{u}_t. \quad (11.1)$$

여기서  $\ell$ 은 language instruction,  $P_t$ 는 plan,  $s_t$ 는 선택된 skill,  $\pi_{s_t}$ 는 skill에 연결된 controller 또는 policy이다. 이 구조의 장점은 module boundary가 분명하다는 것이다. planner가 틀렸는지, skill precondition이 틀렸는지, controller가 실패했는지 분리해서 진단할 수 있다.

또한 이 구조는 data efficiency가 좋다. 로봇이 이미 grasp, place, navigate 같은 skill을 갖고 있으면 LLM/VLM planner는 새로운 task 조합을 비교적 적은 data로 만들 수 있다. 이때 학습 문제는 motor control 전체가 아니라 skill selection과 ordering으로 축소된다.

$$s_t^* = \arg \max_{s_i \in \mathcal{S}} Q_\theta(s_i \mid \ell, \mathbf{b}_t, h_t), \quad (11.2)$$

여기서  $h_t$ 는 history이다. 이 식에서 policy의 action space는  $\mathcal{S}$ 로 제한된다. 이는 장점이자 한계다.

## 11.2 Skill boundary의 병목

Skill library가 강하면 system은 안정적이다. 하지만 skill boundary가 task의 자연스러운 단위를 잘못 자르면 generalization이 막힌다. 예를 들어 “잡기” skill이 fixed top-down grasp만 지원한다면, 컵 손잡이, 얇은 천, 투명 용기, partially occluded object에 대해 planner가 아무리 좋은 sequence를 만들어도 실행이 어렵다.

Skill boundary의 병목은 세 가지로 나타난다.

$$\mathcal{B}_{coverage} = \mathbb{I}[\exists s_i \in \mathcal{S} : s_i \text{ can express the required behavior}], \quad (11.3)$$

$$\mathcal{B}_{parameter} = \mathbb{I}[\exists \alpha_i : \pi_{s_i}(\alpha_i, \mathbf{b}_t) \text{ succeeds}], \quad (11.4)$$

$$\mathcal{B}_{feedback} = \mathbb{I}[\text{skill exposes enough intermediate feedback}]. \quad (11.5)$$

첫 번째는 필요한 행동이 skill set에 있는가의 문제다. 두 번째는 skill은 있지만 parameterization이 충분한가의 문제다. 세 번째는 skill 내부 상태가 planner나 monitor에 충분히 노출되는가의 문제다.

Real VLA 관점에서 중요한 질문은 “skill을 쓸 것인가 말 것인가”가 아니다. 어떤 boundary를 고정하고, 어떤 boundary를 학습할 것인가이다.

표 23: Skill boundary 가 만드는 trade-off

| Boundary choice     | 장점                                        | 병목                                               |
|---------------------|-------------------------------------------|--------------------------------------------------|
| Coarse skill        | 안전 검증과 reuse가 쉽다                          | 새로운 contact mode와 세밀한 조작이 어렵다                    |
| Fine skill          | 조합성이 좋아진다                                 | long-horizon planning과 error accumulation이 커진다   |
| Parameterized skill | 기존 controller를 유지하면서 task variation을 처리한다 | parameter space가 커지면 planner가 사실상 control 문제를 푼다 |
| Learned skill       | data에서 motion pattern을 흡수한다               | 실패 원인과 safety certificate가 불명확해질 수 있다            |

### 11.3 VLA가 흡수하는 responsibility

VLA는 language와 observation에서 robot action을 바로 예측하는 방향으로 skill boundary의 일부를 흡수한다.

$$\mathbf{a}_t = \pi_{\theta}(\ell, \mathbf{o}_{\leq t}, \mathbf{q}_t). \quad (11.6)$$

이 식은 planner+skills 구조와 다르게 skill name을 명시적으로 거치지 않는다. 대신 policy 내부 representation이 object, relation, subgoal, motion primitive를 latent하게 표현할 수 있다.

하지만 모든 responsibility가 하나의 neural network 안으로 사라지는 것은 아니다. 실제 system에서는 VLA output을 다음 중 하나의 interface로 제한한다.

$$\mathbf{a}_t \in \{\text{skill call, waypoint, delta pose, action chunk, joint command, residual}\}. \quad (11.7)$$

각 interface는 control boundary를 다르게 만든다. skill call은 planner+skills에 가깝고, joint command는 end-to-end control에 가깝다. waypoint와 action chunk는 중간 지점이다.

## 11.4 Interface spectrum

Planner+skills에서 VLA로 넘어가는 과정을 하나의 spectrum으로 보자.

$$\mathcal{I} = \{\mathcal{I}_{symbolic}, \mathcal{I}_{skill}, \mathcal{I}_{waypoint}, \mathcal{I}_{chunk}, \mathcal{I}_{lowlevel}\}. \quad (11.8)$$

표 24: Planner+skills와 VLA 사이의 action interface

| Interface         | Output                                  | Engineering implication                                  |
|-------------------|-----------------------------------------|----------------------------------------------------------|
| Symbolic plan     | subgoal sequence, object relation       | interpretability는 높지만 executability gap이 크다              |
| Skill call        | skill id, arguments                     | 안전하고 재사용 가능하나 skill coverage에 묶인다                        |
| Waypoint          | end-effector pose, target pose          | motion planner와 결합하기 쉽고 controller boundary가 남는다         |
| Action chunk      | short horizon action sequence           | visuomotor smoothness를 학습하지만 latency와 stability를 관리해야 한다 |
| Low-level command | joint velocity, torque, gripper command | 표현력은 크지만 safety와 transfer가 어렵다                           |

이 표는 VLA 논문을 읽을 때 가장 먼저 확인해야 하는 항목이다. 같은 VLA라는 이름을 쓰더라도 action interface가 다르면 학습 난이도, 필요한 data, safety checker, controller integration이 모두 달라진다.

## 11.5 Latent skill과 hierarchical policy

Planner+skills 구조에서 skill은 사람이 정의한 discrete object였다. VLA로 넘어가면 skill이 latent variable로 바뀔 수 있다.

$$z_t \sim p_\theta(z_t \mid \ell, \mathbf{o}_t, h_t), \quad (11.9)$$

$$\mathbf{a}_{t:t+H} \sim p_\phi(\mathbf{a}_{t:t+H} \mid z_t, \mathbf{o}_t, \mathbf{q}_t). \quad (11.10)$$

여기서  $z_t$ 는 explicit skill id가 아니라 learned latent skill이다. 이 구조는 skill library의 coverage 범위를 줄일 수 있다. data가 충분하면 latent skill은 사람이 이름 붙이지 않은 motion pattern도 표현할 수 있다.

하지만 latent skill은 검증이 어렵다. explicit skill은 precondition과 termination condition을 붙일 수 있지만, latent variable은 의미가 불명확할 수 있다. 따라서 hierarchical VLA는 보통 두 가지 장치를 함께 둔다.

$$\begin{aligned} z_t &= H_\theta(\ell, \mathbf{o}_t, h_t), \\ \mathbf{a}_t &= L_\phi(z_t, \mathbf{o}_t, \mathbf{q}_t), \\ \text{allow}_t &= S_\eta(z_t, \mathbf{a}_t, \mathbf{b}_t, \mathbf{q}_t). \end{aligned} \quad (11.11)$$

$H_\theta$ 는 high-level reasoning,  $L_\phi$ 는 low-level action generation,  $S_\eta$ 는 safety gate이다. 이 구조는 Chapter 17의 hierarchical VLA와 직접 연결된다.

## 11.6 Residual VLA

Planner+skills와 end-to-end VLA 사이의 실용적인 중간 형태는 residual command이다. 기존 controller가 nominal command를 만들고, learned policy가 보정항을 더한다.

$$\mathbf{u}_t = \mathbf{u}_t^{\text{base}} + \Delta \mathbf{u}_t^{\text{vla}}, \quad \Delta \mathbf{u}_t^{\text{vla}} = f_\theta(\ell, \mathbf{o}_t, \mathbf{b}_t). \quad (11.12)$$

이 구조는 classical control의 안정성을 유지하면서 perception-language 조건을 반영할 수 있다. 예를 들어 nominal trajectory는 motion planner가 만들고, VLA는 grasp approach angle, contact timing, local correction을 보정한다.

Residual 구조의 위험은 learned residual이 base controller의 안정성 가정을 깨는 것이다. 따라서 residual은 norm bound, frequency bound, safety projection을 가져야 한다.

$$\Delta \mathbf{u}_t^{\text{saf}} = \Pi_{\mathcal{U}_{\text{saf}}}(\Delta \mathbf{u}_t^{\text{vla}}), \quad \|\Delta \mathbf{u}_t^{\text{saf}}\| \leq \epsilon. \quad (11.13)$$

이런 형태는 Real VLA에서 중요하다. learned model을 쓰되, controller와 safety boundary를 완전히 버리지 않는다.



## 11.7 Closed-loop feedback의 위치 변화

Planner+skills 구조에서는 feedback이 주로 planner 밖에서 처리된다. skill이 실패하면 monitor가 report를 만들고, planner가 다음 skill을 다시 고른다.

$$h_{t+1} = \text{Update}(h_t, r_t, e_t). \quad (11.14)$$

VLA에서는 feedback의 일부가 policy 내부 observation history로 들어간다.

$$\mathbf{a}_t = \pi_\theta(\ell, \mathbf{o}_{t-L:t}, \mathbf{a}_{t-L:t-1}). \quad (11.15)$$

이 변화는 중요하다. policy가 실패 징후를 직접 보고 local correction을 할 수 있기 때문이다. 그러나 history를 넣는 것만으로 recovery가 보장되지는 않는다. policy가 failure mode를 학습하려면 dataset에 실패, correction, recovery가 포함되어야 한다.

따라서 VLA data engine은 success trajectory만 모아서는 부족하다.

$$\mathcal{D} = \mathcal{D}_{\text{success}} \cup \mathcal{D}_{\text{failure}} \cup \mathcal{D}_{\text{correction}} \cup \mathcal{D}_{\text{intervention}}. \quad (11.16)$$

이 식은 Chapter 14의 data engine과 연결된다. VLA가 planner+skills의 robustness를 넘어서려면 실패와 개입을 학습해야 한다.

## 11.8 Middle ground로서의 hierarchical VLA

Pure planning과 pure end-to-end policy는 양극단이다. 실제 로봇 시스템에서는 다음 objective가 동시에 필요하다.

$$\max_{\theta, \phi} J_{\text{task}} - \lambda_1 J_{\text{unsafe}} - \lambda_2 J_{\text{latency}} - \lambda_3 J_{\text{uninterpretable}}. \quad (11.17)$$

이 objective에서 task success만 최적화하면 unsafe behavior가 생길 수 있고, interpretability만 중시하면 skill boundary가 너무 rigid해진다. hierarchical VLA는 이 trade-off를 조절한다.

**Design principle: choose the learning boundary**

Real VLA 설계에서 핵심 결정은 model size보다 learning boundary이다. 어떤 판단을 LLM/VLM planner에 맡길지, 어떤 motion을 learned action head에 맡길지, 어떤 constraint를 controller와 safety monitor에 고정할지 먼저 정해야 한다.

## 11.9 VLA로 넘어가는 조건

Planner+skills에서 VLA로 넘어갈 필요가 생기는 조건은 비교적 분명하다.

1. task variation이 skill library의 coverage를 넘어선다.
2. perception과 action 사이의 fine-grained coupling이 중요하다.
3. contact나 deformation처럼 symbolic subgoal로 표현하기 어려운 dynamics가 많다.
4. long-horizon task에서 skill 사이의 error accumulation이 커진다.
5. data가 충분해서 end-to-end 또는 hierarchical policy를 학습할 수 있다.

반대로 모든 문제를 VLA로 바꿀 필요는 없다. stable pick-place, fixture가 잘 정의된 industrial task, safety-critical constrained motion은 explicit controller와 verified planner가 여전히 유리할 수 있다. Real VLA는 기존 robotics를 대체하는 이름이 아니라, 기존 robotics boundary를 학습 가능한 representation과 결합하는 설계 방식이다 [28, 29, 34].

## 11.10 Further reading and exercises

**Further reading** Planner+skills 구조와 RT-1 이후의 action-producing transformer 구조를 비교해서 읽어라 [28, 34].

**Review question** skill library가 너무 강하면 generalization이 막히고, end-to-end policy가 너무 강하면 safety/control boundary가 약해지는 이유는 무엇인가?

**Design exercise** 기존 skill library가 있는 robot에 VLA policy를 추가한다고 가정하고, 어떤 boundary를 학습하고 어떤 boundary를 유지할지 표로 정리하라.

## 11.11 요약

Planner+skills 구조는 VLA의 전 단계이면서 동시에 현대 VLA 시스템의 일부로 남아 있다. 이 구조는 해석 가능성, safety gate, controller reuse라는 장점이 있지만, skill coverage, parameterization, feedback exposure에서 병목을 만든다. VLA는 이 병목의 일부를 learned policy로 흡수한다. 그러나 모든 것을 end-to-end로 밀어 넣으면 system diagnosis와 safety assurance가 어려워진다.

따라서 planner+skills에서 VLA로의 전환은 단순한 model replacement가 아니다. action interface와 learning boundary를 재설계하는 문제다. 다음 장에서는 이 전환이 실제로 Robotics Transformer 계열에서 어떻게 나타났는지, robot action이 sequence prediction과 token prediction 문제로 재정의된 과정을 다룬다.

## 제12장

# Robotics Transformer Era

## 학습 목표

이 장은 Robotics Transformer 시대를 VLA의 탄생기로 정리한다. 핵심은 특정 논문 이름을 외우는 것이 아니라, 로봇 action이 transformer의 sequence prediction 문제로 재정의되었다는 점이다. 언어 문장을 token sequence로 보고 다음 token을 예측하듯이, 로봇 trajectory도 observation, language instruction, history가 주어졌을 때 다음 action 또는 action chunk를 예측하는 문제로 볼 수 있다. RT-1, PaLM-E, RT-2, RoboCat은 이 전환을 서로 다른 각도에서 밀어붙인 사례다 [34, 35, 36, 37].

1. robot policy가 task-specific controller에서 generalist transformer policy로 이동한 배경을 설명한다.
2. action tokenization과 multimodal sequence modeling의 의미를 정리한다.
3. BC-Z, Gato, RT-1, PaLM-E, RT-2, RoboCat의 기술적 역할을 비교한다.
4. 왜 이 흐름이 곧바로 완전한 Real VLA가 아니었는지, 다음 장의 action representation 문제로 연결한다.

## 12.1 전환점: action as sequence prediction

고전적인 로봇 제어에서 action은 controller가 계산하는 입력이다. imitation learning에서는 expert trajectory에서 state-action pair를 모아 policy를 학습한다. Robotics Transformer 시대의 전환은 이 문제를 sequence modeling으로 재해석한

데 있다.

$$p_{\theta}(\mathbf{a}_{1:T} \mid \mathbf{o}_{1:T}, \ell) = \prod_{t=1}^T p_{\theta}(\mathbf{a}_t \mid \mathbf{o}_{\leq t}, \mathbf{a}_{< t}, \ell). \quad (12.1)$$

이 식은 behavior cloning의 일반화다. 차이는 입력과 출력이 더 이상 작은 vector pair가 아니라 multimodal token sequence가 된다는 점이다.

Transformer는 다음과 같은 token 집합을 attention으로 처리한다.

$$X_t = [x_{1:N}^{text}, x_{1:M}^{image}, x_{1:R}^{state}, x_{< t}^{action}]. \quad (12.2)$$

policy는 이 token sequence에서 action distribution을 예측한다.

$$p_{\theta}(\mathbf{a}_t \mid X_t) = \text{Head}_{act}(\text{Transformer}_{\theta}(X_t)). \quad (12.3)$$

이 구조는 language, vision, proprioception, action history를 하나의 sequence 처리 문제로 묶는다. VLA라는 이름은 여기서 자연스럽게 나온다. vision-language representation이 action head와 연결되기 때문이다.

## 12.2 Language-conditioned imitation learning

BC-Z류 흐름은 language-conditioned imitation learning이 generalization을 만들 수 있음을 보여주는 초기 단계로 볼 수 있다 [33]. task id 대신 language instruction이나 video context를 condition으로 넣고, policy가 여러 task를 하나의 parameter set으로 학습한다.

$$\mathcal{L}_{BC} = - \sum_{(\mathbf{o}_t, \ell, \mathbf{a}_t^E) \in \mathcal{D}} \log p_{\theta}(\mathbf{a}_t^E \mid \mathbf{o}_t, \ell). \quad (12.4)$$

이 식은 단순하지만 중요한 의미가 있다. task label이 language로 바뀌면, policy는 task 간 공유 구조를 language embedding을 통해 활용할 수 있다. “open drawer”, “close drawer”, “pick red block” 같은 명령이 서로 독립된 class가 아니라 text space 안에서 관련성을 갖는다.

그러나 이 단계의 한계도 분명하다. language-conditioned BC는 dataset coverage에 강하게 의존한다. unseen task generalization이 가능하더라도, 그 generalization은 observation distribution, object set, action interface, robot morphology가

크게 바뀌면 약해진다. 이 문제는 이후 RT-1과 RT-2에서 data scale과 model scale의 문제로 다시 등장한다.

### 12.3 Gato와 generalist agent 관점

Gato류 generalist agent 관점은 하나의 transformer가 서로 다른 modality와 task를 처리할 수 있다는 사고를 강화했다. 여기서 중요한 점은 robot-specific 성능 자체보다 interface 통일이다. text, image, discrete action, continuous value를 token처럼 정리하면 하나의 model family가 여러 domain을 다룰 수 있다.

$$y_k \sim p_\theta(y_k | y_{<k}), \quad y_k \in \mathcal{V}_{text} \cup \mathcal{V}_{image} \cup \mathcal{V}_{action}. \quad (12.5)$$

이 관점은 로봇에 직접 적용할 때 곧바로 어려움에 부딪힌다. 로봇 action은 token prediction처럼 보이더라도 physical system에 작용한다. action error는 다음 token의 문법 오류가 아니라 collision, slip, instability로 나타난다. 그럼에도 generalist agent 관점은 VLA 연구에 중요한 질문을 남겼다. 하나의 model이 language understanding, visual recognition, robot control을 같은 sequence modeling framework 안에서 다룰 수 있는가?

### 12.4 RT-1: real robot data와 action token

RT-1은 Robotics Transformer라는 이름이 보여주듯, real robot manipulation data와 transformer policy를 결합한 대표 사례다 [34]. 핵심은 task diversity, large-scale robot trajectory, language instruction, action tokenization의 결합이다. policy는 observation과 instruction을 입력으로 받아 discretized action을 예측한다.

$$\hat{a}_t^{disc} = \arg \max_{a \in \mathcal{A}_{disc}} p_\theta(a | \mathbf{o}_{\leq t}, \ell). \quad (12.6)$$

discrete action token은 transformer training pipeline과 잘 맞는다. classification head를 붙이고 cross-entropy loss로 학습할 수 있기 때문이다.

$$\mathcal{L}_{act} = - \sum_t \log p_\theta(a_t^{disc} | \mathbf{o}_{\leq t}, \ell). \quad (12.7)$$

이 구조는 VLA 관점에서 두 가지 의미가 있다. 첫째, language-conditioned robot policy가 large-scale data와 transformer architecture에서 실제 robot task를 수행할 수 있음을 보였다. 둘째, action representation이 model architecture만큼 중요하다는 사실을 드러냈다. action을 어떻게 discretize할지, time horizon을 어떻게 잡을지, control frequency를 어떻게 맞출지가 policy 성능과 안정성에 직접 영향을 준다.

## 12.5 PaLM-E: embodied multimodal language model

PaLM-E류 embodied multimodal language model은 language model 내부로 visual observation과 robot state를 넣는 방향을 보여준다 [35]. 이 접근은 robot policy를 단순한 action predictor가 아니라 embodied reasoning system으로 본다.

$$h = \text{LLM}_\theta(x^{text}, E_{vision}(\mathbf{o}_t), E_{state}(\mathbf{q}_t)). \quad (12.8)$$

여기서  $h$ 는 language response, plan, task-relevant representation, 또는 downstream action module의 입력이 될 수 있다.

이 흐름의 장점은 semantic reasoning이다. 대형 언어모델은 instruction decomposition, commonsense, long-horizon context 유지에 강하다. 하지만 action generation이 직접 연결되지 않으면 Chapter 10의 planner problem으로 돌아간다. 즉 embodied multimodal language model은 VLA의 reasoning backbone이 될 수 있지만, action interface와 control boundary를 별도로 설계해야 한다.

## 12.6 RT-2: action을 token처럼 다루기

RT-2류 접근은 VLM을 robot action과 결합한 전환점으로 볼 수 있다 [36]. 핵심 아이디어는 web-scale vision-language knowledge를 robot trajectory data와 함께 fine-tune하고, robot action을 text token과 유사한 방식으로 표현하는 것이다.

$$p_\theta(y_{1:K}, a_{1:T}^{tok} | I, \ell) = \prod_k p_\theta(y_k | y_{<k}, I, \ell) \prod_t p_\theta(a_t^{tok} | y_{1:K}, a_{<t}^{tok}, I, \ell). \quad (12.9)$$

여기서  $y$ 는 language token,  $a^{tok}$ 는 tokenized robot action이다. 이 구조의 의미는 크다. VLM이 갖고 있는 object, relation, semantic knowledge가 action prediction

에 직접 연결될 가능성을 열었기 때문이다.

그러나 action-as-token에는 engineering 한계가 있다. discrete token은 training을 단순하게 만들지만, continuous control의 정밀도와 smoothness를 제한할 수 있다. 또한 token sequence가 길어지면 latency가 증가하고, high-frequency control에는 별도 controller가 필요하다. 따라서 RT-2 이후 연구는 action chunking, diffusion, flow matching, continuous action expert, efficient fine-tuning으로 확장된다. 이 내용은 Chapter 13에서 본격적으로 다룬다.

## 12.7 RoboCat과 multi-embodiment adaptation

RoboCat 류 흐름은 multi-task, multi-embodiment, visual goal-conditioned policy, adaptation loop를 강조한다 [37]. VLA에서 중요한 질문은 하나의 robot에서 많은 task를 하는 것만이 아니다. 다른 robot morphology, 다른 camera, 다른 object distribution으로 policy를 옮길 수 있는가가 중요하다.

$$\pi_{\theta} : (\mathbf{o}_t, \ell, e) \mapsto \mathbf{a}_t, \quad e \in \mathcal{E}_{\text{embodiment}}. \quad (12.10)$$

여기서  $e$ 는 embodiment descriptor이다. robot이 바뀌면 action space, kinematics, camera geometry, controller dynamics가 바뀐다. 따라서 multi-embodiment policy는 단순한 multi-task policy보다 어렵다.

adaptation loop는 다음 형태로 볼 수 있다.

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \text{Collect}(\pi_{\theta_k}, \mathcal{T}_{\text{new}}, \mathcal{E}_{\text{new}}), \quad \theta_{k+1} = \text{Train}(\mathcal{D}_{k+1}). \quad (12.11)$$

이 식은 data engine의 시작점이다. VLA는 model architecture만으로 성장하지 않는다. deployment, failure collection, retraining, evaluation loop가 함께 있어야 한다.

## 12.8 Robotics Transformer 계열 비교

이 표에서 보듯 Robotics Transformer 시대의 성과는 “transformer를 로봇에 적용했다” 정도로 요약하면 부족하다. 더 정확한 요약은 다음과 같다. robot policy가 language-conditioned, multimodal, scalable sequence model로 재정의되었다.



표 25: Robotics Transformer 시대의 주요 흐름

| 흐름      | 핵심 기여                                                                     | Real VLA 관점의 한계                                                 |
|---------|---------------------------------------------------------------------------|-----------------------------------------------------------------|
| BC-Z    | language/video-conditioned imitation learning과 unseen task generalization | dataset coverage와 robot/domain shift에 민감하다                      |
| Gato    | 하나의 transformer로 여러 modality/task를 다루는 generalist agent 관점                | physical execution과 safety boundary가 약하다                        |
| RT-1    | real robot data, task diversity, transformer action policy                | action discretization과 data scale에 의존한다                         |
| PaLM-E  | embodied multimodal language model과 state/image/text 결합                   | action generation과 controller interface는 별도 설계가 필요하다            |
| RT-2    | VLM knowledge와 robot action token을 결합한 VLA 전환점                            | continuous/dexterous control, latency, action precision 문제가 남는다 |
| RoboCat | multi-task, multi-embodiment adaptation과 data loop                        | embodiment transfer와 evaluation protocol이 어렵다                   |

## 12.9 왜 이것이 Real VLA의 시작인가

VLA를 다음 mapping으로 쓰자.

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{o}_t, \ell, h_t). \quad (12.12)$$

Robotics Transformer 시대는 이 mapping을 large-scale transformer로 학습할 수 있다는 가능성을 열었다. 하지만 Real VLA는 이 식 하나로 끝나지 않는다. 실제 시스템은 action representation, controller interface, data engine, evaluation, safety monitor를 필요로 한다.

따라서 이 시대는 첫 번째 완성형이면서 동시에 두 번째 문제 제기의 출발점이다. RT-1/RT-2 계열은 VLA를 가능하게 했지만, 다음 질문들을 남겼다.

1. action을 discrete token으로 둘 것인가, continuous vector로 둘 것인가?

2. 한 시점 action을 예측할 것인가, action chunk를 예측할 것인가?
3. deterministic head를 쓸 것인가, distributional generator를 쓸 것인가?
4. controller가 추적할 reference를 낼 것인가, hardware에 가까운 command를 낼 것인가?
5. web-scale semantic prior가 physical skill로 전이되는 범위는 어디까지인가?

## 12.10 2024–2025 흐름으로 이어지는 질문

RT-2 이후의 중요한 변화는 “VLA를 만들 수 있다”에서 “어떤 action interface와 fine-tuning recipe가 실제 robot에서 빠르고 안전한가”로 질문이 이동했다는 점이다. OpenVLA는 open-source 7B VLA와 대규모 real-world robot demonstrations를 통해 VLA 연구의 재현 가능한 기준점을 제공했다 [42]. 그러나 open model 자체가 deployment-ready interface를 보장하지는 않는다.

OpenVLA-OFT는 같은 OpenVLA 계열에서도 action decoding, continuous action representation, action chunking, objective 선택이 fine-tuning 효율과 inference throughput에 큰 영향을 준다는 점을 보여준다 [45]. FAST는 action tokenization을 단순한 binning이 아니라 high-frequency robot action sequence를 압축하고 복원하는 문제로 재정의한다 [44].  $\pi_0$ 와  $\pi_{0.5}$ 는 pretrained VLM 위에 flow-based action generation과 heterogeneous co-training을 결합하여, action expert와 data mixture가 VLA generalization의 핵심 축임을 보여준다 [43, 48].

### 이 장을 읽을 때의 critical reading point

Robotics Transformer 계열을 읽을 때는 backbone 이름보다 action이 어떤 representation으로 기록되고, 어떤 controller interface로 실행되며, evaluation이 어떤 embodiment와 data split에서 이루어졌는지를 먼저 확인해야 한다. 최신 VLA 논문의 차이는 종종 architecture block 하나가 아니라 action tokenizer, chunk horizon, fine-tuning objective, data mixture, latency budget에 있다.

## 12.11 Further reading and exercises

**Further reading** BC-Z, Gato, RT-1, PaLM-E, RT-2, RoboCat을 history가 아니라 interface evolution으로 읽어라 [33, 32, 34, 35, 36, 37].

**Review question** Robotics Transformer 시대가 VLA를 가능하게 했지만 Real VLA를 완성하지는 못한 이유는 무엇인가?

**Design exercise** RT-1, RT-2, OpenVLA 중 하나를 골라 input, action, controller, data, evaluation evidence를 Chapter 0 기준으로 분해하라.

## 12.12 요약

Robotics Transformer 시대는 VLA의 역사에서 결정적이다. BC-Z는 language-conditioned imitation learning의 generalization 가능성을 보였고, Gato는 single transformer generalist agent 관점을 확산시켰다. RT-1은 real robot data와 transformer policy를 결합했고, PaLM-E는 embodied multimodal language model의 방향을 제시했다. RT-2는 VLM과 robot action token을 결합해 VLA라는 문제 설정을 선명하게 만들었고, RoboCat은 multi-task/multi-embodiment adaptation과 data loop를 강조했다.

하지만 이 장의 결론은 Robotics Transformer가 모든 문제를 해결했다는 것이 아니다. 오히려 이 시기 이후 가장 중요한 질문은 action representation으로 이동한다. action을 어떤 형식으로 표현하느냐가 control boundary, latency, dexterity, safety, data efficiency를 결정한다. 다음 장은 이 책의 기술적 중심부인 action representation으로 들어간다.

## 제13장

# Action Representation Is the Control Boundary

## 학습 목표

이 장은 책 전체의 action API를 고정한다. VLA에서 가장 중요한 질문은 어떤 backbone을 쓰는가만이 아니다. 더 근본적인 질문은 action을 무엇으로 표현하는가이다. 로봇 action은 token일 수도 있고, continuous vector일 수도 있고, waypoint, delta pose, action chunk, diffusion/flow distribution, whole-body command일 수도 있다. 이 선택은 출력 포맷의 문제가 아니라 VLA policy layer와 motion/control layer 사이의 책임 분할을 정하는 control boundary 문제다.

1. action representation을 control boundary로 해석한다.
2. token, vector, waypoint, chunk, distribution, action expert를 구분한다.
3. 각 표현이 latency, safety, recovery, controller compatibility에 미치는 영향을 설명한다.
4. 실제 robot task에 맞는 action interface를 고르는 checklist를 만든다.

## 13.1 Why action representation is the control boundary

VLA policy를 가장 추상적으로 쓰면 다음과 같다.

$$\mathbf{a}_t \sim \pi_{\theta}(\cdot \mid \mathbf{o}_{\leq t}, \ell, \mathbf{q}_t). \quad (13.1)$$

여기서  $\mathbf{a}_t$ 가 무엇인지가 이 장의 핵심이다.  $\mathbf{a}_t$ 가 discrete token이면 decoder와 tokenizer가 중요하다.  $\mathbf{a}_t$ 가 end-effector delta pose이면 frame convention, Cartesian servo, IK가 중요하다.  $\mathbf{a}_t$ 가 action chunk이면 horizon  $H$ , update frequency, interruption rule이 중요하다.  $\mathbf{a}_t$ 가 torque라면 stability와 safety assurance 부담이 급격히 커진다.

따라서 action representation은 다음 계약을 정의한다.

$$\text{Contract} = (\mathcal{A}, f_{VLA}, f_{ctrl}, \mathcal{C}_{safe}, \mathcal{E}_{exec}), \quad (13.2)$$

여기서  $\mathcal{A}$ 는 action space,  $f_{VLA}$ 는 VLA update rate,  $f_{ctrl}$ 은 controller loop rate,  $\mathcal{C}_{safe}$ 는 safety constraint,  $\mathcal{E}_{exec}$ 는 executor/controller이다. 같은 성능표를 가진 두 VLA라도 이 contract가 다르면 실제 시스템은 전혀 다르다.

#### Robotist's takeaway

Action representation은 VLA의 출력 포맷이 아니라, 의미 기반 policy와 물리 controller 사이의 계약이다.

## 13.2 Action representation spectrum

이 책에서는 다음 spectrum을 기본 taxonomy로 사용한다.

$$\mathcal{A}_{VLA} = \{\text{token, vector, waypoint, delta pose, chunk, distribution, expert, whole body}\}. \quad (13.3)$$

이 spectrum은 순위가 아니다. 왼쪽으로 갈수록 semantic abstraction이 크고, 오른쪽으로 갈수록 physical responsibility가 커진다. 좋은 interface는 task, robot, controller, latency, data, safety envelope에 따라 달라진다.

## 13.3 Canonical VLA case comparison

최근 VLA 흐름을 action representation 관점에서 다시 읽으면, 모델 family의 차이는 backbone 크기보다  $\mathbf{a}_t$ 가 어떤 형식으로 생성되고 어떤 controller boundary를 통과하는가에서 선명해진다. RT-1과 RT-2는 action token을 통해 language model

표 26: Action representation taxonomy

| Type              | Downstream interface                        | Main failure mode                              |
|-------------------|---------------------------------------------|------------------------------------------------|
| Action token      | token decoder, post-decoder, action wrapper | quantization, invalid token, decoding latency  |
| Continuous vector | servo, OSC, impedance, action wrapper       | scaling error, saturation, mode averaging      |
| Waypoint          | IK, motion planner, MPC                     | waypoint가 trajectory feasibility를 보장하지 않음      |
| Delta pose        | Cartesian servo, OSC, impedance             | frame convention error, drift, limit violation |
| Action chunk      | chunk executor, low-level tracker           | chunk inertia, interruption difficulty         |
| Distribution      | sampling, diffusion/flow action head        | sampling cost, safety projection difficulty    |
| Action expert     | VLM backbone plus motor head                | semantic-motor mismatch, fine-tuning burden    |
| Whole-body action | WBC, balance, locomotion controller         | balance, self-collision, contact instability   |

training recipe와 robot action을 연결했다 [34, 36]. OpenVLA는 open VLA baseline을 제공했고, OpenVLA-OFT는 continuous action, action chunking, parallel decoding, fine-tuning objective가 inference speed와 success를 바꾼다는 점을 보여준다 [42, 45].  $\pi_0$ 는 flow matching action expert를 사용하고, FAST는 action tokenizer 자체를 고주파 robot action sequence의 압축 문제로 다룬다 [43, 44]. RDT-1B 같은 diffusion transformer 계열은 bimanual manipulation에서 unified action space와 diffusion-based action generation을 결합하는 또 다른 action-interface 사례로 읽을 수 있다 [46].

표 27의 목적은 model ranking이 아니다. 같은 VLA라도 action이 token인지, continuous vector인지, chunk인지, flow-generated trajectory인지에 따라 downstream controller, safety shield, evaluation metric이 달라진다. 그러므로 VLA 논문

표 27: Action representation 관점에서 본 대표 VLA 계열

| Model family | Action representation        | Controller interface             | Latency / safety note                                               |
|--------------|------------------------------|----------------------------------|---------------------------------------------------------------------|
| RT-1         | discrete action token        | robot-specific wrapper           | token rate, decoding cost, quantization, invalid action mapping     |
| RT-2         | action as token              | action tokenizer/wrapper         | autoregressive delay, precision, smoothness, vocabulary restriction |
| OpenVLA      | token/action decoding        | fine-tuned robot policy          | large-model throughput, target-robot adaptation, calibration        |
| OpenVLA-OFT  | continuous action + chunking | faster decoder/wrapper           | fast chunk generation, interruption rule, stale action validation   |
| $\pi_0$      | flow matching action expert  | high-frequency action generation | denoising budget, control budget, action distribution validation    |
| FAST         | compressed action tokenizer  | token-to-action decoder          | efficient AR generation, reconstruction error, control fidelity     |

을 읽을 때 action representation table이 없으면 그 논문은 아직 robot system claim을 충분히 달지 않은 것이다.

#### Case study: OpenVLA-OFT, FAST, $\pi_0$ 를 action boundary로 읽기

OpenVLA-OFT, FAST,  $\pi_0$ 는 모두 “VLA 성능 개선”으로 묶일 수 있지만, Real VLA 관점에서는 서로 다른 병목을 겨냥한다 [45, 44, 43]. OpenVLA-OFT는 continuous action, chunking, faster decoding, fine-tuning objective를 통해 target robot adaptation과 speed/success trade-off를 다룬다. FAST는 high-frequency robot action sequence를 token compression 문제로 읽는다.  $\pi_0$ 는 flow matching action expert를 사용해 action distribution 자체를 생성한다.

| 사례          | 실제로 바뀐 것                                     | 반드시 물어야 할 질문                                                   |
|-------------|----------------------------------------------|----------------------------------------------------------------|
| OpenVLA-OFT | action decoding, continuous action, chunking | chunk interruption과 stale action rejection은 어떻게 되는가?           |
| FAST        | action tokenizer와 reconstruction fidelity    | token compression error가 controller tracking error로 어떻게 나타나는가? |
| $\pi_0$     | flow-based action generation                 | denoising budget과 robot control budget이 동시에 만족되는가?             |

이 세 사례의 공통점은 backbone 논쟁보다 action API 논쟁이 더 중요하다는 점이다. 논문을 읽을 때 “무슨 모델인가”보다 “어떤 action을 어느 rate로 내고, 어떤 controller와 safety layer가 그것을 받아들이는가”를 먼저 확인해야 한다.

## 13.4 Discrete action tokens

Discrete action token은 robot action을 vocabulary item처럼 다룬다. 연속 action  $\mathbf{a}_t \in \mathbb{R}^d$ 를 binning 또는 tokenizer로 변환하면

$$\mathbf{z}_t = \text{Tok}(\mathbf{a}_t), \quad \mathbf{z}_t \in \mathcal{V}_{act}. \quad (13.4)$$

VLA는 다음 token을 예측한다.

$$p_\theta(\mathbf{z}_t \mid \mathbf{o}_{\leq t}, \ell, \mathbf{z}_{< t}). \quad (13.5)$$

이 방식은 VLM/LLM training pipeline과 잘 맞는다. cross-entropy loss, autoregressive decoding, text token과 action token의 통합이 가능하기 때문이다.

하지만 token action에는 quantization error가 있다.

$$\epsilon_q = \|\mathbf{a}_t - \text{Dec}(\text{Tok}(\mathbf{a}_t))\|. \quad (13.6)$$



fine manipulation, dexterous hand, force-sensitive contact에서는 작은 quantization error가 task failure로 이어질 수 있다. 또한 action token sequence가 길어지면 decoding latency가 증가한다. 따라서 token action은 semantic transfer와 training convenience가 강한 대신, high-frequency continuous control에서는 controller나 post-decoder의 보완이 필요하다.

## 13.5 Continuous regression and normalization

Continuous action vector는 VLA가 실수 벡터를 직접 예측하는 방식이다.

$$\hat{\mathbf{a}}_t = f_\theta(\mathbf{o}_{\leq t}, \ell, \mathbf{q}_t), \quad \hat{\mathbf{a}}_t \in \mathbb{R}^d. \quad (13.7)$$

behavior cloning에서는 주로 다음 손실을 쓴다.

$$\mathcal{L}_{reg} = \sum_t \|\hat{\mathbf{a}}_t - \mathbf{a}_t^E\|_1 \quad \text{or} \quad \sum_t \|\hat{\mathbf{a}}_t - \mathbf{a}_t^E\|_2^2. \quad (13.8)$$

continuous action은 controller와 자연스럽게 연결된다. 예를 들어 normalized end-effector delta command를 Cartesian servo에 넣을 수 있다. 그러나 normalization이 잘못되면 같은 vector가 robot마다 전혀 다른 물리 의미를 갖는다.

$$\mathbf{a}_t^{phys} = \sigma_a \odot \mathbf{a}_t^{norm} + \mu_a. \quad (13.9)$$

$\mu_a, \sigma_a$ 는 dataset, robot, controller mode에 의존한다. multi-embodiment dataset에서는 이 값이 성능과 안전성에 직접 영향을 준다.

continuous regression의 또 다른 문제는 mode averaging이다. 가능한 action mode가 여러 개인데 평균을 내면 실제로는 어떤 mode도 아닌 action이 나온다.

$$\hat{\mathbf{a}} = \mathbb{E}[\mathbf{a} \mid \mathbf{o}, \ell] \notin \{\mathbf{a}^{(1)}, \mathbf{a}^{(2)}, \dots\}. \quad (13.10)$$

contact-rich manipulation에서는 mode averaging이 slip, collision, failed insertion으로 이어질 수 있다. 이것이 diffusion/flow action distribution이 중요해지는 이유다.

### 13.6 Waypoint, delta pose, and trajectory target

Waypoint와 delta pose는 continuous vector의 특수한 interface이다. end-effector pose를  $T_t \in SE(3)$ 라고 하자. delta pose command는 다음 target을 만든다.

$$T_{t+1}^{target} = T_t \exp(\Delta \xi_t), \quad (13.11)$$

여기서  $\Delta \xi_t$ 는 twist coordinate로 표현된 상대 motion이다. 이 command는 Cartesian servo, operational-space control, impedance controller와 연결되기 쉽다.

Waypoint는 더 큰 시간 단위의 reference이다.

$$r_t = (p_t^{target}, R_t^{target}, g_t^{gripper}), \quad (13.12)$$

하지만 waypoint는 trajectory가 아니다. waypoint가 collision-free path, joint-limit satisfaction, dynamic feasibility를 보장하지 않는다. 따라서 waypoint interface는 motion planner 또는 MPC와 함께 써야 한다.

Trajectory target은 reference sequence이다.

$$r_{t:t+H} = (r_t, r_{t+1}, \dots, r_{t+H}). \quad (13.13)$$

이때 VLA는 full low-level command를 내는 것이 아니라 controller가 추적할 목표를 낸다. 이 차이를 명확히 하지 않으면 논문에서 말하는 action과 실제 robot driver command가 혼동된다.

### 13.7 Action chunking as temporal interface

Action chunk는 한 시점 action이 아니라 여러 time step의 action sequence를 한번에 예측하는 출력 단위이다.

$$A_t^{(H)} = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}) \sim \pi_\theta(\cdot \mid \mathbf{o}_{\leq t}, \ell). \quad (13.14)$$

chunking은 단순히 inference를 덜 자주 하기 위한 trick이 아니다. temporal abstraction이다. policy가 짧은 motion segment를 한번에 만들면 single-step compounding error를 줄이고, bimanual manipulation처럼 coordination이 필요한 task에서 temporal consistency를 얻을 수 있다.

하지만 chunk horizon  $H$ 는 trade-off를 만든다.  $H$ 가 길수록 motion은 부드러워질 수 있지만, unexpected contact나 perception update가 들어왔을 때 빠르게 수정하기 어렵다. 이를 chunk inertia라고 부를 수 있다.

$$\text{RecoveryDelay} \approx \frac{H_{\text{remain}}}{f_{\text{ctrl}}} + \frac{1}{f_{\text{VLA}}}. \quad (13.15)$$

따라서 chunk executor는 interruption rule을 가져야 한다.

$$\text{Exec}(A_t^{(H)}) = \begin{cases} \text{continue,} & \text{safe}(x_\tau, A_t^{(H)}) = 1, \\ \text{interrupt,} & \text{safe}(x_\tau, A_t^{(H)}) = 0. \end{cases} \quad (13.16)$$

## 13.8 Diffusion policies and multimodal actions

Continuous regression은 여러 가능한 action mode를 평균낼 수 있다. Diffusion policy는 action chunk distribution을 denoising process로 표현한다. 단순화하면 noisy action  $A^K$ 에서 시작해 denoising을 반복한다.

$$A^{k-1} = D_\theta(A^k, k, \mathbf{o}, \ell), \quad k = K, K-1, \dots, 1. \quad (13.17)$$

최종 action chunk는  $A^0$ 이다.

$$A_t^{(H)} = A^0. \quad (13.18)$$

이 방식의 장점은 multimodal action distribution을 표현하기 쉽다는 점이다. 예를 들어 object를 왼쪽에서 잡을 수도 있고 오른쪽에서 잡을 수도 있다. 두 mode의 평균은 실패할 수 있지만, distributional generator는 하나의 mode를 sample할 수 있다.

그러나 diffusion은 sampling cost를 갖는다.

$$T_{\text{infer}} \approx K \cdot T_{\text{denoise}}. \quad (13.19)$$

real-time robot에서는  $T_{\text{infer}}$ 가 controller loop와 맞아야 한다. 또한 sampling diversity는 safety guarantee가 아니다. sample된 action은 여전히 collision, joint limit, force limit, workspace constraint를 통과해야 한다.

### 13.9 Flow matching and action expert

Flow matching은 action space에서 continuous transformation을 학습하는 방식으로 볼 수 있다. noise 또는 prior sample  $A_0$ 에서 data action  $A_1$ 으로 흐르는 vector field를 학습한다.

$$\frac{dA_\tau}{d\tau} = v_\theta(A_\tau, \tau, \mathbf{o}, \ell), \quad \tau \in [0, 1]. \quad (13.20)$$

이 구조는 VLM backbone과 continuous action expert를 분리하기 좋다.

$$h^{vlm} = F_\theta(\mathbf{o}, \ell), \quad A_t^{(H)} = G_\phi(h^{vlm}, \mathbf{o}, \mathbf{q}_t). \quad (13.21)$$

$F_\theta$ 는 semantic representation을 만들고,  $G_\phi$ 는 motor action distribution을 만든다. 이 분리는 Real VLA 관점에서 중요하다. semantic backbone과 motor head의 학습 속도, data requirement, deployment constraint가 다르기 때문이다.

하지만 action expert는 controller와 맞아야 한다. expert가 만든 chunk가 downstream controller의 rate, saturation, smoothness, frame convention과 맞지 않으면 좋은 semantic representation도 실제 robot에서는 실패한다.

### 13.10 Action tokenizers and compression

Action tokenization은 단순 binning으로 끝나지 않는다. robot action은 시간 구조와 주파수 구조를 갖는다. action trajectory  $A_t^{(H)}$ 를 compact token  $z_t$ 로 압축한다고 하자.

$$z_t = \text{Tok}(A_t^{(H)}), \quad \hat{A}_t^{(H)} = \text{Dec}(z_t). \quad (13.22)$$

좋은 tokenizer는 reconstruction error뿐 아니라 smoothness와 control relevance를 보존해야 한다.

$$\mathcal{L}_{tok} = \|A - \hat{A}\|^2 + \lambda_s \|\nabla A - \nabla \hat{A}\|^2 + \lambda_f \|\Phi(A) - \Phi(\hat{A})\|^2. \quad (13.23)$$

여기서  $\Phi$ 는 frequency-domain feature 또는 task-relevant feature이다. high-frequency dexterous behavior를 모두 제거한 token은 language model에는 편해도 robot control에는 부적합할 수 있다.

### 13.11 Choosing an interface

Action representation 선택은 다음 조건으로 결정해야 한다.

표 28: Task 별 action interface 선택

| Task / embodiment        | Preferred representation                                | Safety/evaluation focus                         |
|--------------------------|---------------------------------------------------------|-------------------------------------------------|
| Tabletop pick-and-place  | delta pose, waypoint, short chunk                       | collision, grasp failure, pose tracking         |
| Bimanual manipulation    | action chunk, continuous vector, distribution           | coordination, chunk interruption, contact force |
| Dexterous manipulation   | diffusion/ flow distribution, tactile-conditioned chunk | slip, multimodality, latency, force limit       |
| Mobile manipulation      | subgoal plus waypoint plus local chunk                  | base-arm coordination, obstacle proximity       |
| Humanoid whole-body task | whole-body reference, hierarchy, WBC target             | balance, self-collision, fall recovery          |
| Human-proximity task     | conservative waypoint or skill call with strong shield  | intervention, E-stop, audit log                 |

이 표의 핵심은 universal best action representation은 없다는 것이다. token이 항상 구식인 것도 아니고, diffusion이나 flow가 항상 정답인 것도 아니다. task와 controller가 요구하는 물리적 계약에 맞아야 한다.

### 13.12 Failure modes

### 13.13 Checklist

새 VLA system을 읽거나 설계할 때는 다음 질문에 답해야 한다.

1. action은 token, vector, waypoint, delta pose, chunk, distribution, torque 중 무엇인가?
2. action은 어느 frame에서 정의되는가?

표 29: Action representation failure modes

| Failure mode           | Mechanism                                   | Mitigation                                                       |
|------------------------|---------------------------------------------|------------------------------------------------------------------|
| Quantization loss      | discrete bin 이 fine action 을 파괴             | finer tokenizer, residual controller, continuous head            |
| Mode averaging         | regression 이 여러 mode 의 평균을 냄                | diffusion/ flow, mixture model, goal-conditioned disambiguation  |
| Chunk inertia          | 긴 chunk 가 빠른 recovery 를 막음                  | interruption rule, shorter horizon, monitor-triggered replanning |
| Controller mismatch    | action type 과 downstream controller 가 맞지 않음 | explicit action schema, controller wrapper, rate matching        |
| Latency mismatch       | $f_{VLA}$ 가 $f_{ctrl}$ 보다 너무 낮음             | asynchronous execution, chunking, fallback controller            |
| Embodiment mismatch    | 같은 vector 가 robot 마다 다른 물리 의미를 가짐           | embodiment adapter, normalization card, action calibration       |
| Safety incompatibility | shield 가 action 을 해석하지 못함                   | safety-readable action type, projection, runtime monitor         |

3. VLA update rate  $f_{VLA}$  와 controller rate  $f_{ctrl}$  은 얼마인가?
4. action normalization과 clipping rule은 무엇인가?
5. safety shield는 action 전, action 후, controller reference 후 중 어디에서 작동하는가?
6. action chunk가 있다면 horizon  $H$  와 interruption rule은 무엇인가?
7. diffusion/flow를 쓴다면 sampling time과 safety projection time은 얼마인가?
8. multi-embodiment라면 action adapter 또는 embodiment descriptor가 있는가?

9. evaluation은 success rate뿐 아니라 latency, smoothness, saturation, recovery를 측정하는가?

### 13.14 Running example: mug, rack, and glass

“머그컵을 drying rack에 올리고 유리컵은 건드리지 말라”는 같은 instruction이라도 action representation에 따라 전혀 다른 system claim이 된다. RT-style token action은 “move-left”, “close-gripper” 같은 discrete command로 표현될 수 있지만, rack slot과 유리컵 사이 간격이 작으면 quantization error가 collision margin을 먹을 수 있다. Continuous delta pose는 제어기와 잘 연결되지만, grasp approach가 여러 mode를 가질 때 평균 action이 rack edge를 향할 수 있다. Action chunk는 pick-lift-transport 구간을 부드럽게 만들 수 있지만, 유리컵이 흔들리거나 사람이 끼어들면 interruption rule이 없을 때 위험하다. Diffusion 또는 flow action expert는 여러 가능한 approach trajectory를 표현할 수 있지만, sampling된 chunk가 forbidden region을 침범하지 않는지 safety shield가 해석할 수 있어야 한다.

#### 이 예제에서 action representation이 남겨야 할 기록

좋은 VLA report는 mug pose uncertainty, glass forbidden region, rack placement target, chosen action type, controller reference, chunk horizon, interruption rule, latency, safety rejection 횟수를 함께 남겨야 한다. “성공했다”는 말만으로는 action representation의 공학적 품질을 평가할 수 없다.

### 13.15 Further reading and exercises

**Further reading** RT-1/RT-2, OpenVLA-OFT, FAST,  $\pi_0$ , RDT-1B를 action representation contract로 비교하라 [34, 36, 45, 44, 43, 46].

**Review question** action representation이 output format이 아니라 control boundary라는 말은 무엇을 의미하는가?

**Design exercise** 하나의 contact-rich task에 대해 token, continuous delta, way-point, chunk, diffusion/flow action의 장단점을 controller/safety/evaluation 관점에서 비교하라.

## 13.16 요약

Action representation은 Real VLA의 중심 문제다. token, continuous vector, way-point, delta pose, chunk, diffusion/flow action은 각각 controller interface, latency, recovery granularity, safety projection 조건을 바꾼다. 따라서 좋은 VLA는 좋은 backbone만으로 정의되지 않는다. 좋은 VLA는 자신이 어떤 action contract 위에서 동작하는지 명시한다. 다음 장에서는 이 action representation이 어떤 data engine에서 학습되고 유지되는지 다룬다.



## 제14장

# Data Engines

## 학습 목표

이 장은 VLA 성능을 model architecture 만으로 설명하지 않는다. Real VLA의 성능은 어떤 embodiment에서, 어떤 action representation으로, 어떤 language와 success/failure context를 붙여 데이터를 수집하고, 정제하고, 평가하고, 다시 배치하는지에 의해 결정된다. Open X-Embodiment, BridgeData, DROID, ALOHA, LIBERO, CALVIN, ManiSkill 같은 흐름은 모두 데이터가 단순한 파일 묶음이 아니라 data engine임을 보여준다.

1. robot data가 web data와 다른 이유를 설명한다.
2. demonstration, teleoperation, simulation, synthetic data, human video의 역할과 한계를 구분한다.
3. action representation과 dataset schema가 어떻게 연결되는지 정식화한다.
4. failure, correction, intervention data가 Real VLA에서 왜 필수인지 이해한다.

### 14.1 Robot data is not web data

웹 데이터는 주로 text, image, video의 통계적 co-occurrence를 제공한다. robot data는 여기에 physical execution이 붙는다. 같은 이미지와 같은 instruction이라도 robot embodiment, camera calibration, gripper type, controller mode, action representation이 다르면 데이터의 의미가 달라진다. Open X-Embodiment와 RT-X 계열이 보여준 핵심도 단순히 데이터를 많이 모으는 것이 아니라, 여러 embodiment의

robot trajectory를 공통 schema로 묶어 transfer question을 실험 가능하게 만드는 데 있다 [38].

robot trajectory dataset을 다음과 같이 쓰자.

$$\mathcal{D} = \{(\mathbf{o}_{1:T}^{(i)}, \mathbf{q}_{1:T}^{(i)}, \mathbf{a}_{1:T}^{(i)}, \ell^{(i)}, m^{(i)}, y^{(i)})\}_{i=1}^N. \quad (14.1)$$

여기서  $m^{(i)}$ 는 metadata이고,  $y^{(i)}$ 는 outcome label이다. VLA에서  $m^{(i)}$ 는 선택 사항이 아니다. robot model, camera pose, action normalization, controller frequency, teleoperation interface, success criterion이 들어가야 한다.

robot data가 어려운 이유는 action이 physical system에 묶여 있기 때문이다.

$$\mathbf{a}_t \sim \pi_\theta(\cdot \mid \mathbf{o}_{\leq t}, \ell, m), \quad (14.2)$$

metadata  $m$ 이 빠지면 policy는 서로 다른 robot의 action을 같은 의미로 오해할 수 있다.

## 14.2 Data engine loop

Data engine은 단순히 dataset을 모으는 과정이 아니다. 반복적인 system loop이다.

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \text{Collect}(\pi_{\theta_k}, \mathcal{T}_k, \mathcal{E}_k), \quad \theta_{k+1} = \text{Train}(\text{Curate}(\mathcal{D}_{k+1})). \quad (14.3)$$

여기서  $\mathcal{T}_k$ 는 task distribution,  $\mathcal{E}_k$ 는 environment/embodiment distribution이다. evaluation은 이 loop의 밖에 있지 않다. 평가 결과는 어떤 data를 더 모아야 하는지 결정한다.

$$\Delta \mathcal{D}_k = \text{Prioritize}(\text{Failures}(\pi_{\theta_k}), \text{CoverageGap}(\mathcal{D}_k), \text{SafetyEvents}_k). \quad (14.4)$$

Real VLA에서 좋은 data engine은 많이 모으는 engine이 아니라, 다음 실패를 줄이는 data를 우선순위화하는 engine이다.

### 14.3 Data record schema

VLA data record는 최소한 다음 fields를 가져야 한다.

$$r_i = (\mathbf{o}_{1:T}, \mathbf{q}_{1:T}, \mathbf{a}_{1:T}, \ell, g, e, c, y, q). \quad (14.5)$$

여기서  $g$ 는 goal 또는 subgoal,  $e$ 는 embodiment metadata,  $c$ 는 controller/action metadata,  $y$ 는 outcome,  $q$ 는 quality label이다.

표 30: VLA data record에 필요한 주요 field

| Field                | Example                               | 왜 필요한가                                |
|----------------------|---------------------------------------|---------------------------------------|
| Observation          | image, depth, tactile, proprioception | policy input과 sensor assumption 정의    |
| Action               | token, delta pose, chunk, command     | Chapter 13의 action contract 연결        |
| State/ configuration | joint, gripper, base, calibration     | feasibility와 embodiment grounding     |
| Language             | instruction, correction, annotation   | task semantics와 ambiguity 처리          |
| Metadata             | robot, camera, controller, rate       | multi-embodiment/action normalization |
| Outcome              | success, failure, partial success     | evaluation과 failure mining            |
| Quality              | human rating, anomaly, uncertainty    | curation과 weighting                   |

이 schema가 없으면 dataset은 커져도 해석 가능성이 낮다. 특히 action field가 모호하면 training loss와 deployment command 사이의 의미가 달라진다.

### 14.4 Robot dataset card

출판 가능한 VLA paper 또는 재사용 가능한 robot dataset은 dataset card를 가져야 한다. dataset card는 marketing summary가 아니라 action, controller, embodiment,

failure evidence를 복원하기 위한 최소 기록이다.

표 31: Robot Dataset Card 최소 양식

| Field                   | 반드시 기록할 내용                                                                                |
|-------------------------|-------------------------------------------------------------------------------------------|
| Robot embodiment        | robot model, arm/hand/gripper, mobile base, joint limits, payload limit                   |
| Camera setup            | camera pose, calibration, frame convention, synchronization, occlusion pattern            |
| Action representation   | token, delta pose, joint target, chunk, trajectory, torque, action rate                   |
| Controller              | servo, impedance, OSC, WBC, gripper controller, low-level safety limit                    |
| Teleoperation interface | VR, leader-follower, keyboard, scripted policy, autonomy-assisted collection              |
| Reset policy            | manual reset, scripted reset, object randomization, failure reset rule                    |
| Failure labels          | slip, collision, no grasp, wrong object, timeout, unsafe proximity, operator intervention |
| Human intervention      | intervention trigger, correction action, recovery trajectory, abort reason                |
| Safety events           | rejected action, force limit, emergency stop, watchdog fallback, human near miss          |
| Train/eval split        | task split, object split, scene split, embodiment split, temporal split                   |

BridgeData, DROID, ALOHA/ACT, LIBERO, CALVIN, HuLC/What Matters, ManiSkill 같은 이름을 본문에 나열하는 것만으로는 data claim이 강해지지 않는다 [49, 50, 39, 51, 52, 53, 54]. 중요한 것은 각 dataset이 어떤 embodiment, action interface, controller, reset policy, failure label을 제공하는지이다. 이 표를 채울 수 없는 dataset은 VLA training에는 쓸 수 있어도 Real VLA system claim을 검토하기에는 약하다.

### Case study: dataset 이름보다 dataset card

Open X-Embodiment, DROID, ALOHA는 모두 robot learning data를 제공하지만, Real VLA 관점에서 같은 종류의 evidence가 아니다 [38, 50, 39]. Open X-Embodiment는 multi-embodiment transfer question을 만들고, DROID는 in-the-wild teleoperation scale과 diversity를 제공하며, ALOHA 계열은 bi-manual manipulation과 low-cost teleoperation interface를 강하게 보여준다.

| Dataset family    | 강한 evidence                                               | 약해질 수 있는 지점                                    |
|-------------------|-----------------------------------------------------------|------------------------------------------------|
| Open X-Embodiment | embodiment mixture와 shared training protocol              | robot별 controller/action meta-data의 해석 난도      |
| DROID             | diverse real-world scenes와 teleoperation scale            | task/failure label consistency와 reset protocol |
| ALOHA             | bimanual action, teleoperation, contact-rich manipulation | domain breadth와 long-tail failure coverage     |

따라서 dataset 비교는 sample count로 끝나면 안 된다. action representation, collection interface, failure labels, split rule, intervention policy를 같이 비교해야 VLA training evidence가 deployment evidence로 얼마나 이어지는지 판단할 수 있다.

## 14.5 Demonstration collection

robot demonstration은 여러 방식으로 수집된다.

중요한 것은 source를 섞는 방식이다. data mixture를 다음과 같이 쓸 수 있다.

$$\mathcal{D} = \bigcup_{j=1}^J \mathcal{D}_j, \quad \mathcal{L} = \sum_{j=1}^J w_j \mathbb{E}_{r \sim \mathcal{D}_j} [\ell_{\theta}(r)]. \quad (14.6)$$

weight  $w_j$ 는 단순히 dataset size에 비례하면 안 된다. 작은 failure dataset이나 high-quality correction dataset이 큰 success-only dataset보다 특정 failure mode에는 더 중요할 수 있다.

표 32: Data source comparison

| Source                | 장점                                                 | 한계                                           |
|-----------------------|----------------------------------------------------|----------------------------------------------|
| Teleoperation         | real robot action 과 human strategy를 직접 수집          | operator bias, device mapping, fatigue       |
| Kinesthetic teaching  | contact-rich motion 과 force cue를 직관적으로 전달          | robot type 과 task setup에 제한                  |
| Scripted policy       | 대량 반복과 label consistency가 좋음                       | diversity와 recovery behavior가 부족             |
| Autonomous collection | policy distribution의 실제 failure를 수집 가능             | safety monitor와 reset cost가 필요               |
| Simulation            | scale, controlled variation, rare event generation | sim-to-real gap, contact model mismatch      |
| Human video           | semantic prior와 task diversity가 큼                  | robot action label과 embodiment grounding이 없음 |

## 14.6 Action normalization and embodiment metadata

multi-embodiment VLA에서 가장 흔한 함정은 action normalization이다. robot  $e$ 마다 action scale과 controller mode가 다르면 같은 normalized vector가 다른 physical command가 된다.

$$\mathbf{a}_{t,e}^{phys} = \sigma_e \odot \mathbf{a}_t^{norm} + \mu_e. \quad (14.7)$$

따라서 dataset card에는 최소한 다음 정보를 포함해야 한다.

$$m_e = (\text{robot, kinematics, controller, } f_{ctrl}, \text{action type, } \mu_e, \sigma_e). \quad (14.8)$$

metadata가 빠진 multi-robot dataset은 scale만 커질 수 있다. 반대로 잘 정의된 embodiment metadata는 transfer learning과 adapter 설계의 기반이 된다.

## 14.7 Failure, correction, and recovery data

성공 trajectory만 모으면 policy는 성공 직전의 분포를 많이 보지만, 실패에서 빠져나오는 법을 배우지 못한다. Real VLA data engine은 다음 네 종류를 분리해야 한다.

$$\mathcal{D} = \mathcal{D}_{success} \cup \mathcal{D}_{failure} \cup \mathcal{D}_{correction} \cup \mathcal{D}_{intervention}. \quad (14.9)$$

failure record는 단순히 실패했다는 label이 아니라 원인을 포함해야 한다.

$$f_i = (\text{stage}, \text{symptom}, \text{sensor evidence}, \text{constraint}, \text{recovery action}). \quad (14.10)$$

예를 들어 grasp failure라도 perception error, slip, collision, insufficient force, wrong frame convention은 서로 다른 failure이다. 이 차이를 기록하지 않으면 data engine은 같은 실패를 계속 반복한다.

## 14.8 Simulation, synthetic data, and human video

simulation은 coverage를 준다. rare event, domain randomization, controlled ablation, benchmark repeatability에 강하다. 그러나 contact, sensor noise, deformable object, human proximity에서는 sim-to-real gap이 생긴다.

$$\Delta_{sim2real} = d(p_{sim}(\mathbf{o}, \mathbf{a}, y), p_{real}(\mathbf{o}, \mathbf{a}, y)). \quad (14.11)$$

여기서  $d$ 는 distribution distance를 나타내는 추상 notation이다. 중요한 것은 simulation score가 real deployment evidence로 자동 변환되지 않는다는 점이다.

human video는 task semantics와 object interaction prior를 제공할 수 있다. 그러나 robot action label이 없다.

$$\text{HumanVideo} = (\text{visual task prior}) \neq (\text{robot action supervision}). \quad (14.12)$$

따라서 human video는 pretraining, affordance prior, goal recognition에는 유용하지만, controller-compatible action label을 대신하지 않는다.

## 14.9 Curation and relabeling

data engine의 품질은 수집보다 curation에서 갈리는 경우가 많다. curation은 bad sample을 버리는 과정만이 아니다. annotation schema를 통일하고, action type을 명시하고, failure를 taxonomy로 분류하고, train/eval leakage를 막는 과정이다.

$$\mathcal{D}^{clean} = \text{Filter}(\text{Relabel}(\text{Normalize}(\mathcal{D}^{raw}))). \quad (14.13)$$

language relabeling도 중요하다. 같은 trajectory는 여러 instruction으로 표현될 수 있다. 하지만 relabeling이 실제 goal과 맞지 않으면 policy는 language grounding을 잘못 배운다.

$$\ell' = R(\tau), \quad \text{Accept}(\ell') = \mathbb{I}[\text{GoalConsistent}(\ell', \tau) = 1]. \quad (14.14)$$

## 14.10 Data quality metrics

data engine은 dataset size만 보고 평가하면 안 된다. 최소한 다음 지표가 필요하다.

$$C_{task} = \text{Coverage}(\mathcal{T}_{train}, \mathcal{T}_{target}), \quad (14.15)$$

$$C_{emb} = \text{Coverage}(\mathcal{E}_{train}, \mathcal{E}_{target}), \quad (14.16)$$

$$Q_{action} = \text{Consistency}(\text{action schema}), \quad (14.17)$$

$$Q_{fail} = \frac{|\mathcal{D}_{failure} \cup \mathcal{D}_{recovery}|}{|\mathcal{D}|}, \quad (14.18)$$

$$Q_{leak} = 1 - \text{LeakageRate}(\mathcal{D}_{train}, \mathcal{D}_{eval}). \quad (14.19)$$

이 지표는 완전한 표준이 아니라 data engine을 읽는 최소 lens이다.

## 14.11 Data engine and safety

unsafe data는 버리기만 할 대상이 아니다. 어떤 unsafe event는 policy가 피해야 할 negative evidence이고, 어떤 event는 safety monitor를 학습하거나 test하는 데 필요하다. 하지만 unsafe demonstration을 그대로 imitation target으로 쓰면 위험



하다.

$$\mathcal{D}_{unsafe} \xrightarrow{\text{label}} \{\text{negative, intervention, shield test, discard}\}. \quad (14.20)$$

이 routing은 Chapter 19의 safety and assurance로 이어진다. data engine은 safety의 입력이다.

## 14.12 Further reading and exercises

**Further reading** Open X-Embodiment, BridgeData, DROID, ALOHA/ACT, LIBERO, CALVIN, HuLC, ManiSkill을 dataset name이 아니라 evidence type으로 읽어라 [38, 49, 50, 39, 51, 52, 53, 54].

**Review question** robot data가 web data와 다른 가장 중요한 이유는 embodiment, controller, reset, failure label 중 어디에 있는가?

**Design exercise** 새 robot dataset을 만든다고 가정하고 Robot Dataset Card의 모든 항목을 실제 수집 protocol로 변환하라.

## 14.13 요약

VLA data는 많으면 좋은 파일 묶음이 아니다. Real VLA의 data engine은 수집, 정제, 라벨링, normalization, failure mining, evaluation, deployment log를 반복하는 system이다. robot data는 embodiment와 action representation에 묶여 있으므로 web data처럼 단순히 scale만 키우기 어렵다. teleoperation은 real behavior를 주지만 operator bias가 있고, simulation은 scale을 주지만 sim-to-real gap이 있으며, human video는 semantic prior를 주지만 robot action supervision은 아니다.

다음 장에서는 이 data engine 위에서 모델을 실제 task와 robot에 맞게 적응시키는 방법을 다룬다. action head, adapter, fine-tuning, normalization, inference speed는 모두 data engine과 분리할 수 없다.

## 제15장

# Adaptation and Fine-Tuning

## 학습 목표

범용 VLA가 있다고 해서 곧바로 모든 연구실의 로봇을 잘 움직이는 것은 아니다. robot마다 camera pose, gripper, action frequency, joint limit, calibration, table height, object distribution, controller wrapper가 다르다. 따라서 adaptation과 fine-tuning은 선택 사항이 아니라 deployment process의 일부다. 이 장은 foundation VLA를 특정 robot/task/action space에 맞추는 engineering recipe를 정리한다.

1. VLA adaptation이 단순 마지막 layer 학습이 아님을 이해한다.
2. prompt, sensor, action head, adapter, LoRA, full fine-tuning의 차이를 구분한다.
3. embodiment/action normalization과 catastrophic forgetting 문제를 정식화한다.
4. offline evaluation에서 real-robot validation으로 넘어가는 protocol을 설계한다.

## 15.1 Why adaptation is unavoidable

pretrained VLA policy를 다음과 같이 쓰자.

$$\mathbf{a}_t \sim \pi_{\theta_0}(\cdot \mid \mathbf{o}_{\leq t}, \ell, \mathbf{q}_t, m_0). \quad (15.1)$$

여기서  $m_0$ 는 pretraining data의 implicit metadata이다. 실제 배치 대상 robot은  $m_\star$ 를 갖는다. adaptation은 다음 mismatch를 줄이는 과정이다.

$$\Delta_m = d(m_0, m_\star) = d_{\text{sensor}} + d_{\text{action}} + d_{\text{embodiment}} + d_{\text{task}} + d_{\text{controller}}. \quad (15.2)$$

이 mismatch가 작으면 prompt나 action normalization만으로 충분할 수 있다. mismatch가 크면 action head, vision encoder, language backbone, controller wrapper 까지 조정해야 한다.

adaptation을 너무 약하게 하면 robot-specific behavior를 얻지 못한다. 너무 강하게 하면 pretrained representation을 망가뜨리거나 unsafe overfitting이 생긴다. 이 장의 핵심은 어디를 얼마나 바꿀지 결정하는 것이다.

## 15.2 Adaptation targets

VLA adaptation target은 여러 층에 있다.

Open-source VLA 생태계에서 Octo, OpenVLA, OpenVLA-OFT, FAST, VLA Foundry 같은 흐름은 이 target들을 서로 다른 방식으로 다룬다 [41, 42, 45, 44, 56]. 이 책에서는 이들을 특정 성능 수치의 증거가 아니라, adaptation design space를 보여주는 case study로 취급한다.

## 15.3 Action head adaptation

Chapter 13의 관점에서 가장 중요한 fine-tuning 대상은 action head이다. pretrained representation  $h_t$ 가 주어졌을 때 action head는 다음 mapping을 만든다.

$$\mathbf{a}_t = g_\phi(h_t, \mathbf{q}_t). \quad (15.3)$$

robot이 바뀌면  $g_\phi$ 의 output type이 달라질 수 있다. 예를 들어 source model이 token action을 내는데 target robot은 continuous delta pose를 요구할 수 있다.

$$g_\phi^{\text{source}} : h_t \mapsto z_t, \quad g_\phi^{\text{target}} : h_t \mapsto \Delta x_t. \quad (15.4)$$

이 경우 full backbone보다 action head와 wrapper를 먼저 조정하는 것이 보수적이다. backbone은 semantic perception을 유지하고, target-specific motor interface

표 33: VLA adaptation target

| Target             | 무엇을 바꾸는가                                         | 적합한 상황                                            |
|--------------------|--------------------------------------------------|---------------------------------------------------|
| Prompt/template    | instruction style, task wording                  | model capability는 충분하지만 language convention이 다를 때 |
| Sensor remapping   | camera crop, calibration, proprioception channel | observation schema가 다를 때                          |
| Action wrapper     | normalization, clipping, frame convention        | action space는 유사하지만 scale/rate가 다를 때              |
| Action head        | token/vector/chunk decoder                       | robot action type이 달라질 때                          |
| Adapter/LoRA       | 일부 low-rank parameter update                     | data가 작고 pretrained representation을 보존해야 할 때      |
| Full fine-tuning   | backbone까지 update                                | domain gap이 크고 충분한 data/evaluation이 있을 때          |
| Controller wrapper | safety, tracking, smoothing, rate conversion     | model output과 robot controller가 직접 맞지 않을 때        |

만 바꾼다.

## 15.4 Embodiment adapter

multi-embodiment adaptation에서는 robot metadata를 policy에 명시적으로 넣는 것이 유리하다.

$$e_{\star} = E_{\eta}(m_{\star}), \quad \mathbf{a}_t \sim \pi_{\theta}(\cdot \mid \mathbf{o}_{\leq t}, \ell, \mathbf{q}_t, e_{\star}). \quad (15.5)$$

여기서  $E_{\eta}$ 는 embodiment adapter이다. 이 adapter는 kinematics, action scale, camera calibration, gripper type 같은 정보를 representation에 넣는다.

adapter가 없으면 model은 robot-specific difference를 observation distribution에서 암묵적으로 추론해야 한다. 이는 data가 많을 때는 가능할 수 있지만, small fine-tuning에서는 불안정하다.

## 15.5 LoRA, adapter tuning, and full fine-tuning

parameter-efficient tuning의 한 예는 low-rank update이다.

$$W' = W + \Delta W, \quad \Delta W = BA, \quad \text{rank}(\Delta W) \ll \text{rank}(W). \quad (15.6)$$

이 방식은 pretrained representation을 크게 바꾸지 않고 target data에 적응할 수 있다. small dataset에서는 full fine-tuning보다 안정적일 수 있다.

하지만 parameter-efficient tuning이 항상 충분한 것은 아니다. camera modality가 크게 바뀌거나, action representation이 바뀌거나, target task가 pretraining distribution 밖이면 action head와 일부 encoder를 함께 조정해야 할 수 있다.

표 34: Fine-tuning 방식의 trade-off

| Method                     | 장점                     | 위험                                                   |
|----------------------------|------------------------|------------------------------------------------------|
| Frozen backbone + new head | 안정적이고 data-efficient   | semantic-action mismatch가 남을 수 있음                    |
| Adapter / LoRA             | forgetting을 줄이고 비용이 낮음 | 큰 domain gap에는 부족할 수 있음                              |
| Partial fine-tuning        | 필요한 module만 바꿀 수 있음    | 어떤 layer를 열지 결정이 어려움                                 |
| Full fine-tuning           | target domain에 강하게 적응  | catastrophic forgetting, unsafe overfitting, 큰 검증 비용 |

## 15.6 Fine-tuning objective

target dataset  $\mathcal{D}_{target}$ 에 대해 behavior cloning loss를 쓰면

$$\mathcal{L}_{target} = -\mathbb{E}_{(\mathbf{o}, \ell, \mathbf{a}) \sim \mathcal{D}_{target}} \log \pi_{\theta}(\mathbf{a} \mid \mathbf{o}, \ell). \quad (15.7)$$

그러나 target data가 작으면 overfitting과 forgetting이 생긴다. 따라서 source policy와의 regularization을 추가할 수 있다.

$$\mathcal{L} = \mathcal{L}_{target} + \lambda D_{KL}(\pi_{\theta}(\cdot \mid \mathbf{o}, \ell) \parallel \pi_{\theta_0}(\cdot \mid \mathbf{o}, \ell)) + \beta \mathcal{L}_{safe}. \quad (15.8)$$

여기서  $\mathcal{L}_{safe}$  는 unsafe action penalty, constraint violation penalty, 또는 safety classifier loss 가 될 수 있다.

## 15.7 Data mixture for adaptation

adaptation data는 success-only demonstration으로 충분하지 않다.

$$\mathcal{D}_{adapt} = \mathcal{D}_{demo} \cup \mathcal{D}_{correction} \cup \mathcal{D}_{failure} \cup \mathcal{D}_{intervention}. \quad (15.9)$$

small dataset에서는 data weighting이 중요하다.

$$\mathcal{L}_{adapt} = \sum_j w_j \mathbb{E}_{r \sim \mathcal{D}_j} [\ell_\theta(r)]. \quad (15.10)$$

failure와 correction data는 수가 적어도 target deployment에서는 큰 영향을 줄 수 있다. 특히 action chunking이나 continuous action objective를 쓰는 경우, correction data는 recovery behavior를 학습하는 핵심이다.

## 15.8 Action normalization and calibration

fine-tuning에서 action normalization은 단순 preprocessing이 아니다. target robot의 physical scale을 정하는 calibration step이다.

$$\mathbf{a}_t^{phys} = \sigma_\star \odot \mathbf{a}_t^{norm} + \mu_\star, \quad \mathbf{a}_t^{safe} = \text{Clip}(\mathbf{a}_t^{phys}, \mathcal{U}_{safe}). \quad (15.11)$$

이 식에서  $\mu_\star, \sigma_\star$ 와  $\mathcal{U}_{safe}$ 는 robot-specific이다. 잘못된 scale은 성공률 문제에 safety issue가 된다.

calibration은 반드시 offline sanity check와 real robot low-speed test를 거쳐야 한다.

$$\text{Ready} = \mathbb{I}[\text{SchemaCheck} \wedge \text{ScaleCheck} \wedge \text{SafetyCheck} \wedge \text{LatencyCheck}]. \quad (15.12)$$

## 15.9 Validation protocol

fine-tuning된 policy는 곧바로 full-speed real robot에 올리면 안 된다. 최소 세 단계가 필요하다.

1. offline validation: held-out trajectory, action distribution, invalid action rate, latency를 본다.
2. simulation or replay validation: sensor/action wrapper와 controller interface를 검증한다.
3. guarded real-robot validation: low speed, restricted workspace, human intervention, emergency stop, logging을 사용한다.

성공률 하나만으로는 충분하지 않다.

$$M_{deploy} = (R_{success}, R_{invalid}, R_{saturation}, T_{infer}, E_{tracking}, R_{intervention}, R_{recovery}). \quad (15.13)$$

이 지표들은 Chapter 16의 evaluation protocol로 이어진다.

## 15.10 Case-study roles

이 장에서 최신 오픈소스/프로젝트 사례는 다음 역할로 읽는다.

이름을 외우는 것보다 중요한 것은 각 사례가 어떤 adaptation boundary를 건드리는지 보는 것이다.

### Case study: OpenVLA-OFT as action-interface adaptation

OpenVLA-OFT는 단순히 OpenVLA를 더 학습한 사례가 아니라 continuous action, action chunking, decoding speed, fine-tuning objective가 target robot deployment claim을 어떻게 바꾸는지 보여주는 사례다 [45]. Real VLA 관점의 질문은 성능 수치 하나가 아니라, target controller가 어떤 action을 받고 chunk interruption과 stale action을 어떻게 처리하는가이다.

## 15.11 Deployment checklist

1. target robot의 action type, frame, frequency, controller를 명시했는가?
2. source model의 action representation과 target action representation이 같은가?

표 35: Adaptation ecosystem의 case-study 역할

| Case        | 이 장에서의 역할                                                     | 읽을 때의 주의점                                         |
|-------------|---------------------------------------------------------------|---------------------------------------------------|
| Octo        | generalist robot policy와 빠른 sensor/action adaptation의 사례      | dataset/action schema가 target robot과 맞는지 확인       |
| OpenVLA     | open-source VLA backbone과 robot action fine-tuning의 기준점       | token/action interface와 latency를 함께 봐야 함          |
| OpenVLA-OFT | continuous action, chunking, practical fine-tuning recipe의 사례 | project/report claim과 independent validation 구분   |
| FAST        | efficient action tokenization과 compression의 사례                | tokenizer가 physical frequency structure를 보존하는지 확인 |
| VLA Foundry | LLM/ VLM/ VLA training stack 통합의 engineering 사례               | framework capability와 model capability를 구분        |

- 3. action normalization과 clipping이 robot-specific으로 검증되었는가?
- 4. fine-tuning data에 failure, correction, intervention이 포함되어 있는가?
- 5. LoRA/adapter/full fine-tuning 중 선택 이유가 data size와 domain gap에 맞는가?
- 6. offline metric, replay/sim metric, guarded real metric을 분리했는가?
- 7. deployment log가 다음 data engine loop로 들어가는가?

15.12 Further reading and exercises

**Further reading** Octo, OpenVLA, OpenVLA-OFT, FAST, VLA Foundry를 adaptation target과 evidence tier 관점에서 비교하라 [41, 42, 45, 44, 56].



**Review question** pretrained VLA를 target robot에 맞출 때 sensor remapping보다 action normalization이 더 위험할 수 있는 이유는 무엇인가?

**Design exercise** source VLA와 target robot이 다를 때 sensor adapter, action head, normalization, validation split, safety wrapper를 포함한 adaptation plan을 작성하라.

## 15.13 요약

Adaptation은 VLA를 특정 robot system에 연결하는 과정이다. 이는 prompt engineering도 아니고 마지막 layer 학습만도 아니다. sensor remapping, action normalization, embodiment adapter, action head tuning, LoRA/full fine-tuning, data mixture, safety wrapper, validation protocol이 모두 포함된다. small data에서는 pretrained representation을 보존해야 하지만, target robot의 action/control boundary를 충분히 바꾸지 못하면 task는 실패한다. aggressive fine-tuning은 성능을 올릴 수 있지만 forgetting과 unsafe overfitting을 만든다.

다음 장에서는 fine-tuned VLA가 실제로 잘하는지 어떻게 주장할 수 있는지, benchmark와 evaluation protocol을 다룬다.

## 제 IV 편

# Deployment Evidence

## 제16장

# Evaluation and Benchmarking

## 학습 목표

이 장은 VLA 평가의 evidence standard를 정의한다. 로봇이 task를 성공했다는 말은 생각보다 복잡하다. 한 번 성공했는가, 여러 초기조건에서 성공했는가, 실패했을 때 안전하게 멈췄는가, 사람 개입 없이 성공했는가, latency와 controller tracking은 괜찮았는가? Benchmark score는 capability의 증거이지만 capability 자체는 아니다.

1. success rate가 숨기는 failure type과 deployment cost를 설명한다.
2. benchmark family와 real-robot protocol을 구분한다.
3. action representation 별 추가 평가 지표를 정의한다.
4. reproducibility, benchmark artifact, statistical confidence를 평가 설계에 포함한다.

## 16.1 Why success rate is not enough

가장 흔한 metric은 success rate이다.

$$R_{success} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y_i = \text{success}]. \quad (16.1)$$

이 지표는 필요하지만 충분하지 않다. 같은 success rate라도 실패 양상, 개입 횟수, 충돌 위험, latency, recovery ability가 다를 수 있다.

Real VLA 평가에서는 최소한 다음 vector를 함께 봐야 한다.

$$M = (R_{success}, R_{failure\ type}, R_{intervention}, R_{unsafe}, T_{completion}, T_{infer}, E_{tracking}, R_{recovery}). \quad (16.2)$$

success rate 하나만 보고 model을 비교하면, benchmark에서 빠르게 성공하지만 실제 배치에서는 위험한 policy를 좋은 policy로 오해할 수 있다.

## 16.2 Benchmark families

Benchmark는 모두 같은 역할을 하지 않는다. 일부는 language-conditioned long-horizon manipulation을 본다. 일부는 lifelong learning, simulation coverage, open-vocabulary instruction, real-robot robustness를 본다.

표 36: VLA benchmark family

| Family            | 무엇을 보기 좋은가                                | 한계                                              |
|-------------------|-------------------------------------------|-------------------------------------------------|
| Sim manipulation  | controlled comparison, repeatability      | contact/sensor/latency reality gap              |
| Long-horizon task | subgoal sequencing, compounding error     | reset/protocol dependency가 큼                    |
| Lifelong/transfer | task variation과 knowledge transfer        | benchmark-specific overfitting 가능               |
| Open-vocabulary   | language grounding과 object generalization | visual grounding이 physical feasibility를 보장하지 않음 |
| Real-robot eval   | hardware interaction과 deployment friction | cost, variance, reproducibility difficulty      |
| Safety benchmark  | unsafe instruction/ event recognition     | semantic safety와 physical safety gap            |

LIBERO, CALVIN, ManiSkill, SimplerEnv, real-robot suites, evaluation harness 들은 이 family들 안에서 읽어야 한다 [51, 52, 54, 55]. 특정 benchmark의 높은 score는 그 protocol에서의 evidence이지 모든 robot capability의 증명이 아니다.

### 16.3 Evaluation protocol schema

evaluation episode는 다음 tuple로 정의할 수 있다.

$$e_i = (s_0, \ell, \mathcal{R}, \pi_\theta, \tau, y, f, h). \quad (16.3)$$

여기서  $s_0$ 는 initial condition,  $\ell$ 은 instruction,  $\mathcal{R}$ 은 reset distribution,  $\tau$ 는 rollout,  $y$ 는 outcome,  $f$ 는 failure label,  $h$ 는 human intervention log이다.

report에는 최소한 다음 항목이 있어야 한다.

1. initial condition sampling rule
2. instruction generation and ambiguity rule
3. reset and termination condition
4. allowed human intervention
5. controller and action frequency
6. success/failure annotation rule
7. number of trials and confidence interval

### 16.4 Action-dependent metrics

Chapter 13에서 action representation이 달라지면 evaluation도 달라진다.

따라서 VLA 평가표는 model name과 success rate만으로 충분하지 않다. action interface와 control loop 정보를 함께 보고해야 한다.

### 16.5 Robustness and distribution shift

generalization은 하나의 단어가 아니다. object shift, viewpoint shift, lighting shift, language paraphrase, embodiment shift, dynamics shift는 서로 다르다.

$$\Delta_{eval} = (\Delta_{obj}, \Delta_{scene}, \Delta_{lang}, \Delta_{emb}, \Delta_{dyn}, \Delta_{human}). \quad (16.4)$$

평가 protocol은 어떤 shift를 넣었는지 명시해야 한다.

표 37: Action type별 추가 evaluation metric

| Action type          | 추가 metric                                                                          | 해석 주의점                                             |
|----------------------|------------------------------------------------------------------------------------|----------------------------------------------------|
| Action token         | invalid token rate, decoding latency, discretization granularity                   | token confidence 는 safety confidence가 아니다          |
| Continuous vector    | saturation rate, smoothness, tracking error, scale report                          | mean action error가 task success를 대체하지 않는다          |
| Waypoint/ delta pose | IK feasibility, collision-free rate, frame error, recovery after infeasible target | waypoint는 trajectory feasibility가 아니다              |
| Action chunk         | horizon $H$ , interruption latency, chunk smoothness, chunk override rate          | long chunk success와 fast recovery는 trade-off가 있다   |
| Diffusion/flow       | sampling time, sample variance, safety projection rate, mode collapse              | multimodality는 safety guarantee가 아니다               |
| Whole-body action    | balance, foot contact, self-collision, fall/recovery, human proximity              | manipulation success만으로 whole-body safety를 말할 수 없다 |

robustness는 다음과 같이 조건부 success로 볼 수 있다.

$$R_{robust}(\Delta) = P(success \mid \Delta_{eval} = \Delta). \quad (16.5)$$

평균 success rate가 높아도 특정 shift에서 무너지면 deployment risk가 크다.

## 16.6 Failure taxonomy

failure를 하나의 실패로 묶으면 data engine이 개선 방향을 찾지 못한다.

$$f \in \{\text{perception, grounding, planning, action, control, safety, human interface}\}. \quad (16.6)$$

각 failure type은 다른 대응을 요구한다. perception failure는 data augmentation이나 sensor change가 필요할 수 있고, action failure는 action representation이나 controller wrapper를 바꿔야 할 수 있다.

## 16.7 Statistical reporting

robot experiment는 비용이 커서 trial 수가 작아지기 쉽다. 따라서 uncertainty reporting이 중요하다. success count를 binomial로 보면 confidence interval을 함께 보고해야 한다.

$$\hat{p} = \frac{k}{N}, \quad SE = \sqrt{\frac{\hat{p}(1 - \hat{p})}{N}}. \quad (16.7)$$

작은  $N$ 에서 1-2회 성공 차이는 과도하게 해석하면 안 된다. 특히 real robot evaluation에서는 seed, object placement, operator, reset quality가 variance를 만든다.

## 16.8 Evaluation harnesses and reproducibility

VLA 평가가 어려운 이유 중 하나는 benchmark 별 script, model server, action wrapper, observation preprocessing이 다르기 때문이다. evaluation harness는 이 fragmentation을 줄인다.

$$\text{Harness} = (\text{EnvAPI}, \text{ModelAPI}, \text{ActionWrapper}, \text{Logger}, \text{Metric}). \quad (16.8)$$

vla-eval류 harness는 여러 model과 benchmark를 같은 interface에서 비교하려는 engineering effort로 볼 수 있다. 그러나 harness가 metric validity를 자동으로 해결하지는 않는다. harness는 reproducibility를 돕지만, 무엇을 측정해야 하는지는 여전히 evaluation design의 문제다.

## 16.9 Benchmark artifacts

Benchmark artifact는 benchmark 설계가 실제 capability보다 쉬운 shortcut을 만드는 현상이다.

표 38: Benchmark artifact examples

| Artifact                   | 생기는 이유                          | 대응                                            |
|----------------------------|---------------------------------|-----------------------------------------------|
| Memorized layout           | scene variation 이 작음            | randomized layout, held-out scene             |
| Language template shortcut | instruction pattern 이 제한 됨      | paraphrase, ambiguity, negation test          |
| Success-only blindness     | failure type 을 기록하지 않음          | failure taxonomy, near-miss log               |
| Simulator comfort          | sim dynamics 가 단순함              | real-robot spot check, dynamics randomization |
| Reset leakage              | reset state 가 train/eval 에 반복 됨 | split by object/scene/operator/time           |
| Human intervention hidden  | 사람이 암묵적으로 도와줌                   | intervention log 와 allowed action 명시          |

16.10 Real robot reporting standard

real robot evaluation은 다음 standard를 포함해야 한다.

- 1. robot hardware, controller, action representation
- 2. sensor setup and calibration
- 3. task distribution and held-out split
- 4. number of trials and initial condition rule
- 5. success, partial success, failure, unsafe event definition
- 6. latency, control frequency, saturation, tracking error
- 7. human intervention and reset protocol
- 8. deployment logs and failure examples



이 standard가 없으면 real robot demo는 설득력 있는 anecdote일 수는 있어도 재현 가능한 evidence가 되기 어렵다.

### Case study: LIBERO/SimplerEnv reporting audit

LIBERO와 SimplerEnv 류 benchmark는 VLA와 robot policy 평가를 더 재현 가능하게 만들기 위한 중요한 시도이다 [51, 55]. 그러나 benchmark가 있다는 사실만으로 Real VLA claim이 닫히지는 않는다. benchmark가 무엇을 통제하고 무엇을 생략하는지 따로 읽어야 한다.

| 항목             | 확인할 것                                            | 빠지면 약해지는 claim                             |
|----------------|--------------------------------------------------|--------------------------------------------|
| Task split     | object, scene, language, temporal split이 분리되는가   | memorized layout 이나 template shortcut 가능성  |
| Action wrapper | token/vector/chunk가 simulator command로 어떻게 변환되는가 | action representation 비교의 공정성              |
| Reset protocol | 실패 후 reset과 initial condition이 기록되는가             | success rate의 variance 해석                  |
| Real transfer  | real robot spot check 또는 dynamics stress가 있는가    | simulator comfort를 real capability로 오해할 위험 |

따라서 benchmark 결과를 인용할 때는 score와 함께 protocol card를 읽어야 한다. Real VLA 평가의 핵심은 “몇 점인가”가 아니라 “어떤 조건에서 그 점수가 의미를 갖는가”이다.

## 16.11 Further reading and exercises

**Further reading** LIBERO, CALVIN, ManiSkill, SimplerEnv 를 benchmark score가 아니라 protocol evidence로 읽어라 [51, 52, 54, 55].

**Review question** 같은 success rate를 가진 두 VLA system이 서로 다른 robot capability를 가질 수 있는 이유는 무엇인가?

**Design exercise** mug/rack/glass task에 대해 success, unsafe event, intervention, recovery, latency, tracking error를 포함한 evaluation report template을 작성하라.

## 16.12 요약

Evaluation의 목적은 높은 success rate를 기록하는 것이 아니라, 어떤 capability가 어떤 조건에서 재현 가능하고 어떤 조건에서 무너지는지를 분리해서 증명하는 것이다. Benchmark는 필요하다. 하지만 benchmark score는 특정 protocol의 evidence이지 실제 robot capability 그 자체가 아니다. Real VLA 평가에서는 success rate와 함께 failure taxonomy, robustness shift, latency, tracking, safety intervention, recovery, statistical confidence를 보고해야 한다.

다음 장에서는 evaluation에서 드러난 한계가 hierarchical VLA와 embodied reasoning 구조로 어떻게 이어지는지 다룬다.

## 제17장

# Hierarchical VLA and Embodied Reasoning

## 학습 목표

이 장은 Real VLA를 단일 거대 policy가 아니라 여러 시간 척도와 책임 경계를 가진 계층형 system으로 본다. 상위 layer는 장면과 명령을 해석하고, task를 subgoal로 나누고, 진행 상황과 실패를 판단한다. 하위 VLA policy는 주어진 subgoal 또는 instruction에 대해 action chunk, waypoint, trajectory를 생성한다. controller와 safety monitor는 물리적 실행을 안정화하고 제한한다.

1. embodied reasoning layer의 역할과 출력 형식을 정의한다.
2. hierarchical policy를 options와 belief-state 관점에서 정식화한다.
3. reasoning layer, VLA policy, controller, safety monitor 사이의 interface contract를 설계한다.
4. progress detection, success estimation, replanning trigger를 평가 가능한 형태로 만든다.

## 17.1 Why hierarchy returns

초기 VLA 논의는 end-to-end policy의 가능성을 강조했다. 관측과 언어를 넣으면 action이 직접 나오는 구조는 data가 충분하고 task horizon이 짧을 때 강력하다 [34, 42, 43].

$$\mathbf{a}_t \sim \pi_\theta(\cdot \mid \mathbf{o}_{\leq t}, \ell). \quad (17.1)$$

그러나 실제 배치에 가까운 task에서는 이 단순한 interface만으로 부족한 경우가 많다. 이유는 네 가지다.

1. task horizon이 길어질수록 action error보다 subgoal error가 더 중요해진다.
2. safety constraint는 policy 내부의 implicit preference가 아니라 외부에서 검증 가능한 조건이어야 한다.
3. reasoning과 action generation은 요구하는 시간 척도와 compute budget이 다르다.
4. 실패 후 복구는 단순 next action prediction이 아니라 상태 재해석과 계획 수정이 필요하다.

따라서 real deployment에서는 다음 구조가 자연스럽다.

$$z_t = \rho_\psi(\ell, \mathbf{o}_{\leq t}, \mathbf{b}_t, m_t), \quad \mathbf{a}_{t:t+H-1} \sim \pi_\theta(\cdot \mid \mathbf{o}_{\leq t}, z_t, \mathbf{q}_t). \quad (17.2)$$

여기서  $\rho_\psi$ 는 embodied reasoning layer,  $z_t$ 는 subgoal, constraint, success criterion, tool call, recovery instruction을 포함하는 중간 표현이다. 하위 VLA는  $z_t$ 를 실행 가능한 action sequence로 변환한다.

### Hierarchical VLA

Hierarchical VLA는 language-conditioned reasoning layer와 action-generating VLA policy를 분리하되, 두 layer가 동일한 관측, task context, progress state를 공유하도록 설계한 robot policy stack이다. 핵심은 모듈을 많이 나누는 것이 아니라 각 layer의 책임과 출력 검증 조건을 명확히 하는 것이다.

## 17.2 Embodied reasoning is not text reasoning

LLM의 text reasoning은 문장 사이의 논리 관계를 다룬다. embodied reasoning은 물리 세계 안에서 실행될 action의 조건을 다룬다. 같은 명령이라도 robot embodiment, object pose, gripper state, scene constraint에 따라 의미가 달라진다.

명령  $\ell$ 이 주어졌을 때 embodied reasoning layer가 추정해야 할 변수는 다음과 같이 쓸 수 있다.

$$r_t = (g_t, c_t, \sigma_t, q_t, \eta_t). \quad (17.3)$$

여기서  $g_t$ 는 현재 subgoal,  $c_t$ 는 constraint set,  $\sigma_t$ 는 success criterion,  $q_t$ 는 필요한 clarification 또는 perception query,  $\eta_t$ 는 recovery mode이다.

표 39: Embodied reasoning의 주요 출력

| Output                       | 의미               | 예시                                    |
|------------------------------|------------------|---------------------------------------|
| Subgoal $g_t$                | 현재 실행해야 할 물리 목표  | 컵을 오른쪽 gripper workspace 안으로 이동       |
| Constraint $c_t$             | 실행 중 지켜야 할 조건    | 칼날 방향 회피, 사람 근처 속도 제한                 |
| Success criterion $\sigma_t$ | subgoal 완료 판정 조건 | object centroid가 bin 내부이고 gripper가 열림 |
| Query $q_t$                  | 추가 관측 또는 사람 질문   | 가려진 물체 확인, ambiguous instruction 확인   |
| Recovery mode $\eta_t$       | 실패 후 전환할 행동 범주   | regrasp, re-detect, ask, abort        |

중요한 점은 reasoning output이 자연어 설명으로 끝나면 안 된다는 것이다. 하위 VLA와 controller가 사용할 수 있는 contract로 내려와야 한다. 예를 들어 “파란 컵을 집어라”는 문장은 robot stack 내부에서 object id, target pose distribution, allowed approach direction, gripper command convention, success predicate로 변환되어야 한다.

### 17.3 Options view of hierarchical VLA

계층형 VLA는 options framework로 정식화할 수 있다. 상위 policy는 option  $\omega_t$ 를 선택하고, 하위 policy는 선택된 option 아래에서 action을 생성한다.

$$\omega_t \sim \mu_\psi(\cdot \mid \mathbf{b}_t, \ell), \quad \mathbf{a}_t \sim \pi_\theta(\cdot \mid \mathbf{o}_t, \omega_t). \quad (17.4)$$

option은 단순 skill 이름일 수도 있고, continuous subgoal, constraint set, language instruction을 함께 가진 구조화된 object일 수도 있다.

각 option은 termination model을 갖는다.

$$\beta_{\omega}(\mathbf{b}_t) = P(\text{terminate} = 1 \mid \mathbf{b}_t, \omega). \quad (17.5)$$

termination은 성공뿐 아니라 실패, unsafe condition, ambiguity, timeout도 포함한다. 따라서 hierarchical VLA의 상위 loop는 다음과 같이 쓸 수 있다.

$$\omega_{t+1} = \begin{cases} \mu_{\psi}(\mathbf{b}_t, \ell) & \text{if } \beta_{\omega_t}(\mathbf{b}_t) = 1, \\ \omega_t & \text{otherwise.} \end{cases} \quad (17.6)$$

이 관점은 VLA를 classical skill library로 되돌리는 것이 아니다. option 내부의 하위 policy가 여전히 visuomotor foundation model일 수 있고, option 자체도 language-conditioned reasoning model이 생성할 수 있다. 차이는 action prediction을 무한히 길게 늘리는 대신, 긴 horizon을 검증 가능한 subproblem으로 나누는 데 있다.

## 17.4 Interface contract between reasoning and VLA

계층 구조가 잘 작동하려면 layer 사이의 interface가 느슨한 자연어 prompt가 아니라 typed contract에 가까워야 한다. 한 time step에서 reasoning layer가 VLA에 전달하는 command를 다음 tuple로 정의하자.

$$\mathbf{z}_t = (\ell_t^{sub}, \mathcal{O}_t, \mathcal{G}_t, \mathcal{C}_t, \Sigma_t, H_t). \quad (17.7)$$

여기서  $\ell_t^{sub}$ 는 sub-instruction,  $\mathcal{O}_t$ 는 relevant object set,  $\mathcal{G}_t$ 는 target geometry 또는 pose distribution,  $\mathcal{C}_t$ 는 constraint,  $\Sigma_t$ 는 success predicate,  $H_t$ 는 action horizon이다.

contract가 없으면 상위 model은 그럴듯한 subgoal을 말하고, 하위 VLA는 그 subgoal을 다르게 해석하며, controller는 불가능한 target을 받는다. 이 mismatch는 성능 문제가 아니라 system integration 문제다.

표 40: Reasoning-VLA interface contract

| Field             | 필요 이유                                | 검증 방법                              |
|-------------------|--------------------------------------|------------------------------------|
| Sub-instruction   | 하위 policy 의 local objective 지정       | instruction grounding check        |
| Object set        | attention과 affordance 범위 제한          | detector id, mask, pose confidence |
| Geometry          | motion target 또는 relation 지정         | frame convention, pose covariance  |
| Constraint        | collision, force, human proximity 제한 | safety monitor predicate           |
| Success predicate | option termination 조건 제공             | state estimator 또는 verifier        |
| Horizon           | action chunk 길이와 interruptibility 결정 | latency budget, override test      |

## 17.5 Progress detection and success estimation

long-horizon task에서 핵심은 다음 action보다 progress state이다. progress를 scalar로만 두면 불충분하다. subgoal별 completion, constraint violation, uncertainty, recovery need를 함께 추정해야 한다.

$$p_t = \chi_\xi(\mathbf{o}_{\leq t}, z_t, \tau_{0:t}) = (P_{done}, P_{fail}, P_{unsafe}, U_{state}, U_{intent}). \quad (17.8)$$

여기서  $\chi_\xi$ 는 progress detector 또는 verifier이다.  $U_{state}$ 는 state uncertainty,  $U_{intent}$ 는 instruction ambiguity를 나타낸다.

replanning trigger는 다음 조건으로 정의할 수 있다.

$$\text{replan}_t = \mathbb{I}[P_{done} > \alpha \vee P_{fail} > \gamma \vee P_{unsafe} > \delta \vee U_{state} > \epsilon \vee t - t_0 > T_{\max}]. \quad (17.9)$$

이 식은 중요한 engineering principle을 말한다. reasoning은 매 step마다 반드시 호출될 필요가 없다. 그러나 완료, 실패, 위험, 불확실성, timeout이 감지되면 상위 layer가 다시 개입해야 한다.

## 17.6 VLA plus TAMP

Task and Motion Planning은 symbolic precondition과 geometric feasibility를 함께 다룬다. VLA와 TAMP를 결합할 때 TAMP는 반드시 모든 action을 결정하는 central planner일 필요가 없다. 세 가지 역할이 가능하다.

1. 상위 reasoning이 symbolic subgoal sequence를 만들고, TAMP가 feasibility를 검증한다.
2. TAMP가 candidate plan skeleton을 만들고, VLA가 각 step의 visual execution을 담당한다.
3. VLA가 proposed action을 만들고, TAMP 또는 geometric checker가 constraint violation을 걸러낸다.

이를 optimization 형태로 쓰면 다음과 같다.

$$\max_{\omega_{0:K}, \mathbf{a}_{0:T}} \mathbb{E} \left[ \sum_{t=0}^T r(\mathbf{s}_t, \mathbf{a}_t, \ell) - \lambda \sum_{t=0}^T \mathbb{I}[\mathbf{s}_t \notin \mathcal{S}_{safe}] \right] \quad \text{s.t.} \quad \mathbf{a}_t \sim \pi_{\theta}(\cdot \mid \mathbf{o}_t, \omega_k). \quad (17.10)$$

여기서  $\omega_{0:K}$ 는 subgoal 또는 option sequence이고,  $\mathcal{S}_{safe}$ 는 safety set이다. TAMP의 가치는 reward를 크게 만드는 것보다 infeasible branch를 조기에 제거하는 데서 더 크게 나타날 수 있다.

## 17.7 Tool and function calling

embodied reasoning layer는 종종 외부 tool을 호출해야 한다. 예를 들어 object detector, pose estimator, mapping module, grasp sampler, collision checker, memory retrieval, human query interface가 tool이 될 수 있다.

$$\mathbf{y}_t^{tool} = \mathcal{T}_{k_t}(\mathbf{x}_t), \quad k_t \sim \rho_{\psi}(\cdot \mid \ell, \mathbf{o}_{\leq t}, \mathbf{b}_t). \quad (17.11)$$

tool calling은 LLM application에서 흔하지만, robot에서는 tool output이 physical action의 전제 조건이 된다. 따라서 tool result에는 confidence, frame convention, timestamp, validity region이 포함되어야 한다.



**Tool call의 실패 모드**

tool call이 성공했다는 software-level return code와 robot task가 성공할 조건은 다르다. pose estimator가 값을 반환했더라도 frame이 틀리거나 오래된 observation이면 하위 VLA는 잘못된 target을 실행한다. 따라서 tool result는 controller가 쓰기 전에 freshness와 consistency를 검사해야 한다.

## 17.8 World models and memory

hierarchical VLA에서 world model은 두 수준에서 쓰인다. 첫째, short-horizon action consequence를 예측하여 unsafe 또는 infeasible action을 걸러낸다. 둘째, long-horizon task state를 유지하여 이미 완료된 subgoal과 남은 subgoal을 구분한다.

$$\hat{s}_{t+1} = f_{\varphi}(\mathbf{b}_t, \mathbf{a}_t), \quad m_{t+1} = \mathcal{M}(m_t, \mathbf{o}_t, \mathbf{a}_t, \hat{s}_{t+1}). \quad (17.12)$$

여기서  $m_t$ 는 scene memory 또는 episodic memory이다. memory가 없으면 robot은 같은 drawer를 반복해서 열거나 이미 옮긴 object를 다시 찾을 수 있다.

world model은 완벽할 필요가 없다. 실용적으로는 다음 세 질문에 답하면 충분한 경우가 많다.

1. 이 action이 가까운 미래에 collision 또는 joint limit violation을 만들 가능성이 큰가?
2. 현재 subgoal이 이미 완료되었는가?
3. 실패했을 때 이전에 성공한 recovery pattern이 있는가?

## 17.9 Monolithic versus hierarchical design

monolithic VLA와 hierarchical VLA는 서로 배타적인 연구 방향이 아니다. task와 deployment condition에 따라 다른 점이 유리하다.

공학적 결론은 단순하다. 짧고 반복적인 visuomotor task는 monolithic policy가 더 나을 수 있다. 그러나 multi-step manipulation, mobile manipulation, human-

표 41: Monolithic VLA와 hierarchical VLA의 비교

| 기준           | Monolithic VLA               | Hierarchical VLA                   |
|--------------|------------------------------|------------------------------------|
| Short task   | data가 충분하면 단순하고 빠름           | overhead가 생길 수 있음                  |
| Long horizon | compounding error와 memory 약점 | subgoal과 progress 관리에 유리           |
| Safety       | implicit behavior에 의존하기 쉬움   | external monitor와 constraint 삽입 가능 |
| Debugging    | failure 원인 분해가 어려움           | layer별 log와 failure taxonomy 가능    |
| Latency      | 한 model call로 끝낼 수 있음        | reasoning call과 tool call 비용 존재    |
| Adaptation   | data scale이 크면 강력함           | robot-specific wrapper 교체 가능       |

in-the-loop task, safety-critical task는 hierarchical structure가 더 많은 관측 가능성과 개입 지점을 제공한다.

## 17.10 Safety gates

hierarchical VLA에서 safety는 마지막에 붙이는 filter가 아니다. 각 layer에 gate가 있어야 한다.

$$\mathbf{a}_t^{exec} = \mathcal{P}_{safe}(\mathbf{a}_t^{raw}; \mathbf{b}_t, c_t, \mathcal{S}_{robot}, \mathcal{S}_{human}). \quad (17.13)$$

$\mathcal{P}_{safe}$ 는 raw action을 실행 가능한 safe action으로 projection하거나, projection이 불가능하면 halt 또는 ask로 전환한다.

이 구조는 model을 덜 신뢰하자는 말이 아니다. 어떤 model도 모든 physical constraint를 내부화했다는 evidence를 쉽게 제공할 수 없으므로, 검증 가능한 constraint는 system level에서 표현해야 한다는 뜻이다.

표 42: 계층별 safety gate

| Layer           | Gate                 | 예시                                                   |
|-----------------|----------------------|------------------------------------------------------|
| Reasoning       | instruction validity | 금지된 행동, ambiguous command, missing object            |
| Planning        | feasibility          | collision-free path, reachable grasp, precondition   |
| VLA policy      | action sanity        | saturation, invalid token, large pose jump           |
| Controller      | physical limit       | torque, velocity, joint limit, contact force         |
| Runtime monitor | emergency override   | human proximity, unexpected contact, perception loss |

## 17.11 Logging and evaluation

hierarchical VLA를 평가하려면 final success만 기록해서는 안 된다. 각 layer의 decision이 남아야 한다.

$$L_t = (\ell, z_t, p_t, \mathbf{a}_t^{raw}, \mathbf{a}_t^{exec}, s_t^{monitor}, y_t). \quad (17.14)$$

이 log는 failure attribution에 필요하다. 실패가 reasoning error인지, object grounding error인지, VLA action error인지, controller tracking error인지, safety gate의 conservative rejection인지 구분해야 다음 iteration의 data collection이 가능하다.

평가 metric도 계층별로 나누어야 한다.

1. reasoning accuracy: subgoal decomposition, object relation, constraint recognition
2. progress accuracy: completion, failure, unsafe event detection
3. action execution: tracking, smoothness, interruption latency
4. recovery quality: replan success, ask/abort correctness, repeated failure reduction

5. system cost: reasoning latency, tool call count, compute, human intervention

## 17.12 Design checklist

hierarchical VLA를 설계할 때 다음 질문을 먼저 정해야 한다.

1. 상위 reasoning layer는 매 step 호출되는가, event 기반으로 호출되는가?
2. reasoning output은 자연어인가, typed command 인가, hybrid 인가?
3. 하위 VLA는 action chunk, waypoint, low-level command 중 무엇을 출력하는가?
4. progress detector는 visual model 인가, state estimator 인가, rule verifier 인가?
5. 실패 시 recovery option은 누가 선택하는가?
6. safety gate는 action 전, controller 전, runtime 중 어디에 있는가?
7. log는 layer 별 decision과 rejected action을 보존하는가?

### Case study: embodied reasoning layer를 과대해석하지 않기

high-level reasoning layer는 task planning, spatial logic, success detection을 강화할 수 있지만, 그 자체가 low-level VLA policy나 verified controller는 아니다. Real VLA 관점에서는 reasoning output이 typed subgoal인지, action constraint인지, progress signal인지 분리하고, 하위 VLA와 controller가 무엇을 검증하는지 확인해야 한다.

## 17.13 Further reading and exercises

**Further reading** SayCan/VoxPoser식 planner, OpenVLA/ $\pi_0$ 식 action policy, embodied reasoning layer를 hierarchy 관점에서 비교하라 [28, 31, 42, 43].

**Review question** 상위 reasoning layer가 실패했는지 하위 VLA action이 실패했는지 구분하려면 어떤 log가 필요한가?

**Design exercise** long-horizon mobile manipulation task에 대해 reasoning layer, VLA policy, controller, monitor의 interface contract를 설계하라.

## 17.14 요약

계층형 VLA는 고전 modular robotics로의 단순 회귀가 아니다. embodied reasoning model은 task semantics, scene relation, progress, failure, tool use를 다루고, 하위 VLA는 fluid한 visuomotor action generation을 담당하며, controller와 safety monitor는 physical execution을 검증한다. real deployment에서 중요한 것은 어느 layer가 더 지능적인가가 아니라, 어떤 정보가 어느 layer에서 결정되고 어떻게 검증되는가이다.

다음 장에서는 이 계층 구조가 실제 하드웨어와 시간 제약을 만날 때 어떤 문제가 생기는지 다룬다. VLA는 좋은 action을 예측하는 것만으로 충분하지 않다. 정해진 latency 안에, 제한된 compute와 sensor bandwidth 안에서, controller가 받아들일 수 있는 형태로 action을 내야 한다.

## 제18장

# Real-Time and Hardware-Aware VLA

## 학습 목표

이 장은 VLA를 robot hardware 위에서 실행되는 real-time system으로 다룬다. 좋은 model은 benchmark에서 높은 success rate를 내는 것만으로 충분하지 않다. 정해진 latency budget 안에서 action을 내고, controller 주기와 맞으며, memory와 power 제약 안에서 안정적으로 실행되어야 한다. 지연이 생기면 안전하게 degrade하고, 통신이 끊기면 fallback controller로 넘어가야 한다.

1. VLA inference latency가 closed-loop manipulation과 locomotion에 미치는 영향을 정식화한다.
2. reasoning, VLA, trajectory generator, low-level controller의 time scale을 구분한다.
3. action chunking, asynchronous execution, fallback controller의 역할을 설명한다.
4. on-device inference와 cloud inference의 trade-off를 hardware-aware 관점에서 평가한다.

## 18.1 Latency is part of the dynamics

로봇에서 latency는 단순한 user experience 문제가 아니다. action이 늦게 도착하면 controller는 현재 상태가 아니라 과거 상태에 맞는 명령을 실행한다. discrete-

time dynamics를 다음과 같이 쓰자 [8, 60].

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{u}_{t-d}) + \mathbf{w}_t, \quad d = \left\lceil \frac{T_{lat}}{T_s} \right\rceil. \quad (18.1)$$

여기서  $T_{lat}$ 는 end-to-end latency,  $T_s$ 는 control sampling period이다.  $d = 0$ 인 system과  $d = 3$ 인 system은 같은 policy라도 다른 closed-loop system이다.

VLA policy가 출력하는 command를  $\mathbf{a}_t$ 라고 하고 controller가 이를  $\mathbf{u}_t$ 로 변환한다고 하자.

$$\mathbf{a}_t = \pi_\theta(\mathbf{o}_{\leq t}, \ell), \quad \mathbf{u}_t = \kappa(\mathbf{s}_t, \mathbf{a}_{t-d}). \quad (18.2)$$

이때 latency는 action quality와 별개의 axis가 아니라 action quality의 일부다. 좋은 action도 너무 늦으면 나쁜 action이 된다.

### Real-time VLA

Real-time VLA는 fixed deadline 안에서 action 또는 action chunk를 생성하고, deadline miss가 발생했을 때 controller-level fallback과 safety monitor가 정의되어 있는 VLA system이다. 여기서 real-time은 항상 hard real-time을 뜻하지 않는다. 중요한 것은 timing behavior가 측정되고 system design에 반영된다는 점이다.

## 18.2 Latency budget

VLA의 end-to-end latency는 model inference 시간 하나로 설명되지 않는다.

$$T_{lat} = T_{sense} + T_{pre} + T_{infer} + T_{decode} + T_{comm} + T_{ctrl}. \quad (18.3)$$

camera exposure, image transfer, resize, tokenization, model forward pass, action decoding, network communication, controller handoff가 모두 포함된다. benchmark 논문에서 inference time만 보고 real robot timing을 판단하면 위험하다.

real robot report에는 평균 latency뿐 아니라 tail latency가 필요하다.

$$T_{p95} = \inf\{t \mid P(T_{lat} \leq t) \geq 0.95\}. \quad (18.4)$$

평균이 낮아도  $T_{p95}$ 가 크면 intermittent failure가 생긴다. 특히 contact-rich manip-

표 43: Latency budget 구성 요소

| Component    | 의미                                      | 주요 risk                          |
|--------------|-----------------------------------------|----------------------------------|
| $T_{sense}$  | sensor exposure 와 frame transfer        | rolling shutter, dropped frame   |
| $T_{pre}$    | resize, crop, calibration, tokenization | CPU bottleneck, copy overhead    |
| $T_{infer}$  | VLA forward pass와 decoding              | model size, batch, KV cache      |
| $T_{decode}$ | token 또는 latent action을 command로 변환     | invalid token, IK failure        |
| $T_{comm}$   | process, network, middleware delay      | cloud jitter, ROS queue delay    |
| $T_{ctrl}$   | controller handoff와 tracking start      | stale target, frequency mismatch |

ulation에서는 tail event 한 번이 전체 episode를 망칠 수 있다.

### 18.3 Time-scale hierarchy

모든 layer가 같은 frequency로 돌 필요는 없다. 오히려 그렇게 설계하면 compute를 낭비하거나 불안정해질 수 있다.

이 hierarchy는 Chapter 17의 계층형 VLA와 직접 연결된다. 상위 reasoning은 느려도 된다. 그러나 safety monitor와 controller는 느리면 안 된다. VLA가 직접 50 Hz action을 내는 구조와, VLA가 5 Hz action chunk를 내고 low-level controller가 500 Hz로 추적하는 구조는 완전히 다른 design point이다.



표 44: Robot VLA stack의 시간 척도

| Layer                | Typical rate            | 역할                                          |
|----------------------|-------------------------|---------------------------------------------|
| Embodied reasoning   | event-based 또는 0.2–2 Hz | task decomposition, progress 판단, recovery   |
| VLA policy           | 1–10 Hz                 | action chunk, waypoint, local trajectory 생성 |
| Trajectory generator | 10–100 Hz               | smooth interpolation, constraint handling   |
| Low-level controller | 100–1000 Hz             | joint torque, velocity, impedance tracking  |
| Safety monitor       | 50–1000 Hz              | limit, collision, human proximity 감시        |

## 18.4 Action chunking

action chunking은 VLA latency를 완화하는 대표적 방법이다. model이 한 번에  $H$  개의 action을 출력한다고 하자.

$$A_t^{(H)} = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}) \sim \pi_\theta(\cdot \mid \mathbf{o}_{\leq t}, \ell). \quad (18.5)$$

chunk 실행 중 model은 다음 chunk를 비동기적으로 계산할 수 있다. effective planning interval은 줄어들지만, interruption latency는 커진다.

$$T_{interrupt} \approx \min(HT_s, T_{monitor}) \quad \text{if override is enabled.} \quad (18.6)$$

override가 없으면  $H$ 가 클수록 위험하다. 따라서 action chunk는 반드시 monitor-triggered cancellation과 함께 설계되어야 한다.

### Long chunk의 위험

긴 action chunk는 inference latency를 숨길 수 있지만, scene이 바뀌었을 때 오래된 계획을 계속 실행하게 만들 수 있다. 특히 사람과 가까운 manipulation, moving object, contact transition에서는 chunk horizon을 task phase에 따라 줄이거나 safety monitor가 즉시 중단할 수 있어야 한다.

## 18.5 Asynchronous execution

실제 system에서는 sensing, inference, decoding, control이 같은 thread에서 순차적으로 실행되지 않는다. 비동기 pipeline이 필요하다.

$$\text{Pipeline} = (Q_{obs}, Q_{act}, \pi_{\theta}, \kappa, \mathcal{W}). \quad (18.7)$$

$Q_{obs}$ 는 timestamped observation queue,  $Q_{act}$ 는 action queue,  $\mathcal{W}$ 는 watchdog이다.

비동기 system의 핵심 문제는 stale action이다. action이 생성된 observation time을  $t_o$ , 실행 time을  $t_e$ 라고 하면 freshness는 다음과 같이 정의할 수 있다.

$$F = \mathbb{I}[t_e - t_o \leq T_{fresh}]. \quad (18.8)$$

$F = 0$ 이면 action은 실행하지 않거나 revalidate해야 한다. stale action을 그대로 실행하는 것은 latency를 줄인 것이 아니라 timing bug를 숨긴 것이다.

## 18.6 Hardware constraints

robot hardware에서는 compute resource가 고정되어 있지 않다. GPU load, memory pressure, power mode, thermal throttling, sensor bandwidth가 시간에 따라 변한다. hardware state를  $h_t$ 라고 하면 VLA runtime은 다음 조건부 system이 된다.

$$\mathbf{a}_t \sim \pi_{\theta}(\cdot \mid \mathbf{o}_{\leq t}, \ell, h_t), \quad h_t = (M_t, C_t, P_t, \Theta_t, B_t). \quad (18.9)$$

여기서  $M_t$ 는 memory,  $C_t$ 는 compute availability,  $P_t$ 는 power budget,  $\Theta_t$ 는 temperature,  $B_t$ 는 bandwidth이다.

hardware-aware 설계는 모델을 작게 만드는 작업만이 아니다. 어떤 layer를 on-device에 두고, 어떤 layer를 remote에 돌지, 어떤 frequency로 호출할지 결정하는 architecture problem이다.

## 18.7 On-device versus cloud inference

cloud inference는 큰 model과 최신 weights를 쉽게 쓸 수 있다는 장점이 있다. 그러나 robot control에서는 network jitter와 connectivity failure가 치명적이다.

표 45: Hardware-aware VLA의 제약

| Constraint | VLA에 미치는 영향                                     | 대응                                                   |
|------------|-------------------------------------------------|------------------------------------------------------|
| Memory     | model size, image resolution, context length 제한 | quantization, adapter loading, streaming             |
| Compute    | inference rate와 batch size 제한                   | smaller policy head, early exit, distillation        |
| Power      | mobile robot runtime 제한                         | duty cycle, dynamic model selection                  |
| Thermal    | long-run latency drift                          | throttling-aware scheduler, cooling margin           |
| Bandwidth  | sensor streaming과 cloud inference 제한            | compression, local preprocessing, keyframe selection |

표 46: On-device와 cloud VLA의 비교

| 기준          | On-device                | Cloud                                 |
|-------------|--------------------------|---------------------------------------|
| Latency     | 낮고 예측 가능할 수 있음           | network jitter와 queue delay 존재        |
| Model scale | hardware에 제한됨            | 큰 model 사용 가능                         |
| Privacy     | sensor data 외부 전송 감소     | video와 scene data 전송 risk             |
| Reliability | local failure mode 중심    | connectivity와 service availability 의존 |
| Update      | 배포와 version 관리 필요        | centralized update 쉬움                 |
| Safety      | local watchdog과 밀접 결합 가능 | remote action은 local validation 필수    |

현실적인 구조는 hybrid인 경우가 많다. 상위 reasoning이나 semantic query는 cloud model을 쓰고, low-level VLA 또는 fallback policy는 on-device에 둔다. 이때 cloud output은 직접 motor command가 아니라 local validator를 통과해야 하는 subgoal 또는 plan이어야 한다.

## 18.8 Compression and its robot-specific risks

quantization, pruning, distillation, low-rank adaptation은 VLA deployment에 필요할 수 있다. 그러나 robotics에서는 language benchmark와 다른 failure mode가 생긴다.

$$\theta' = \mathcal{C}(\theta), \quad \Delta_\pi = \mathbb{E}[d(\pi_\theta(\mathbf{o}, \ell), \pi_{\theta'}(\mathbf{o}, \ell))]. \quad (18.10)$$

문제는  $\Delta_\pi$ 가 작아도 rare contact state나 safety-critical state에서 error가 커질 수 있다는 점이다.

표 47: Model compression의 robotics-specific risk

| 방법                | 장점                        | 주의점                                               |
|-------------------|---------------------------|---------------------------------------------------|
| Quantization      | memory와 latency 감소        | small action difference가 contact outcome을 바꿀 수 있음 |
| Distillation      | 작은 student policy 확보      | teacher의 uncertainty와 failure behavior가 사라질 수 있음  |
| Pruning           | compute 감소                | rare skill 또는 object category 성능 저하               |
| Early exit        | easy frame에서 빠른 inference | hard state를 제대로 감지하지 못하면 위험                       |
| Adapter swap-ping | robot별 specialization     | load time, version mismatch, calibration mismatch |

따라서 compressed VLA는 평균 success rate만으로 검증하면 안 된다. latency gain과 함께 action distribution shift, unsafe event, recovery performance를 다시 측정해야 한다.

## 18.9 Watchdog and fallback controller

real-time VLA에는 watchdog이 필요하다. watchdog은 model이 늦거나 이상한 command를 내면 system을 안전한 상태로 전환한다.

$$m_t^{run} = \begin{cases} \text{VLA} & T_{lat} \leq T_{max} \wedge \mathbf{a}_t \in \mathcal{A}_{safe}, \\ \text{fallback} & T_{lat} > T_{max} \vee \mathbf{a}_t \notin \mathcal{A}_{safe}, \\ \text{halt} & \mathbf{s}_t \notin \mathcal{S}_{safe}. \end{cases} \quad (18.11)$$

fallback은 단순 정지가 아닐 수 있다. gripper force를 낮추고 retract하거나, current trajectory를 부드럽게 stop하거나, base를 멈추고 arm을 safe pose로 보내는 controller가 될 수 있다.

### Graceful degradation

Graceful degradation은 VLA가 deadline을 놓치거나 uncertainty가 커졌을 때 system이 갑자기 실패하지 않고, 더 보수적이고 검증 가능한 mode로 전환되는 성질이다. real robot에서는 peak performance보다 degradation path가 더 중요할 수 있다.

## 18.10 Timing report standard

VLA 논문이나 system report가 real deployment를 주장하려면 다음 timing 정보를 포함해야 한다.

1. sensor frequency, image resolution, preprocessing device
2. model size, precision, hardware accelerator, batch size
3. mean, median, p95, p99 end-to-end latency
4. action output rate와 low-level controller rate
5. action chunk horizon, interruption rule, watchdog threshold
6. on-device/cloud boundary와 communication delay

7. thermal throttling 또는 long-run latency drift 측정

8. deadline miss가 발생했을 때 fallback behavior

이 정보가 없으면 같은 model score라도 deployment risk를 비교할 수 없다. robotics에서 timing은 implementation detail이 아니라 reproducibility condition이다.

표 48: Real-time VLA 논문에서 자주 빠지는 timing evidence

| 항목                       | 빠지면 약해지는 claim                                             |
|--------------------------|------------------------------------------------------------|
| Mean latency             | 평균만 있으면 tail failure를 볼 수 없다.                              |
| p95/p99 latency          | real-time 안정성과 deadline miss risk를 판단할 수 없다.               |
| Stale action rejection   | 오래된 action을 실행하지 않는지 확인할 수 없다.                             |
| Watchdog behavior        | model timeout 또는 invalid action에서 system이 어떻게 전환되는지 불명확하다. |
| Fallback controller      | 실패 시 stop, retract, hold, safe pose 중 무엇을 하는지 알 수 없다.      |
| Cloud/on-device boundary | network jitter가 safety-critical path에 들어가는지 판단할 수 없다.      |
| Thermal throttling       | 장시간 실행에서 latency drift가 생기는지 알 수 없다.                       |

## 18.11 Design checklist

hardware-aware VLA를 설계할 때는 다음 순서가 실용적이다.

1. task가 요구하는 control frequency와 reaction time을 먼저 정한다.
2. VLA가 직접 출력할 action abstraction과 chunk horizon을 정한다.
3. latency budget을 sensing, preprocessing, inference, decoding, communication, control로 분해한다.
4. p95와 p99 latency를 측정하고 deadline miss policy를 정한다.
5. on-device와 cloud boundary를 safety-critical path 기준으로 나눈다.

6. compression 후에는 success rate뿐 아니라 unsafe event와 action distribution drift를 재측정한다.
7. watchdog, fallback controller, stale action rejection을 integration test에 넣는다.

### Case study: cloud VLA와 stale action

cloud inference를 쓰면 큰 model을 사용할 수 있지만 network jitter가 safety-critical path에 들어온다. action이 생성된 observation이 이미 오래되었다면, controller는 의미상 맞지만 물리적으로 stale한 command를 실행할 수 있다. 따라서 cloud/on-device 비교는 average success가 아니라 p95/p99 latency, stale-action rejection, watchdog, fallback controller를 함께 보고해야 한다.

## 18.12 Further reading and exercises

**Further reading** MPC와 predictive safety filter를 real-time VLA의 deadline, fallback, stale-action rejection 문제와 함께 읽어라 [8, 60].

**Review question** 평균 inference latency가 낮아도 robot deployment claim이 약할 수 있는 이유는 무엇인가?

**Design exercise** edge/cloud hybrid VLA system에 대해 sensing, preprocessing, inference, decoding, communication, control의 latency budget과 watchdog policy를 설계하라.

## 18.13 요약

Real-time VLA의 핵심은 model을 빠르게 만드는 것 하나가 아니다. latency는 dynamics에 들어가며, control frequency hierarchy, action chunking, asynchronous execution, hardware state, cloud boundary, fallback controller가 모두 함께 설계되어야 한다. 큰 model이 더 좋은 model일 수는 있지만, robot 위에서는 deadline을

지키지 못하는 큰 model보다 안정적으로 닫힌 루프를 유지하는 작은 model이 더 나올 수 있다.

다음 장에서는 이 timing-aware system이 사람과 물리 환경 안에서 어떤 safety와 assurance 문제를 갖는지 다룬다. VLA가 action을 생성할 수 있다는 사실과, 그 action을 신뢰하고 배치할 수 있다는 사실은 다른 문제다.



## 제19장

# Safety and Assurance

## 학습 목표

이 장은 VLA safety를 prompt filter나 모델의 선의로 다루지 않는다. VLA가 물리 로봇을 움직이는 순간 safety는 perception, planning, control, runtime monitoring, data curation, evaluation이 함께 만드는 stack property가 된다. 안전한 VLA는 위험한 명령을 거절할 뿐 아니라, 애매하면 묻고, 불확실하면 멈추고, 실패하면 보수적으로 복구해야 한다.

1. VLA safety를 semantic, geometric, dynamic, system, data safety로 나눈다.
2. unsafe success와 safe failure를 구분한다.
3. safety shield, runtime monitor, fallback policy를 수식으로 정의한다.
4. assurance case와 safety reporting standard를 설계한다.

## 19.1 Safety is a stack property

VLA model이 위험한 문장을 거절한다고 해서 robot이 안전해지는 것은 아니다. 반대로 model이 안전한 instruction을 받았더라도 perception error, wrong frame, stale action, controller saturation 때문에 unsafe event가 발생할 수 있다. 따라서 safety는 다음 stack 전체의 성질이다 [9, 59, 60, 61, 62].

$$\mathcal{S}_{VLA} = (\mathcal{S}_{sem}, \mathcal{S}_{geo}, \mathcal{S}_{dyn}, \mathcal{S}_{sys}, \mathcal{S}_{data}). \quad (19.1)$$

각 항은 semantic safety, geometric safety, dynamic safety, system safety, data safety를 뜻한다.

표 49: VLA safety taxonomy

| Safety layer | 보는 것                                     | 대표 failure                                       |
|--------------|------------------------------------------|--------------------------------------------------|
| Semantic     | instruction, intent, policy rule         | 위험한 명령 수행, ambiguity 무시                          |
| Geometric    | workspace, collision, reachability       | 충돌 경로, unreachable target                        |
| Dynamic      | speed, force, torque, contact stability  | 과도한 힘, slip, fall, unstable contact              |
| System       | monitor, fallback, override, logging     | watchdog 부재, emergency stop 실패                   |
| Data         | demonstration, label, distribution, bias | unsafe behavior imitation, missing negative case |

이 taxonomy의 목적은 책임을 분리하는 것이다. semantic filter로 collision을 막을 수 없고, collision checker로 위험한 human instruction을 이해할 수 없다. 안전은 여러 gate가 서로 다른 failure mode를 막을 때 생긴다.

## 19.2 Unsafe success and safe failure

robot benchmark에서 success는 task outcome만 보는 경우가 많다. 그러나 safety 관점에서는 success와 safety를 분리해야 한다.

$$y_i = (y_i^{task}, y_i^{safe}), \quad y_i^{task}, y_i^{safe} \in \{0, 1\}. \quad (19.2)$$

네 가지 경우가 있다.

Real VLA에서는 unsafe success가 특히 중요하다. robot이 컵을 빠르게 건네는데 성공했지만 사람 손에 과도한 힘으로 밀었다면, success rate는 높아도 system은 unsafe이다.

표 50: Task outcome과 safety outcome의 분리

| Task    | Safety | 해석                                  |
|---------|--------|-------------------------------------|
| success | safe   | 원하는 deployment outcome              |
| success | unsafe | unsafe success; task는 되었지만 배치하면 안 됨 |
| failure | safe   | safe failure; 멈춤, 질문, 보수적 abort     |
| failure | unsafe | 가장 나쁜 경우; task도 실패하고 위험도 발생         |

### 19.3 Risk model

hazard event를  $H_j$  라고 하자. 하나의 episode에서 risk는 hazard probability와 severity의 조합으로 볼 수 있다.

$$R = \sum_{j=1}^J P(H_j \mid \ell, \mathbf{o}_{0:T}, \pi_\theta) \cdot C(H_j). \quad (19.3)$$

$C(H_j)$ 는 hazard severity이다. 같은 probability라도 사람과의 접촉, sharp object, hot object, high-speed motion은 cost가 다르다.

deployment 조건은 다음과 같이 쓸 수 있다.

$$\pi_\theta \in \Pi_{\text{deploy}} \quad \text{only if} \quad R \leq R_{\text{max}} \wedge P(H_{\text{critical}}) \leq \epsilon. \quad (19.4)$$

이 식은 risk를 완전히 정확히 계산할 수 있다는 뜻이 아니다. 최소한 어떤 hazard를 줄이려는지, 어떤 threshold를 기준으로 배치하는지 명시해야 한다는 뜻이다.

### 19.4 Semantic safety

semantic safety는 language instruction과 context가 허용 가능한지 판단한다. instruction  $\ell$ 에 대해 semantic safety classifier를 다음과 같이 정의할 수 있다.

$$s_{\text{sem}} = \sigma_\phi(\ell, \mathbf{o}_t, m_t) \in \{\text{allow, ask, refuse, escalate}\}. \quad (19.5)$$

중요한 것은 context dependence이다. “칼을 가져와”는 주방 정리 task에서는 안전할 수 있지만, 사람을 향해 빠르게 전달하는 상황에서는 위험할 수 있다. semantic safety는 단어 금지 목록이 아니라 intent, object, user, scene, robot capability를 함께 보는 판단이다.

### Ambiguity is a safety signal

instruction이 애매할 때 임의로 해석해서 실행하는 것은 unsafe behavior이다. VLA가 모를 때 물어보는 능력은 대화 기능이 아니라 safety mechanism이다.

## 19.5 Geometric and dynamic safety

geometric safety는 target과 path가 workspace constraint를 만족하는지 본다.

$$\mathbf{q}_t \in \mathcal{Q}_{free}, \quad x_{ee,t} \in \mathcal{X}_{task}, \quad d(\mathcal{B}_{robot}, \mathcal{B}_{human}) \geq d_{min}. \quad (19.6)$$

dynamic safety는 속도, 힘, 토크, contact constraint를 본다.

$$\|\dot{\mathbf{q}}_t\| \leq v_{max}, \quad \|\tau_t\| \leq \tau_{max}, \quad \|F_{contact,t}\| \leq F_{max}. \quad (19.7)$$

VLA action은 이런 constraint를 직접 만족하지 않을 수 있다. 따라서 raw action은 실행 전에 projection 또는 rejection을 거쳐야 한다.

## 19.6 Safety shield

safety shield는 learned policy가 낸 raw action을 safety constraint 안으로 넣는 layer이다.

$$\mathbf{a}_t^{exec} = \arg \min_{\mathbf{a} \in \mathcal{A}_{safe}(\mathbf{b}_t)} \|\mathbf{a} - \mathbf{a}_t^{raw}\|_W^2. \quad (19.8)$$

projection이 가능하면 가장 가까운 safe action을 실행한다. projection이 불가능하면 halt, ask, fallback 중 하나를 선택한다.

control barrier function 형태로 safety set을 표현할 수도 있다. safety condition

을  $h(\mathbf{s}) \geq 0$ 로 두면 continuous control에서 다음 constraint를 둘 수 있다.

$$\nabla h(\mathbf{s})f(\mathbf{s}) + \nabla h(\mathbf{s})g(\mathbf{s})\mathbf{u} + \alpha h(\mathbf{s}) \geq 0. \quad (19.9)$$

VLA가 직접 torque를 내지 않더라도, downstream controller는 이 constraint를 만족하는 command만 실행해야 한다.

## 19.7 Runtime monitor

runtime monitor는 policy 밖에서 system state를 감시한다.

$$s_t^{mon} = \mathcal{M}(\mathbf{b}_t, \mathbf{a}_t^{raw}, \mathbf{a}_t^{exec}, h_t, p_t), \quad (19.10)$$

여기서  $h_t$ 는 hardware state,  $p_t$ 는 progress state이다. monitor output은 다음 mode 중 하나가 될 수 있다.

$$s_t^{mon} \in \{\text{continue, slow, replan, ask, fallback, estop}\}. \quad (19.11)$$

표 51: Runtime monitor가 감시해야 할 신호

| Signal              | 위험 의미                                 | 대응                            |
|---------------------|---------------------------------------|-------------------------------|
| Pose uncertainty 증가 | object 또는 robot state 불확실             | slow, re-detect, ask          |
| Action saturation   | policy-controller mismatch            | scale down, fallback          |
| Contact force spike | unexpected collision 또는 grasp failure | stop, retract                 |
| Human proximity 감소  | 사람과 너무 가까움                            | speed limit, halt             |
| Deadline miss       | stale action 가능성                      | reject action, hold, fallback |
| Repeated failure    | recovery loop에 빠짐                     | replan, human handoff         |

## 19.8 Human supervision

human-in-the-loop은 단순히 사람이 옆에 있다는 뜻이 아니다. human authority boundary가 정의되어야 한다.

$$a_t^{final} = \mathcal{H}(a_t^{exec}, s_t^{mon}, u_t^{human}). \quad (19.12)$$

$u_t^{human}$ 은 approval, correction, emergency stop, teleoperation takeover일 수 있다. supervision design에서 중요한 질문은 세 가지다.

1. 어떤 event에서 robot이 사람에게 물어보는가?
2. 어떤 event에서 사람이 robot을 즉시 override할 수 있는가?
3. human intervention이 data engine에 어떻게 기록되는가?

human override가 log에 남지 않으면 safety가 실제 model capability처럼 보이는 reporting bias가 생긴다.

## 19.9 Data safety

VLA는 demonstration에서 behavior를 배운다. demonstration이 항상 안전하다는 보장은 없다. operator가 shortcut을 쓰거나, near-miss를 무시하거나, 안전하지 않은 speed로 task를 수행할 수 있다.

$$\mathcal{D}_{safe} = \{\tau_i \in \mathcal{D} \mid r_{safe}(\tau_i) \geq \rho\}. \quad (19.13)$$

unsafe demonstration을 단순히 제거하는 것도 항상 답은 아니다. robot은 무엇을 하지 말아야 하는지도 배워야 한다. 따라서 data에는 positive success, safe failure, unsafe near-miss, human correction이 구분되어야 한다.

## 19.10 Red-teaming and safety benchmarks

VLA red-teaming은 단순히 이상한 prompt를 넣어보는 것이 아니다. semantic ambiguity, adversarial instruction, physical uncertainty, perception blind spot, timing failure를 함께 만들어야 한다.

표 52: Safety-aware data label

| Label            | 의미                               | 학습에서의 사용                           |
|------------------|----------------------------------|------------------------------------|
| Safe success     | 성공하고 안전함                         | behavior cloning, positive example |
| Safe failure     | 실패했지만 안전하게 중단                    | abstention, recovery, ask 학습       |
| Unsafe near-miss | 사고 직전 또는 constraint violation 근접 | shield training, negative mining   |
| Human correction | 사람이 개입해 수정                       | preference, recovery data          |
| Unsafe success   | task는 되었지만 위험함                   | success metric 에서 제외, penalty data |

1. unsafe instruction: 명시적으로 위험한 행동 요구
  2. ambiguous instruction: object, recipient, intent가 불분명한 명령
  3. context reversal: 같은 문장이 안전한 context와 위험한 context에서 다르게 작동하는지 확인
  4. perception stress: occlusion, reflective object, poor lighting
  5. dynamic stress: moving human, moving object, contact transition
  6. system stress: latency spike, dropped frame, network disconnect
- safety benchmark score는 다음 vector로 보고해야 한다.

$$M_{safe} = (R_{unsafe}, R_{near\ miss}, R_{ask}, R_{refuse}, R_{fallback}, T_{detect}, T_{stop}). \quad (19.14)$$

unsafe rate만 낮고 ask/refuse가 과도하면 robot은 쓸모가 줄어든다. 반대로 task success가 높아도 near-miss가 많으면 배치할 수 없다.

# 19.11 Assurance case

assurance는 “안전할 것 같다”는 인상이 아니라 claim, evidence, argument의 구조이다.

$$\mathcal{A} = (C, E, G). \tag{19.15}$$

$C$ 는 safety claim,  $E$ 는 evidence,  $G$ 는 evidence가 claim을 지지하는 argument이다.

표 53: VLA assurance case 예시

| 구성 요소         | 예시                                                            |
|---------------|---------------------------------------------------------------|
| Claim         | robot은 사람 30 cm 이내에서 gripper 속도를 제한한다                         |
| Evidence      | proximity sensor log, controller limit test, red-team episode |
| Argument      | monitor가 distance를 측정하고 shield가 speed command를 projection함    |
| Assumption    | sensor calibration이 유지되고 occlusion이 threshold 이하임             |
| Residual risk | sensor blind spot에서는 emergency stop이 필요함                      |

VLA assurance의 어려움은 learned model이 open-ended instruction을 다룬다는 점이다. 따라서 assurance case는 model 자체를 완전히 증명하려 하기보다, 배치 범위, monitor, fallback, human override, logging을 함께 묶어야 한다.

# 19.12 Incident report standard

safety incident가 발생했을 때 다음 정보가 남아야 한다.

1. instruction, scene snapshot, robot state, human proximity
2. raw VLA action과 shield 이후 executed action
3. monitor state, watchdog event, fallback 여부
4. latency, dropped frame, hardware state
5. contact force, collision distance, joint limit margin



6. human intervention time과 intervention type
7. root cause label: semantic, perception, planning, action, control, system, data
8. 재현 가능 조건과 mitigation action

incident log가 없으면 safety improvement는 anecdote가 된다. real deployment에 서는 near-miss도 failure data로 취급해야 한다.

#### Case study: unsafe success를 별도 label로 남기기

로봇이 컵을 rack에 놓았지만 지나가는 동안 유리컵과 2 mm까지 접근했고 사람이 손을 뻗어 멈출 준비를 했다면, final success만으로는 좋은 결과처럼 보인다. Real VLA report에서는 이 episode를 unsafe success 또는 near-miss로 분리해야 한다. 그래야 safety shield와 data engine이 같은 실패를 다시 학습할 수 있다.

## 19.13 Further reading and exercises

**Further reading** Control barrier functions, shielding, predictive safety filter, ISO safety standards를 VLA safety stack의 서로 다른 evidence tier로 읽어라 [9, 59, 60, 61, 62].

**Review question** VLA가 위험한 instruction을 거절해도 robot safety가 보장되지 않는 이유는 무엇인가?

**Design exercise** unsafe success, safe failure, human intervention, near-miss를 구분하는 incident report form을 작성하라.

## 19.14 요약

VLA safety는 prompt engineering으로 끝나지 않는다. semantic safety는 위험한 명령과 ambiguity를 다루고, geometric safety는 collision과 reachability를 다루며, dynamic safety는 force와 stability를 제한한다. system safety는 monitor, watchdog,

fallback, human override로 구현되고, data safety는 unsafe behavior가 학습되는 것을 막는다. 안전한 VLA는 모든 명령을 수행하는 robot이 아니라, 수행하면 안 되는 상황을 알고 멈출 수 있는 robot이다.

다음 장에서는 이런 safety와 real-time 제약이 humanoid와 whole-body control로 확장될 때 어떤 추가 문제가 생기는지 다룬다. humanoid VLA에서는 manipulation만이 아니라 balance, locomotion, gaze, bimanual coordination, human proximity가 하나의 action space 안에서 결합된다.

제 V 편

## **Beyond Manipulation**

## 제20장

# Humanoids and Whole-Body VLA

## 학습 목표

이 장은 humanoid VLA를 arm-only manipulation VLA의 단순 확장으로 보지 않는다. humanoid에서는 torso, head, arms, hands, legs, balance, locomotion, gaze, bimanual coordination, whole-body contact가 하나의 closed-loop system 안에서 결합된다. 따라서 VLA는 semantic goal을 물리적으로 실행 가능한 whole-body objective로 변환해야 하고, whole-body controller는 그 objective를 balance와 contact constraint 안에서 실행해야 한다.

1. humanoid VLA가 arm-only VLA와 근본적으로 다른 이유를 설명한다.
2. whole-body state, action, constraint를 정식화한다.
3. VLA policy와 whole-body controller의 책임 경계를 정의한다.
4. human video, teleoperation, simulation이 humanoid data engine에서 맡는 역할을 구분한다.

## 20.1 Humanoid VLA is not high-dimensional arm control

tabletop manipulator에서는 end-effector pose와 gripper command만으로 task를 상당 부분 표현할 수 있다. humanoid에서는 같은 “pick up” instruction도 base pose,

foot placement, torso lean, arm reachability, gaze, hand posture, balance margin을 함께 결정해야 한다.

arm-only VLA를 다음과 같이 쓰자.

$$\mathbf{a}_t^{arm} = (\Delta x_{ee,t}, \Delta R_{ee,t}, g_t). \quad (20.1)$$

humanoid whole-body action은 훨씬 넓다.

$$\mathbf{a}_t^{hum} = (\mathbf{a}_t^{base}, \mathbf{a}_t^{torso}, \mathbf{a}_t^{head}, \mathbf{a}_t^{left\ arm}, \mathbf{a}_t^{right\ arm}, \mathbf{a}_t^{hands}, \mathbf{a}_t^{legs}). \quad (20.2)$$

그러나 핵심은 dimension 수가 아니라 coupling이다. arm target이 balance를 바꾸고, foot placement가 reachability를 바꾸며, gaze가 perception quality를 바꾸고, hand contact가 locomotion stability를 바꾼다.

### Whole-body VLA

Whole-body VLA는 language-conditioned policy가 whole-body objective, contact mode, locomotion-manipulation coordination, gaze, hand task를 함께 결정하고, downstream whole-body controller가 balance와 physical constraint 안에서 이를 실행하는 robot policy stack이다.

## 20.2 Whole-body state

humanoid state는 joint configuration만으로 충분하지 않다. base pose, contact state, center of mass, support region, object relation, human proximity가 포함되어야 한다.

$$\mathbf{s}_t^{hum} = (\mathbf{q}_t, \dot{\mathbf{q}}_t, x_{base,t}, x_{com,t}, \mathcal{C}_t, \mathcal{O}_t, h_t). \quad (20.3)$$

$\mathcal{C}_t$ 는 contact set,  $\mathcal{O}_t$ 는 object state,  $h_t$ 는 human/context state이다.

belief state도 중요하다.

$$\mathbf{b}_t^{hum} = P(\mathbf{s}_t^{hum} \mid \mathbf{o}_{\leq t}, \mathbf{u}_{< t}). \quad (20.4)$$

humanoid는 occlusion이 많고 self-motion이 크다. head, arm, torso가 움직이면 camera extrinsic과 visible region이 계속 바뀐다. 따라서 gaze control과 state esti-

mation은 action policy의 일부가 된다.

## 20.3 Balance and support constraints

humanoid control에서 balance는 선택 기능이 아니다. center of mass projection이 support polygon 안에 있거나, dynamic balance condition을 만족해야 한다.

$$\Pi_{xy}(x_{com,t}) \in \mathcal{P}_{support} \quad \text{for quasi-static tasks.} \quad (20.5)$$

dynamic task에서는 momentum과 contact wrench까지 봐야 한다.

$$\dot{h}_{mom} = \sum_{i \in \mathcal{C}_t} J_i(\mathbf{q}_t)^\top \lambda_i + \tau_{ext}. \quad (20.6)$$

여기서  $\lambda_i$ 는 contact wrench,  $J_i$ 는 contact Jacobian이다.

VLA가 whole-body joint command를 직접 낸다면 이런 constraint를 모두 내부화해야 한다. 실제로는 VLA가 objective를 내고 whole-body controller가 constraint를 만족시키는 구조가 더 안전하다.

## 20.4 Interface to whole-body controller

VLA와 whole-body controller 사이의 interface는 다음 tuple로 정의할 수 있다.

$$z_t^{wbc} = (x_{ee}^L, x_{ee}^R, x_{gaze}, x_{base}, \mathcal{C}_{des}, \mathcal{W}_{task}, \mathcal{C}_{safe}). \quad (20.7)$$

왼손과 오른손 목표, gaze target, base target, desired contact mode, task weight, safety constraint가 포함된다.

whole-body controller는 보통 다음 quadratic program 형태로 쓸 수 있다.

$$\begin{aligned} \min_{\ddot{\mathbf{q}}, \tau, \lambda} \quad & \sum_k \|J_k(\mathbf{q})\ddot{\mathbf{q}} + \dot{J}_k\dot{\mathbf{q}} - \ddot{x}_k^{des}\|_{W_k}^2 + \|\tau\|_{W_\tau}^2 \\ \text{s.t.} \quad & M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{b}(\mathbf{q}, \dot{\mathbf{q}}) = S^\top \tau + J_c^\top \lambda, \\ & \lambda \in \mathcal{K}_{friction}, \quad \mathbf{q} \in \mathcal{Q}_{limit}, \quad \tau \in \mathcal{T}_{limit}. \end{aligned} \quad (20.8)$$

VLA의 역할은 이 QP를 대체하는 것이 아니라, 어떤 task objective와 contact mode를 줄지 결정하는 것이다.

표 54: VLA policy와 whole-body controller의 책임 경계

| 구성 요소   | VLA가 결정하기 좋은 것             | Controller가 보장해야 할 것                       |
|---------|----------------------------|--------------------------------------------|
| Base    | 어디로 이동할지, 어느 방향을 볼지        | footstep feasibility, collision, stability |
| Arms    | 어느 손으로 어떤 object를 다룰지      | IK, joint limit, smooth tracking           |
| Hands   | grasp type, release timing | force limit, contact stability             |
| Gaze    | 무엇을 봐야 하는지                 | camera motion smoothness, calibration      |
| Contact | 기대 contact phase           | friction cone, wrench feasibility          |
| Safety  | task-level 금지 조건           | torque, speed, distance hard limit         |

## 20.5 Locomotion-manipulation coupling

humanoid는 조작하기 위해 걸어야 하고, 걷는 동안 조작 대상과 시야를 유지해야 한다. 따라서 locomotion과 manipulation은 분리된 문제가 아니다.

$$\begin{aligned}
 \max_{\xi_{base}, \xi_{arm}} \quad & R_{task}(\xi_{base}, \xi_{arm}, \ell) \\
 & - \lambda_{bal} C_{balance}(\xi_{base}, \xi_{arm}) \\
 & - \lambda_{col} C_{collision}(\xi_{base}, \xi_{arm}).
 \end{aligned} \tag{20.9}$$

base trajectory  $\xi_{base}$  와 arm trajectory  $\xi_{arm}$  는 함께 최적화되어야 한다. 예를 들어 선반 위 물체를 잡는 task에서 base가 너무 가까우면 arm collision이 생기고, 너무 멀면 reachability가 부족하다.

### Arm-first planning의 한계

humanoid에서 arm trajectory를 먼저 만들고 나중에 base와 balance를 맞추려 하면 실패가 많아진다. reachability, visibility, support polygon은 task planning 초기에 들어가야 한다.

## 20.6 Bimanual and dexterous control

bimanual manipulation은 두 팔을 독립적으로 제어하는 것이 아니다. 양손의 역할 분담, force closure, object frame, relative motion이 필요하다.

$$x_{rel} = (x_{ee}^L)^{-1} x_{ee}^R, \quad x_{obj} = \mathcal{G}(x_{ee}^L, x_{ee}^R, c_L, c_R). \quad (20.10)$$

한 손은 fixture 역할을 하고 다른 손은 manipulation을 할 수 있다. 또는 두 손이 같은 object를 들고 force를 분배해야 할 수 있다.

dexterous hand는 더 어렵다. action이 finger joint command로 내려가면 contact state가 급격히 늘어난다.

$$c_t^{hand} = \{(i, j, n_{ij}, f_{ij}) \mid \text{finger } i \text{ contacts object } j\}. \quad (20.11)$$

VLA가 직접 finger-level command를 내는 것보다 grasp primitive, in-hand manipulation objective, tactile feedback policy를 결합하는 방식이 현실적일 수 있다.

## 20.7 Dual-system architecture

humanoid VLA는 빠른 visuomotor action과 느린 embodied reasoning을 분리하는 dual-system 구조가 자연스럽다.

$$z_t^{high} = \rho_\psi(\ell, \mathbf{o}_{\leq t}, \mathbf{b}_t^{hum}, m_t), \quad z_t^{wbc} = \pi_\theta(\mathbf{o}_{\leq t}, z_t^{high}). \quad (20.12)$$

상위 reasoning은 task, subgoal, tool use, recovery를 담당한다. 하위 VLA는 whole-body objective를 생성한다. WBC와 locomotion controller는 physics constraint를 만족시킨다.



GR00T N1은 이 관점을 읽기 좋은 최신 사례다 [47]. 해당 계열은 vision-language module이 환경과 instruction을 해석하고, diffusion transformer motor module이 real-time motor action을 생성하는 dual-system humanoid VLA 구조를 제안한다. 또한 real robot trajectories, human videos, synthetic datasets를 섞는 data mixture를 사용한다. Real VLA 관점에서 중요한 점은 humanoid foundation model을 단일 policy head로 보지 않고, semantic interpretation, motor generation, data mixture, deployment embodiment를 함께 봐야 한다는 것이다.

#### GR00T N1을 읽을 때 확인할 것

Humanoid VLA claim은 “generalist humanoid”라는 이름만으로 평가하면 안 된다. vision-language module과 motor module의 handoff, action frequency, whole-body controller interface, balance constraint, human-video와 robot-trajectory data의 mixture ratio, real robot evaluation task를 함께 확인해야 한다.

#### Case study: GR00T N1을 humanoid system contract로 읽기

GR00T N1의 중요한 점은 humanoid foundation model을 단일 거대 policy로만 제시하지 않는다는 데 있다 [47]. vision-language module과 diffusion transformer motor module의 분리는 Chapter 17의 hierarchy, Chapter 18의 real-time budget, Chapter 19의 safety envelope와 직접 연결된다.

| 계약 항목             | 읽어야 할 내용                                      | Real VLA 질문                                       |
|-------------------|-----------------------------------------------|---------------------------------------------------|
| High-level module | instruction, scene, memory, subgoal 처리        | semantic plan 이 언제 motor objective로 바뀌는가?         |
| Motor module      | diffusion transformer 기반 action generation    | action frequency와 WBC interface가 명시되는가?           |
| Data mixture      | robot trajectory, human video, synthetic data | embodiment mismatch와 retargeting error를 어떻게 다루는가? |
| Safety/eval       | whole-body task 와 human proximity             | balance failure, near miss, intervention 이 보고되는가? |

이 사례의 결론은 “humanoid도 VLA가 된다”가 아니다. 더 정확한 결론은 humanoid VLA 일수록 model claim이 controller, balance, data mixture, safety case와 더 강하게 묶인다는 것이다.

표 55: Humanoid VLA stack

| Layer                 | 입력                              | 출력                                     |
|-----------------------|---------------------------------|----------------------------------------|
| Reasoning             | language, scene, memory         | subgoal, hand choice, recovery mode    |
| VLA policy            | images, proprioception, subgoal | WBC target, gaze, contact mode         |
| Whole-body controller | target, state, constraint       | torque, velocity, acceleration command |
| Locomotion controller | base target, terrain, balance   | footstep, gait, base motion            |
| Safety monitor        | state, human proximity, force   | slow, stop, fallback, estop            |

이 구조에서 가장 중요한 것은 handoff이다. reasoning이 “상자를 내려라”라고 말하는 것과 VLA가 “오른손으로 잡고 왼발을 앞으로 내딛어라”라고 구체화

하는 것 사이에는 많은 state abstraction이 필요하다.

## 20.8 Humanoid data engine

humanoid data는 arm-only manipulation보다 훨씬 비싸다. human video, teleoperation, simulation, robot autonomous rollout이 서로 다른 역할을 맡는다.

표 56: Humanoid VLA data source

| Source             | 제공하는 것                                                    | 한계                                                |
|--------------------|-----------------------------------------------------------|---------------------------------------------------|
| Human video        | semantic prior, body-object relation, long-horizon script | embodiment mismatch, missing force/proprioception |
| Teleoperation      | robot-valid action, contact, recovery example             | cost, operator bias, limited scale                |
| Simulation         | rare failure, safety stress, large variation              | sim-to-real gap, contact fidelity                 |
| Autonomous rollout | actual policy distribution                                | unsafe exploration, reset cost                    |
| Motion capture     | whole-body kinematic prior                                | robot dynamics와 actuator limit mismatch           |

human video를 robot action으로 쓰려면 retargeting이 필요하다.

$$\xi_{robot}^* = \arg \min_{\xi} d(\mathcal{R}_{robot}(\xi), \mathcal{R}_{human}(\xi_{human})) + \lambda C_{feas}(\xi). \quad (20.13)$$

retargeting은 imitation만이 아니라 feasibility projection이다. human은 넘어지지 않고 할 수 있는 motion이라도 robot morphology와 actuator limit에서는 불가능할 수 있다.

## 20.9 Safety envelope

humanoid는 사람 크기의 mass와 reach를 갖기 때문에 safety envelope가 더 엄격해야 한다.

$$\mathcal{E}_{safe}^{hum} = \{s, u \mid d_{human} \geq d_{min}, \|v_{ee}\| \leq v_{max}, \\ \|F_{contact}\| \leq F_{max}, m_{balance} \geq m_{min}\}. \quad (20.14)$$

$m_{balance}$ 는 balance margin이다. manipulation success가 높아도 balance margin이 낮거나 human proximity constraint를 자주 위반하면 humanoid VLA는 배치할 수 없다.

### 20.10 Evaluation

humanoid VLA 평가는 task success만으로 부족하다.

$$M_{hum} = (R_{task}, R_{balance}, R_{fall}, R_{contact}, \\ R_{bimanual}, R_{handover}, T_{lat}, R_{intervention}). \quad (20.15)$$

여기서  $R_{balance}$ 는 balance constraint 만족률,  $R_{fall}$ 은 fall 또는 near-fall rate,  $R_{contact}$ 는 unsafe contact rate,  $R_{handover}$ 는 human handover safety를 뜻한다.

평가 protocol에는 다음 항목이 필요하다.

1. robot morphology, actuator limit, controller rate
2. task environment와 human proximity condition
3. fall, near-fall, unsafe contact definition
4. teleoperation 또는 human assistance 여부
5. object mass, size, temperature, fragility
6. balance margin, foot slip, self-collision log
7. recovery behavior와 emergency stop event

## 20.11 Design checklist

humanoid VLA를 설계할 때는 다음 순서가 실용적이다.

1. VLA가 joint command를 직접 낼지, WBC objective를 낼지 정한다.
2. manipulation task가 base motion과 footstep planning을 요구하는지 먼저 본다.
3. gaze와 perception action을 action space에 포함할지 정한다.
4. bimanual task에서는 object frame과 relative hand motion을 명시한다.
5. teleoperation data에는 balance, contact, recovery label을 함께 저장한다.
6. human video prior는 robot feasibility projection을 거친다.
7. safety monitor는 human proximity, balance margin, contact force를 high rate로 감시한다.

## 20.12 Further reading and exercises

**Further reading** GR00T N1과  $\pi_{0.5}$ 를 humanoid dual-system, data mixture, whole-body controller interface 관점에서 읽어라 [47, 48].

**Review question** humanoid VLA에서 manipulation action과 locomotion/balance action을 분리해서 평가하기 어려운 이유는 무엇인가?

**Design exercise** humanoid handover task에 대해 high-level module, motor module, WBC, balance constraint, human-proximity safety envelope를 포함한 system contract를 설계하라.

## 20.13 요약

Humanoid VLA는 VLA의 모든 어려움이 한 embodiment에 모인 형태다. state estimation, contact, action representation, hierarchical reasoning, real-time inference,

safety, data scaling이 동시에 필요하다. 핵심은 VLA가 모든 joint command를 직접 예측하는 것이 아니라, semantic goal을 whole-body objective로 바꾸고, controller가 physics constraint 안에서 실행하도록 하는 책임 경계를 세우는 것이다.

다음 장에서는 책 전체를 마무리하면서 Real VLA가 앞으로 어떤 방향으로 발전해야 하는지 정리한다. 중요한 질문은 더 큰 모델 하나가 모든 문제를 해결하느냐가 아니라, data, embodiment, action interface, evaluation, safety가 함께 어떻게 성숙하느냐다.

## 제21장

# What We Have Learned

## 학습 목표

이 장은 책 전체의 기술 내용을 다시 압축한다. 목적은 감상적 결론을 쓰는 것이 아니라, Real VLA를 읽고 설계할 때 반복해서 써야 하는 system-level vocabulary를 정리하는 것이다. 앞 장들에서 배운 핵심은 단순하다. VLA는 더 큰 VLM이 아니라, language-conditioned perception과 learned action을 state estimation, control, planning, data, evaluation, safety assurance에 연결하는 책임 구조이다.

1. Part I-V가 어떤 순서로 Real VLA system claim을 닫는지 설명한다.
2.  $\mathbf{o}_t, \mathbf{s}_t, \mathbf{b}_t, \mathbf{a}_t, \mathbf{u}_t, \mathbf{q}_t$ 의 역할을 다시 정리한다.
3. state에서 action, control, data, evaluation, safety로 이어지는 연결 구조를 정식화한다.
4. mug/rack/glass running example을 책 전체의 stack 관점에서 다시 해석한다.
5. VLA 논문과 시스템을 읽을 때 반드시 확인해야 할 질문을 checklist로 정리한다.

## 21.1 The thesis revisited

이 책의 첫 명제는 다음과 같았다.

### Real VLA thesis

Real VLA는 특정 모델 이름이 아니다. Real VLA는 language-conditioned perception과 learned action을 실제 로봇의 물리 실행, monitoring, evaluation, safety assurance로 연결하는 책임 구조이다.

책 전체를 지나온 뒤 이 명제는 다음 식으로 다시 쓸 수 있다.

$$\mathcal{R} = (\mathcal{O}, \mathcal{B}, \mathcal{A}, \mathcal{U}, \mathcal{D}, \mathcal{E}, \mathcal{S}), \quad (21.1)$$

여기서  $\mathcal{O}$ 는 observation interface,  $\mathcal{B}$ 는 state/belief representation,  $\mathcal{A}$ 는 policy-level action space,  $\mathcal{U}$ 는 controller command space,  $\mathcal{D}$ 는 data engine,  $\mathcal{E}$ 는 evaluation protocol,  $\mathcal{S}$ 는 safety assurance layer이다.

따라서 VLA system claim은 다음 질문으로 환원된다.

$$\text{Claim} = f(\text{model, interface, data, control, evidence, safety}). \quad (21.2)$$

model 향이 강해도 나머지 향이 비어 있으면 claim은 닫히지 않는다. 반대로 backbone이 보통 수준이어도 action interface, data record, evaluation protocol, safety gate가 명확하면 system contribution은 강할 수 있다.

## 21.2 Part I: robot substrate

Part I은 Real VLA가 올라가는 물리적 기반을 고정했다. 이 부분의 결론은 VLA policy가 image와 language만으로 로봇을 움직이는 것이 아니라는 점이다. 실제 실행은 state, dynamics, controller, contact, planning 위에서 일어난다.

$$\mathbf{b}_t = P(\mathbf{s}_t \mid \mathbf{o}_{\leq t}, \mathbf{u}_{< t}), \quad \mathbf{a}_t \sim \pi_{\theta}(\cdot \mid \mathbf{o}_{\leq t}, \mathbf{b}_t, \ell). \quad (21.3)$$

여기서  $\mathbf{o}_t$ 는 sensor input이고,  $\mathbf{s}_t$ 는 물리 상태이며,  $\mathbf{b}_t$ 는 uncertainty를 포함한 internal estimate이다. VLA가 직접 보는 것은 대개  $\mathbf{s}_t$ 가 아니라  $\mathbf{o}_t$ 와 그로부터 추정된 representation이다. 이 차이를 무시하면 perception success와 robot success를 혼동한다.



control interface는 다음 mapping이다.

$$\mathbf{u}_t = \kappa(\mathbf{a}_t, \hat{\mathbf{s}}_t, C_t), \quad (21.4)$$

여기서  $C_t$ 는 collision, joint limit, force limit, workspace, human proximity 같은 constraint이다. 좋은 action이라도 controller가 추적할 수 없거나 constraint를 위반하면 robot action이 아니다.

표 57: Part I에서 고정한 핵심 구분

| 구분                               | 의미                                                     | VLA에서의 위험                                        |
|----------------------------------|--------------------------------------------------------|--------------------------------------------------|
| Observation vs state             | sensor input과 실제 물리 상태의 차이                             | 보이는 것과 가능한 것을 혼동                                 |
| Policy action vs control command | learned policy output과 actuator/controller command의 차이 | $\mathbf{a}_t$ 를 곧바로 $\mathbf{u}_t$ 로 오해         |
| Geometry vs contact              | 위치 관계와 접촉/힘 제약의 차이                                     | grasp, insertion, pushing 실패를 semantic error로 오해 |
| Plan vs executable trajectory    | plausible subgoal과 feasible motion의 차이                 | LLM plan을 robot plan으로 과대해석                      |

## 21.3 Part II: learning foundations

Part II는 imitation learning, reinforcement learning, visuomotor policy를 통해 VLA policy의 직접 조상을 정리했다. 핵심은 VLA가 갑자기 새로 생긴 문제가 아니라, robot learning의 오래된 실패 모드가 language와 foundation model 위에서 다시 나타난다는 것이다.

behavior cloning은 다음 손실을 줄인다.

$$\mathcal{L}_{BC} = \sum_t \ell(\pi_\theta(\mathbf{o}_{\leq t}, \ell), \mathbf{a}_t^E). \quad (21.5)$$

그러나 rollout에서는 policy가 만든 action이 다음 observation distribution을 바꾼

다.

$$\mathbf{o}_{t+1} \sim P(\cdot \mid \mathbf{s}_t, \mathbf{a}_t), \quad \mathbf{a}_t \sim \pi_\theta. \quad (21.6)$$

이것이 covariate shift이다. VLA가 language instruction을 이해하더라도, 실패 직전 state와 recovery action을 보지 못했다면 실제 rollout은 무너질 수 있다.

reinforcement learning과 optimal control은 reward, cost, constraint를 명시한다.

$$J(\pi) = \mathbb{E}_\pi \left[ \sum_{t=0}^T r(\mathbf{s}_t, \mathbf{a}_t, \ell) - \lambda c(\mathbf{s}_t, \mathbf{a}_t) \right]. \quad (21.7)$$

VLA 시대에도 이 식은 사라지지 않는다. 단지 reward가 language-conditioned task progress, physical safety, intervention, recovery, latency를 함께 포함해야 한다.

## 21.4 Part III: from planner to VLA

Part III는 VLM/LLM, planner+skills, robotics transformer, action representation, data engine, adaptation을 연결했다. 여기서 책의 중심 장은 action representation이다. VLA의 진짜 interface는 prompt가 아니라  $\mathbf{a}_t$ 이다.

$$\mathbf{a}_t \in \{\text{token, vector, waypoint, delta pose, chunk, distribution, whole body}\}. \quad (21.8)$$

각 action type은 downstream contract를 바꾼다.

$$\text{ActionContract} = (\mathcal{A}, f_{VLA}, f_{ctrl}, \kappa, \mathcal{C}_{safe}, \text{interrupt}). \quad (21.9)$$

token action은 language-model training pipeline과 잘 맞지만 quantization과 decoding latency 문제가 있다. continuous action은 controller와 연결하기 쉽지만 normalization과 frame convention이 중요하다. chunk와 diffusion/flow action은 temporal consistency와 multimodality를 주지만 interruption과 safety projection이 어려워진다.

표 58: Action representation 이 바꾸는 system contract

| Action type    | 얻는 것                                          | 반드시 확인할 것                                       |
|----------------|-----------------------------------------------|-------------------------------------------------|
| Token          | sequence modeling 과 semantic transfer         | quantization, invalid token, decoder latency    |
| Delta pose     | controller 연결과 빠른 servo                       | frame, scale, saturation, collision projection  |
| Waypoint       | planning/control integration                  | IK, feasibility, local planner assumption       |
| Chunk          | temporal consistency 와 bi-manual coordination | horizon, stale action, interruption rule        |
| Diffusion/flow | multimodal action distribution                | sampling budget, safety validation, smoothness  |
| Whole-body     | humanoid task coupling 표현                     | balance, contact, self-collision, fall recovery |

## 21.5 Part IV: deployment evidence

Part IV는 evaluation, hierarchy, real-time runtime, safety assurance를 다루었다. 이 부분의 핵심은 capability claim과 deployment evidence를 분리하는 것이다. benchmark success는 필요하지만 충분하지 않다.

$$\mathcal{M}_{deploy} = (R_{success}, R_{safe}, T_{p95}, E_{track}, N_{intervention}, R_{recovery}). \quad (21.10)$$

여기서  $R_{success}$ 만 있으면 성공률이다.  $R_{safe}$ , latency, tracking error, intervention, recovery가 함께 있어야 robot capability claim에 가까워진다.

hierarchical VLA는 이 claim을 layer별로 분해한다.

$$\text{Decision} = (d^{reason}, d^{skill}, \mathbf{a}_t, \mathbf{u}_t, d^{monitor}). \quad (21.11)$$

어느 layer가 무엇을 결정했는지 기록하지 않으면 실패 원인을 알 수 없다. 상위 reasoning이 틀렸는지, skill selection이 틀렸는지, action representation이 틀렸는지, controller tracking이 틀렸는지, monitor가 늦었는지 구분해야 한다.

## 21.6 Part V: beyond manipulation

Part V는 humanoid와 future direction으로 범위를 넓힌다. humanoid VLA는 arm-only VLA의 단순 고차원 버전이 아니다. base, torso, head, arms, hands, legs, gaze, balance, locomotion, contact가 하나의 closed-loop system 안에서 결합된다.

$$\mathbf{a}_t^{hum} = (\mathbf{a}_t^{base}, \mathbf{a}_t^{torso}, \mathbf{a}_t^{head}, \mathbf{a}_t^{arms}, \mathbf{a}_t^{hands}, \mathbf{a}_t^{legs}). \quad (21.12)$$

중요한 것은 dimension이 아니라 coupling이다. arm target은 balance를 바꾸고, foot placement는 reachability를 바꾸며, gaze는 perception quality를 바꾼다. 따라서 humanoid VLA claim에는 whole-body controller와 safety envelope이 반드시 포함되어야 한다.

Future Directions는 이 복습 위에서 읽어야 한다. world model, self-improving data loop, online adaptation, multi-embodiment transfer, safety assurance는 새로운 유행어가 아니다. 지금까지 정리한 interface와 evidence requirement가 커진 결과이다.

## 21.7 Notation recap

## 21.8 The system loop

책 전체를 하나의 loop로 쓰면 다음과 같다.

$$\begin{aligned} \mathbf{b}_t &= \text{Estimate}(\mathbf{o}_{\leq t}, \mathbf{u}_{< t}), \\ \mathbf{a}_t &\sim \pi_\theta(\cdot \mid \mathbf{o}_{\leq t}, \mathbf{b}_t, \ell), \\ \tilde{\mathbf{a}}_t &= \text{Shield}(\mathbf{a}_t, \mathbf{b}_t, C_t), \\ \mathbf{u}_t &= \kappa(\tilde{\mathbf{a}}_t, \hat{\mathbf{s}}_t, C_t), \\ y_t &= \text{Evaluate}(\mathbf{s}_t, \mathbf{a}_t, \mathbf{u}_t, \ell), \\ \mathcal{D}_{k+1} &= \mathcal{D}_k \cup \text{Log}(\mathbf{o}_t, \mathbf{b}_t, \mathbf{a}_t, \mathbf{u}_t, y_t). \end{aligned} \quad (21.13)$$

이 loop에서 하나라도 빠지면 Real VLA claim은 약해진다. Estimate가 없으면 perception uncertainty를 모른다. Shield가 없으면 unsafe action을 구분하기 어

표 59: 이 책에서 반복해서 쓰는 핵심 notation

| 기호    | 의미                                            | 확인해야 할 질문                                                        |
|-------|-----------------------------------------------|------------------------------------------------------------------|
| $o_t$ | observation, sensor input, language context   | 무엇을 실제로 보았는가? calibration과 delay는?                               |
| $s_t$ | physical state                                | 무엇이 관측되지 않았고 추정되어야 하는가?                                          |
| $b_t$ | uncertainty를 포함한 belief                       | uncertainty가 action과 safety에 반영되는가?                              |
| $q_t$ | robot configuration 또는 generalized coordinate | reachability, joint limit, kinematic feasibility는?               |
| $a_t$ | policy-level action                           | token, pose, delta, chunk, distribution 중 무엇인가?                  |
| $u_t$ | controller/driver command                     | 어떤 controller가 어떤 rate로 실행하는가?                                   |
| $C_t$ | constraint set                                | collision, force, human proximity, workspace가 hard constraint인가? |
| $m_i$ | data/embodiment metadata                      | robot, camera, controller, normalization이 기록되는가?                 |

럽다.  $\kappa$ 가 명시되지 않으면 action의 물리 의미가 없다. Evaluate와 Log가 없으면 성공과 실패가 다음 data engine으로 돌아가지 않는다.

## 21.9 Running example revisited

running example은 “머그컵을 drying rack에 올리고 유리컵은 건드리지 말라”였다. 이 예제는 책 전체를 관통하는 작은 benchmark로 읽을 수 있다.

이 예제에서 좋은 VLA는 단순히 “mug”와 “rack”을 이해하는 모델이 아니다. 좋은 VLA는 forbidden region, grasp approach, rack clearance, controller reference, interruption rule, safety log, evaluation metadata를 함께 남기는 system이다.

표 60: mug/rack/glass 예제를 stack 전체로 읽기

| Layer        | 이 예제에서 해야 할 일                                   | 실패하면 보이는 현상                             |
|--------------|-------------------------------------------------|-----------------------------------------|
| Perception   | mug, rack slot, glass, free space를 찾음           | object는 찾지만 glass clearance를 모름         |
| State/belief | pose uncertainty와 occlusion을 추정                 | rack slot 위치가 불확실한데 바로 접근               |
| Planning     | pick, lift, transport, place, verify 순서를 구성     | plausible plan은 있지만 executable path가 없음 |
| Action       | delta pose, waypoint, chunk 중 interface 선택      | action quantization 또는 frame error      |
| Control      | gripper, arm, contact force를 추적                 | overshoot, slip, rack collision         |
| Evaluation   | success, no-glass-contact, latency, recovery 기록 | 성공률은 높지만 near-miss가 숨겨짐                 |
| Safety       | forbidden region과 force limit을 강제               | 유리컵을 건드리고도 success로 집계                  |
| Data engine  | failure/ correction/ intervention을 다음 학습으로 보냄   | 같은 failure를 다음 rollout에서 반복             |

## 21.10 Twelve questions for reading VLA papers

VLA 논문을 읽을 때 다음 12문항을 먼저 확인한다.

1. observation은 무엇이며, camera/proprioception/tactile calibration이 명시되는가?
2. language instruction은 task goal인가, constraint인가, preference인가?
3. state 또는 belief를 쓰는가, 아니면 raw observation만 쓰는가?
4. action representation은 token, vector, waypoint, chunk, distribution 중 무엇인가?
5. policy-level action이 어떤 controller command로 변환되는가?

6. VLA inference rate와 controller rate는 각각 얼마인가?
7. action normalization, frame convention, clipping rule이 robot-specific으로 정의되는가?
8. data에는 success뿐 아니라 failure, correction, intervention, reset이 들어가는가?
9. evaluation은 trial count, randomization, confidence, failure taxonomy를 제공하는가?
10. safety는 prompt rule인가, geometric/dynamic/runtime shield인가?
11. latency, stale action, watchdog, fallback controller가 보고되는가?
12. deployment log가 다음 data collection 또는 adaptation으로 돌아가는가?

이 질문 중 절반 이상이 비어 있으면, 그 논문은 VLA model claim은 할 수 있어도 Real VLA system claim은 아직 약하다.

## 21.11 Ten propositions

표 61: 이 책의 핵심 명제 10개

| 번호 | 명제                                                                                                     |
|----|--------------------------------------------------------------------------------------------------------|
| 1  | VLA는 더 큰 VLM이 아니라 robot execution stack에 연결되는 책임 구조이다.                                                 |
| 2  | observation, state, belief를 구분하지 않으면 perception success와 robot capability를 혼동한다.                       |
| 3  | $a_t$ 와 $u_t$ 는 다르며, 그 사이에는 controller contract가 필요하다.                                                 |
| 4  | contact-rich manipulation은 geometry만으로 설명되지 않는다. force, compliance, slip, recovery가 필요하다.              |
| 5  | LLM/VLM plan은 executable plan이 아니다. feasibility, IK, collision, recovery가 따로 필요하다.                     |
| 6  | imitation learning의 covariate shift는 VLA fine-tuning에서도 핵심 failure mode이다.                             |
| 7  | action representation은 formatting 문제가 아니라 control boundary 문제이다.                                       |
| 8  | robot data는 web data가 아니다. embodiment, controller, action schema, reset policy가 의미를 만든다.               |
| 9  | benchmark success는 robot capability의 일부 증거일 뿐이다. metadata와 failure taxonomy가 필요하다.                     |
| 10 | safety는 prompt policy가 아니라 semantic, geometric, dynamic, runtime, data, human layer의 stack property이다. |

## 21.12 Further reading and exercises

**Further reading** 이 장의 12 questions를 책 전체의 final paper-review protocol로 사용하라. 특히 Chapter 13, 14, 16, 19와 함께 다시 읽는 것이 좋다.

**Review question** 이 책의 10개 핵심 명제 중 실제 VLA 논문 리뷰에서 가장 자



주 빠지는 명제는 무엇인가?

**Design exercise** VLA 논문 하나를 골라 12 questions checklist로 short review를 작성하고, claim strength를 strong/moderate/weak으로 분류하라.

## 21.13 What remains

이 장은 책의 결론이 아니라 압축이다. 지금까지 배운 것은 현재의 Real VLA system contract이다. 다음 질문은 이 contract가 앞으로 어떻게 확장되는가이다. 더 강한 world model은 어떤 state를 예측해야 하는가? Online adaptation은 어떤 release gate를 통과해야 하는가? Multi-embodiment transfer는 semantic, geometric, control transfer를 어떻게 분리해야 하는가? Humanoid VLA는 balance와 manipulation을 어떤 hierarchy로 결합해야 하는가?

따라서 다음 장의 Future Directions는 공상적 전망이 아니라, 지금까지 정리한 책임 구조 위에서 남는 연구 문제의 목록이다. 이제 우리가 배운 현재의 system contract 위에서, 앞으로 어떤 연구 문제가 남는가를 본다.

## 제22장

## Future Directions

## 학습 목표

이 장은 Real VLA의 미래를 단순한 모델 크기 경쟁으로 보지 않는다. 앞으로의 VLA는 world model, online adaptation, self-improving data loop, multi-embodiment transfer, formal safety assurance, hardware-aware deployment가 결합된 robot system으로 발전할 가능성이 크다. 이 책의 결론은 VLA가 제어, 계획, 모션플래닝, 로봇러닝을 대체하지 않는다는 것이다. 오히려 VLA는 이들을 language와 vision을 통해 더 넓은 task space로 연결한다.

1. Real VLA의 미래 시스템 구성 요소를 정리한다.
2. world model, data engine, safety assurance, deployment loop의 역할을 설명한다.
3. multi-embodiment transfer와 online adaptation의 조건을 정식화한다.
4. foundation model 시대의 roboticist가 가져야 할 시스템 관점을 정리한다.

## 22.1 Beyond bigger VLMs

VLA의 미래가 더 큰 VLM에 action head를 붙이는 방향만으로 정해지지 않는 것이다. 큰 model은 semantic prior와 generalization을 제공하지만, robot deployment는 physical state, latency, controller, safety, data feedback loop를 요구한다. 미래의 Real VLA system은 다음 tuple에 가깝다.

$$\mathcal{R}_{VLA} = (\rho_\psi, \pi_\theta, \mathcal{W}_\varphi, \kappa, \mathcal{D}, \mathcal{M}, \mathcal{S}, \mathcal{E}). \quad (22.1)$$

여기서  $\rho_\psi$  는 embodied reasoning,  $\pi_\theta$  는 action policy,  $\mathcal{W}_\varphi$  는 world model,  $\kappa$  는 controller,  $\mathcal{D}$  는 data engine,  $\mathcal{M}$  은 runtime monitor,  $\mathcal{S}$  는 safety case,  $\mathcal{E}$  는 evaluation protocol이다.

### Future Real VLA

Future Real VLA는 language-conditioned action model 하나가 아니라, world model, policy, controller, data loop, safety assurance, hardware runtime이 닫힌 루프를 이루는 robot system이다. 핵심 성능은 benchmark score뿐 아니라 배치 후 실패를 측정하고 안전하게 개선하는 능력이다.

## 22.2 World models

현재 VLA는 imitation과 pattern recognition에서 강하지만, hidden state, long-horizon consequence, physical causality, object permanence, tool use, failure recovery를 안정적으로 다루기 어렵다. world model은 이 한계를 줄이는 방향이다.

$$p_\varphi(\mathbf{s}_{t+1}, \mathbf{o}_{t+1}, r_t \mid \mathbf{b}_t, \mathbf{a}_t) \quad (22.2)$$

는 action이 world state와 observation, reward 또는 task progress를 어떻게 바꾸는지 예측한다.

world model은 video prediction model과 같지 않다. robot에 필요한 것은 모든 pixel을 잘 예측하는 것이 아니라 task-relevant state와 safety-relevant state를 예측하는 것이다.

$$\mathcal{Z}_{rel} = (x_{obj}, x_{ee}, c_{contact}, h_{human}, u_{uncertainty}, y_{success}). \quad (22.3)$$

미래의 VLA world model은 세 가지 기능을 가져야 한다.

1. consequence prediction: 이 action이 task와 safety에 어떤 결과를 만드는가?
2. information gathering: 불확실할 때 어떤 관측을 더 해야 하는가?
3. counterfactual evaluation: 다른 action을 했으면 failure를 피할 수 있었는가?

## 22.3 Information gathering

robot은 모르면 행동하기 전에 봐야 한다. 불확실성을 줄이는 action도 policy의 일부다. 다음 action을 task reward 만으로 고르지 않고 information gain을 함께 고려할 수 있다.

$$\mathbf{a}_t^* = \arg \max_{\mathbf{a}} \mathbb{E}[R(\mathbf{s}_t, \mathbf{a}, \ell) + \lambda I(\mathbf{s}_{t+1}; \mathbf{o}_{t+1} \mid \mathbf{b}_t, \mathbf{a}) - \mu C_{risk}(\mathbf{s}_t, \mathbf{a})]. \quad (22.4)$$

이 식은 VLA가 항상 물체를 움직이는 action만 내야 하는 것이 아님을 보여준다. head를 돌려서 더 잘 보기, camera view를 바꾸기, drawer 안을 확인하기, 사람에게 질문하기도 embodied action이다.

## 22.4 Self-improving data loop

static pretrained VLA는 배치 후 새로운 object, scene, user, failure를 만난다. 따라서 future VLA는 deployment data를 다시 model improvement로 연결해야 한다.

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \mathcal{F}(\tau_{1:N}^{deploy}, h_{1:N}^{human}, f_{1:N}^{failure}). \quad (22.5)$$

여기서  $\mathcal{F}$ 는 rollout, human feedback, failure label을 curated training data로 바꾸는 data engine이다.

이 loop는 자동화될 수 있지만 완전히 무감독이면 안 된다. robot data는 physical risk를 갖기 때문에 release gate가 필요하다.

## 22.5 Safe online adaptation

online adaptation은 매력적이지만 위험하다. policy update가 안전성 regression을 만들 수 있기 때문이다. update  $\theta_k \rightarrow \theta_{k+1}$ 는 다음 gate를 통과해야 한다.

$$\begin{aligned} \text{Accept}(\theta_{k+1}) &= \mathbb{I}[M_{task}(\theta_{k+1}) \geq M_{task}(\theta_k) - \epsilon_t \wedge \\ &\quad M_{safe}(\theta_{k+1}) \geq M_{safe}(\theta_k) - \epsilon_s \wedge \\ &\quad R(\theta_{k+1}) \leq R_{max}]. \end{aligned} \quad (22.6)$$

task metric이 조금 좋아져도 safety metric이 나빠지면 release해서는 안 된다.

표 62: Self-improving VLA data loop

| Stage    | 하는 일                                    | 주요 risk                                |
|----------|-----------------------------------------|----------------------------------------|
| Deploy   | policy rollout과 log 수집                  | unsafe exploration, logging gap        |
| Diagnose | failure taxonomy 와 root cause 분석        | superficial label, hidden intervention |
| Curate   | correction, near-miss, recovery data 구성 | biased sampling, duplicate shortcut    |
| Train    | adapter, action head, policy update     | forgetting, overfit, safety regression |
| Gate     | offline, sim, real-robot validation     | weak benchmark, untested shift         |
| Release  | versioned deployment와 rollback 준비       | config drift, calibration mismatch     |

safe online adaptation은 다음 세 가지를 요구한다.

1. update scope 제한: action head, adapter, calibration parameter 등부터 조정한다.
2. rollback 가능성: 새 model이 실패하면 즉시 이전 version으로 돌아간다.
3. audit log: 어떤 data와 어떤 metric으로 update가 승인되었는지 남긴다.

## 22.6 Multi-embodiment transfer

미래 VLA의 핵심 질문 중 하나는 여러 robot embodiment 사이의 transfer이다. 같은 instruction도 gripper, arm length, camera pose, base mobility, controller가 다르면 action이 달라진다.

$$\mathbf{a}_t^{(e)} \sim \pi_{\theta}(\cdot \mid \mathbf{o}_{\leq t}^{(e)}, \ell, a_e), \quad (22.7)$$

여기서  $e$ 는 embodiment,  $a_e$ 는 embodiment descriptor 또는 adapter이다.

transfer는 다음 세 layer로 나누어야 한다.

1. semantic transfer: 같은 task instruction과 object relation을 이해한다.
2. geometric transfer: robot morphology에 맞는 reachability와 frame을 계산한다.
3. control transfer: action scale, frequency, controller wrapper를 맞춘다.

표 63: Multi-embodiment transfer의 실패 모드

| Failure                     | 원인                                  | 대응                                   |
|-----------------------------|-------------------------------------|--------------------------------------|
| Same word, different action | gripper와 controller convention 차이   | embodiment descriptor, action schema |
| Reachability mismatch       | arm length와 base mobility 차이        | geometric feasibility layer          |
| Sensor mismatch             | camera pose, FOV, depth quality 차이  | sensor adapter, calibration report   |
| Dynamics mismatch           | payload, compliance, speed limit 차이 | controller-aware action scaling      |
| Safety mismatch             | human proximity와 force limit 차이     | robot-specific safety envelope       |

## 22.7 Safety assurance as a research primitive

VLA가 실제 세계로 갈수록 safety assurance는 부록이 아니라 research primitive가 된다. 앞으로의 논문은 model score와 함께 safety case를 제시해야 한다.

$$\text{PaperClaim} = (M_{\text{benchmark}}, M_{\text{real}}, M_{\text{safe}}, \mathcal{A}_{\text{assurance}}, \mathcal{L}_{\text{logs}}). \quad (22.8)$$

benchmark success만으로는 deployment claim을 할 수 없다. real robot protocol, safety metric, incident log, failure taxonomy가 함께 있어야 한다.

formal verification은 모든 VLA behavior를 증명하는 방식으로 당장 쓰이기는 어렵다. 그러나 bounded subsystem에는 유용하다. 예를 들어 speed limit, human

distance, controller saturation, collision checker, watchdog deadline은 formal 또는 semi-formal assurance가 가능하다.

## 22.8 Hardware-aware scaling

model scaling은 계속 중요하다. 그러나 robotics에서는 scaling law가 hardware와 닫힌 루프 안에서 해석되어야 한다.

$$J = M_{task} - \lambda_1 T_{lat} - \lambda_2 C_{compute} - \lambda_3 R_{unsafe} - \lambda_4 R_{intervention}. \quad (22.9)$$

큰 model이  $M_{task}$ 를 올려도 latency와 unsafe rate를 같이 올리면 deployment objective는 나빠질 수 있다.

앞으로 중요한 연구 방향은 다음과 같다.

1. large reasoning model과 small real-time policy의 분업
2. action head와 controller wrapper의 hardware-aware 설계
3. quantization 후 safety regression 측정
4. edge accelerator에 맞는 vision-action architecture
5. deadline miss를 고려한 benchmark와 reporting

## 22.9 Evaluation becomes infrastructure

VLA evaluation은 논문 마지막 표가 아니라 infrastructure가 되어야 한다. 같은 model을 여러 embodiment, task, controller, latency condition에서 비교하려면 standard harness가 필요하다.

$$\mathcal{E} = (\mathcal{T}, \mathcal{R}, \mathcal{B}, \mathcal{M}, \mathcal{P}). \quad (22.10)$$

$\mathcal{T}$ 는 task suite,  $\mathcal{R}$ 는 robot/embodiment set,  $\mathcal{B}$ 는 benchmark protocol,  $\mathcal{M}$ 은 metric,  $\mathcal{P}$ 는 reporting package이다.

좋은 evaluation infrastructure는 model ranking만 만들지 않는다. 실패를 data engine으로 돌려보내고, safety case를 강화하며, adaptation target을 알려준다.

# 22.10 Research agenda

표 64: Real VLA 연구 agenda

| Agenda              | 핵심 질문                                    | 필요한 evidence                                     |
|---------------------|------------------------------------------|--------------------------------------------------|
| World model         | action consequence와 hidden state를 예측하는가? | counterfactual, recovery, uncertainty test       |
| Data engine         | failure가 다음 data collection으로 이어지는가?     | failure taxonomy, correction data, release gate  |
| Action interface    | policy output이 controller와 맞는가?          | tracking, saturation, interruption latency       |
| Embodiment transfer | robot이 바뀌어도 task semantics가 유지되는가?       | adapter ablation, cross-robot validation         |
| Safety assurance    | unsafe success를 줄였는가?                    | near-miss log, safety benchmark, incident report |
| Real-time runtime   | hardware budget 안에서 동작하는가?               | p95 latency, watchdog, thermal drift             |
| Humanoid extension  | balance와 manipulation이 함께 되는가?           | whole-body metrics, fall/near-fall reporting     |

# 22.11 What should roboticists learn?

foundation model 시대의 roboticist는 model API를 호출하는 사람도, 고전 control 만 고집하는 사람도 아니다. 앞으로 필요한 감각은 system-level integration이다.

1. model이 무엇을 알고 무엇을 모르는지 evidence로 판단한다.
2. action representation을 controller, safety, latency와 함께 설계한다.
3. data collection을 model training의 전처리가 아니라 closed-loop improvement mechanism으로 본다.
4. benchmark score와 real deployment claim을 분리해서 읽는다.



5. failure log를 연구 자산으로 다룬다.
6. safety를 prompt policy가 아니라 robot stack의 hard constraint로 본다.

## 22.12 Further reading and exercises

**Further reading** Future direction을 model scale, data engine, action interface, evaluation, safety assurance로 나누어 읽어라. 최신 system은 evidence tier를 함께 적어야 한다.

**Review question** VLA의 다음 발전이 단순히 더 큰 VLM이 아니라 더 강한 system contract여야 하는 이유는 무엇인가?

**Design exercise** 향후 1년 연구계획을 하나 작성하라. 최소한 target task, action representation, data engine, evaluation protocol, safety evidence를 포함해야 한다.

## 22.13 Closing synthesis

이 책의 출발점은 “VLA는 더 큰 VLM이 아니다”였다. 마지막 결론도 같다. Real VLA는 vision, language, action이 결합된 model이지만, model만으로 완성되지 않는다. state estimation, dynamics, control, contact, planning, imitation learning, reinforcement learning, transformer architecture, data engine, adaptation, evaluation, real-time runtime, safety, humanoid embodiment가 함께 작동해야 한다.

VLA는 기존 로봇공학을 대체하지 않는다. 오히려 기존 로봇공학이 다루던 문제를 더 넓은 task distribution과 language interface 위에서 다시 묻는다. controller는 여전히 필요하고, planning은 여전히 필요하며, dynamics와 contact는 사라지지 않는다. 달라지는 것은 이 모든 요소가 foundation model과 data loop를 중심으로 다시 연결된다는 점이다.

Real VLA를 연구한다는 것은 하나의 거대 모델을 기다리는 일이 아니다. 로봇이 실제 세계에서 명령을 이해하고, 볼 것을 보고, 할 수 있는 일을 하고,

모르면 멈추고, 실패하면 배우고, 사람과 환경을 해치지 않도록 만드는 system을 설계하는 일이다. 이것이 로봇공학자의 관점에서 본 VLA의 핵심이다.

# Instructor and Editorial Note

## 강의 운영과 원고 검토를 위한 기준

이 부록은 독자가 본문을 읽는 데 필요한 필수 장은 아니다. 강의 담당자, reading group 운영자, 원고 검토자가 이 책을 수업 또는 출판 원고로 사용할 때 확인해야 할 기준을 정리한다.

편집자나 강의 담당자가 이 원고를 검토할 때 가장 먼저 보아야 할 것은 주제의 최신성보다 구조의 검증 가능성이다. 최신 모델 이름은 빠르게 바뀌지만, 실제 로봇 시스템에서 남는 질문은 더 오래 간다. 이 원고가 책으로 성립하려면 각 장은 단순한 배경 설명이 아니라 다음 네 가지 산출물을 남겨야 한다.

### 각 장의 최소 산출물

각 장은 하나의 central claim, 그 claim을 지탱하는 최소 정식화, 다음 layer로 넘기는 interface, 그리고 그 claim이 실패하는 조건을 명시해야 한다. 이 네 가지 중 하나가 빠지면 장은 survey paragraph에 머물고, 강의 단위로는 충분히 닫히지 않는다.

이 기준에서 보면 이 원고의 강점은 분명하다. VLA를 model taxonomy가 아니라 system responsibility로 재정의하고, classical robotics와 embodied foundation model을 하나의 interface 문제로 묶는다. 반대로 가장 위험한 부분도 분명하다. 각 장이 논문 이름과 시스템 원리를 충분히 연결하지 못하면, 원고는 최신 분야 소개에 머물 수 있다. 따라서 강의나 개정에서는 각 장마다 대표 논문, 구현 interface, 평가 protocol, 실패 사례를 같은 구조로 정렬해야 한다.

수업 관점에서는 한 장을 다음 순서로 운영하는 것이 적절하다.

1. 먼저 그 장의 system claim을 제시한다.

표 65: 원고 검토자가 각 장에서 확인해야 할 quality gate

| 검토 항목             | 통과 기준                                                                          | 약한 원고의 신호                           |
|-------------------|--------------------------------------------------------------------------------|-------------------------------------|
| Chapter claim     | 그 장이 Real VLA system claim의 어느 부분을 닫는지 한 문장으로 말할 수 있다.                         | 기술 소개는 많지만 왜 이 장이 필요한지 불명확하다.       |
| Formal object     | 핵심 변수가 action, state, belief, constraint, data record, metric 중 어디에 속하는지 드러난다. | 용어는 많지만 수식이나 interface가 남지 않는다.     |
| System interface  | 앞 장에서 받은 입력과 뒤 장으로 넘기는 출력이 구분된다.                                               | chapter order는 있으나 dependency가 약하다. |
| Evidence standard | benchmark, real-robot log, ablation, failure case 중 무엇이 claim을 지지하는지 말한다.      | 성능 수치만 있고 실험 조건과 실패 조건이 분리되지 않는다.   |
| Teaching unit     | 강의 후 학생이 풀 수 있는 engineering question이 남는다.                                     | 읽을 수는 있지만 과제, 토론, 설계 문제로 바뀌지 않는다.   |

2. 그 claim을 표현하는 최소 변수와 mapping을 고정한다.
3. 실제 robot stack에서 그 mapping이 어느 module 사이의 interface인지 확인한다.
4. 마지막으로 그 claim이 어떤 data, benchmark, safety log 없이는 성립하지 않는지 따진다.

이 운영 방식은 책의 난도를 낮추기 위한 장치가 아니다. 학생이 model architecture를 외우는 데서 멈추지 않고, action이 controller command로 내려가는 순간 어떤 가정이 추가되는지, benchmark success가 real deployment claim으로 올라갈 때 어떤 evidence가 더 필요한지를 따지게 만드는 장치이다.

## 참고 문헌

- [1] R. E. Kalman. A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 1960.
- [2] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [3] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer, 2009.
- [4] Mark W. Spong, Seth Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, 2006.
- [5] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- [6] Oussama Khatib. A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation. *IEEE Journal on Robotics and Automation*, 1987.
- [7] Neville Hogan. Impedance Control: An Approach to Manipulation. *ASME Journal of Dynamic Systems, Measurement, and Control*, 1985.
- [8] David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. *Constrained Model Predictive Control: Stability and Optimality*. Automatica, 2000.
- [9] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control Barrier Functions: Theory and Applications. *European Control Conference*, 2019.

- [10] Matthew T. Mason. *Mechanics of Robotic Manipulation*. MIT Press, 2001.
- [11] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [12] Lydia E. Kavraki, Petr Svestka, Jean-Claude Latombe, and Mark H. Overmars. Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces. *IEEE Transactions on Robotics and Automation*, 1996.
- [13] Steven M. LaValle and James J. Kuffner. Randomized Kinodynamic Planning. *International Journal of Robotics Research*, 2001.
- [14] Leslie Pack Kaelbling and Tomas Lozano-Perez. Hierarchical Task and Motion Planning in the Now. *IEEE International Conference on Robotics and Automation*, 2011.
- [15] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. *AISTATS*, 2011.
- [16] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018.
- [17] Tuomas Haarnoja et al. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ICML*, 2018.
- [18] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-Learning for Offline Reinforcement Learning. *NeurIPS*, 2020.
- [19] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-End Training of Deep Visuomotor Policies. *Journal of Machine Learning Research*, 2016.
- [20] Frederik Ebert et al. Visual Foresight: Model-Based Deep Reinforcement Learning for Vision-Based Robotic Control. *arXiv:1812.00568*, 2018.

- [21] Andy Zeng, Pete Florence, Jonathan Tompson, Stefan Welker, Jonathan Chien, Maria Attarian, Travis Armstrong, Ivan Krasin, Dan Duong, Ayzaan Wahid, Vikas Sindhwani, and Johnny Lee. Transporter Networks: Rearranging the Visual World for Robotic Manipulation. Conference on Robot Learning, 2020; PMLR, 2021. arXiv:2010.14406.
- [22] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. CLIPort: What and Where Pathways for Robotic Manipulation. Conference on Robot Learning, 2021; PMLR 164, 2022. arXiv:2109.12098.
- [23] Ashish Vaswani et al. Attention Is All You Need. NeurIPS, 2017.
- [24] Alexey Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR, 2021.
- [25] Alec Radford et al. Learning Transferable Visual Models From Natural Language Supervision. ICML, 2021.
- [26] Jean-Baptiste Alayrac et al. Flamingo: a Visual Language Model for Few-Shot Learning. NeurIPS, 2022.
- [27] Junnan Li et al. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. ICML, 2023.
- [28] Michael Ahn et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. Conference on Robot Learning, 2022; PMLR 205, 2023. arXiv:2204.01691.
- [29] Jacky Liang et al. Code as Policies: Language Model Programs for Embodied Control. arXiv:2209.07753, 2022.
- [30] Wenlong Huang et al. Inner Monologue: Embodied Reasoning through Planning with Language Models. Conference on Robot Learning, 2022; PMLR 205, 2023. arXiv:2207.05608.

- [31] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. Conference on Robot Learning, 2023; PMLR 229, 2023. arXiv:2307.05973.
- [32] Scott Reed et al. A Generalist Agent. arXiv:2205.06175, 2022.
- [33] Eric Jang, Alex Irpan, Mohi Khansari, Daniel Kappler, Frederik Ebert, Corey Lynch, Sergey Levine, and Chelsea Finn. BC-Z: Zero-Shot Task Generalization with Robotic Imitation Learning. Conference on Robot Learning, 2021; PMLR 164, 2022. arXiv:2202.02005.
- [34] Anthony Brohan et al. RT-1: Robotics Transformer for Real-World Control at Scale. Robotics: Science and Systems, 2023. arXiv:2212.06817.
- [35] Danny Driess et al. PaLM-E: An Embodied Multimodal Language Model. International Conference on Machine Learning, 2023; PMLR 202. arXiv:2303.03378.
- [36] Anthony Brohan et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818, 2023.
- [37] Konstantinos Bousmalis et al. RoboCat: A Self-Improving Generalist Agent for Robotic Manipulation. Transactions on Machine Learning Research, 2023. arXiv:2306.11706.
- [38] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. arXiv:2310.08864, 2023.
- [39] Tony Z. Zhao et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. arXiv:2304.13705, 2023.
- [40] Cheng Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. arXiv:2303.04137, 2023.



- [41] Octo Model Team et al. Octo: An Open-Source Generalist Robot Policy. arXiv:2405.12213, 2024.
- [42] Moo Jin Kim et al. OpenVLA: An Open-Source Vision-Language-Action Model. Conference on Robot Learning, 2024; PMLR 270:2679–2713, 2025. arXiv:2406.09246.
- [43] Kevin Black et al.  $\pi_0$ : A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164, 2024.
- [44] Karl Pertsch et al. FAST: Efficient Action Tokenization for Vision-Language-Action Models. arXiv:2501.09747, 2025.
- [45] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success. arXiv:2502.19645, 2025.
- [46] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. RDT-1B: a Diffusion Foundation Model for Bimanual Manipulation. arXiv:2410.07864, 2024.
- [47] NVIDIA et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. arXiv:2503.14734, 2025.
- [48] Physical Intelligence et al.  $\pi_{0.5}$ : a Vision-Language-Action Model with Open-World Generalization. arXiv:2504.16054, 2025.
- [49] Homer Walke et al. BridgeData V2: A Dataset for Robot Learning at Scale. arXiv:2308.12952, 2023.
- [50] Alexander Khazatsky et al. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. arXiv:2403.12945, 2024.
- [51] Bo Liu et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. NeurIPS Datasets and Benchmarks, 2023. arXiv:2306.03310.

- [52] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. *IEEE Robotics and Automation Letters*, 2022. arXiv:2112.03227.
- [53] Oier Mees, Lukas Hermann, and Wolfram Burgard. What Matters in Language Conditioned Robotic Imitation Learning over Unstructured Data. arXiv:2204.06252, 2022.
- [54] Jiayuan Gu et al. ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills. *ICLR*, 2023.
- [55] Moo Jin Kim et al. SimplerEnv: Simulated Manipulation Policy Evaluation Environments for Real-World Robots. arXiv:2405.05941, 2024.
- [56] Jean Mercat, Sedrick Keh, Kushal Arora, Isabella Huang, Paarth Shah, Haruki Nishimura, Shun Iwase, and Katherine Liu. VLA Foundry: A Unified Framework for Training Vision-Language-Action Models. arXiv:2604.19728, 2026.
- [57] Ajay Mandlekar et al. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation. *Conference on Robot Learning*, 2021.
- [58] Yuke Zhu et al. robosuite: A Modular Simulation Framework and Benchmark for Robot Learning. arXiv:2009.12293, 2020.
- [59] Mohammed Alshiekh et al. Safe Reinforcement Learning via Shielding. *AAAI*, 2018.
- [60] Kim P. Wabersich, Lukas Hewing, and Melanie N. Zeilinger. A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems. *Automatica*, 2021.
- [61] International Organization for Standardization. ISO 10218: Robots and Robotic Devices – Safety Requirements for Industrial Robots. *ISO*, 2011.

- [62] International Organization for Standardization. ISO/TS 15066: Robots and Robotic Devices – Collaborative Robots. ISO, 2016.